
AD14N AD15N AD17N AD18N AC104N

发行版本 v1.6

liujie

2025 年 12 月 25 日

1	文档版本说明	2
1.1	v1.6 说明	2
1.2	v1.5 说明	2
1.3	v1.4 说明	3
1.4	v1.3 说明	3
1.5	v1.2 说明	4
1.6	v1.1 说明	5
1.7	v1.0 说明	5
2	常见问题快速索引	6
2.1	外挂资源 flash 如何烧写资源	7
2.2	保持 IO 状态	7
3	SDK 介绍	8
3.1	新版 SDK 命名说明	8
3.2	SDK 工程与配置	9
3.2.1	应用配置文件《app_config.c》介绍	11
3.2.2	应用配置文件《app_config.h》介绍	14
3.2.3	应用 APP 配置文件《app_modules.h》介绍	17
3.2.4	CodeBlocks 工程宏定义介绍	19
3.3	新版本 SDK 工程裁切（2023 年 7 月发布的 SDK 起始）	20
3.4	旧版本 SDK 工程裁切	20
3.4.1	如何将解码格式从工程中移除	20
3.4.2	如何将压缩格式从工程中移除	22
4	芯片框图和硬件环境介绍	25
4.1	AD14N 芯片框图和硬件开发环境	25
4.2	AD15N 芯片框图和硬件开发环境	28
4.3	AD17N 芯片框图和硬件开发环境	30
4.4	AD18N 芯片框图和硬件开发环境	32
5	下载目录介绍	35
5.1	下载目录介绍	35
5.2	dowload_bat.c 文件介绍	36
5.3	外挂资源 flash 下载（AD23N 暂不支持）	37
5.4	isd_config.ini 配置文件说明链接	38
5.5	isd_config.ini 文件介绍	38

5.6	AD23N/24N 的 isd_config.ini 文件重要变化介绍	38
5.7	isd_config.ini 文件指令介绍	38
6	杰理科技一拖二烧录器用户手册链接	42
7	工具使用环境介绍	43
7.1	开发工具介绍	43
7.2	音频转换工具	44
7.3	资源打包工具	47
7.4	多算法授权简单介绍	49
8	时钟驱动	50
8.1	AD14N / AC104N 系统时钟超频说明	51
8.2	内置系统 flash spi 时钟说明	51
8.2.1	AD14N / AC104N SPI 时钟限制	52
8.2.2	AD15N SPI 时钟限制	53
8.2.3	AD17N SPI 时钟限制	54
8.2.4	AD24N SPI 时钟限制	54
8.3	时钟相关函数	55
8.3.1	函数 void pll_sel(u32 pll_clock, _PLL_DIV pll_div, _PLL_B_DIV pll_b_div)	55
8.3.2	函数 int clk_get(const char *name);	55
8.3.3	函数 int clk_set(const char *name, int clk)	55
8.3.4	函数 int clk_early_init(enum pll_ref_source pll_ref, u32 ref_frequency, u32 pll_frequency);	56
9	异常	57
9.1	异常分析	58
9.2	常见异常	60
10	tick_timer	61
10.1	注意事项	62
11	内置系统 flash 写保护	63
11.1	A0 芯片如何添加写保护参数	64
11.1.1	flash 写保护工具说明	64
11.1.2	新增 flash 写保护参数与指令	65
11.1.3	写保护参数下载说明	70
11.1.4	第一点相关函数说明	71
11.2	内封 flash 芯片如何添加写保护参数（旧）	71
11.2.1	配置烧写器获取内封 flash 写保护参数	71
11.2.2	配置强制下载工具获取内封 flash 写保护参数	72
11.2.3	第三点相关函数说明	72
11.3	内封 flash 芯片如何添加写保护参数（新）	73
11.4	程序中开启与关闭写保护功能	73
11.4.1	第五点相关函数说明	73
11.4.2	常见问题说明	74
12	中断系统	76
12.1	中断的注册与注销	76
12.1.1	函数 void HWI_Install(unsigned char index, unsigned int isr, unsigned char priority)	79
12.1.2	函数 void HWI_Uninstall(unsigned char index)	80
12.2	软中断说明	80
12.2.1	函数 void bit_set_swi(unsigned char index)	82
12.2.2	函数 void bit_clr_swi(unsigned char index)	82
12.3	软件不可屏蔽中断	82
12.3.1	函数 void _OS_ENTER_CRITICAL(u32 bit_list)	83

12.3.2	函数 void _OS_ENTER_CRITICAL(u32 bit_list_l, u32 bit_list_h)	83
12.3.3	函数 void _OS_ENTER_CRITICAL(IRQ_IDX_BITMAP *p_irq_bitmap)	84
12.3.4	函数 void _OS_EXIT_CRITICAL(void)	84
12.3.5	函数 void vm_isr_response_index_register(u8 index)	84
12.3.6	函数 void vm_isr_response_index_unregister(u8 index)	84
12.3.7	函数 u32 get_vm_isr_response_index_l(void)	84
12.3.8	函数 u32 get_vm_isr_response_index_h(void)	85
12.4	硬件不可屏蔽中断	85
12.4.1	函数 void irq_unmask_set(u8 index)	85
12.4.2	函数 void irq_unmask_disable(u8 index)	85
13	AD14N 与 AD15N 的内核异常管理单元	86
13.1	内核异常示例 (非对齐访问):	87
13.1.1	第 1, 2 行	88
13.1.2	第 4, 5 行堆栈边界	88
13.1.3	第 6~13 行 CPU 瞬态	88
13.1.4	第 15~25 行	90
13.1.5	第 27~28 行	90
13.1.6	第 29 行异常信息种类	90
13.1.7	第 31~45 行通用寄存器信息	91
14	AD17N 的内核异常管理单元	92
14.1	异常示例 (非对齐访问):	93
14.1.1	第 1, 2 行	94
14.1.2	第 4, 5 行	94
14.1.3	第 6~14 行 CPU 瞬态	95
14.1.4	第 16~28 行 reti 与 rets	96
14.1.5	第 30~45 行 sp	96
14.1.6	第 47 行异常信息种类	97
14.1.7	第 49~63 行通用寄存器信息	97
14.2	外部异常:	98
14.2.1	示例-AUDIO 触发外部异常	98
14.2.2	示例-看门狗 WDT 触发异常	99
15	GPIO 接口说明	100
15.1	DACP, DACN 口设置 (AC104/AD14/15/17)	100
15.2	普通 IO 口 (AC104/AD14/15/17/18)	101
15.3	模块 IO 映射	103
15.3.1	AC104/AD14/AD15	103
15.3.2	AD17	104
15.3.3	AD18	105
15.4	常见问题和注意事项	105
15.4.1	AC104/AD14/AD15/AD17 生产不良率高	105
15.4.2	AD18 部分 IO 下拉使用寄存器设置无效	105
16	设备管理	106
16.1	设备注册表	107
16.2	设备函数	110
16.2.1	函数 int devices_init()	110
16.2.2	函数 bool dev_online(const char* name)	110
16.2.3	函数 void* dev_open(const char* name, void* arg)	110
16.2.4	函数 int dev_bulk_read(void* _device, void* buf, u32 offset, u32 len)	111
16.2.5	函数 int dev_bulk_write(void* _device, void* buf, u32 offset, u32 len)	111
16.2.6	函数 int dev_byte_read(void* _device, void* buf, u32 offset, u32 len)	111
16.2.7	函数 int dev_byte_write(void* _device, void* buf, u32 offset, u32 len)	112

16.2.8	函数 int dev_ioctl(void* device, int cmd, u32 arg)	112
16.2.9	函数 int dev_close(void* device)	113
17	文件系统管理	114
17.1	文件系统介绍	116
17.2	文件系统使用一般流程	116
17.3	通用文件系统管理函数	117
17.3.1	函数 void vfs_init(void)	117
17.3.2	函数 u32 vfs_mount(void **ppvfs, void *device, void *type)	117
17.3.3	函数 u32 vfs_openbypath(void *pvfs, void **ppvfile, const char *path)	118
17.3.4	函数 u32 vfs_openbyindex(void *pvfs, void **ppvfile, u32 index)	118
17.3.5	函数 u32 vfs_createfile(void *pvfs, void **ppvfile, u32 *pindex)	119
17.3.6	函数 u32 vfs_read(void *pvfile, void *buf, u32 len)	120
17.3.7	函数 u32 vfs_write(void *pvfile, void *buf, u32 len)	120
17.3.8	函数 u32 vfs_seek(void *pvfile, u32 offset, u32 mode)	120
17.3.9	函数 u32 vfs_file_close(void **ppvfile)	121
17.3.10	函数 u32 vfs_fs_close(void **ppvfs)	122
17.3.11	函数 u32 vfs_file_name(void *pvfile, void *name, u32 len)	122
17.3.12	函数 int vfs_get_attrs(void *pvfile, void *pvfs_attr)	122
17.3.13	函数 int vfs_ioctl(void *pvfile, int cmd, int arg)	123
18	sydfs 文件系统	125
18.1	函数 int vfs_file_crc(void *pvfile)	127
19	norfs 文件系统	128
20	fat 文件系统	130
20.1	FAT 文件系统管理函数	131
20.1.1	函数 int fs_ftell(void *pvfile, void *parm)	131
20.1.2	函数 int vfs_get_fsize(void *pvfile, void *parm)	131
20.1.3	函数 struct vfscan *vfs_fscan_new(void *pvfs, const char *path, const char *arg, u8 max_depth, int (*callback)(void), struct vfscan *fsn, struct vfscan_reset_info *info)	131
20.1.4	函数 void fs_fscan_release(void *pvfs, struct vfscan *fs)	132
20.1.5	函数 int fs_get_folderinfo(void **ppvfile, struct vfscan *fs, int *start_num, int *end_num)	132
20.1.6	函数 int fs_get_encfolder_info(void *pvfs, char *folder, char *ext, u32 *last_num, u32 *total_num)	132
20.1.7	函数 u32 vfs_file_name(void *pvfile, void *name, u32 len)	133
20.1.8	函数 int fs_select(void *pvfs, void **ppvfile, struct vfscan *fs, int sel_mode, int arg)	133
20.1.9	函数 int vfs_mk_dir(void *pvfs, char *folder, u8 mode)	133
20.1.10	函数 int vfs_delete_dir(void *pvfs, char *path)	134
20.1.11	函数 int vfs_get_encfolder_info(void *pvfs, char *folder, char *ext, u32 *last_num, u32 *total_num)	134
20.1.12	函数 u32 vfs_file_delete(void *pvfile)	134
20.1.13	函数 int vfs_select(void *pvfs, void **ppvfile, struct vfscan *fs, int sel_mode, int arg)	135
20.1.14	函数 int vfs_ioctl(void *pvfile, int cmd, int arg)	135
20.1.15	文件夹删除函数 int vfs_delete_dir(void *pvfs, char *path)	136
20.1.16	格式化接口 int vfs_format(void **ppvfs, const char *dev_name, const char *type, u32 clust_size, u8 create_new)	136
20.2	FAT 文件系统简易驱动 & FAT 文件系统完整驱动区别	140
21	freefs 文件系统	141
22	解码接口	144
22.1	解码使用一般流程	145
22.2	数据流获取	146

22.3	解码管理接口函数	147
22.3.1	函数 <code>dec_obj *decoder_io(void *pfile, u32 dec_ctl, dp_buff * dbuff, u8 loop)</code>	147
22.3.2	函数 <code>void irq_decoder_ret(dec_obj *obj, u32 ret)</code>	148
22.4	解码功能控制函数	149
22.4.1	函数 <code>void decoder_pause(dec_obj *obj)</code>	149
22.4.2	函数 <code>void decoder_stop(dec_obj *obj, DEC_STOP_WAIT wait)</code>	149
22.5	各个格式的解码函数	149
22.5.1	统一介绍 <code>decode_api</code>	149
23	F1X 格式解码流程说明	150
23.1	简介	150
23.2	结构体 / 函数说明	150
23.2.1	结构体 <code>f1x_ctl</code>	150
23.2.2	函数 <code>void *f1x_play_api(void *pfile, u8 *loop_tab, u32 size, u8 index, u32 addr)</code>	151
23.2.3	函数 <code>int f1x_parsing(void *pfile, u32 *buff, u32 sum)</code>	151
23.2.4	函数 <code>u32 set_play_file(void *priv, u32 *startpos, u32 *endpos)</code>	151
23.3	解码流程	151
24	编码接口	153
24.1	编码应用流程	155
24.2	编码功能控制接口函数	155
24.2.1	<code>void encoder_io(u32(* fun)(void*), void* pfile)</code>	155
24.2.2	<code>void stop_encode(void* pfile, u32 dlen)</code>	155
25	JL_MIO 功能	156
25.1	JLmio 原理	156
25.2	JLmio 一般流程	157
25.2.1	使用 <code>JLmio.exe</code> 创建 <code>mio</code> 文件	157
25.2.2	使用打包工具将音频文件与所制成的 <code>mio</code> 文件放于同一目录下	157
25.2.3	在 <code>download.bat</code> 中添加相应 <code>mio</code> 文件夹	158
25.2.4	在 <code>app_music.c</code> 中添加相应 <code>mio</code> 文件夹	158
25.2.5	在 <code>mio_api.c</code> 中将相应宏开关打开	159
25.3	JLmio 接口介绍	160
25.3.1	<code>u32 mio_a_pwm_init(u32 chl)</code>	160
25.3.2	<code>u32 mio_a_pwm_run(u32 chl, u32 pwm_var)</code>	160
25.3.3	<code>u32 mio_a_pwm_uninit(u32 chl)</code>	160
25.3.4	<code>u32 mio_a_io_init(u32 mask)</code>	161
25.3.5	<code>u32 mio_a_io_run(u32 mask, u32 io_ver)</code>	161
25.3.6	<code>u32 mio_a_io_uninit(u32 mask)</code>	161
26	玩具应用篇 (voice_toy)	162
26.1	简介	162
26.1.1	模式相关函数	163
26.2	<code>toy_music</code> 多路解码模式	163
26.2.1	模式说明	163
26.2.2	解码播放函数	164
26.3	<code>toy_midi</code> <code>midi</code> 解码模式	166
26.3.1	模式说明	166
26.3.2	相关函数	166
26.4	<code>toy_midi_keyboard</code> <code>midi</code> 琴模式	167
26.4.1	模式说明	167
26.4.2	相关函数	167
26.5	<code>toy_speaker</code> 扩音模式	170
26.5.1	模式说明	170
26.5.2	相关函数	170

26.6	toy_linein linein 模式	170
26.6.1	模式说明	170
26.7	toy_record 录音模式	170
26.7.1	模式说明	170
26.7.2	相关函数	171
27	玩具应用篇 (旧工程 - app_mg)	172
27.1	简介	172
27.2	模式初始化函数	173
27.2.1	函数 int app_mg_init(APP_T app,void* param)	173
27.2.2	函数 void app(void)	173
27.3	模式切换函数	174
27.3.1	函数 int app_switch(APP_T app,void* param)	174
27.3.2	函数 int app_switch_next(void)	174
27.3.3	函数 int app_switch_priv(void)	174
27.3.4	函数 u8 app_get(void)	174
27.4	常用工程配置文件	174
27.4.1	“app_config.c” 文件	174
27.4.2	函数 u8 app_get(void)	174
27.4.3	code block 工程配置	175
28	小音箱应用	176
29	AUDIO DAC	178
29.1	AUDIO DAC 各芯片的音频性能指标	179
29.2	AUDIO DAC 配置参数	180
29.2.1	AD14N / AD15N / AC104N DAC 驱动模式介绍	180
29.2.2	AD17N APA 驱动模式介绍	181
29.2.3	AD18N AUDIO 驱动模式介绍	183
29.3	AUDIO DAC 接口介绍	187
29.3.1	函数 void dac_mode_init(void)	187
29.3.2	函数 void dac_init_api(u32 sr)	187
29.3.3	函数 void dac_power_on(u32 sr, bool delay_flag)	187
29.3.4	函数 void dac_off_api(void)	187
29.3.5	函数 bool dac_mute(bool mute)	188
29.3.6	函数 u8 dac_vol(char set, u8 vol)	188
29.3.7	函数 bool regist_dac_channel(void *psound, void *kick)	188
29.3.8	函数 bool unregist_dac_channel(void *psound)	188
29.3.9	函数 void fill_dac_fill(u8 *buf, u32 len)	188
29.3.10	函数 void single_apa_startup_mute_cb(void)	189
29.3.11	函数 void single_apa_mute(u8 mute)	189
30	AUDIO ADC (AD14N / AC104N)	190
30.1	AUDIO ADC 性能指标	191
30.2	AUDIO ADC 接口介绍	191
30.2.1	函数 void audio_adc_mode_init(void)	191
30.2.2	函数 u32 audio_adc_init_api(u32 sr, ADC_MODE mode, u32 ch)	191
30.2.3	函数 void audio_adc_off_api(void)	191
30.2.4	函数 bool regist_audio_adc_channel(void *psound, void *kick)	191
30.2.5	函数 bool unregist_audio_adc_channel(void *psound)	192
30.2.6	函数 void fill_audio_adc_fill(u8 *buf, u32 len)	192
31	Audio 指标测试数据 (AD14N / AC104N)	193
31.1	测试机指标名词说明	194
31.2	AD14/AC104N Audio_ADC 音频性能指标测试说明	194

31.2.1	AD14/AC104N Audio_ADC 性能测试条件	194
31.2.2	AD14/AC104N Audio_ADC 性能测试数据	195
31.3	AD14/AC104N Audio_DAC 音频性能指标测试说明	196
31.3.1	AD14/AC104N Audio_DAC(RDAC 输出) 性能指标测试说明	196
31.3.2	AD14/AC104N Audio_DAC(APA 输出) 性能指标测试说明	197
31.4	AD14/AC104N Audio_AD2DA 音频性能指标测试说明	198
31.4.1	AD14/AC104N Audio_AD2DA(RDAC 输出) 性能指标测试说明	199
31.4.2	AD14/AC104N Audio_AD2DA(APA 输出) 性能指标测试说明	200
32	音量	203
32.1	底层音量接口	203
32.1.1	函数 void dac_mode_init(u16 vol, void *phy_vol_set_func)	204
32.1.2	函数 u8 dac_vol(char set,u8 vol)	204
32.2	音量应用介绍	204
33	混响 (echo)	206
33.1	1 配置参数说明	206
33.2	相关函数	207
33.2.1	函数 void *link_echo_sound(void *p_sound_out, void *p_dac_cbuf, void **pp_effect, u32 sr)	207
33.2.2	函数 void *echo_api(void *obuf, u32 sr, void **ppsound)	207
33.2.3	函数 void echo_release(void **ppeffect)	207
34	移频啸叫抑制 (pitchshifter_howling)	208
34.1	1 配置参数说明	208
34.2	2 相关函数	209
34.2.1	2.1 函数 void *link_pitchshift_howling_sound(void *p_sound_out, void *p_dac_cbuf, void **pp_effect, u32 sr)	209
34.2.2	2.2 函数 void *pitchshift_howling_api(void *obuf, u32 sr, void **ppsound)	209
34.2.3	2.3 函数 void update_howling_parm_fs_api(u32 sr, s16 new_fs)	210
35	自适应陷波啸叫抑制 (notch_howling)	211
35.1	配置参数说明	211
35.2	相关函数	212
35.2.1	函数 void *link_notch_howling_sound(void *p_sound_out, void *p_dac_cbuf, void **pp_effect, u32 sr)	212
35.2.2	函数 void *notch_howling_api(void *obuf, u32 sr, void **ppsound)	212
35.2.3	函数 void notchhowling_release(void **ppeffect)	212
35.3	资源使用说明	212
36	均衡器 (PCM_EQ)	213
36.1	1 配置参数说明	213
36.2	2 相关函数	214
36.2.1	2.1 函数 void *link_pcm_eq_sound(void *p_sound_out, void *p_dac_cbuf, void **pp_effect, u32 sr, u32 channel)	214
36.2.2	2.2 函数 void *pcm_eq_api(void *obuf, u32 sr, u32 channel, void **ppsound)	214
36.3	3 EQ 工具使用	215
36.3.1	3.1 EQ 工具使用介绍	215
36.3.2	3.2 SDK 配置说明	217
37	变音音效 (vo_pitch)	222
37.1	可选音效	222
37.2	配置参数说明	223
37.3	相关结构体及函数	223
37.3.1	结构体 VOICE_PITCH_PARA_STRUCT	223

37.3.2	函数 void *link_voice_pitch_sound(sound_out_obj *p_curr_sound, void *p_dac_cbuf, void **pp_effect, u32 cmd)	224
37.3.3	函数 void voicepitch_release(void **ppeffect)	224
37.3.4	函数 void *voice_pitch_api(void *obuf, u32 cmd, void **ppsound)	224
37.3.5	函数 void rap_callback(void *priv, int pos)	224
37.3.6	函数 void rap_reopen(void)	224
37.4	RAP 和 RAP_REALTIME 模式文件替换说明	224
38	混合变音音效 (voicechanger)	227
38.1	可选音效	227
38.2	配置参数说明	228
38.3	相关函数接口	228
38.3.1	函数 void *link_voice_changer_sound(void *p_sound_out, void p_dac_cbuf, void **pp_effect, u32 in_sr)	228
38.3.2	函数 void *voice_changer_api(void *obuf, u32 sr, void **ppsound)	229
38.3.3	函数 void update_voice_changer_parm(VOICECHANGER_AV_PARM *new_vc_parm, VOICESYN_AV_PARM *new_vsyn_ctrl)	229
38.3.4	函数 void voicechanger_release(void **ppeffect)	229
39	变速变调 (speed)	230
39.1	配置参数说明	230
39.2	相关函数	231
39.2.1	函数 void *speed_api(void *obuf, u32 insample, void **ppsound)	231
39.2.2	函数 void sp_release(void **ppeffect)	231
39.2.3	函数 int music_play_speed_set_para(music_play_obj *hdl, SPEED_PITCH_PARA_STRCUT *sp_parm)	231
40	分贝检测 (energe_detect)	232
40.1	配置参数 (energe_param) 说明	232
40.2	使用 demo	233
41	音频降噪算法 (ANS)	234
41.1	涉及文件	235
41.2	相关参数	236
41.3	相关函数	237
41.3.1	函数 void *link_ans_sound(void *p_sound_out, void *p_ans_obuf, void **pp_effect, u32 sr)	237
41.3.2	函数 int ans_run(void *hld, short *inbuf, int len)	238
41.3.3	函数 void ans_release(void **ppeffect)	238
41.4	ans 算法和 MMU 的联动情况	238
42	低功耗休眠	239
42.1	关机与唤醒相关参数配置	240
42.1.1	A0 芯片 flash 使用 VDDIO 供电配置	240
42.1.2	VDDIO 电压档位配置	241
42.2	低功耗休眠相关函数	242
42.2.1	函数 void sys_softoff()	242
42.2.2	函数 void sys_power_down(u32 usec)	242
42.2.3	函数 void board_set_soft_poweroff(void)	243
43	低功耗唤醒	244
43.1	初始化唤醒口	244
43.1.1	AD14 / AC104 / AD15 初始化唤醒口配置	245
43.1.2	AD17 / AD18 初始化唤醒口配置	245
43.1.3	AD23 / AD24 初始化唤醒口配置	246
43.1.4	保护唤醒口	246

43.1.5	AD14 / AC104 / AD15 保护唤醒口	247
43.1.6	AD17 / AD18 保护唤醒口	247
43.1.7	AD24 保护唤醒口	248
43.2	唤醒源获取	248
44	PMU 复位功能介绍	249
44.1	主动复位	249
44.2	长按复位	249
44.2.1	AD14N / AD15N / AC104N 长按复位功能	250
44.2.2	AD17N / AD18N 长按复位功能	251
44.2.3	AD24N / AD23N 长按复位功能	254
44.3	MCLR 短接复位	254
44.4	复位源获取	255
45	低功耗复用功能	256
45.1	低功耗复用示例	256
45.1.1	打开低功耗复用功能宏	256
45.1.2	修改链接文件	257
45.1.3	修改应用层与低功耗复用的功能	259
45.1.4	最终	259
46	lvd 低电管理	261
46.1	lvd 配置 lvd_config	261
46.2	获取 lvd 电压 get_lvd_vol	262
47	部分芯片低功耗调试配置	263
47.1	各个系列芯片支持的配置情况	263
47.1.1	修改后禁止用于生产的配置	263
47.1.2	允许修改的配置	264
47.1.3	严禁修改的配置	264
47.2	第一部分：打印调试控制	264
47.2.1	空闲状态调试	265
47.3	第二部分：实际运行代码控制	265
47.3.1	PMU 相关模块开关	265
48	VDDIO 电压配置说明	266
48.1	VDDIO 作用域	266
48.2	如何配置每个阶段的 VDDIO 档位	266
48.2.1	上电过程：	266
48.2.2	Boot-Loader：	267
48.2.3	SDK：	267
48.3	常见问题介绍	268
48.3.1	VDDIO 偏差	268
49	单备份升级说明	269
49.1	单备份升级流程	270
49.2	单备份升级相关功能开关	271
49.3	单备份升级相关函数接口	272
49.3.1	u32 try_to_upgrade(char *dev_name, char *up_file_path)	272
49.3.2	u32 check_ufw_file(char *dev_name, char *up_file_path);	272
49.3.3	u8 *user_get_ufw_vid(u8 *len);	273
49.3.4	u8 *user_get_flash_vid(u8 *len);	273
50	双备份升级	274
50.1	支持双备份升级的芯片以及对应 SDK 版本	274

50.2	SDK 相关配置	274
50.3	相关接口说明	275
50.3.1	函数 u32 jlfs_get_idle_bank_info(u32 *bank_addr, u32 *bank_size)	275
50.3.2	函数 u32 jlfs_updata_dual_bank_info(u32 bank_addr, u16 data_crc)	276
50.3.3	函数 u32 jlfs_check_dual_bank_info(u32 update_bank_addr)	276
50.4	常见客户问题	276
51	VM 掉电存储说明	278
51.1	VM 记忆的基本原理	278
51.2	VM 记忆的系统局限性	278
51.3	NEW VM 的预擦除	279
51.4	NEW VM 常见 BUG	279
51.5	VM 掉电存储使用说明	279
51.6	VM 掉电存储版本简单说明	280
51.7	新旧 VM 文件组成及版本选择	280
51.8	VM 相关参数介绍	282
51.8.1	ID 号 (vm_api.h)	283
51.8.2	读/写入数据长度	283
51.8.3	vm 存储区域大小	284
51.8.4	新版 VM 格式整理时缓存空间设置 new_vm_buff (new_vm.h)	284
51.8.5	新版 vm 格式整理后是否立即擦除旧半区区域功能	285
51.8.6	新版 vm 是否支持多段读功能	286
51.9	VM 掉电记忆存储接口	286
51.9.1	函数 u32 syscfg_vm_init(u32 addr, u32 size)	286
51.9.2	函数 u32 vm_write(u32 id, u8 *buf, u32 len)	286
51.9.3	函数 u32 vm_read(u32 id, u8 *buf, u32 len)	287
51.9.4	函数 u32 vm_pre_erase(void)	287
51.9.5	函数 nvm_pre_erase_next(NEW_VM_OBJ *p_nvm, u16 using_next, u16 idle_next)	287
51.10	VM 常见问题介绍	288
51.10.1	VM 简单使用 demo:	288
51.10.2	调整 VM 区域大小设置:	288
51.10.3	新版 VM 在关机或者掉电后可能存在数据丢失 BUG:	289
51.10.4	VM 格式整理原理:	289
52	消息与事件	290
52.1	1 消息与事件使用一般流程	291
52.2	2 消息与事件函数	292
52.2.1	2.1 void event_init(void)	292
52.2.2	2.2 void message_init()	292
52.2.3	2.3 u32 event2msg_api(u32 event)	292
52.2.4	2.4 int post_event(int event)	293
52.2.5	2.5 int post_msg(int argc, ...)	294
52.2.6	2.6 bool get_event_status(u32 event)	295
52.2.7	2.7 函数 int get_msg(int len, int *msg)	296
52.2.8	2.8 函数 void clear_all_message(void)	296
53	按键系统 key	297
53.1	按键类型介绍	298
53.1.1	IO 按键	298
53.1.2	AD 按键	299
53.1.3	IR 按键	300
53.1.4	MATRIX 按键	300
53.1.5	touch 按键:	300
53.2	按键接口介绍:	303

53.2.1	函数 void key_init(void)	303
53.2.2	函数 void key_table_sel(void *msg_filter)	303
53.2.3	函数 void key_scan(void)	303
54	Reserved_Area 区域说明	304
54.1	认证码相关信息	304
54.2	认证码功能 isd_config_ini.c 配置	304
54.3	认证码信息接口	306
54.3.1	void rsv_auth_analysis(void)	306
54.4	认证码文件说明	306
55	多录音分区 (AD14N、AD24N)	307
55.1	1 多录音分区一般流程	307
55.1.1	1.1 创建录音分区初始化	308
55.1.2	1.2 创建录音切换分区消息事件并设置键位	309
55.1.3	1.3 在 app 录音任务中实现录音分区切换操作	310
56	虚拟 RTC	311
56.1	功能接口介绍	311
56.1.1	int rtc_simulate_init(const struct vir_rtc_data *arg)	311
56.1.2	void read_sys_time(struct sys_time *curr_time);	311
56.1.3	void write_sys_time(struct sys_time *curr_time);	312
56.1.4	void read_alarm(struct sys_time *alarm_time);	312
56.1.5	void write_alarm(struct sys_time *alarm_time);	312
56.1.6	void vir_set_alarm_ctrl(u8 set_alarm);	312
56.1.7	u8 vir_get_alarm_ctrl(void);	312
57	类型感知内存管理单元	313
57.1	本内存管理驱动特性与限制	313
57.1.1	内存管理方式	313
57.1.2	内存释放机制	313
57.1.3	工作流程示例	314
57.2	函数接口说明	314
57.2.1	函数 void vPortInit(void *pAddr, uint32_t xLen)	314
57.2.2	函数 void vPort_disable()	314
57.2.3	函数 void *pvPortMalloc(size_t xWantedSize, mm_type type)	314
57.2.4	函数 void vPortFree(void *pv)	315
58	以指针方式直接访问系统 Flash 中的资源文件	316
58.1	示例代码	316
58.2	关键变量说明	316
58.3	重要注意事项	317
58.4	相关资源	317
59	Indices and tables	318

开源社区仓库链接:

<https://gitee.com/Jieli-Tech/fw-AD15N>

<https://github.com/Jieli-Tech/fw-AD15N>

开源社区交流群:

AD系列32位芯片 杰理开源社区 



此二维码365天内有效 (2026-04-21前)

 钉钉扫一扫群二维码, 立即加入群聊

开源社区钉钉群二维码

作者: 刘杰

本页介绍历史版本

1.1 v1.6 说明

日期：2025 年 12 月 25 日

- 文档勘误 config_asser 量产值描述笔误->“*ADI7N* 的内核异常管理单元”；
- 文档勘误 stack_magic0、stack_magic 备注笔误”->“第 4, 5 行堆栈边界”；
- 文档勘误“非对其访问”改正为“非对齐访问”；

1.2 v1.5 说明

日期：2025 年 12 月 24 日

- 新增函数 vfs_ioctl 获取文件系统总容量与剩余容量的命令”->“函数 *vfs_ioctl* 获取总容量命令 *FS_IOCTL_GET_PARTITION_INFO*”；
- 新增 fat 文件系统文件夹删除函数”->“文件夹删除函数 *int vfs_delete_dir(void *pvfs, char *path)*”；
- 新增 fat 文件格式化接口函数”->“格式化接口 *int vfs_format(void **ppvfs, const char *dev_name, const char *type, u32 clust_size, u8 create_new)*”；
- 新增章节《部分芯片低功耗调试配置》”->“部分芯片低功耗调试配置”；
- 新增章节《类型感知内存管理单元》”->“类型感知内存管理单元”；
- 新增章节《*freefs* 文件系统》”->“*freefs* 文件系统”；

- 新增章节《以指针方式直接访问系统 Flash 中的资源文件》->“以指针方式直接访问系统 *Flash* 中的资源文件”;
- 修改章节《长按复位》->“*长按复位*”;
- 添加章节《isd_config.ini 配置文件说明链接》->“*isd_config.ini* 配置文件说明链接”;
- 添加章节《杰理科技一拖二烧录器用户手册链接》->“杰理科技一拖二烧录器用户手册链接”;
- 添加章节《AD14N 与 AD15N 内核异常管理单元》->“*AD14N* 与 *AD15N* 的内核异常管理单元”;
- 添加章节《AD17N 内核异常管理单元》->“*AD17N* 的内核异常管理单元”;

1.3 v1.4 说明

日期: 2024 年 11 月 4 日

- 新增 Audio 指标测试数据章节 (AD14N / AC104N) ”->“*Audio* 指标测试数据 (*AD14N* / *AC104N*) ”;
- 添加语句“download.bat 脚本由 download_bat.c 文件编译生成”->“*download_bat.c* 文件介绍”;
- 修正编码接口说明”->“*编码接口*”;
- 补充中断系统优先级说明”->“*中断系统*”;
- 新增 AD14/AD15 长按复位功能说明->“*PMU* 复位功能介绍”;
- 新增 AD14 升级说明->“单备份升级说明”;
- 新增异常章节说明->“*异常*”;
- 新增 tick_timer 章节说明->“*tick_timer*”;
- 新增分贝检测章节说明->“分贝检测 (*energe_detect*) ”;
- 新增低功耗唤醒章节说明->“*低功耗唤醒*”;
- 修改环境介绍章节部分的硬件环境介绍和 SDK 介绍说明->“*SDK* 介绍”“芯片框图和硬件环境介绍”;

1.4 v1.3 说明

日期: 2023 年 9 月 11 日

对应 SDK 版本: AD1x-4578_AC104-release_v1.8.0

1、新增 AD18N 芯片相关内容:

- 添加环境介绍->“*AD18N* 芯片框图和硬件开发环境”;
- 更新 SDK 工程与配置说明->“*SDK* 工程与配置”;
- 更新外挂资源 flash 下载说明->“外挂资源 *flash* 下载 (*AD23N* 暂不支持) ”;
- 更新 isd_config.ini 配置说明->“*isd_config.ini* 文件介绍”;
- 更新 AUDIO_DAC 相关说明->“*AUDIO DAC*”;
- 更新软关机和低功耗相关说明->“*低功耗休眠*”;

- 更新内封 flash 写保护说明 (使用了新策略) -> “内封 flash 芯片如何添加写保护参数 (新)”;

2、新增章节说明:

- 新增双备份升级相关内容介绍-> “双备份升级”;
- 新增了中断说明-> “中断系统”;
- 新增变音音效 (vo_pitch)rap 文件替换说明-> “RAP 和 RAP_REALTIME 模式文件替换说明”;
- 新增 GPIO 接口章节-> “GPIO 接口说明”;
- 新增 FIX 解码格式说明-> “FIX 格式解码流程说明”;
- 新增 VDDIO 章节说明-> “VDDIO 电压配置说明”;

3、更新章节说明:

- 修改虚拟 RTC 章节说明, 概述添加注意事项-> “虚拟 RTC”;
- 新增各芯片启动时间指标-> “低功耗休眠”;
- 新增 AD17N 单端推功放原理图-> “AD17N APA 驱动模式介绍”;
- 删除 AD15N 单端推功放原理图, 该方式底噪较大;
- 更新 flash 单双线原理图以及软件配置-> “isd_config.ini 文件介绍”;
- 更新 VM 常见问题介绍-> “VM 常见问题介绍”;
- 更新 speed 变速变调参数限制说明-> “变速变调 (speed) ”;

1.5 v1.2 说明

日期: 2023 年 7 月 11 日

对应 SDK 版本: AD1x-457_AC104-release_v1.7.0

1、新增 AD17N 芯片相关内容:

- 添加环境介绍->” AD17N 芯片框图和硬件开发环境 “
- 更新 SDK 工程与配置说明->” SDK 工程与配置 “;
- 添加应用配置文件《app_modules.h》说明->” 应用 APP 配置文件《app_modules.h》介绍 “;
- 更新工程裁切说明->” 新版本 SDK 工程裁切 (2023 年 7 月发布的 SDK 起始) “;
- 更新外挂资源 flash 下载说明->” 外挂资源 flash 下载 (AD23N 暂不支持) “;
- 更新 isd_config.ini 配置说明->” isd_config.ini 文件介绍 “;
- 更新 AUDIO_DAC 相关说明->” AUDIO DAC “;
- 更新软关机和低功耗相关说明->” 低功耗休眠 “;

2、新增发声玩具应用工程 voice_toy 相关内容->” 玩具应用篇 (voice_toy) “;

3、新增 new vm 相关内容->” VM 掉电存储说明 “;

4、新增 speed 变速变调说明->” 变速变调 (speed) “;

5、新增 clock 时钟说明->” 时钟驱动 “;

- 6、新增虚拟 RTC 章节说明->“[虚拟 RTC](#)”；
- 7、新增内置系统 flash 写保护说明->“[内置系统 flash 写保护](#)”；

1.6 v1.1 说明

日期：2023 年 3 月 30 日

- 1、将 MP3、WAV 的说明加入到《解码接口》的章节中

1.7 v1.0 说明

日期：2023 年 3 月 18 日

建立文档，文档内容包括：

环境介绍
下载目录&工具使用环境介绍
设备管理
文件系统管理
解码接口
编码接口
JL_MIO 功能
玩具应用篇
小音箱应用篇
AUDIO DAC
AUDIO ADC (AD14N)
音量
混响 (echo)
移频啸叫抑制 (pitchshifter_howling)
自适应陷波啸叫抑制 (notch_howling)
均衡器 (PCM_EQ)
变音音效 (vo_pitch)
vm 虚拟存储
消息与事件
按键系统 key
Reserved_Area 区域说明
多录音分区 (AD14N)
软关机 Soft off 和低功耗 Power down

常见问题快速索引

本章节为一些常见问题快速索引。

AD23N 开发环境快速上手：

1. SDK 与开发板功能介绍（快速上手指南）-> “SDK 与开发板功能介绍（快速上手指南）”；

玩具系列 SDK 常见问题：

1. 外挂资源 flash 如何烧写资源 -> “[外挂资源 flash 如何烧写资源](#)”；
2. 保持 IO 状态 -> “[保持 IO 状态](#)”；
3. AD24N 工程编译时选择的编译器以及对应工具链版本 -> <https://gitee.com/Jieli-Tech/AD24N/issues/IBVYTN>
4. AD24N 开机卡死问题处理方法 -> <https://gitee.com/Jieli-Tech/AD24N/issues/IBUAVB>
5. AD24N 在低功耗休眠时，PA0 出现漏电原因以及处理方法 -> <https://gitee.com/Jieli-Tech/AD24N/issues/IBZ3GR>

相关 ISSUE：

1. FAQ：flash 概念，外挂 & 内封、系统 flash& 数据 flash 的关系。
 2. FAQ：用户在访问外挂 flash 时候，mount 返回值出现 8008 问题解决办法总结。
 3. FAQ：如何烧写资源文件到外挂数据 flash？。
-

2.1 外挂资源 flash 如何烧写资源

可参考文档以下部分：

1. 外挂资源 *flash* 下载 (*AD23N* 暂不支持)
2. *download_bat.c* 文件介绍

相关 ISSUE：

1. FAQ：flash 概念，外挂 & 内封、系统 flash& 数据 flash 的关系。
 2. FAQ：用户在访问外挂 flash 时候，mount 返回值出现 8008 问题解决办法总结。
 3. FAQ：如何烧写资源文件到外挂数据 flash？。
-

2.2 保持 IO 状态

可参考文档以下部分：

1. 保护唤醒口
 2. 保护唤醒口
-
-

3.1 新版 SDK 命名说明

2023 年 9 月发布的 SDK 整合了 AD14N / AC104N / AD15N / AD17N / AD18N 的工程，SDK 命名为 *AD1x-4578_AC104-release_v1.8.0*，其含义为：

- AD1x-4578：代表 AD14、AD15、AD17、AD18N
- AC104：代表 AC104
- v1.8.0：表示当前 SDK 版本号



新版本 SDK 名称说明

3.2 SDK 工程与配置

目前 AD14N / AD15N / AC104N / AD17N 支持的工程如下：

芯片/工程	app_mg	simple	mbox_mg	mcu	voice_toy
AD14N / AC104N	√	√	√	√	√
AD15N	√	√		√	√
AD17N				√	√
AD18N				√	√

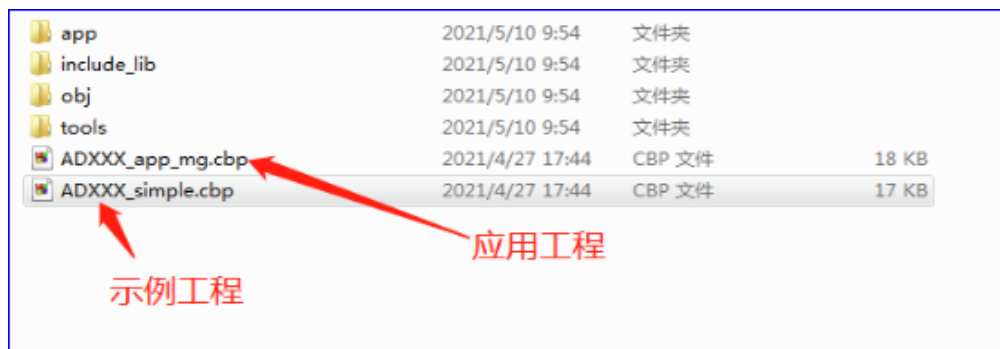
- **app_mg 应用工程：**

该工程有完整玩具功能，能实现解码、编码录音、MIDI、LINEIN 等功能；同时将 VM、按键、休眠应用在工程中。

- **simple 示例工程：**

该工程可实现解码、扩音功能等功能；未将 VM、按键、休眠应用在工程中，但是相关代码能在工程中直接调用使用。

芯片/工程	支持的 SDK 版本
AD14N	ad140-release_v1.0.0 ~ v1.4.0、开源 sdk_v1.0.0 ~ v1.6.0
AD15N	ad150-release_v1.0.0 ~ v1.4.0、开源 sdk_v1.0.0 ~ v1.6.0



- **mbox_mg 应用工程：**

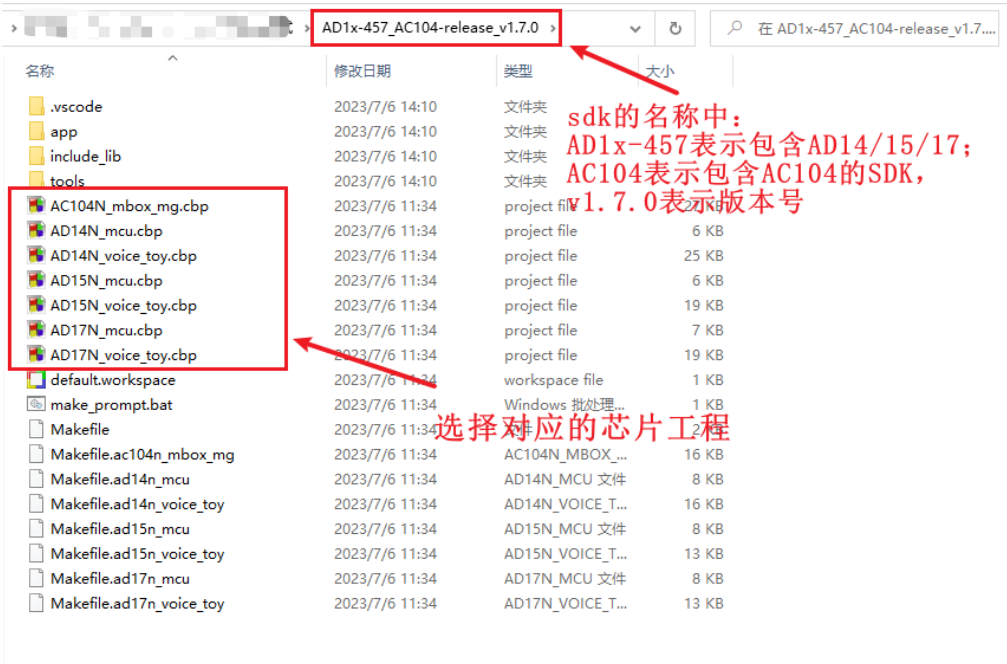
该工程有完整插卡音箱功能，能实现标准 MP3、WAV 解码、FM、USB 从机、录音、扩音功能等功能；同时将 VM、升级、按键、休眠应用在工程中。

芯片/工程	支持的 SDK 版本
AC104N	ac104-release_v1.0.0 ~ v1.4.0、开源 SDK_v1.6.0 ~ 至今最新版本



- **voice_toy 应用工程:**
该工程有完整发声玩具功能，在 simple 工程基础上集成了 app_mg 功能；能实现解码、MIDI 功能；同时将 VM、按键、休眠应用在工程中。

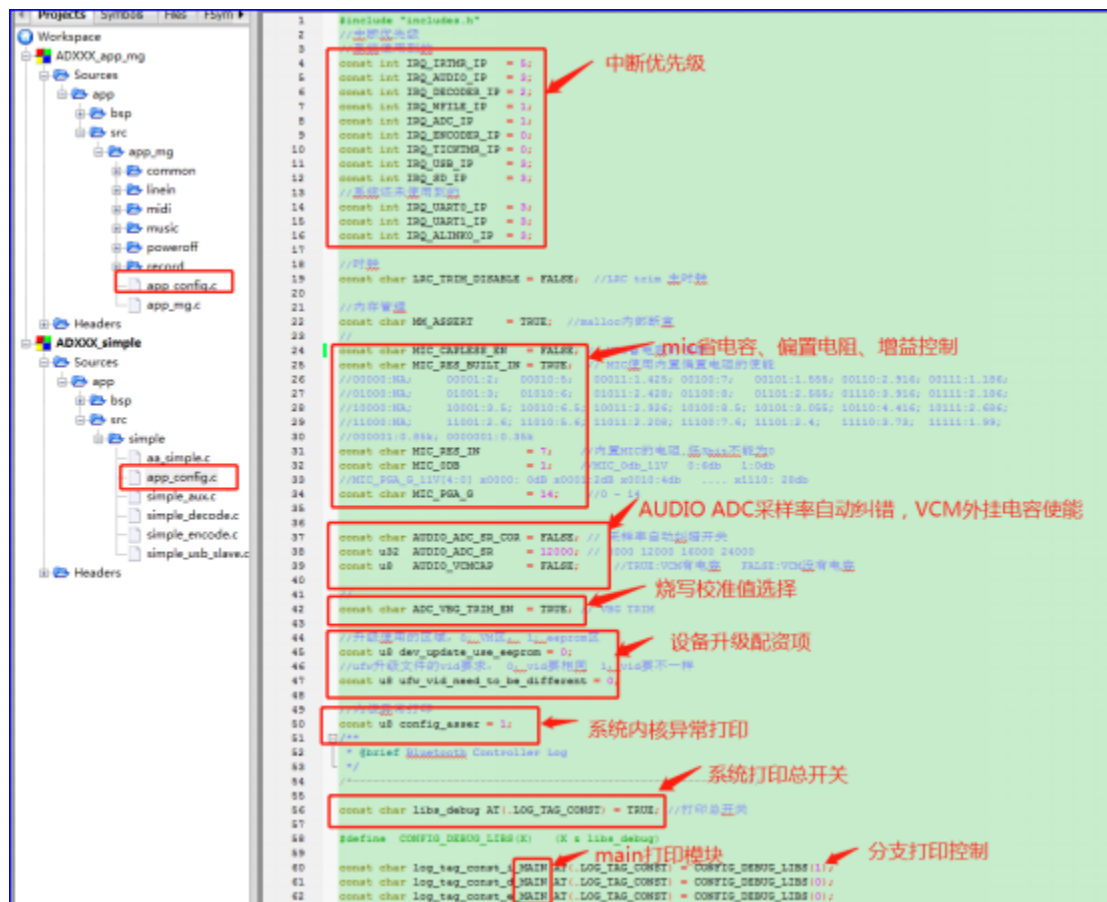
芯片/工程	支持的 SDK 版本
AD14/15/17/18/AC104	AD1x-457_AC104-release_v1.7.0 ~ 至今最新版本



- **mcu 应用工程:**
该工程为 MCU 工程，仅初始化串口打印、时钟、电源、异常等模块，不包含其他功能。

3.2.1 应用配置文件《app_config.c》介绍

应用配置文件《app_config.c》能配置与系统相关的配置：



1、中断优先级；

注：控制各个模块的中断优先级！

2、LRC_TRIM_DISABLE：LRC 校准时钟，LRC 必须经过了烧写器校准后才能使用

- ① FALSE：使用 LRC 校准时钟偏差，生产必须选择 FALSE；
- ② TRUE：不使用 LRC 校准时钟偏差；

注：默认开启 LRC 校准时钟，时钟不准的芯片一定要过烧写器校准！

3、MM_ASSERT：MALLOC 内部断言使能

- ① FALSE：关闭 malloc 内部断言；
- ② TRUE：打开 malloc 内部断言，默认打开；

4、MIC_CAPLESS_EN：MIC 省电容使能

- ① FALSE：不省电容；
 - ② TRUE：省电容；
- 注：省电容方案底噪较大，不建议使用！

5、MIC_RES_BUILT_IN: MIC 电路偏置方式

- ① 0: 使用 MICLDO (PA15) 提供偏置电压, 需短接 V_MIC 和 MICLDO;
- ② 1: 使用 MICBIAS (PA13) 提供偏置电压, 需短接 MIC 和 MICBIAS;
- ③ 2: 不使用内部偏置 (不占用 I/O 口);

6、MIC_LDO_V: MICLDO 增益 “MIC_0DB”

注: 此选项只在 *MIC_RES_BUILT_IN* == 0 时起作用, 以下为千欧姆

- ① 0: 1.8V;
- ② 1: 2.1V;
- ③ 2: 2.4V;
- ④ 3: 2.7V;

7、MIC_RES_IN: MICBIAS 电路内置偏置电阻挡位选择

注: 此选项只在 *MIC_RES_BUILT_IN* == 1 时起作用, 以下为千欧姆

value	kkilohm	value	kkilohm
0b00100	7;	0b00101	1.555;
0b00110	2.916;	0b00111	1.186;
0b01000	NA;	0b01001	3;
0b01010	6;	0b01011	2.428;
0b00000	NA;	0b00001	2;
0b00010	5;	0b00011	1.425;
0b01100	8;	0b01101	2.555;
0b01110	3.916;	0b01111	2.186;
0b10000	NA;	0b10001	3.5;
0b10010	6.5;	0b10011	2.926;
0b10100	8.5;	0b10101	3.055;
0b10110	4.416;	0b10111	2.686;
0b11000	NA;	0b11001	2.6;
0b11010	5.6;	0b11011	2.208;
0b11100	7.6;	0b11101	2.4;
0b11110	3.73;	0b11111	1.99;
0b10000	0.85k;	0b1000000	0.375k
0b1100000	0.26k		

8、MIC_0DB: MIC 前级增益

value	db
0	6db
1	0db

9、MIC_PGA_G: MIC 后级增益

value	db
0b0000	0db
0b0001	2db
0b0010	4db
...	...
0b1110	28db

10、AUDIO_ADC_SR_COR: 录音采样率纠错使能

- ① FALSE: 关闭自动纠错, 当录音设置的采样率不在 AUDIO ADC 工作的范围内时, 返回错误, 启动 AUDIO ADC 失败;
- ② TRUE: 开启纠错, 当录音设置的采样率不在 AUDIO ADC 工作的范围内时, 将采样率设置在 AUDIO_ADC_SR 对应的值! 如果 AUDIO_ADC_SR 正确那么启动成功, 否则失败;

11、AUDIO_ADC_SR: 录音采样率纠错值

- ① 采样率: 8000
- ② 采样率: 12000
- ③ 采样率: 16000
- ④ 采样率: 24000
- ⑤ 其他: 非法值

12、AUDIO_VCMCAP: VCM 是否挂电容, 挂了电容后音质会有较大提升:

- ① FALSE: 没有挂电容;
 - ② TRUE: 有挂电容;
- 注: 只有部分封装有将 VCMCAP 引脚引出, 详情请看具体芯片封装原理图!

13、ADC_VBG_TRIM_EN: 内部 VBG 校准使能

- ① FALSE: 使用默认值校准;
- ② TRUE: 使用烧写值校准, 生产必须选择 TRUE;

14、GPIO_CRITICAL_OPT: GPIO 中断临界保护使能

- ① FALSE: 关闭 GPIO 中断临界保护;
- ② TRUE: 打开 GPIO 中断临界保护;

15、MAINTRACK_USE_CHN: MIDI 乐谱使用主轨道选择

- ① 0: 用 track 号来区分;
- ② 1: 用 channel 号来区分;

16、MAX_DEC_PLAYER_CNT: MIDI 乐谱解码最大同时发声 key 数

- ① 范围: 1~31, 会影响到 midi 乐谱解码运算 buff 大小;

17、MAX_CTR_PLAYER_CNT: MIDI 琴最大同时发声 key 数

- ① 范围: 1~31, 会影响到 midi 琴解码运算 buff 大小;

18、dev_update_use_eeprom：设备升级占用区域，占用的区域数据是失效

- ① 0：占用 VM 区域（一般用于 AD14N 的内置 flash 录音）；
- ② 1：占用 EEPROM 区域（一般用于 VM 记忆）；

19、dev_update_keep_io_status：设备升级 I/O 保持使能

- ① 0：不保持 I/O，升级时复位 I/O；
- ② 1：保持 I/O，升级过程中 I/O 保持升级前状态；

20、ufw_vid_need_to_be_different：设备升级对 ufw 升级文件的 vid 要求

- ① 0：升级文件 vid 与当前设备 vid 相同才可升级；
 - ② 1：升级文件 vid 与当前设备 vid 不同才可升级；
 - ③ 2：升级文件 vid > 当前设备 vid 才可升级；
 - ④ 3：升级文件 vid < 当前设备 vid 才可升级；
- 注：vid 在 *isd_config.ini* 文件中修改！

21、config_asser：内核异常打印“config_asser”。

- ① 0：关闭内核异常打印；
- ② 1：开启内核异常打印；

注：生产的工程要关闭内核异常打印，内核异常打印会关闭看门狗并且 while(1) 停留!!!

注：生产的工程要关闭内核异常打印，内核异常打印会关闭看门狗并且 while(1) 停留!!!

注：生产的工程要关闭内核异常打印，内核异常打印会关闭看门狗并且 while(1) 停留!!!

22、libs_debug：打印总开关

- ① FALSE：关闭打印；
- ② TRUE：开启打印；

23、log_tag_const_x_yyyy：分支打印开关，仅仅在打印总开关打开的时候有效

- ① x=i：控制打印函数（log_info / log_info_hexdump）的打印开关；
 - ② x=d：控制打印函数（log_debug / log_debug_hexdump）的打印开关；
 - ③ x=e：控制打印函数（log_error / log_error_hexdump）的打印开关；
 - ④ x=c：控制打印函数（log_char）的打印开关；
 - ⑤ yyyy：控制相应的打印块；
- 注：若要打开相应打印，将 *CONFIG_DEBUG_LIBS(0)* 改为 *CONFIG_DEBUG_LIBS(1)* 即可！

3.2.2 应用配置文件《app_config.h》介绍

应用配置文件《app_config.h》能配置与系统相关的配置：

① DISABLE: 使用固定引脚输出, 默认串口引脚 AD14 / AC104: PA4, AD15N: PA6;
② ENABLE: 使能串口任意引脚输出, 输出到;UART_OUTPUT_CH_PORT 定义的引脚;
注: 使能串口输出任意引脚会占用一路 *output channel* 通路!

按键配置

注：按键部分功能需要在 `key.h` 中配置！

配置项目	含义
AD_KEY_CH_SEL	AD 按键采集引脚选择, 可查看 adc_drv.h 中支持的引脚通道
IR_KEY_IO_SEL	红外按键捕获引脚选择, 会占用一路 input channel 通路
IR_KEY_IO_SEL	红外按键捕获引脚选择, 会占用一路 input channel 通路
X_ADC_CH_SEL & Y_PORT_SEL	X 轴要求选择支持 ADC 的 I/O;
TOUCH_KEY_SEL	Y 轴选择普通 I/O
KEY_VOICE_EN	触摸按键 I/O 口选择
	触摸按键 I/O 口选择

低功耗配置

- 1、POWER_WAKEUP_IO: 低功耗唤醒 I/O 选择, 请查看 power_api.c 中的配置

SD 模块配置

- 1、TFG_SD_EN: SD 卡模块使能 (新版本 SDK 在 app_modules.h 中定义该宏)
- 2、SPI_SD_IO_REUSE: 外挂 FLASH 与 SD 卡模块 I/O 复用使能

USB 模块配置

- 1、TCFG_PC_ENABLE: PC 模块使能
- 2、TCFG_USB_EXFLASH_UDISK_ENABLE: 外挂 FLASH 模拟 U 盘使能
- 3、TCFG_UDISK_ENABLE: U 盘模块使能

外挂 FLASH 配置 (FLASH Configuration)

AD17N / AD18N 支持任意 I/O 映射, 外挂 flash I/O 需要在 device_list.c 中指定;

1、TCFG_FLASH_SPI_TYPE_SELECT: spi 选择

- ① 1: 选择硬件 spi, 固定 I/O 口, 可查看 device_list.c 或者芯片原理图的 spi 模块引脚;
- ② 0: 选择软件 spi, 可指定任意 I/O 口;

2、TFG_SPI_UNIDIR_MODE_EN: 外挂 flash 运行 1bit 模式选择

- ① 1: 使能外挂 flash 1bit 模式;
- ② 0: 不使能外挂 flash 1bit 模式;

3、HW_SPI_WORK_MODE & SPI_READ_DATA_WIDTH: spi 模式

- ① SPI_MODE_BIDIR_1BIT: 全双工, di 接收, do 发送;
- ② SPI_MODE_UNIDIR_1BIT: 半双工, do 分时接收/发送;
- ③ SPI_MODE_UNIDIR_2BIT: 半双工, di & do 共 2bit 分时接收/发送;

4、SPI_CS_PORT_SEL: spi cs 引脚选择

5、软件 spi 配置:

- ① A_CLK_BIT & A_CLK_PORT(x): 软件 spi CLK 引脚选择;
- ② A_DO_BIT & A_DO_PORT(x): 软件 spi DO 引脚选择;
- ③ A_DI_BIT & A_DI_PORT(x): 软件 spi DI 引脚选择;
- ④ SOFT_SPI_WORK_MODE: 软件 spi 模式, 可选择 1bit 或者 2bit 模式;

Mbox_mg 工程独有的配置

宏定义	含义
LED_5X7	LED 数码管使能，会占用部分 I/O，请查看 mbox_main.h 中的引脚分布
BREAK_POINT_AUTO_SAVE	实时断点保存使能
POWER-DOWN_UDISK_MODE_EN	U 盘进入 powerdown 使能
FM_SD_IO_REUSE	FM 与 SD 卡模块 I/O 复用使能
LINEIN_DET_MULT_SD	SD 卡模块与 LINEIN 检测 I/O 复用使能

3.2.3 应用 APP 配置文件《app_modules.h》介绍

各个工程下有对应 cpu 的应用配置文件《app_modules.h》，其中的宏定义能配置支持的编解码格式、以及 APP 模式相关的配置；每一个配置项都可独立开关，关闭没有用到的宏定义，可以 * 减少生成的代码量，节省程序空间和运行空间 *。

该文件中出现的宏，即该工程拥有的功能；对于默认为 0 的宏，使能为 1 即可正常使用；

解码模块

宏定义	含义
DECODER_UMP3_EN	UMP3 解码使能；
DECODER_MP3_ST_EN	标准 MP3 解码使能（mbox 独有）；
DECODER_WAV_EN	WAV 解码使能（mbox 独有）；
DECODER_F1A_EN	F1A 解码使能；
DECODER_MIDI_EN	MIDI 解码使能；
DECODER_MIDI_KEYBOARD_EN	MIDI 琴解码使能；
DECODER_A_EN	A 格式解码使能；
HAS_MIO_EN	MIO 功能使能；

音频算法模块

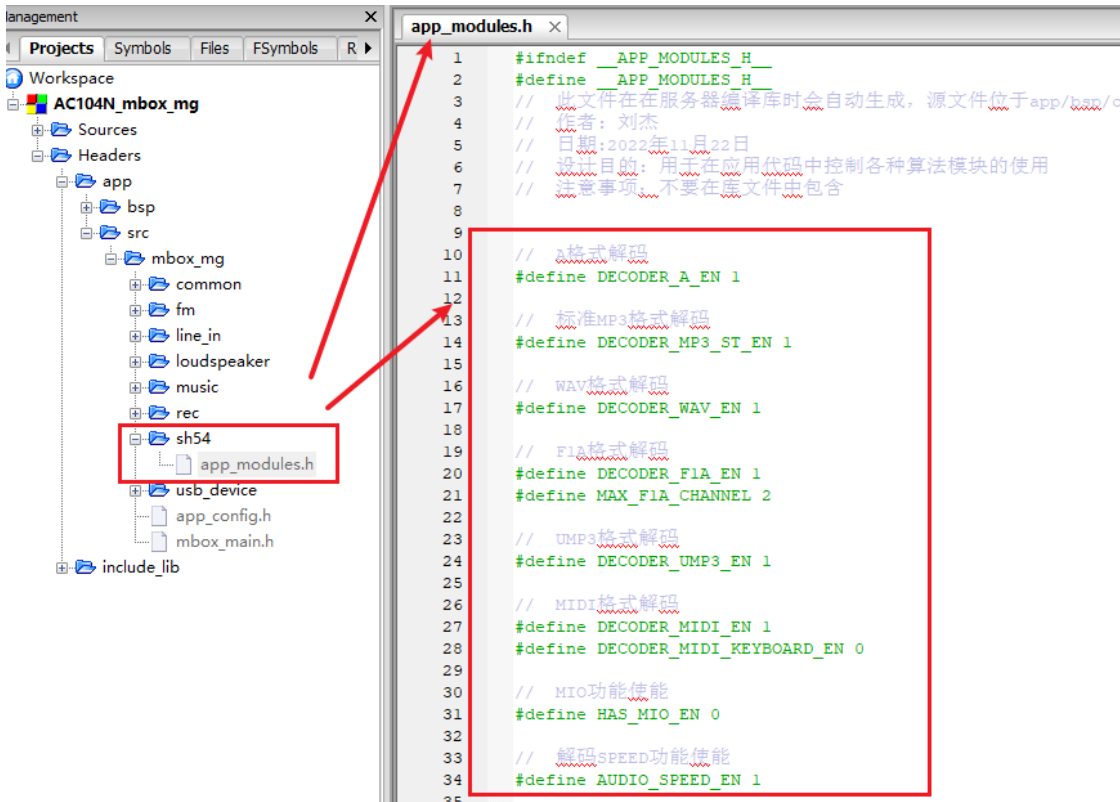
宏定义	含义
AUDIO_SPEED_EN	变速变调算法使能
SPEAKER_EN	扩音使能；
VO_PITCH_EN	变音算法使能；
PCM_EQ_EN	EQ 算法使能；
ECHO_EN	ECHO 算法模块使能；

驱动模块功能

宏定义	含义
EXT_FLASH_EN	外挂 FLASH 使能，使用外挂 FLASH 需要打开该宏；
TFG_SD_EN	SD 卡模块使能，使用 SD 卡需要打开该宏；
UART_UPDATE_EN	带电池串口升级使能；

芯片应用功能

宏定义	含义
RECORD_MODE_EN	录音模式使能；
LINEIN_MODE_EN	LINEIN（AUX）模式使能；
SIMPLE_DEC_EN	多路解码模式使能；
LOUDSPEAKER_EN	扩音模式使能；
FM_EN	FM 模块使能；



app_modules.h 配置

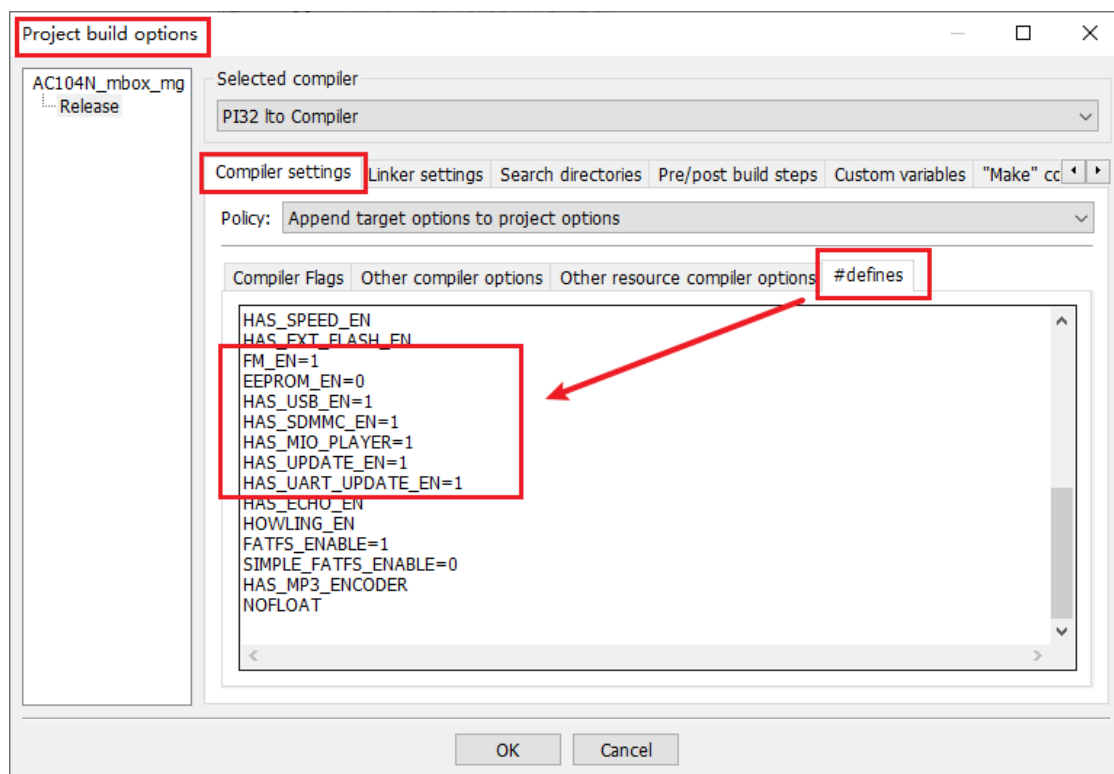
3.2.4 CodeBlocks 工程宏定义介绍

除了 app_config.c、app_config.h 和 app_modules.h 中的配置外，部分模块总开关放在 CodeBlocks 工程的宏定义中，不使用某些模块时，将宏关闭可以节约代码量；

在没有 app_modules.h 的旧版本 SDK 中，模块宏需要在 CodeBlocks 工程的宏定义中关闭。

芯片 Codeblocks 宏定义功能

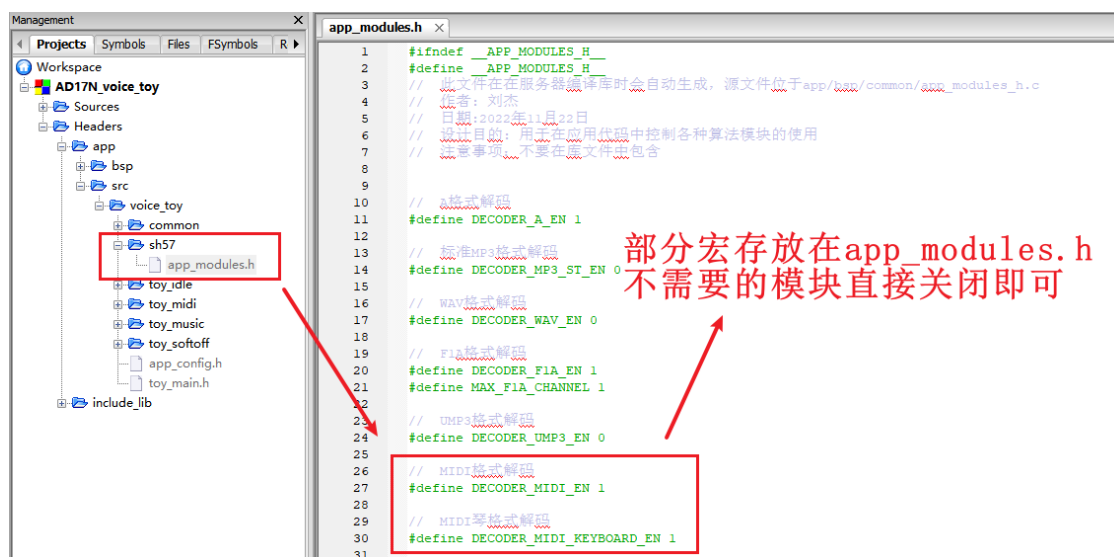
宏定义	含义
AU-DIO_ADC_EN	AUDIO ADC 使能；
EN-CODER_EN	编码使能，有该配置即支持编码功能；
EEPROM_EN	EEPROM 使能，使用 EEPROM 需要打开该宏，并在 SDK 工程中添加相应的 c 文件以及 h 文件；



3.3 新版本 SDK 工程裁切（2023 年 7 月发布的 SDK 起始）

从 2023 年 7 月发布的 AD1x-457_AC104-release_v1.7.0 起始，SDK 添加了 app_modules.h 模块管理机制；

裁切工程只需将 app_modules.h 中需要关闭的宏定义配置为 0 即可；



3.4 旧版本 SDK 工程裁切

3.4.1 如何将解码格式从工程中移除

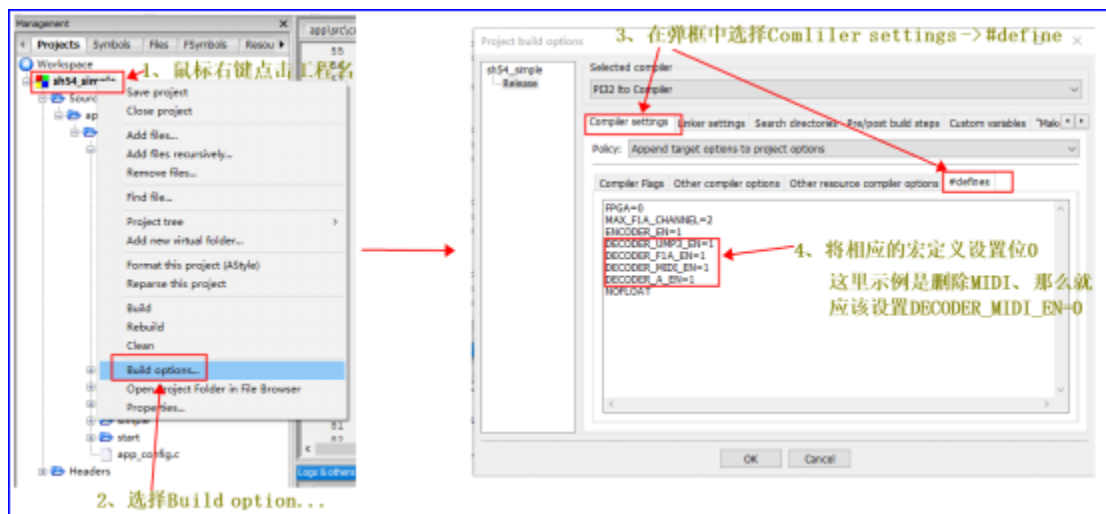
旧版本发布的 SDK 里面支持 F1A、UMP3、A、midi 四种格式，AD15N 支持 F1A、A、MIDI 三种格式，在开发过程中如果不需要支持某些格式，那么可以将相关的代码删除。这样可以极大的节省生成的代码大小；

下面以移除 MIDI 解码格式为例介绍，主要分四步：

- ① 将相关的宏定义清 0；
- ② 移除相关的库文件；
- ③ 移除相关的源文件；
- ④ 删除其资源定义；

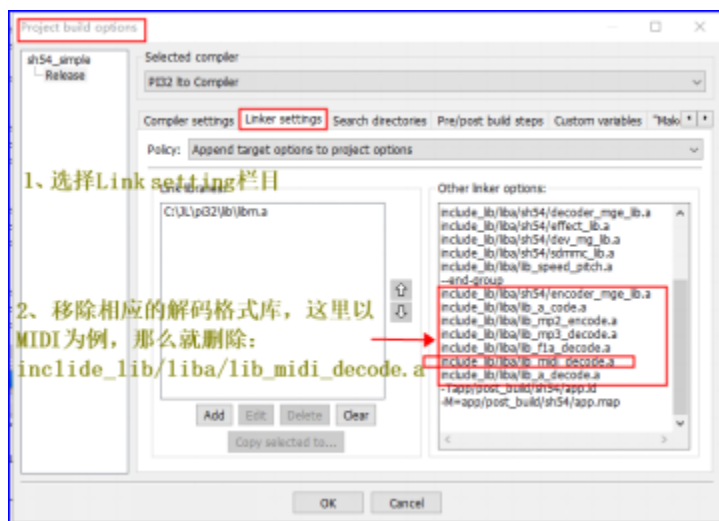
裁切解码第一步

打开 <Project build option> 窗口，将需要 MIDI 格式相应的宏定义设置为 0:



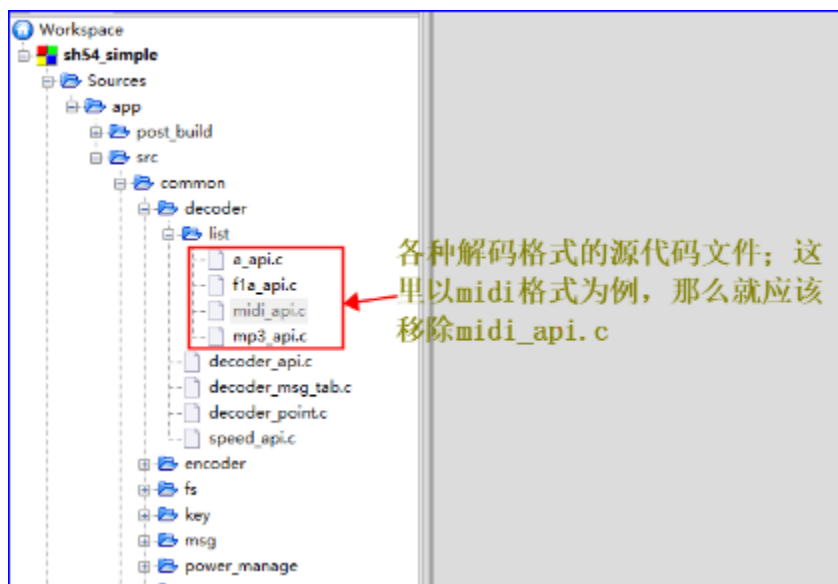
裁切解码第二步

上一步打开的 <Project build option> 窗口中删除 MIDI 的库包含:



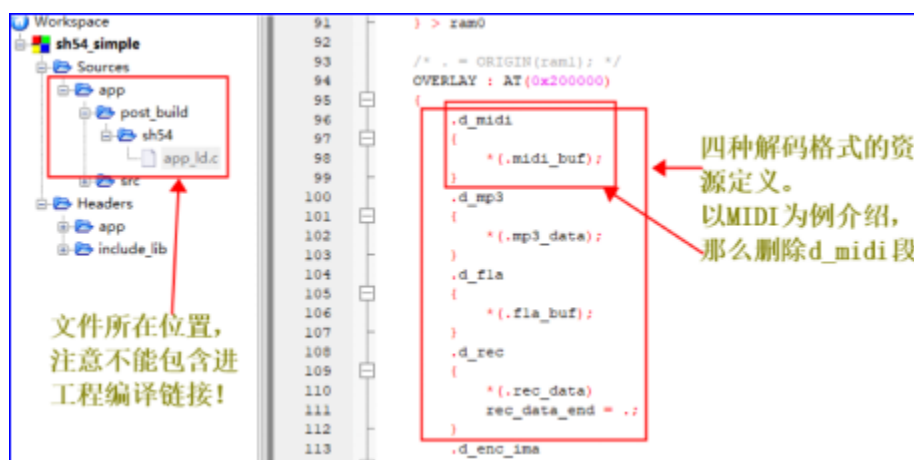
裁切解码第三步

移除工程目录 app\src\common\decoder\list\下的解码源文件;



裁切解码第四步

在文件 app\post_build\sh54\app_ld.c 中删除相关格式用到的资源定义，切记此文件在不可放在工程中编译，如果添加进了工程，一定要将其编译和链接包含取消掉。



3.4.2 如何将压缩格式从工程中移除

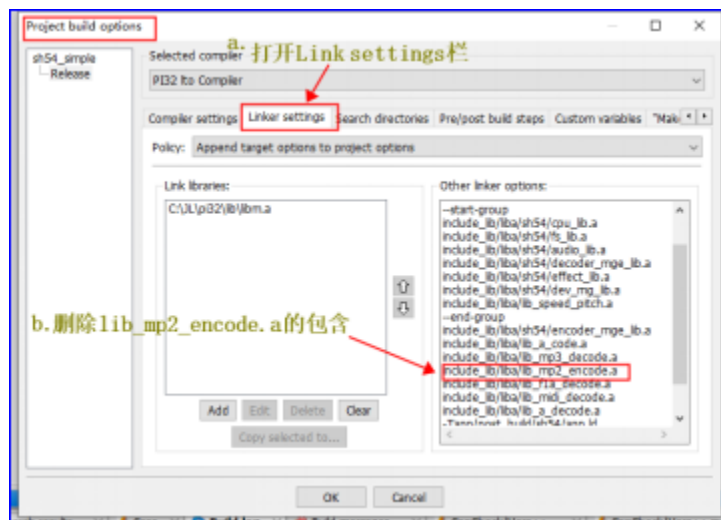
目前发布的 AD14N SDK 里面支持 UMP3、A 两种格式的压缩，AD15N 不支持录音压缩，在开发过程中如果不需要支持某些格式，那么可以将相关的代码删除。这样可以极大的节省生成的代码大小；

下面以移除 UMP3 压缩格式为例介绍，主要分三步：

- ① 移除相关的库文件；
- ② 移除相关的源文件；
- ③ 删除源码使用到的地方定义；

裁切压缩第一步

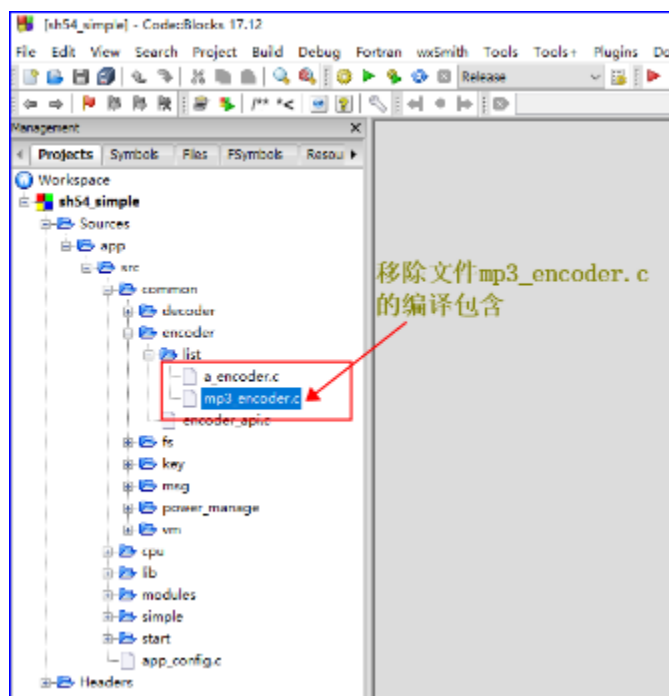
打开 <Project build option> 窗口，删除 mp3 编码库的库包含：



注：部分宏配置需要在《app_modules.h》中关闭！详情请见上文《裁切解码第一步》

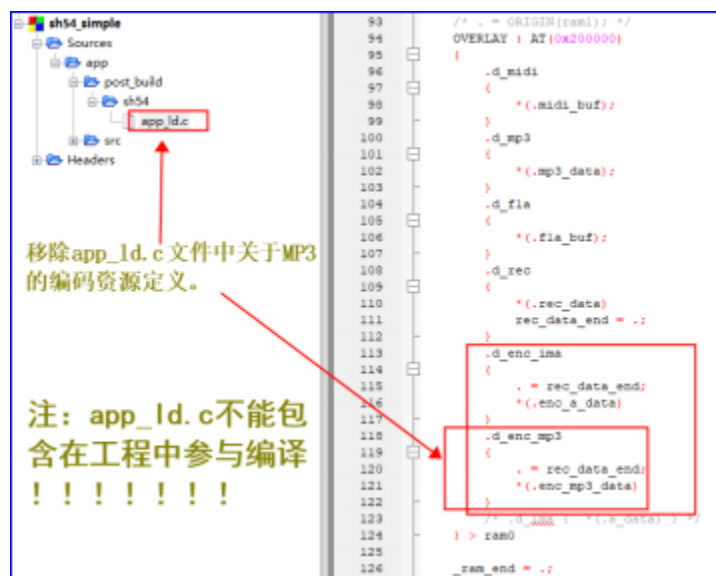
裁切压缩第二步

移除工程目录 app\src\common\decoder\list 下的压缩源文件：



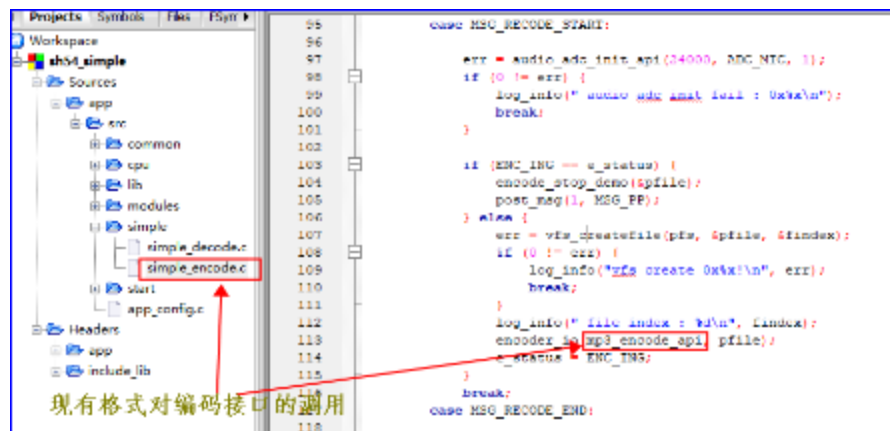
裁切压缩第三步

在文件 app\post_build\sh54\app_ld.c 中删除相关格式用到的资源定义，切记此文件在不可放在工程中编译，如果添加进了工程，一定要将其编译和链接包含取消掉。



裁切压缩第四步

修改一些编译错误，这里主要指在在程序中对编码接口的调用：



芯片框图和硬件环境介绍

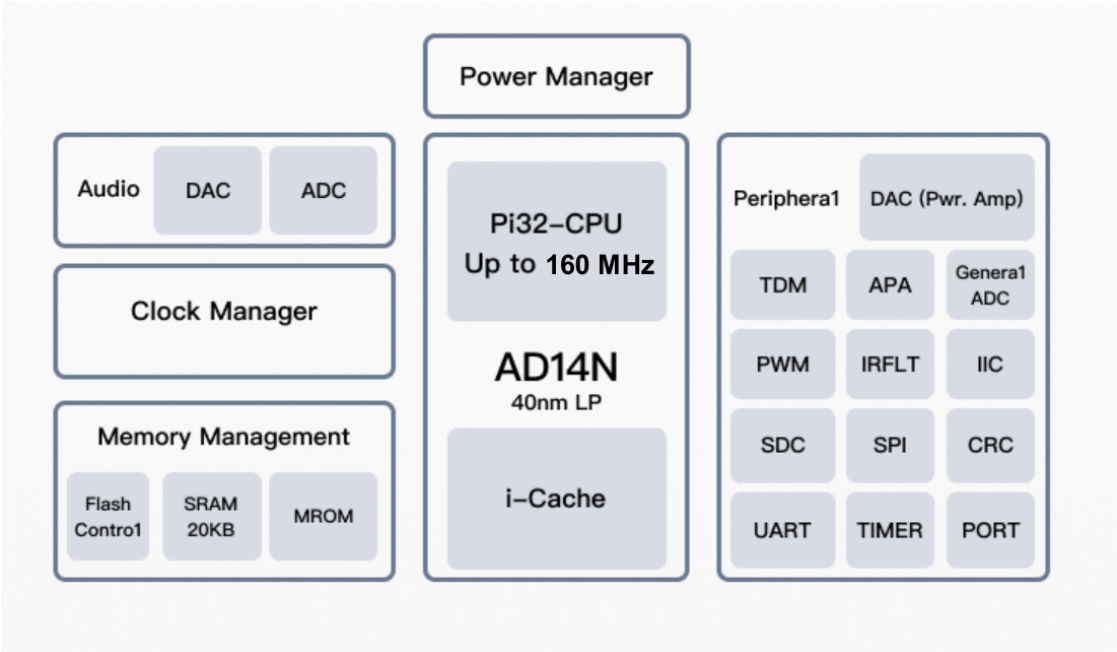
AD 系列硬件开发环境使用不同的开发板，包括不同底板和顶板。

本章节主要介绍芯片的芯片框图和软硬件开发环境。

1. “*AD14N* 芯片框图和硬件开发环境”
2. “*AD15N* 芯片框图和硬件开发环境”
3. “*AD17N* 芯片框图和硬件开发环境”
4. “*AD18N* 芯片框图和硬件开发环境”

4.1 AD14N 芯片框图和硬件开发环境

一、AD14N 芯片的芯片框图



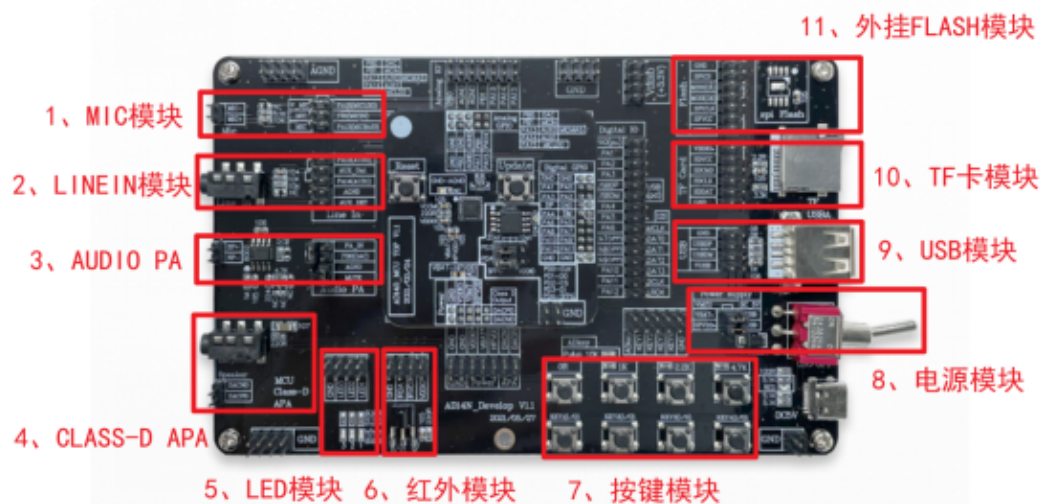
二、AD14N 芯片的开发板硬件环境

AD14N 使用的硬件环境开发板由底板和顶板两部分组成。

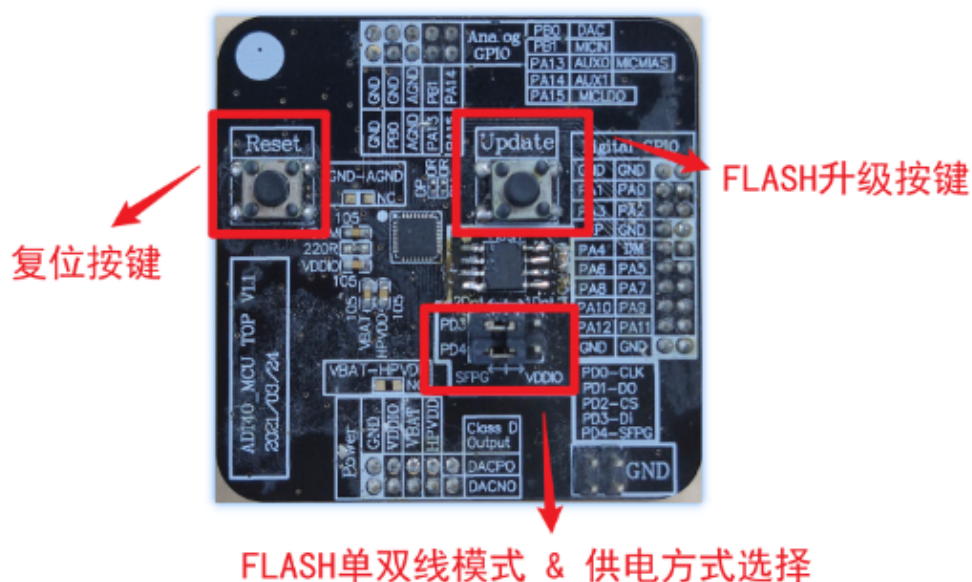
其中模块：

模块	模块详情
MIC 模块	MIC 输入引脚固定为 PB01，偏置电压可由 MICLDO (PA15) 或 MICBIAS (PA13) 提供，也可以使用外部偏置，不占用芯片 I/O 口；
LINEIN 模块	LINEIN 输入引脚可选 AUX0 (PA13) 或 AUX1 (PA14)，需将对应引脚与 AUX_OUT 短接；
Audio PA 功放模块	单端 DAC 输出引脚为 PB0，需将 PA_IN 与 PB0 短接；
Class-D APA 模块	差分 DAC 输出引脚为 DACN 和 DACP；
LED 模块	该模块的 3 个 LED 灯都是高电平点亮；
红外模块	该模块用于接收红外信号，需将 IRVDD 与 VDDIO 短接；
按键模块	该模块有 4 个 AD 按键以及 4 个 I/O 按键，其中 AD 按键外接 10k 上拉电阻；
电源模块	供电可通过正极接“VUSB”，负极接“GND”实现，且需要将 VBAT 和 VUSB 短接、HPVDD 和 VBAT 短接；
USB 模块	该模块可以用来通信 USB 设备，或给开发板下载程序以及供电；
TF 卡模块	该模块支持驱动一线模式的 TF 卡；
外挂 FLASH 模块	该模块可挂载一颗存放资源的 FLASH；

底板包含的模块如下图所示：



顶板包含的模块如下图所示：



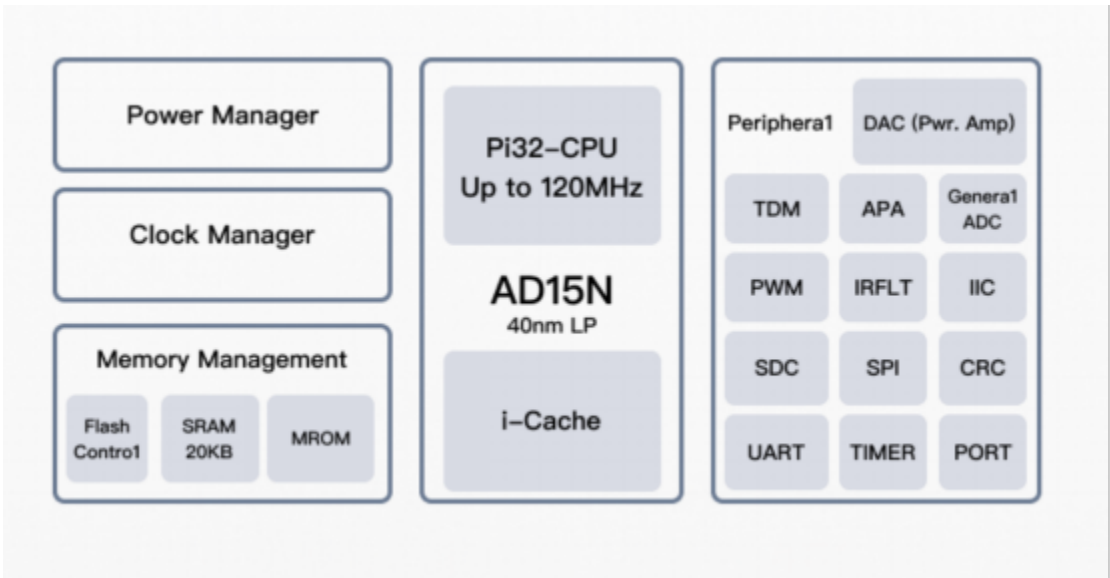
顶板的“FLASH 工作模式以及供电方式选择区”可以选择 FLASH 是跑 1 线模式还是 2 线模式，并且可以选择 POWER 引脚供电使用 VDDIO 还是 PD4 引脚。

系统 flash 单双线配置及 flash 电源配置：

使用 USB 升级 FLASH 的时候，插入 USB 线连电脑和开发板，按住“FLASH 短路按键”不松手之后再短按“复位按键”复位系统，之后松开“FLASH 短路按键”就可以进入升级，此时电脑可以正常出现设备盘符即是操作成功了。

4.2 AD15N 芯片框图和硬件开发环境

一、AD15N 芯片的芯片框图



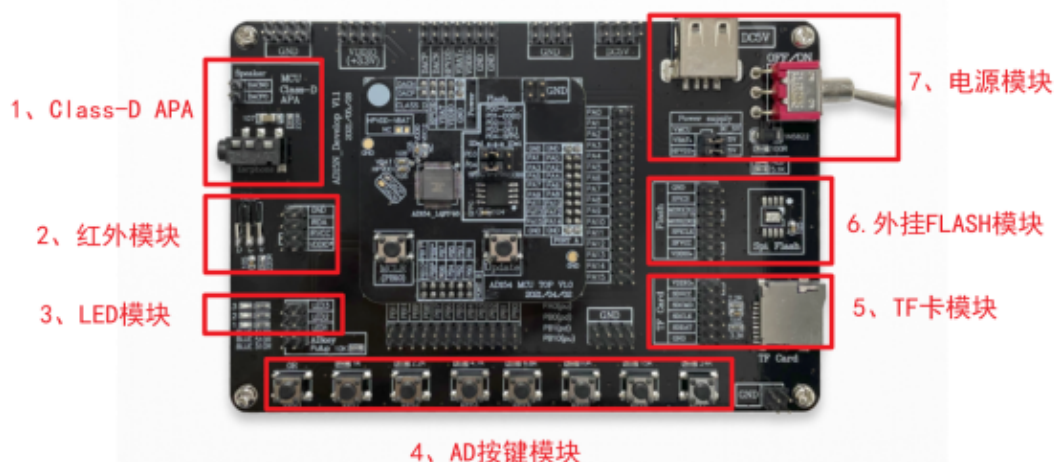
二、AD15N 芯片的开发板硬件环境

AD15N 使用的硬件环境开发板由底板和顶板两部分组成。

其中模块：

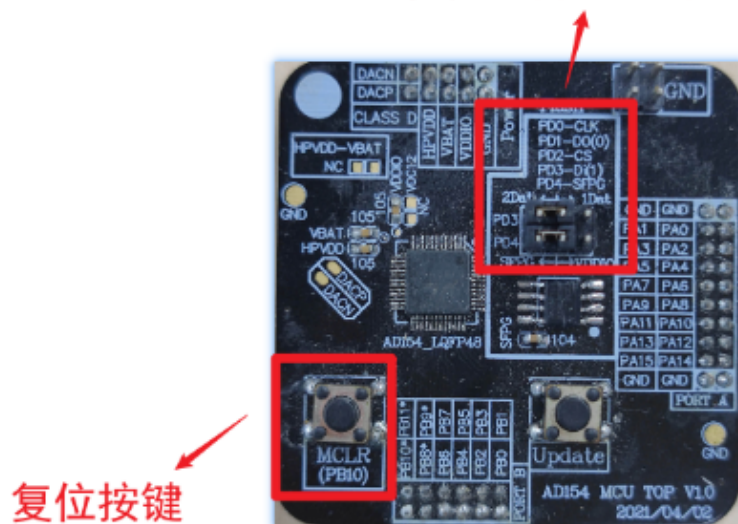
模块	模块详情
Class-D APA 模块	差分 DAC 输出引脚为 DACN 和 DACP；
红外模块	该模块用于接收红外信号，需将 IRVDD 与 VDDIO 短接；
LED 模块	该模块的 3 个 LED 灯都是高电平点亮；
按键模块	该模块有 8 个 AD 按键，外接 10k 上拉电阻；
TF 卡模块	该模块支持驱动一线模式的 TF 卡；
外挂 FLASH 模块	该模块可挂载一颗存放资源的 FLASH；
电源模块	AD15N USB 口只用来供电，芯片自身并没有 USB 模块。特别地，AD150 开发烧写使用串口升级方式，供电使用 USB UPDATER 插入 USB 口供电，并且 USB UPDATER TX 接到开发板的 PB9。

底板包含的模块如下图所示：



顶板包含的模块如下图所示：

FLASH单双线模式 & 供电方式选择

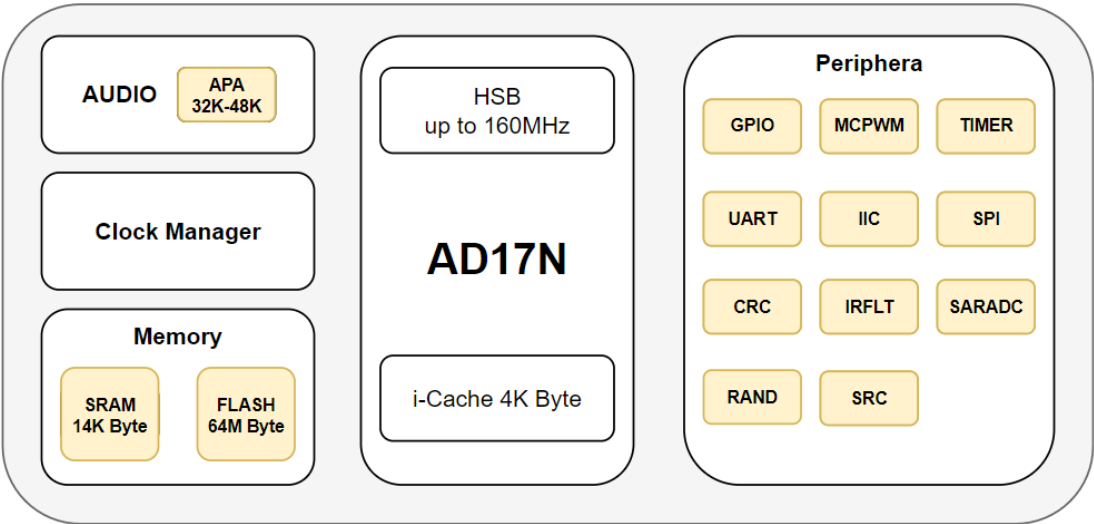


AD15N 使用串口方式升级，升级时候并不需要按着 Update，只需要点击运行升级脚本，USB UPDATER 就会让开发板自动进入升级模式了。

顶板的“FLASH 工作模式以及供电方式选择区”可以选择 FLASH 是跑 1 线模式还是 2 线模式，并且可以选择 POWER 引脚供电使用 VDDIO 还是 PD4 引脚。

4.3 AD17N 芯片框图和硬件开发环境

一、AD17N 芯片的芯片框图



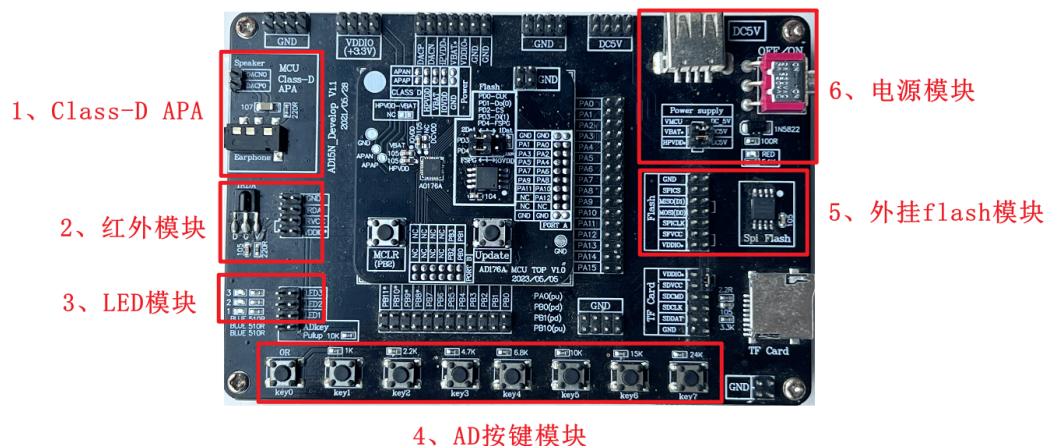
二、AD17N 芯片的开发板硬件环境

AD17N 使用的硬件环境开发板由底板和顶板两部分组成。

其中模块：

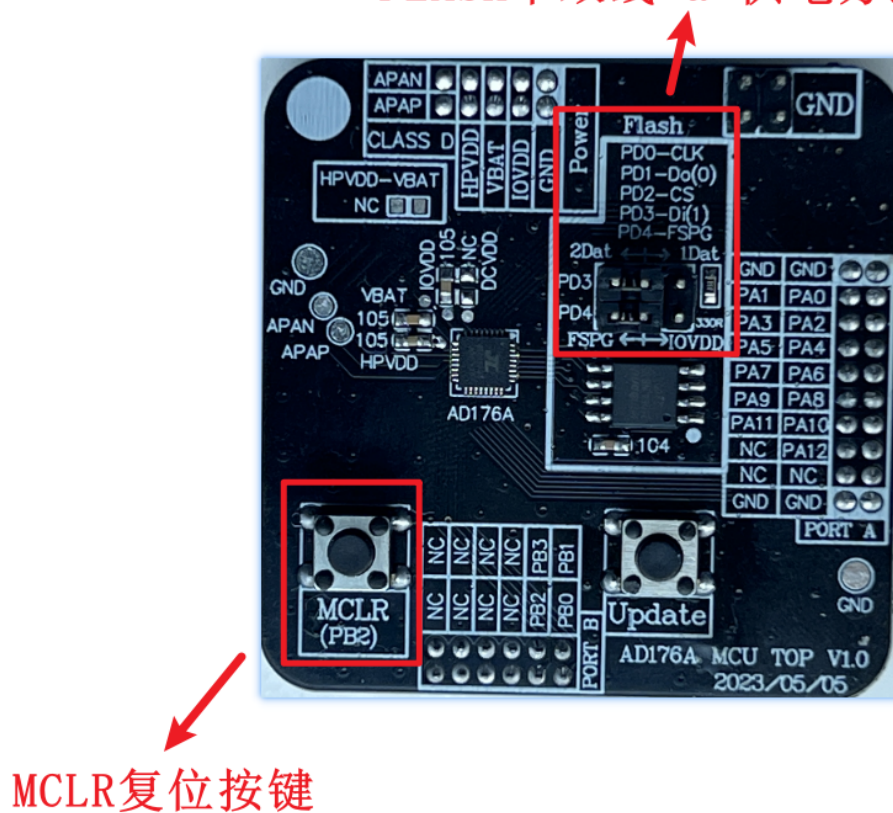
模块	模块详情
Class-D APA 模块	差分输出引脚为 APAN 和 APAP；
红外模块	该模块用于接收红外信号，需将 IRVDD 与 VDDIO 短接；
LED 模块	该模块的 3 个 LED 灯都是高电平点亮；
按键模块	该模块有 8 个 AD 按键，外接 10k 上拉电阻；
外挂 FLASH 模块	该模块可挂载一颗存放资源的 FLASH；
电源模块	AD17N USB 口只用来供电，芯片自身并没有 USB 模块。特别地，AD17N 开发烧写使用串口升级方式，供电使用 USB 强制升级工具插入 USB 口供电，并且 USB 强制升级工具 TX 接到开发板的 PB1。

底板包含的模块如下图所示：



顶板包含的模块如下图所示：

FLASH单双线 & 供电方式选择

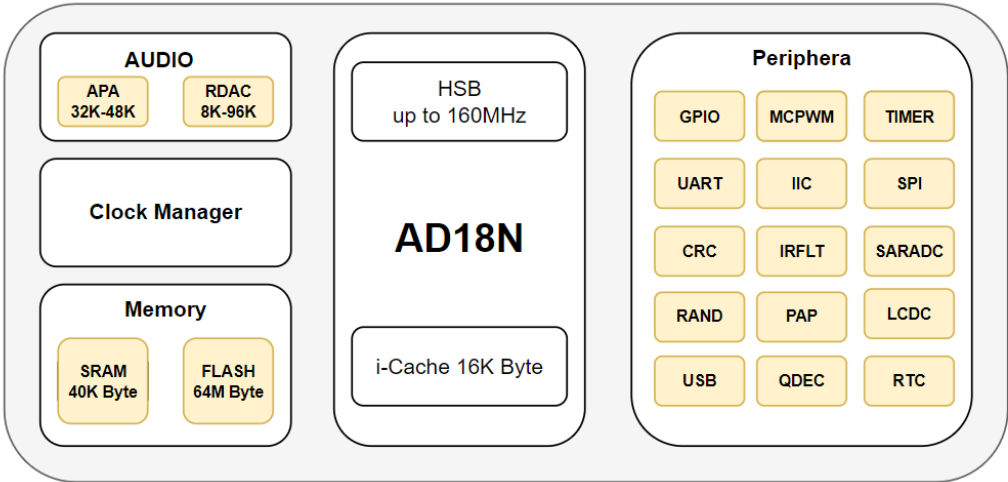


AD17N 使用串口方式升级，升级时候并不需要按着 Update，只需要点击运行升级脚本，USB 强制下载工具就会复位开发板让其进入升级模式。

顶板的“FLASH 工作模式以及供电方式选择区”可以选择 FLASH 是跑 1bit 模式还是 2bit 模式，并且可以选择 POWER 引脚供电使用 VDDIO 还是芯片 PD4(SPFG) 引脚。

4.4 AD18N 芯片框图和硬件开发环境

一、AD18N 芯片的芯片框图



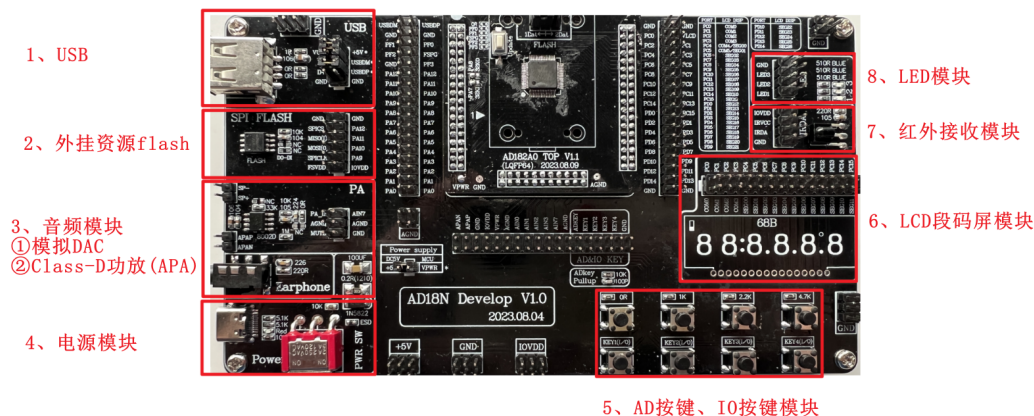
二、AD18N 芯片的开发板硬件环境

AD18N 使用的硬件环境开发板由底板和顶板两部分组成。

其中模块：

模块	模块详情
USB 模块	该模块可以用来通信 USB 设备，或给开发板下载程序以及供电；
外 挂 FLASH 模块	该模块可挂载一颗存放资源的 FLASH；
Audio PA 功放模块	单端模拟 DAC 输出引脚为 AIN7，需将 PA_IN 与 AIN7 短接；
Class-D 功放模块	差分输出引脚为 APAN 和 APAP；（耳机座子接 APA）
电源模块	供电可通过接 USB 线或者 Type-C 线实现，且需要将 +5V 和 VPWR 短接、HPVDD 和 VPWR 短接；
按键模块	该模块有 4 个 AD 按键，外接 10k 上拉电阻与不同阻值的下拉电阻；同时还有 4 个独立 IO 按键；
LCD 断码屏模块	该模块支持最多 4com * 12seg 的段码屏；
红外模块	该模块用于接收红外信号，需将 IRVDD 与 IOVDD 短接；
LED 模块	该模块的 3 个 LED 灯都是高电平点亮；

底板包含的模块如下图所示：

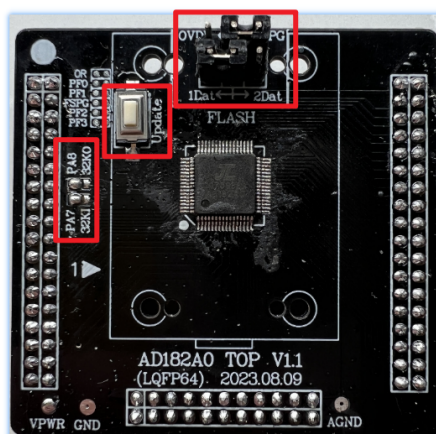


顶板包含的模块如下图所示：

1、FLASH单双线 & 供电方式选择

2、USB升级按键

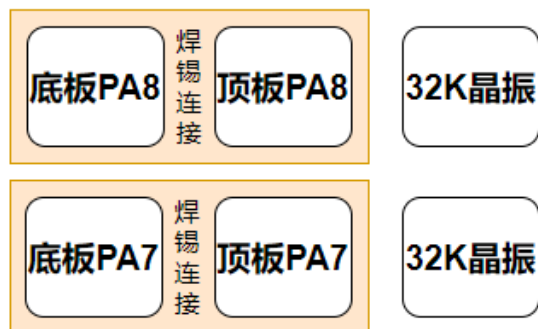
3、PA7 & PA8 外接32K晶振选择



AD18N 使用 USB 升级，升级前需接入 USB 强制下载工具并进入下载模式。（下载器红灯常亮即进入下载模式），再使用 Codeblock 编译下载即可；

顶板的“FLASH 工作模式以及供电方式选择区”可以选择 FLASH 是跑 1bit 模式还是 2bit 模式，并且可以选择 POWER 引脚供电使用 VDDIO 还是芯片 PD4(FSPG) 引脚。

顶板的“PA7 & PA8 外接 32K 晶振选择”可以选择顶板的 PA7、PA8 直接接到底板的 PA7、PA8，还是直接接到外挂 32K 晶振引脚；



PA7 PA8 用作普通I/O



PA7 PA8 外接32K晶振

下载目录介绍

本章节主要介绍下载目录。

SDK 编译完成后,会在下载目录生成代码文件,并与资源文件整合形成烧录文件,通过 CodeBlocks 调用下载工具烧写到芯片系统 FLASH 中;

5.1 下载目录介绍

下载目录位于 `app\post_build\sh5x\`对应工程(工程名称,如: `voice_toy`),一共分为三种文件:

(一) 生产相关文件

- `app.bin`: 编译生成的代码文件
- `jl_isd.bin`: FLASH 烧录文件,包含代码文件和资源文件(AD23N 没有)
- `jl_isd.fw`: 用于我司一拖二/八工具批量生产的 FLASH 烧录文件

(二) 资源文件

- `dir` 文件: 通过打包工具生成的打包文件,通常用于存放音频资源

(三) 下载工具

- `download_bat.c`: 下载批处理文件,可指定内置和外挂 FLASH 烧写内容
- `isd_config.ini`: 工程配置文件
- 下载目录工具更新 `jlproj`: 该工具可在线更新下载目录的 `exe` 工具

本小节着重介绍下载工具的 `download_bat.c` 以及 `isd_config.ini` 文件;

download_bat.c 文件介绍:

- 指定 `app` 大小 `-app`
- 添加资源文件 `-res`

- 擦除 *flash* 部分区域或所有区域 *-format*
- 添加 *key* 文件 *-key*
- 外挂资源 *flash* 下载

isd_config.ini 文件介绍:

- 系统 *flash SPI* 配置 *-SPI*
- *Cache* 空间配置 *-CACHE_WAY*
- *UBOOT* 启动烧 *key* 配置 *-BOOT_TYPE*
- 程序偏移配置 *-EOFFSET & EOFFSET2UP*
- *LRC* 时钟源校准配置 *-LRC*
- 外挂资源 *flash* 下载引脚配置 *-EX_FLASH & EX_FLASH_IO*
- 开辟自定义空间配置

外挂资源 *flash* 下载需要 (AD23N 暂不支持):

1. 外挂资源 *flash* 下载
 2. 外挂资源 *flash* 下载引脚配置 *-EX_FLASH & EX_FLASH_IO*
-

5.2 download_bat.c 文件介绍

download.bat 脚本由 download_bat.c 文件编译生成, download_bat.c 文件支持把代码和资源文件以及部分配置烧写到系统 FLASH, 也支持将指定资源文件烧写到外挂 FLASH

具体参数如下:

- -app app.bin 0x20000:

该指令指定了烧写代码文件 app.bin, 并且指定了代码存放的系统 FLASH 区域大小固定为 0x20000, 即无论 app.bin 大小为多少, 代码区域都会暂用指定的大小, 预留出空间, 方便后续代码修改后升级使用; 用户也可以去掉 0x20000, 不指定代码存放区域的大小, 系统会根据 app.bin 大小自动分配, 但代码改动可能导致资源位置变化, 需要重新下载所有资源。

- -res xxx:

该指令会把指定的资源文件烧写进入系统 FLASH, 不同资源文件以空格隔开。

- -format vm:

该指令会擦除 isd_config.ini 文件中指定的“VM”区域。注: 需要注意区分“VM”区域以及 VM 掉电存储功能。前者为 isd_config.ini 文件中指定的一块名为“VM”的 FLASH 空间区域, 一般用于存放 AD14N 内置 FLASH 的录音数据; 后者为代码中的 VM 断电保存功能, 其实际存放在 isd_config.ini 文件中指定的一块名为“EEPROM”的 FLASH 空间区域中!

- -format all:

该指令会擦除整颗系统 FLASH 数据。

- -key *.key:

该指令会加密烧写文件。参数 *.key 为申请的 key 文件, 不同系列不可混用 key 文件。注: 该指令 AD23N 不可用

- -ckey *.ckey:

该指令用于 AD23N 加密烧写文件。参数 *.ckey 为申请的 ckey 文件，不同系列不可混用 ckey 文件。

```

27 cd app_mg
28
29 isd_download.exe -tonorflash -dev sh54 -boot 0xb00 -div8 -wait 300 -uboot uboot.boot -uboot compress -app app.bin 0x20000
30 -res dir_a dir_song dir_eng dir_poetry dir_story dir_notice dir_bin_flash -wflash dir_song 0 [PA05_1B_NULL]
31
32 @REM 2、添加烧写到内置FLASH的资源文件，以空格隔开
33 @rem -format vm
34 @rem -format all
35 @rem -reboot 500
36
37 @REM //烧写外置flash 命令说明:
38 @rem -wflash dir_song 0 [PA05_1B_NULL]
39 @rem // dir_song : 要烧写的文件名 (文件需在download.bat文件夹下)
40 @rem // 0 : 文件烧录到外置flash的起始地址
41 @rem // [PA05_1B_NULL]: PA05: 外置flash片选引脚 (注意: 不能选USBDP/USBDM)
42 @rem // 1B : spi1, B端口
43 @rem // NULL: power_io & spi1_data_width, power_io连接到外置flash vcc引脚
44 @rem // 可控制flash电源; spi1_data_width: 0: 单线; 1: 双线
45 @rem // 例: NULL/PA00: power_io: 无/pa0; spi1: 双向模式 (注: power_io不能选USBDP/USBDM)
46 @rem // NULL/B010: power_io: 无/pb1; spi1: 单线模式
47 @rem // NULL/A081: power_io: 无/pa8; spi1: 双向模式
48 @rem //注意: spi端口只能选B/C (B: PA11, PA12, PA10[CLK, DO, DI]; C: PA4, PA5, PA6[CLK, DO, DI])
49
50 @REM //生成固件升级文件
51 fw add.exe -noenc -fw jl_isd.fw -add ota.bin -type 100 -out jl_isd.fw
52 ufw_maker.exe -fw to ufw jl_isd.fw
53 copy jl_isd.ufw update.ufw
54 del jl_isd.ufw

```

download.bat.c 介绍

5.3 外挂资源 flash 下载 (AD23N 暂不支持)

download.bat.c 支持将资源文件下载到外挂资源 flash 中，不同芯片配置参数不同，其中：

一、AD14N / AD15N / AC104N 下载外挂 Flash 命令

- -wflash dir_XXX 0 [PA05_1B_NULL]:
 1. dir_XXX: 要烧写的文件名 (文件需在 download.bat.c 所在的目录下);
 2. 0: 文件烧录到外挂资源 flash 的起始地址;
 3. [PA05]: 外挂资源 flash 的 CS 片选引脚 (注意, 不能选 USBDP、USBDM);
 4. [1B]: 选择芯片的 spi1, B 端口引脚, 端口只能选择 B/C;
 5. [NULL]: 选择“NULL”时为双线, DO、DI 分开连接, 选择“NUL0”时为单线, DO&DI 短接;

二、AD17N / AD18N / AD24N 下载外挂 Flash 命令

- -ex_flash dir_song_ext:

该指令会根据 isd_config.ini 中定义的外挂 flash 引脚 (详情请看 EX_FLASH 与 EX_FLASH_IO 说明), 将指定的资源文件下载到外挂 flash 中, 外挂 flash 引脚配置将于后文 isd_config.ini 文件介绍中说明。

注意: 准备下载到外挂 flash 的 dir 文件, 需要经过 packres.exe 指定下载地址再次打包生成新的 dir 文件, 才可使用 -ex_flash 命令。例如将 dir_song 文件下载到外挂 flash, 需要通过 packres 脚本添加 “packres.exe -n res -o dir_song_ext dir_song 0 -normal” 命令, 指定 0 地址将 dir_song 文件打包成 dir_song_ext 文件, 在 download.bat.c 中添加 -ex_flash dir_song_ext 指令即可将 dir_song 文件完整烧写至外挂 flash 中。

5.4 isd_config.ini 配置文件说明链接

关于工具链配置的详细信息，请参考：

杰理开发工具链 INI 配置文档详细说明 https://doc.zh-jieli.com/Tools/zh-cn/dev_tools/toolchains/ini_cfg.html。

5.5 isd_config.ini 文件介绍

isd_config.ini 是一个工程配置文件，用于配置程序中的烧写和启动相关参数，并在下载和生成生产烧写文件时起作用。

5.6 AD23N/24N 的 isd_config.ini 文件重要变化介绍

AD23N/24N 的 isd_config.ini 文件是通过 isd_config.ini.c 在工程的预编译过程中生成的。

如果开发人员需要修改相关参数，应修改 isd_config.ini.c 文件。

isd_config.ini.c 及其生成的 isd_config.ini 文件，位于 app/post_build/ 目录下的芯片子目录中的工程目录。

例如，以 AD24N 的 voice_toy 工程为例，文件路径为：app/post_build/sh58/voice_toy/。

5.7 isd_config.ini 文件指令介绍

(一) EXTRG_CFG_PARAM

- PID: 工程标识，不可超过 16 个字节；
- VID: 工程版本号，用于升级判断；
- NEED_RESERVED_4K: 4K 保留区配置，详情请看《RESERVED_AREA 区域说明》；
 NEED_RESERVED_4K = YES: 保留 4K 配置区；
 NEED_RESERVED_4K = NO: 不保留 4K 配置区；
- SPECIAL_OPT: 选择是否实时根据 FLASH 支持的对齐方式组织文件；
 SPECIAL_OPT = 0: 根据下载时所连样机 FLASH 的对齐方式组织文件，只生成 1 份 4K 或者 256 对齐的烧录文件；
 SPECIAL_OPT 不存在: 生成多份不同对齐方式的烧录文件；
 注：烧录文件在组织过程中支持按 4K 或者 256 字节对齐；若用 256 字节对齐的烧录文件，去生产最小擦除单位为 4K 字节的芯片内置 FLASH，会出现开不了机的问题!!! 注释该配置会关闭该功能，默认生成多份不同对齐方式的文件，由烧写器判断烧写哪一份。
- FORCE_4K_ALIGN: 选择是否强制只生成 1 份 4K 字节对齐的 FLASH 烧录文件
 注：如果方案需要在线升级，建议使用强制 4K 对齐，SDK 默认只生成 1 份 4K 对齐的烧录文件！
- FLASH_SIZE: 指定 FLASH 大小，用于生成指定大小的烧录文件

- ① 0x800000: 按照 8M 字节的 FLASH 的空间生成烧录文件, 如果在线下载时发现实际的 FLASH 与指定的大小不匹配, 则会报错;

注: 在没有直接指定 FLASH 大小的情况下, 最后生成的下载文件会以市面 FLASH 能支持的最小文件的形式组织; 例如:

- 1、当所有文件小于 256K 字节时, 系统会以总大小为 256K 为标准为录音分配剩余空间;
- 2、当所有文件大小等于 300K 时, 系统会以 512K 为标准为录音分配剩余空间;

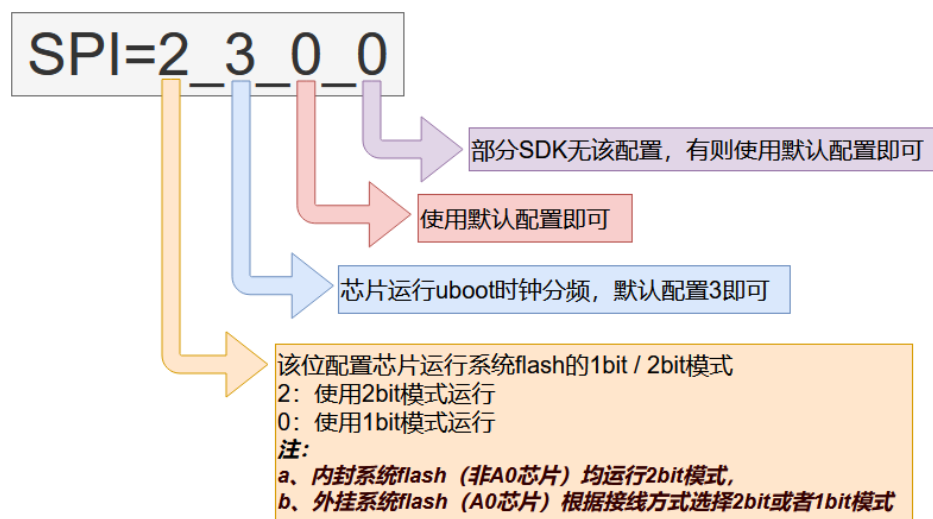
(二) SYS_CFG_PARAM

- SPI: 芯片内置 FLASH 的 SPI 配置, 配置说明如下图所示;

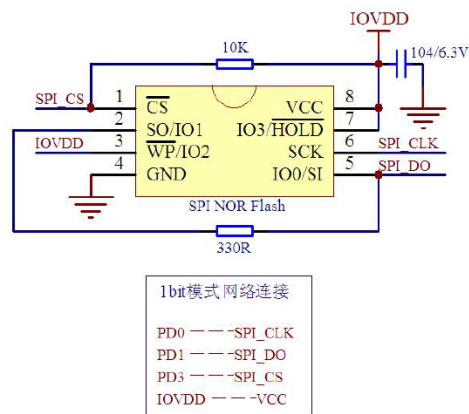
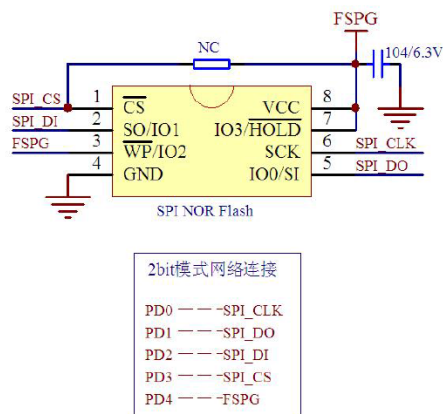
注意: 对于主频速度要求高的方案, 尽量使用两线模式, 有利于系统稳定性和功耗控制。

- ① 0_3_0: 内置 FLASH 使用单线数据模式;
- ② 2_3_0: 内置 FLASH 使用双线数据模式;
- ③ 4_3_0: 内置 FLASH 使用四线数据模式;

芯片	AD14N / AC104N	AD15N	AD17N	AD18N	AD24N	AD23N
是否支持该功能	支持	支持	支持	支持	支持	支持



SPI 配置参数说明



系统 flash 2 线/1 线接法

- **CACHE_WAY**: 芯片使用 cache 的 WAY 数 (1 WAY = 4K 字节); 多出来的 cache 可以作为普通 ram 使用;

注意: Cache 会优先使用地址靠前的空间; 例如 AC104N 有 4 条 WAY 共 16K 字节, 若 **CACHE_WAY** 配置为 2, 系统会占用 16K 字节的前 8K 用于 cache 缓存, 后 8K 字节可用作普通 RAM, 具体请参考 AC104N SDK 的 *ld* 文件;

芯片	AD14N AC104N	/ AD15N	AD17N	AD18N	AD24N	AD23N
是否支持该功能	支持, 可选 1~4	支持, 可选 1~2	不支持	不支持	不支持	不支持

- **BOOT_TYPE**: AD14N / AD15N 在 2021 年 5 月 18 日之后发布的 SDK 已经使用了新的工具生成芯片密钥。新的密钥系统支持芯片用户派生密钥 (leaf key) 为空的芯片; 原厂密钥 (root key) 加上客户派生密钥 (leaf key) 组成完整的芯片密钥, 方便客户备货;

- ① **BOOT_TYPE** = 0: 芯片启动时发现 KEY 不一致, 不启动工程;
- ② **BOOT_TYPE** = 非零: 芯片启动时发现 KEY 不一致, 新系统启动烧写芯片密钥的功能, 之后再尝试启动系统; 如果芯片已有的 KEY 和现有的要烧进去的 KEY 冲突, 有烧坏芯片密钥的可能!!!

芯片	AD14N / AC104N	AD15N	AD17N	AD18N	AD24N	AD23N
是否支持该功能	支持	支持	不支持	不支持	支持	支持

- **EOFFSET & EOFFSET2UP**: 代码偏移配置, 部分芯片 FLASH 需要配置偏移参数;

芯片	AD14N / AC104N	AD15N	AD17N	AD18N	AD24N	AD23N
是否支持该功能	支持	支持	不支持	不支持	支持	支持

- **LRC**: LRC=-1 时, 烧写器烧写时校准芯片 LRC 时钟, 默认开启;

芯片	AD14N / AC104N	AD15N	AD17N	AD18N	AD24N	AD23N
是否支持该功能	支持	支持	支持	支持	支持	支持

- EX_FLASH: 外挂 flash 烧写配置 (使用外挂 flash 下载资源需要打开该配置)

- ① PA05_1A_NULL: 配置顺序为 “CS-SPI 端口-电源口”;
- ② 用 VDDIO 供电电源口参数为 NULL, 使用时配置 CS 脚 I/O 即可;

芯片	AD14N / AC104N	AD15N	AD17N	AD18N	AD24N	AD23N
是否支持该功能	不支持	不支持	支持	支持	支持	暂不支持

- EX_FLASH_IO: 外挂 flash 烧写引脚配置 (使用外挂 flash 下载资源需要打开该配置)

- ① 2_PA01_PA02_PA03_NULL_NULL: 配置顺序为 “数据宽度-CLK-DO-DI-D2-D3”;
- ② 双线模式时数据宽度为 2, D2、D3 口为 NULL, 即 2_PA01_PA02_PA03_NULL_NULL;
- ③ 单线模式时数据宽度为 0, D2、D3 口为 NULL, 即 0_PA01_PA02_PA03_NULL_NULL;
- ④ 四线模式时数据宽度为 4, 需要配置 D2、D3 口;

芯片	AD14N / AC104N	AD15N	AD17N	AD18N	AD24N	AD23N
是否支持该功能	不支持	不支持	支持	支持	支持	暂不支持

(三) RESERVED_CONFIG

SDK 默认开辟了以下区域:

- VM 区域: 用于内置 flash 录音
- EEPROM 区域: 用于 VM 掉电存储功能

需要分清楚 VM 区域与 VM 掉电存储功能! VM 区域是 flash 一片空间的名称, VM 掉电存储是一种功能

- XXXX_ADR: 区域起始地址
 - ① 0: VM 区域起始位置;
 - ② AUTO: 由工具自动分配起始地址;
- XXXX_LEN: 区域大小
- XXXX_OPT: 区域操作属性
 - ① 0: 下载代码时擦除指定区域;
 - ② 1: 下载代码时不操作指定区域;

注: VM 区域的起始地址为 0, 且实际大小为系统剩余的空间, 扩展录音空间可查看 (一) 中 FLASH_SIZE 配置的说明; EEPROM 区域一般用于代码中的 VM 断电记忆存储, 区域大小不少于 8K, 否则 VM 初始化会失败; 用户可自行定义其他空间区域;

杰理科技一拖二烧录器用户手册链接

关于杰理科技一拖二烧录器的详细信息，请参考：

杰理科技一拖二烧录器用户手册 https://doc.zh-jieli.com/Tools/zh-cn/mass_prod_tools/burner_1tuo2/index.html。

工具使用环境介绍

同时我司有许多工具支持压缩转换和打包音频，将资源文件下载到内置或外挂 FLASH 中。

7.1 开发工具介绍

AD24N 支持的开发工具如下：

1. 资源打包工具：能将多个文件打包成一个文件；
2. 音频文件转换工具：能将音频文件转换为系统能支持的格式；
3. MIDI 转换工具：能将 midi 文件转换为系统能支持的格式以及制作 midi 音色库；
4. MIO 工具：能制作波形随音频律动的 MIO 文件；
5. SYD 系统制作工具：能格式化 SD 卡/U 盘并挂载我司 SYD 文件系统的工具；
6. 多算法授权；

以上工具可在 AD14N_AD15N_gitee 开源网站上或询问我司深圳办人员获取，部分工具内含说明文档。

本节将介绍以下工具；

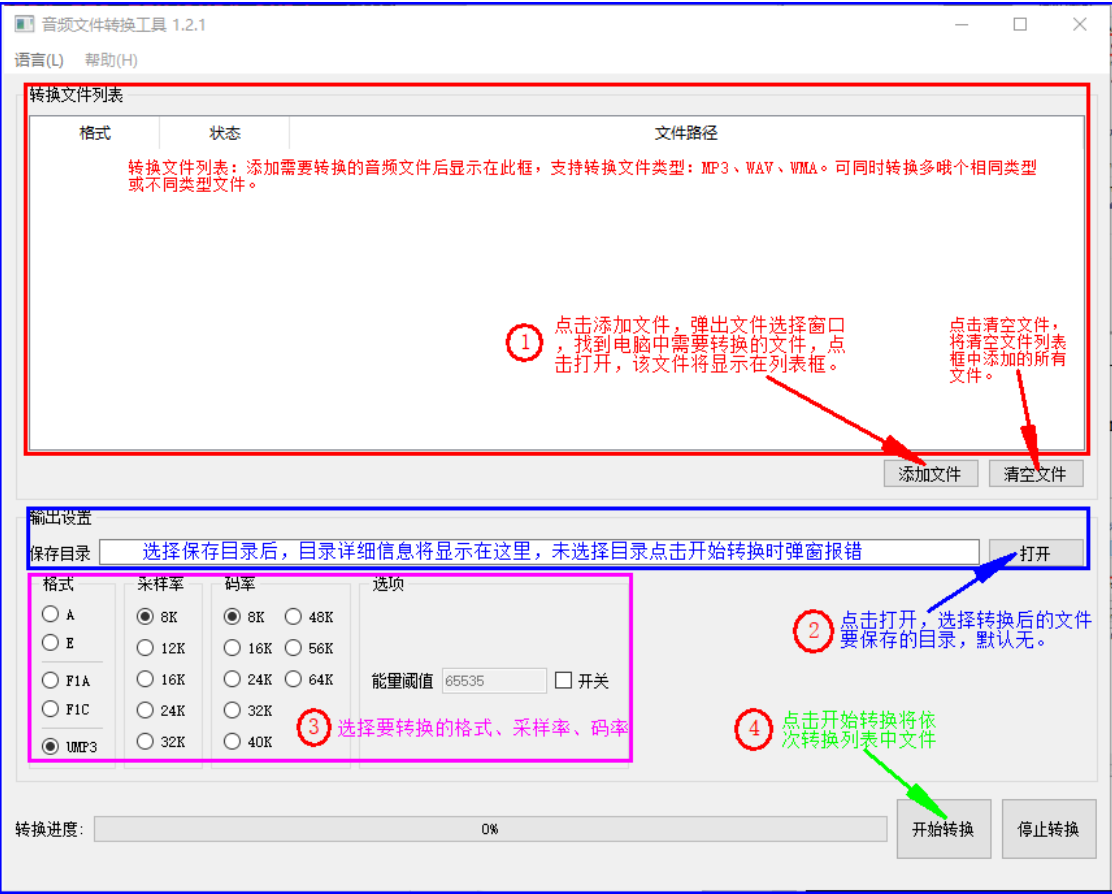
1. 音频转换工具
2. 资源打包工具
3. 多算法授权简单介绍

7.2 音频转换工具

音频文件转换工具 _1.2.20.exe 是我司用于压缩转换 WAV、MP3 和 WMA 音频的工具，可将音频转换成我司的 A（B、E）格式、F1A（F1B、F1C）格式、F1X 格式以及 UMP3 格式；

（一）音频转换步骤

- 1、添加需要转换的音频文件；
- 2、选择输出音频的保存目录；
- 3、选择需要转换的格式、采样率、码率以及其他选项；
- 4、点击开始转换；



音频文件转换步骤

（二）目标格式选择

转换工具可生成三种类型文件：

- 第一类 *.a、*.b、*.e；
- 第二类 *.fla、*.flb、*.flc；
- 第三类 *.ump3。

每种类型都有采样率或码率供自由选择，一般采样率高了对应选择的码率也会高些，采样率跟码率都影响音质。

类型格式选择：选哪个格式，一般考虑音质、设备资源以及系统运行速度。

备注：

例如需要音质高而 flash 大小够用，系统资源够跑，就选用高压缩比的类型。
一般要低码率的话，可以先选 f1a/f1b/f1c；
系统需要速度快，就选 a。
如果 f1a/f1c 没法满足要求，如压音乐之类的，可以选 ump3。
如需要重复播报的音频段或者音频间无缝衔接的，可以选择 f1x。

• a、b、e 格式：

该格式转换压缩比较低，速度快，资源小。可用于人声或者背景音乐。
该格式特点是音质较差，但占用内存小。

格式	采样率
☒ A	☒ 8K
☐ E	☐ 16K
☐ F1A	
☐ F1C	
☐ UMP3	

a、b、e 格式配置图

• f1a、f1b、f1c、f1x 格式：

该格式转换压缩比较高，比较适用人声和音乐，音质一般，其中 f1a、f1b、f1c 可用于无缝衔接，f1x 格式可用于音频拼接。

格式	采样率	码率
☐ A	☒ 8K	☐ 8K ☐ 48K
☐ E	☐ 12K	☐ 16K ☒ 56K
☒ F1A	☐ 16K	☐ 24K ☐ 64K
☐ F1C	☐ 24K	☐ 32K
☐ F1X	☐ 32K	☐ 40K
☒ UMP3		

f1a、f1b、f1c、f1x 格式配置图

• ump3 格式：

m 该格式有较多的采样率和码率可以选择，可以按情况来压缩音乐或者人声，
该格式特点是占用内存大，音质较好。

格式	采样率	码率	选项
☐ A	☒ 8K	☒ 8K ☐ 48K	能量阈值 65535 ☐ 开关
☐ E	☐ 12K	☐ 16K ☐ 56K	
☐ F1A	☐ 16K	☐ 24K ☐ 64K	
☐ F1C	☐ 24K	☐ 32K	
☒ UMP3	☐ 32K	☐ 40K	

ump3 格式配置图

(三) 其他参数说明

1、f1a / f1b / f1c 无缝衔接转换和提高小文件压缩比

(1) 是否无缝循环

如果勾选是否无缝循环，则可以首尾无缝对接循环重复播放同一个音频。其能量转换阈值范围 0 到 65536，代表 0 到 1，默认参数配成 10，即砍掉接近完全静音的部分。

运算过程中的音频也是归一化到 0 到 1 来计算的。

在实际应用中，调整的阈值范围一般是选取 0 到 0.1，即约 0 到 6000。可以设成 2048。试下循环效果。

(2) 循环附加预处理

如果勾选循环附加预处理，会对 f1a/f1b/f1c 文件再做多一次处理，分析计算文件前后的帧叠位置，计算前后的砍帧个数，然后对文件前后做一次前后自适应砍帧的动作。

(3) 提高小文件压缩比

如果勾选提高小文件压缩比，会使压缩后的目标文件码率较为接近目标码率。

f1a 是 cbr 的编码。需要长时间来平均来达到接近目标码率。

如果文件都很小的时候，压出来的目标文件码率可能会距离目标码率差距较大。

勾选提高小文件压缩比，会接近些。

备注：注：建议无缝循环的时候，只勾选是否无缝循环选项，听得出卡顿的文件，再勾选循环附加预处理，因为这个步骤的砍帧可能会出现有些文件前后会被砍比较多的情况！

2、f1x 制作说明

制作 f1x 音频时，需要选择音频头字节数和头尾去除帧数。

音频头字节数用于文件寻址，与原始转换文件大小有关。头字节数为 3 时，生成的 f1x 文件超过 16M 字节会报错，文件较多时可节约空间；头字节数为 4 时，则无限制。

头尾去除帧数用于选择去除的 f1x 内各个音频文件头尾的音频数据包数，可以调整衔接效果。

但不可完全依赖该功能，对于衔接效果不理想的音频，需要从音源文件去改善。

3、ump3 的能量阈值

ump3 格式有能量阈值选项，勾选开关便可以设置阈值，音频前后端会根据这个值砍掉一些低能量的音频，使播放开始和结束时更柔和。

阈值范围：0-2147483648，默认参数配成 65535。即砍掉接近完全静音的部分，值越大能量剪切越多。

鼠标悬停在能量阈值输入框无任何操作两秒以上会显示阈值范围。

7.3 资源打包工具

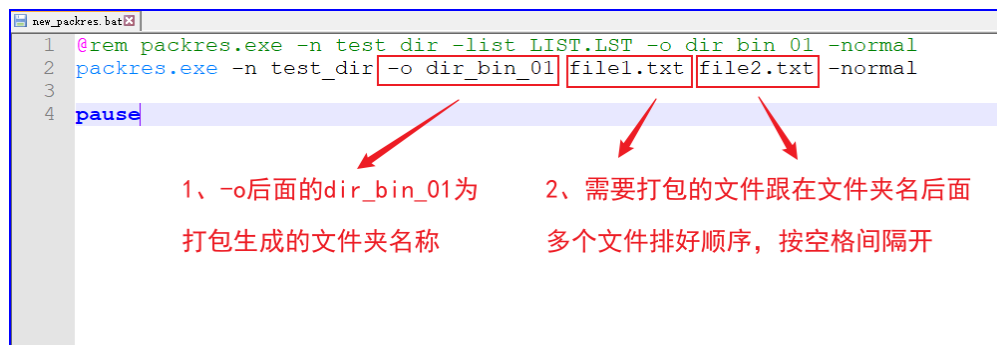
下载到系统 FLASH 的资源文件需要经过资源打包工具处理之后才能使用，SDK 下载目录中的 dir 文件就是经过该工具打包后的文件。

打包工具包括：new_packres.bat、packres.exe 和 pRFiles.exe；

其中 new_packres.bat 支持的命令详情请看同目录下的 readme.md 文件。

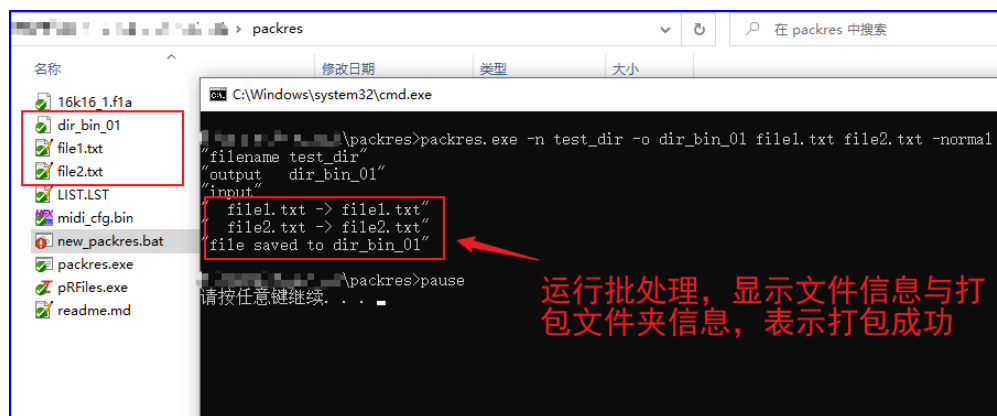
(一) 少量文件直接打包

1、将需要打包的文件放在工具根目录下，打开 new_packres.bat 文件，指定需要打包的文件以及输出的文件夹名；



new_packres.bat 资源打包批处理说明

2、保存 new_packres.bat 文件，并双击该批处理，工具根目录下生成打包文件夹表示打包成功；



new_packres.bat 运行结果图

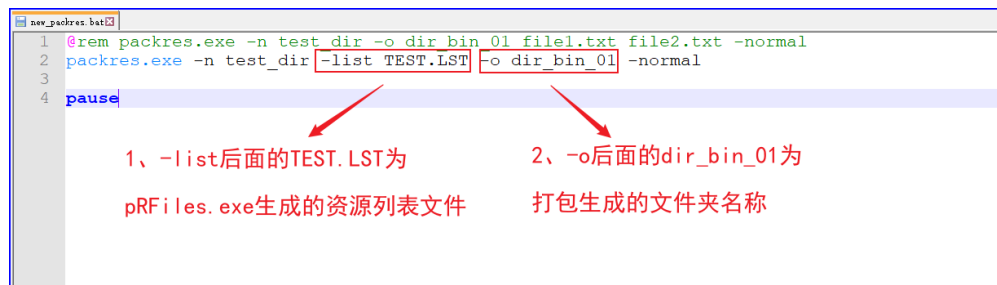
(二) 大量文件通过 pRFiles.exe 批量打包

1、将大量文件放在文件夹中，打开 pRFiles.exe，如下图所示进行操作，保存为 LST 文件，以保存为 TEST.LST 为例；



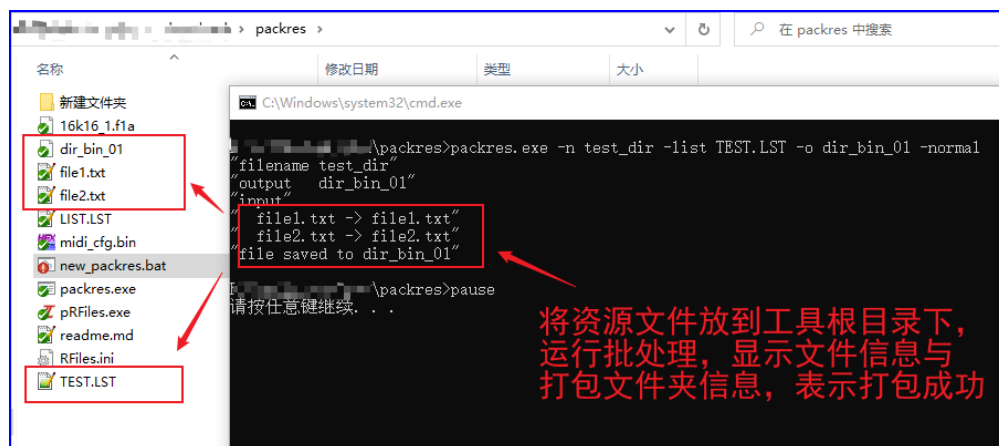
pRFiles.exe 打包工具操作图

2、将大量文件放在文件夹中，打开 pRFiles.exe，如下图所示进行操作，保存为 LST 文件，以保存为 TEST.LST 为例；



new_packres.bat 资源打包批处理说明

3、保存 new_packres.bat 文件，把资源文件全部放到工具根目录下，并双击该批处理，工具根目录下生成打包文件夹表示打包成功；



new_packres.bat 运行结果图

7.4 多算法授权简单介绍

使用该功能前请咨询杰理助理；

多算法授权，指的是：在一个 SDK 中，使用了算法提供商的库函数，而算法提供商可以控制芯片的烧录次数。

（一）使用场景

1. 算法提供商想要控制生产商使用算法库的次数（即：算法提供商按生产的样机收取费用）。

具体详细说明可以点击下方链接：

[多算法授权使用说明](#)

时钟驱动

AD14N / AD15N / AC104N / AD17N / AD18N / AD24N / AD23N 支持动态设置系统 HSB、LSB 时钟，支持多种分频组合；

时钟驱动中限制着内置系统 flash 的 spi 时钟速度，SDK 为兼容大多数 flash，默认的 spi 时钟不超过一定的限制，限制值见下文说明；

最高系统时钟如下：

最 高 系 统 时 钟 / CPU	AD14N / AC104N	AD15	AD17	AD18	AD24N	AD23N
HSB (系统 时钟)	192MHz (超频)	160M	160M	160M	240MHz	288MHz (PLL 时钟需 要配到 576MHz)
LSB	96MHz (超频)	80MH	80MH	96MH	120MHz (前提是系统 时钟运行到 240MHz)	192MHz (前提是系统 时钟运行到 288MHz)

AD14/15/17 以及 AC104 时钟相关函数：

```
void pll_sel(u32 pll_clock, _PLL_DIV pll_div, _PLL_B_DIV pll_b_div);
int clk_get(const char *name);
```

AD18 时钟相关函数：

```
int clk_set(const char *name, int clk);
int clk_get(const char *name);
```

AD24/23 时钟相关函数：

```
int clk_early_init(enum pll_ref_source pll_ref, u32 ref_frequency, u32 pll_
↪frequency);
int clk_set(const char *name, int clk);
int clk_get(const char *name);
```

8.1 AD14N / AC104N 系统时钟超频说明

AD14N / AC104N 支持最高 192MHz 的系统时钟，在运行大于 160MHz 系统时钟前需要提高系统内核电压，否则可能会导致运行异常；

支持 192MHz 时钟的内核电压，需要调用以下接口修改，需要在系统时钟初始化前调用：

```
VDC13_VOL_SEL(VDC13_VOL_SEL_140V);
```

```
SYSVDD_VOL_SEL(SYSVDD_VOL_SEL_129V);
```

系统时钟不超过	内核电压挡位	内置系统 Flash	A0 芯片外挂系统 Flash
160mhz	默认值	支持	支持
192Mhz	1.29v	支持	支持

表系统时钟支持情况

8.2 内置系统 flash spi 时钟说明

当使用 A0 芯片时，系统 Flash 需要外挂，由于 Flash 的型号以及性能，加上 PCB 布线等情况的影响，不能预测系统 Flash 波特率运行在超过 50Mhz、甚至是到 96MHZ 的具体情况；在此种情况下，客户如果需要系统 Flash 波特率运行在超过 50Mhz，需要自行评估。

系统 Flash 波特率	内置系统 Flash	A0 芯片外挂系统 Flash
"<=60Mhz"	支持	支持
">=60mhz" & "<=96Mhz"	支持	不能直接支持
">96MHZ"	不支持	不支持

表系统 Flash 波特率支持情况

8.2.1 AD14N / AC104N SPI 时钟限制

```

121 AT(.sfc.text.cache.L2)
122 static u32 sfc_max_baud(u32 pll_clock, _PLL_DIV pll_div)
123 {
124     u32 t_pll_clk = 0;
125     /* log_info("pll set, pll_clock %d, pll_div %d \n", pll_clock, pll_div); */
126     if (0 == (3 & pll_clock)) {
127         if (0b0100 == (0x0c & pll_clock)) {
128             t_pll_clk = 107;
129         } else if (0b1000 == (0x0c & pll_clock)) {
130             t_pll_clk = 137;
131         } else if (0b1100 == (0x0c & pll_clock)) {
132             t_pll_clk = 192;
133         }
134     } else if (1 == (0x03 & pll_clock)) {
135         t_pll_clk = 96;
136     } else if (2 == (0x03 & pll_clock)) {
137         t_pll_clk = 320;
138     } else if (3 == (0x03 & pll_clock)) {
139         t_pll_clk = 480;
140     }
141     u32 a_div = pll_div & 0x3;
142     a_div = a_div * 2 + 1;
143     u32 b_div = (pll_div >> 2) & 0x3;
144     if (0 == b_div) {
145         b_div = 1;
146     } else if (1 == b_div) {
147         b_div = 2;
148     } else if (2 == b_div) {
149         b_div = 4;
150     } else if (3 == b_div) {
151         b_div = 8;
152     }
153     u32 div = a_div * b_div;
154     sfc_clk = t_pll_clk / div;
155     u32 baud = 0;
156     while ((sfc_clk / (baud + 1)) > 60) {
157         baud++;
158     }
159     sfc_clk *= 1000000;
160     /* log_info(" PLL_SYS_CLK %d; SFC BAUD %d\n", sfc_clk, baud); */
161     return baud;
162 }
163

```

AD14N / AC104N SPI 时钟限制在60MHz以内

app/bsp/cpu/sh54/clock.c

AD14N / AC104N SPI 时钟限制

8.2.2 AD15N SPI 时钟限制

```

121 AT(.ram_code)
122 static u32 sfc_max_baud(u32 pll_clock, _PLL_DIV pll_div)
123 {
124     u32 t_pll_clk = 0;
125     /* log_info("pll set, pll_clock %d, pll_div %d \n", pll_clock, pll_div); */
126     if (0 == (3 & pll_clock)) {
127         if (0b0100 == (0x0c & pll_clock)) {
128             t_pll_clk = 107;
129         } else if (0b1000 == (0x0c & pll_clock)) {
130             t_pll_clk = 137;
131         } else if (0b1100 == (0x0c & pll_clock)) {
132             t_pll_clk = 192;
133         }
134     } else if (1 == (0x03 & pll_clock)) {
135         t_pll_clk = 96;
136     } else if (2 == (0x03 & pll_clock)) {
137         t_pll_clk = 320;
138     } else if (3 == (0x03 & pll_clock)) {
139         t_pll_clk = 480;
140     }
141     u32 a_div = pll_div & 0x3;
142     a_div = a_div * 2 + 1;
143     u32 b_div = (pll_div >> 2) & 0x3;
144     if (0 == b_div) {
145         b_div = 1;
146     } else if (1 == b_div) {
147         b_div = 2;
148     } else if (2 == b_div) {
149         b_div = 4;
150     } else if (3 == b_div) {
151         b_div = 8;
152     }
153     u32 div = a_div * b_div;
154     sfc_clk = t_pll_clk / div;
155     u32 baud = 0;
156     while ((sfc_clk / (baud + 1)) > 50) {
157         baud++;
158     }
159     sfc_clk *= 1000000;
160     /* log_info(" PLL_SYS_CLK %dMhz; SFC BAUD %d\n", pll_sys_clk, baud); */
161     return baud;
162 }
163
164

```

AD15N SPI时钟
限制在50MHz以内

app/bsp/cpu/sh55/clock.c

AD15N SPI 时钟限制

8.2.3 AD17N SPI 时钟限制

```

1 #include "config.h"
2 #include "hwi.h"
3 #include "clock.h"
4 #include "math.h"
5 #include "common.h"
6 #include "gpio.h"
7 #include "asm/power_interface.h"
8
9
10 #define LOG_TAG_CONST    NORM
11 #define LOG_TAG          "[clock]"
12 #include "log.h"
13 void sfc_suspend(u32 enable_spi);
14 void sfc_resume(u32 disable_spi);
15
16 static u32 sys_clock = 48000000;
17 static u32 sfc_clk;
18
19 #define SPI_TSHSL    40
20 #define LSB_MAX_CLK  (80 * MHz_UNIT)
21 #define SPI_MAX_CLK  (50 * MHz_UNIT)
22
23 #define RANGE_MAX    3

```

AD17N spi时钟限制在50MHz内

app/bsp/cpu/sh57/clock.c

AD17N SPI 时钟限制

8.2.4 AD24N SPI 时钟限制

```

124 void clk_out2_close(u8 gpio);
125
126 void delay(u32 cnt);
127 void udelay(u32 us);
128 void mdelay(u32 ms);
129 void rc_udelay(u32 us);
130 u32 get_sys_us_cnt(void);
131 void sys_pll_ldo_trim_check();
132
133 #define PLL_192M_LIMIT 192000000
134 #define PLL_240M_LIMIT 240000000
135 #define PLL_MAX_LIMIT PLL_192M_LIMIT
136
137 #define SPI_MAX_CLK (64 * MHz_UNIT)
138
139 #define FUSB_TRIM_CON0    JL_SYSPLL->TRIM_CON0
140 #define FUSB_TRIM_CON1    JL_SYSPLL->TRIM_CON1
141 #define FUSB_TRIM_PND     JL_SYSPLL->TRIM_PND
142 #define FUSB_FRQ_CNT      JL_SYSPLL->FRQ_CNT

```

AD24N SPI时钟限制在64MHz内

include_lib/cpu/sh58/clock.h

8.3 时钟相关函数

8.3.1 函数 void pll_sel(u32 pll_clock, _PLL_DIV pll_div, _PLL_B_DIV pll_b_div)

此函数用于 AD14/15/17 以及 AC104，实现动态配置系统时钟源和分频，系统时钟配置关系如下：

$\text{sys_clk} = \text{pll_clock} / \text{pll_div} / \text{pll_b_div}$;

注：选择系统时钟时，优先调整二级分频 *pll_b_div*，无法满足所需要的时钟下再调整一级分频 *pll_div*！

AD17N 的二级分频参数命名为 *hsb_div*，枚举类型为 *_HSB_CLK_DIV*，效果与其他芯片一致！

其中参数：

1. *pll_clock*: PLL 时钟源
 - ①、PLL_96M;
 - ①、PLL_107M;
 - ①、PLL_137M;
 - ①、PLL_192M;
 - ①、PLL_320M;
 - ①、PLL_480M;
2. *pll_div*: PLL 时钟一级分频
 - ①、可选 PLL_DIV1~PLL_DIV56，具体请参考 *_PLL_DIV* 枚举；
3. *pll_b_div*: PLL 时钟二级分频
 - ①、可选 PLL_B_DIV1~PLL_B_DIV8，具体请参考 *_pll_b_div* 枚举；

8.3.2 函数 int clk_get(const char *name);

此函数用于实现获取各时钟域的频率，具体参数支持” sys”、” hsb”、” lsb”。

8.3.3 函数 int clk_set(const char *name, int clk)

此函数用于 AD18N、AD24N，AD23N 实现动态配置系统时钟源和分频

其中参数：

1. *name*: 时钟名，可配置以下时钟
 - ①、*sys*: 系统时钟，即 *hsb* 时钟；
 - ①、*lsb*: 低速时钟；
 - ①、*sfc*: *sfc* 时钟；
2. *clk*: 时钟设置目标频率

8.3.4 函数 `int clk_early_init(enum pll_ref_source pll_ref, u32 ref_frequency, u32 pll_frequency);`

此函数用于 AD24N/23N 初始化时钟，main 函数固定要使用，如果需要修改相关流程和参数需要联系开发人员。

其中参数：::

1. `pll_frequency`; PLL 时钟频率。例：AD24 如果需要将系统时钟运行至 240M，则需要先将 `pll_frequency` 设置成 240M，然后再通过 `clk_set` 将系统时钟调整至 240M

异常

芯片在运行代码时，由于软件 / 硬件状态出错，当该错误状态未在硬件 / 软件程序设计覆盖的容错范围时，就会引起系统处于未知状态的异常。

在触发异常后，会有两种后果：

- 一、系统复位；
- 二、进入系统异常中断函数并死循环住；

备注：用户可以通过配置 `config_asser` 来选择触发异常后是系统复位还是进入系统异常中断函数卡死。

```

37 const int IRQ_AES_IP          = 3;    //aes
38
39 //内存管理malloc内部断言
40 const char MM_ASSERT          = TRUE;
41 //内核异常打印
42 const u8 config_asser = 1;
43
44 const u8 config_spi_code_user_cache = 1; //sfc放code区
45
46 const u8 config_audio_adc_enable = 1;
47
48 #if (DECODER_MIDI_EN || DECODER_MIDI_KEYBOARD_EN)
49 //midi主轨选择方式
50 const int MAINTRACK_USE_CHN    = 0;    //0:用track号来区分 1:用channel号来区分。
51 const int MAX_DEC_PLAYER_CNT  = 8;    //midi乐谱解码最大同时发声的key数,范围[1,31]
52 const int MAX_CTR_PLAYER_CNT  = 8;    //midi琴最大同时发声的key数,范围[1,31]
53 const int NOTE_OFF_TRIGGER    = 0;    //midi琴note off回调 1: time传0时, 不会回调 0:
54 const int MIDI_MAX_MARK_CNT   = 0;    //midi解码最大支持的mark数
55 #endif
56
57 #if RTC_EN

```

0: 系统复位

1: 进入异常中断函数卡死

玩具系列芯片统一

`app_config.c`

系统异常触发后果选择

9.1 异常分析

用户在触发异常后，可以观察异常信息里的 `reti` 和 `rets` 的地址，在下载目录里的 `sdk.lst` 文件查看对应地址的代码是出现在那里。

下面例子为 SDK 操作了一个指向空白地址的指针给他赋值导致的 `cpu` 写空白地址异常。（其他异常分析均可类似分析）

1. 分析异常信息

```

[09:03:56.673] [Info] [update]update module init ok
[09:03:56.673] [Info] [PMU]trim_flag:0, adp_trim_flag:0, vm_rd_err:0,
trim_err:0, vm_wr_err:0
[Info] [PMU]wvdd_level: 3, wvdd_level_now: 3
[Info] [PMU]except_cnt: 0
E
[09:03:56.673] X
[09:03:56.673] C
[09:03:56.673] <Error> [EMU]
[09:03:56.673] -----Err: function:exception_analyze() @line:625
[09:03:56.673] <Error> [EMU]_EMU_CON : 0xc000001f
<Error> [EMU]_EMU_MSG : 0x80000000
<Error> [EMU]CPU0 4023892 --> 4024258 --> 4012df4 --> 402136a
[09:03:56.673] <Error> [EMU]usp : 0x3f005b0
<Error> [EMU]ssp : 0x3f01f40
<Error> [EMU]sp : 0x3f02a58
<Error> [EMU]Stack Top: 0x3f02ae0
<Error> [EMU]reti : 0x4021380
<Error> [EMU]rets : 0x4012df4
<Error> [EMU]rete : 0x0
<Error> [EMU]psr : 0x0
<Error> [EMU]icfg : 0x7010a80
<Error> [EMU]--reti---
7F BC FA E1 2F 41 F8 E1 EF C7 00 EA 04 70 80 EA
[09:03:56.673] 30 00 91 80 01 78 01 EA 03 70 81 EA 30 00 A2 81
[09:03:56.673] 0A 78 02 EA 02 70 82 EA 30 00 B3 82 13 78 03 EA
[09:03:56.673] 01 70 83 EA 30 00 C6 83 1E 78 09 58 12 58 41 F6
[09:03:56.673] <Error> [EMU]--rets---
49 57 82 EA 00 04 28 C6 31 C6 08 E1 30 A2 7B 5C
[09:03:56.686] 02 EA 6C 57 82 EA 00 04 28 C6 31 C6 08 E1 A0 A1
[09:03:56.686] 10 CD 19 C0 60 80 FF E1 CF 7A 98 20 00 81 FF E1
[09:03:56.686] 8F 7A 58 20 08 81 FF E1 4F 7A 00 98 5F 10 00 EA
[09:03:56.686] <Error> [EMU]-- sp ---
04 70 30 00 03 70 30 00 34 00 00 00 01 00 00 00
[09:03:56.686] 57 05 00 04 5E 05 00 04 C8 2A F0 03 CC 47 05 04
[09:03:56.686] 14 42 ED 03 0C 02 EE 03 04 05 EE 03 1C 04 EE 03
[09:03:56.686] 14 04 EE 03 40 33 F0 03 B0 22 F0 03 80 13 02 04
[09:03:56.686] 00 00 00 00 F4 2D 01 04 00 00 00 00 40 1F F0 03
[09:03:56.686] 80 0A 01 07 B0 05 F0 03 DF 05 00 04 26 1C F0 03
[09:03:56.686] 1E 2E 04 04 00 00 00 00 00 00 00 00 00 00 00
[09:03:56.686] DF FF 00 00 FF FF 00 00 F0 FF 00 00 03 00 00 00
[09:03:56.686] 00 B0 71 0B 49 A2 2C 08
[09:03:56.686]
[09:03:56.686] <Error> [EMU] [0-CPU] emu err msg : sys exception
<Error> [EMU]JL_CEMU->MSG0 : 0x20000000
<Error> [EMU]JL_CEMU->MSG1 : 0x0
<Error> [EMU]JL_CEMU->MSG2 : 0x0
<Error> [EMU] [1-HCORE] hmem err msg : cpu write hmem exception
[09:03:56.695] <Error> [EMU]x0 = 0x30000000
  
```

1.cpu运行地址

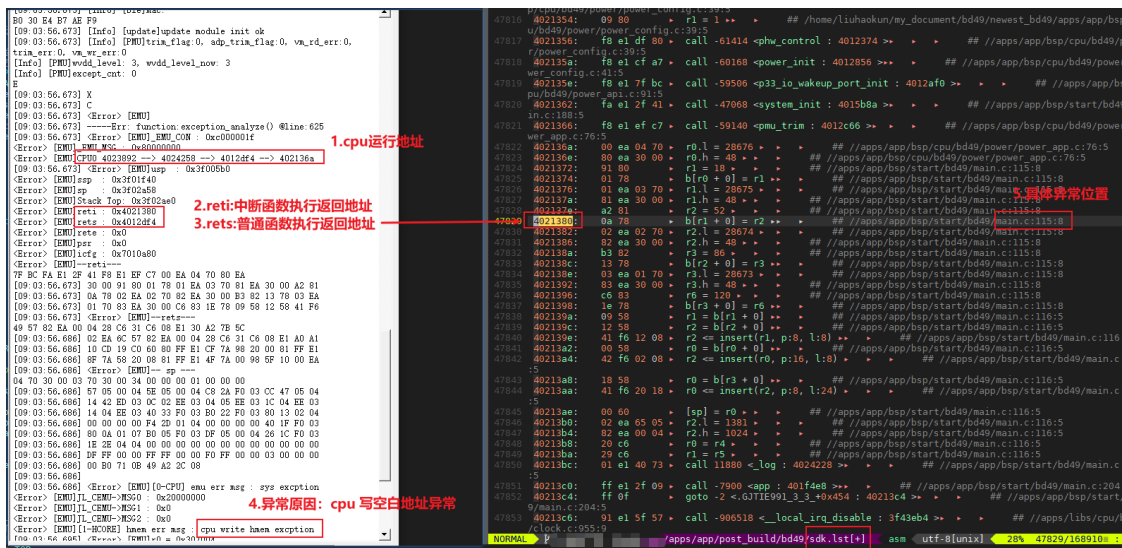
2.reti:中断函数执行返回地址

3.rets:普通函数执行返回地址

4.异常原因: cpu 写空白地址异常

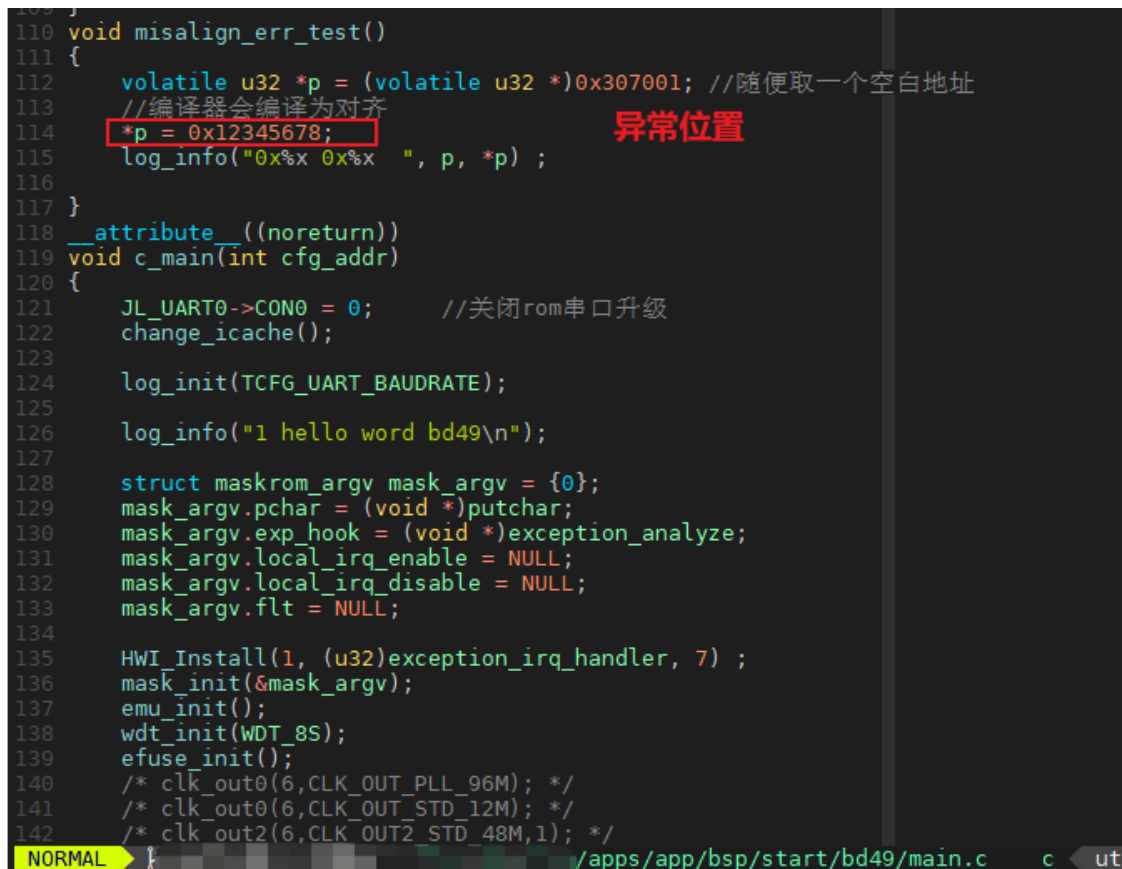
异常信息

2. 在下载目录里的 `sdk.lst` 文件查找 `rets` 和 `reti` 的位置（若分析该两处地址分析不出具体异常原因，可查看异常信息里的 `cpu` 的运行地址来反推异常位置）



异常信息在 sdk.lst 位置

3. 追踪到 SDK 具体异常位置。SDK 操作了一个指向空白地址的指针给他赋值导致的 cpu 写空白地址异常。



异常位置

备注：有一些异常信息往往不在具体异常代码那一行，而是在附近

9.2 常见异常

常见异常信息有：

1. pc_limit: 程序跑飞，指 CPU 的 PC 指针跑到了不允许操作的地址；
2. misalign_err: 非对齐访问，指 CPU 访问内存数据时，寻址指针要求地址 4 字节对齐访问；
3. cpu read / write hmem exception : cpu 读 / 写空白地址访问异常，指 CPU 读/写了内存里的空白地址；
4. watchdog time out : 看门狗异常，指系统未在看门狗规定范围内进行清狗操作。

更多异常信息和 demo 例子可查看下方链接的《系统异常信息》部分：

[系统异常分析](#)

tick_timer 是一个简单的定时器，在该 tick_timer 初始化后，会以 2ms 的间隔起来一次中断，优先级为 IRQ_TTICKTMR_IP.

具体 tick_timer 中断如下：

```
SET(interrupt(""))
void tick_timer_isr()
{
    /* 用户的timer函数不能加入到这里,加到tick_timer_loop */
    j32CPU(core_num())->TTMR_CON |= BIT(6);

    tick_timer_ram_loop();
    if (tick_timer_close) {
        if (tick_timer_close()) {
            return ;
        }
    }

    tick_timer_loop();
}
```

在该中断里面主要分为两部分：

- 1、tick_timer_ram_loop()：该函数里面需要全部放入 ram 中执行；
- 2、tick_timer_loop()：该函数用户正常处理；

设计原因为（不包括 AD23N）：

相关底层驱动在操作 flash 时会关闭总中断，又想在这段时间里执行 tick_timer 中断

那么就需要满足两个条件：

- 1、关总中断时额外放开 tick_timer 中断不受影响。

2、执行的代码都放在 ram 中；（避免关闭了 flash，中断又需要访问 flash 来执行代码导致卡死问题）

那么在系统底层驱动操作 flash 时，虽然关闭了总中断，但是放开了 **tick_timer** 中断，**tick_timer** 中断运行了 **tick_timer_ram_loop()** 部分后，**tick_timer_close()** 判断为真而不执行后面的 **tick_timer_loop()**；

这样用户可以 flash 关闭期间仍旧执行放在 tick_timer 的代码操作。

10.1 注意事项

1. tick_timer 实际是一个定时器触发中断的行为，所以用户在 tick_timer 函数里不要执行过长时间以免中断执行过久。
2. tick_timer 中断的默认优先级为 3，存在会被优先级比他高的中断打断或者被同级优先级的中断延迟起中断时间的问题。

内置系统 flash 写保护

本说明适用于开源 SDK，主要介绍如何将内置 flash 的代码与资源区域设置为写保护状态，包括以下几点：

- 对于 A0 芯片，介绍用户如何根据 flash 手册添加写保护参数；
- 对于内封 flash 的芯片，如何使用烧写器自动获取对应 flash 型号的写保护参数；
- 介绍程序中如何打开与关闭内置 flash 写保护，介绍写保护相关接口；

支持的 SDK 版本

芯片	支持的 SDK 版本
AD14N	ad140-release_v1.4.0 开源 sdk_v1.6.0 以及之后的 SDK
AC104N	ac104-release_v1.4.0 开源 sdk_v1.6.0 以及之后的 SDK
AD15N	ad150-release_v1.4.0 开源 sdk_v1.6.0 以及之后的 SDK
AD17N	开源 sdk_v1.7.0 以及之后的 SDK
AD18N	开源 sdk_v1.8.0 以及之后的 SDK

本章节分为以下几个部分进行介绍说明：

1. “A0 芯片如何添加写保护参数”
 2. “第一点相关函数说明”
 3. “内封 flash 芯片如何添加写保护参数（旧）”
 4. “第三点相关函数说明”
 5. “内封 flash 芯片如何添加写保护参数（新）”
 6. “第五点相关函数说明”
 7. “常见问题说明”
-

11.1 A0 芯片如何添加写保护参数

11.1.1 flash 写保护工具说明

SDK 的下载目录（路径：app\post_build\sh5x\voice_toy(工程)\）中存放有 “flash_write_protect” 文件夹，文件夹中包含以下文件：

- flash_wp_info.csv：该文件以文本形式记录各种 flash 的写保护参数
- csv2dir.bat：该脚本实现将上述 csv 文件转成小机可识别的 dir_sys_info 文件

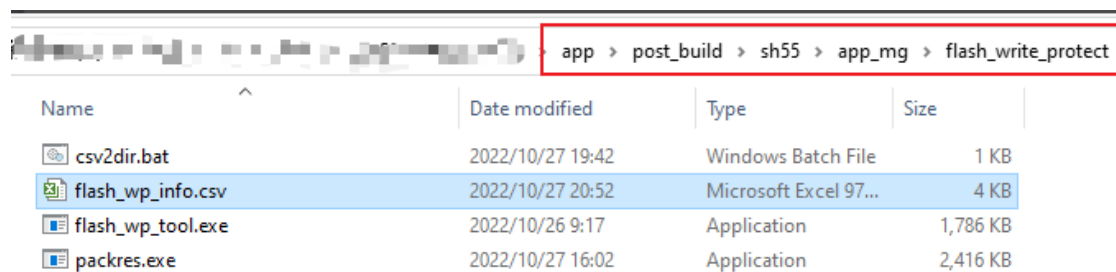


图 2.1 flash 写保护工具

其中，flash_wp_info.csv 文件中涵盖了内封 flash 的绝大部分型号。对于 A0 芯片，用户可自行根据使用的 flash 型号 datasheet 添加相应的写保护参数以及指令。

添加完成后，点击 csv2dir.bat 脚本，即可生成 dir_sys_info 文件，将该文件放在下载目录中，添加到 download.bat 的资源文件中下载到小机中，小机即可识别并添加写保护。

运行 csv2dir.bat 脚本时，需要注意是否有执行成功。

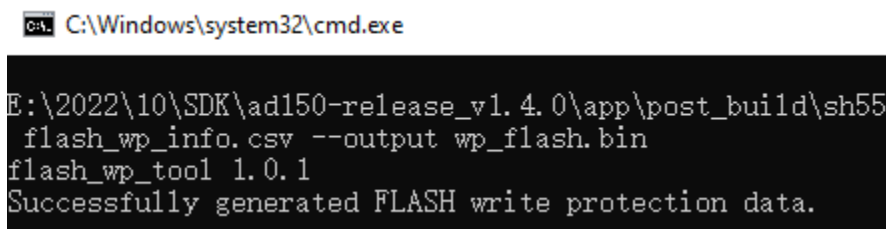


图 2.2 csv2dir.bat 脚本执行成功显示信息

没有执行成功时，需要关注错误信息，重新修改 flash_wp_info.csv 文件；下图的错误信息显示同时存在两组相同的 flash 写保护参数。

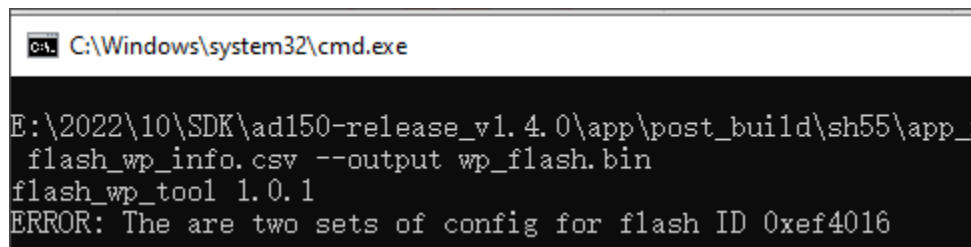


图 2.3 csv2dir.bat 脚本执行失败显示信息

11.1.2 新增 flash 写保护参数与指令

如下对于 A0 的芯片，用户可自行根据需要使用的 flash 的 datasheet，在 flash_wp_info.csv 文件中添加写保护参数以及相关指令，下文以华邦 W25Q32 FLASH 为例，介绍根据手册添加参数。

Flash SR1、SR2 寄存器和写保护范围

在 flash datasheet 中，可通过查找关键字“BP1”快速找到 SR1 寄存器，查找关键字“LB1”快速找到 SR2 寄存器；

检查 SR1 寄存器中是否有 SEC、TB、BP2、BP1、BP0 位，检查 SR2 寄存器是否有 CMP 位；

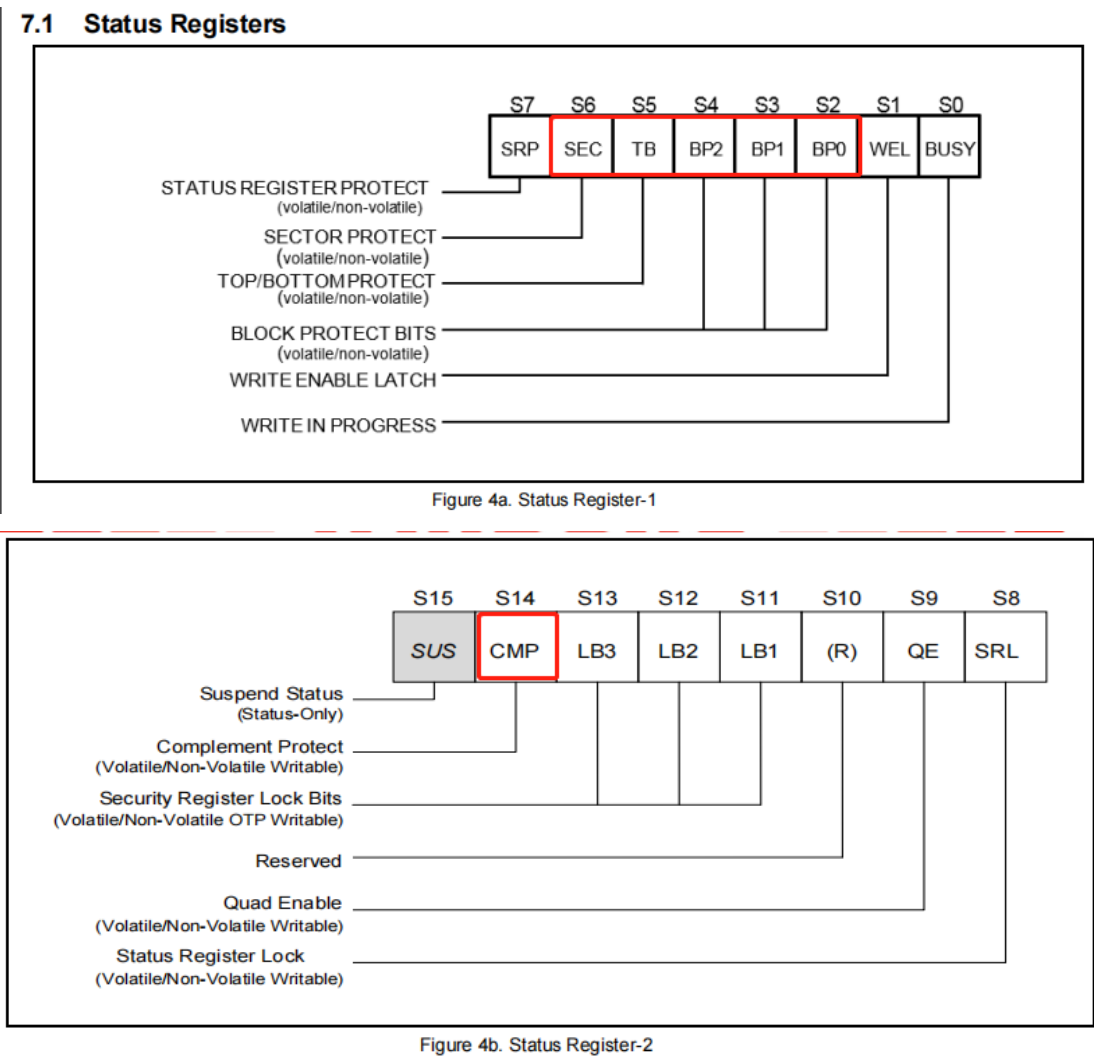


图 2.4 flash SR1 与 SR2 寄存器

通过查找关键字“Density”快速搜索写保护列表，并查看 flash 支持的写保护范围。

7.1.8 Status Register Memory Protection (WPS = 0, **CMP = 0**)

STATUS REGISTER ⁽¹⁾					W25Q32JV (32M-BIT) MEMORY PROTECTION ⁽³⁾			
SEC	TB	BP2	BP1	BP0	PROTECTED BLOCK(S)	PROTECTED ADDRESSES	PROTECTED DENSITY	PROTECTED PORTION ⁽²⁾
X	X	0	0	0	NONE	NONE	NONE	NONE
0	0	0	0	1	63	3F0000h – 3FFFFFFh	64KB	Upper 1/64
0	0	0	1	0	62 and 63	3E0000h – 3FFFFFFh	128KB	Upper 1/32
0	0	0	1	1	60 thru 63	3C0000h – 3FFFFFFh	256KB	Upper 1/16
0	0	1	0	0	56 thru 63	380000h – 3FFFFFFh	512KB	Upper 1/8
0	0	1	0	1	48 thru 63	300000h – 3FFFFFFh	1MB	Upper 1/4
0	0	1	1	0	32 thru 63	200000h – 3FFFFFFh	2MB	Upper 1/2

7.1.9 Status Register Memory Protection (WPS = 0, **CMP = 1**)

STATUS REGISTER ⁽¹⁾					W25Q32JV (32M-BIT) MEMORY PROTECTION ⁽³⁾			
SEC	TB	BP2	BP1	BP0	PROTECTED BLOCK(S)	PROTECTED ADDRESSES	PROTECTED DENSITY	PROTECTED PORTION ⁽²⁾
X	X	0	0	0	0 thru 63	000000h – 3FFFFFFh	4MB	ALL
0	0	0	0	1	0 thru 62	000000h – 3EFFFFh	4,032KB	Lower 63/64
0	0	0	1	0	0 and 61	000000h – 3DFFFFh	3,968KB	Lower 31/32
0	0	0	1	1	0 thru 59	000000h – 3BFFFFh	3,840KB	Lower 15/16
0	0	1	0	0	0 thru 55	000000h – 37FFFFh	3,584KB	Lower 7/8
0	0	1	0	1	0 thru 47	000000h – 2FFFFFFh	3MB	Lower 3/4
0	0	1	1	0	0 thru 31	000000h – 1FFFFFFh	2MB	Lower 1/2

图 2.5 flash 写保护范围

如何添加 flash 写保护参数

打开 flash_wp_info.csv 文件并根据 3.1.1 中的信息填写参数，我司配置 flash 写保护范围从 0 地址开始，向后覆盖。

表格找到上一组 flash 数据空两至三行后，在第一行填写 flash 标号、flash id 以及写使能（Write Enable）指令。

	A	B	C	D	E	F	G	H
1	addr	cmp	BP4	BP3	BP2	BP1	BP0	
222	flash标号	id	写使能命令					
223	W25Q32	0xef4016	06H					
224	128	0	0					
225	256	0	0					
226	512	0	0					
227	1024	0	0					
228	2048	0	0					
229	3072	1	0	0	1	0	1	
230	3584	1	0	0	1	0	0	
231	3968	1	0	0	0	1	0	
232	4032	1	0	0	0	0	1	
233	RDSR1	05H						
234	RDSR2	35H						
235	WRSR	01H	sr1	sr2				
236								
237								

flash 标号、ID 以及写使能指令信息

第二行开始填写 flash 写保护参数，最大支持 9 组 flash 写保护参数。

通过保护范围的大小，设置 SEC、TB、BP0、BP1、BP2（不同的 flash SEC 和 TB 标识可能是其它，如 BP4、BP3）和 CMP 的值。

例如：当 CMP = 0 时，SEC TB BP2 BP1 BP0 为 0b01101，可保护 000000h - 0ffffh 范围内 1MB 的数据。

	A	B	C	D	E	F	G
1	addr	cmp	BP4	BP3	BP2	BP1	BP0
222							
223	W25Q32	0xef4016	06H				
224	128	0	0	1	0	1	0
225	256	0	0	1	0	1	1
226	512	0	0	1	1	0	0
227	1024	0	0	1	1	0	1
228	2048	0	0	1	1	1	0
229	3072	1	0	0	1	0	1
230	3584	1	0	0	1	0	0
231	3968	1	0	0	1	0	0
232	4032	1	0	0	1	0	0
233	RDSR1						
234	RDSR2						
235	WRSR						
236							
237							
238							
239							
240							
241							
242							
243							

图 2.7 flash 写保护范围填写说明 1

当 CMP = 1 时，SEC TB BP2 BP1 BP0 为 0b00001，可保护 000000h - 3effffh 范围内 4032KB 的数据。

	A	B	C	D	E	F	G	I
1	addr	cmp	BP4	BP3	BP2	BP1	BP0	
222								
223	W25Q32	0xef4016	06H					
224	128	0	0	1	0	1	0	
225	256	0	0	1	0	1	1	
226	512	0	0	1	1	0	0	
227	1024	0	0	1	1	0	1	
228	2048	0	0	1	1	1	0	
229	3072	1	0	0	1	0	1	
230	3584	1	0	0	1	0	0	
231	3968	1	0	0	0	1	0	
232	4032	1	0	0	0	0	1	
233	RDSR1	05H	7.1.9 Status Register Memory Protection (WPS = 0, CMP = 1)					
234	RDSR2	35H						
235	WRSR	01H						
236								
237								
238								
239								
240								
241								
242								
243								
244								

STATUS REGISTER ⁽¹⁾					W25Q32JV (32M-BIT) MEMORY PROTECTION ⁽²⁾			
SEC	TB	BP2	BP1	BP0	PROTECTED BLOCK(S)	PROTECTED ADDRESSES	PROTECTED DENSITY	PROTECTED PORTION ⁽²⁾
X	X	0	0	0	0 thru 63	000000h – 3FFFFFFh	4MB	ALL
0	0	0	0	1	0 thru 62	000000h – 3EFFFFh	4,032KB	Lower 63/64
0	0	0	1	0	0 and 61	000000h – 3DFFFFh	3,968KB	Lower 31/32
0	0	0	1	1	0 thru 59	000000h – 3BFFFFh	3,840KB	Lower 15/16
0	0	1	0	0	0 thru 55	000000h – 37FFFFh	3,584KB	Lower 7/8
0	0	1	0	1	0 thru 47	000000h – 2FFFFFFh	3MB	Lower 3/4
0	0	1	1	0	0 thru 31	000000h – 1FFFFFFh	2MB	Lower 1/2

图 2.8 flash 写保护范围填写说明 2

flash 读写 SR1、SR2 指令

Flash 配置写保护时，需要读写 SR1、SR2 寄存器。查找手册找到读写 SR1、SR2 的指令，并填入表格中。

如果 datasheet 上有 31H 命令，优先使用分开写 SR1、SR2 模式；

addr	cmp	BP4	BP3	BP2	BP1	BP0		
W25Q32	0xef4016	06H						
128	0	0	1	0	1	0		
256	0	0	1	0	1	1		
512	0	0	1	1	0	0		
1024	0	0						
2048	0	0						
3072	1	0						
3584	1	0						
3968	1	0						
4032	1	0						
RDSR1	05H							
RDSR2	35H							
WRSR	01H	sr1						

8.2.4 Read Status Register-1 (05h) Status Register-2 (35h) & Status Register-3 (15h)
The Read Status Register instructions allow the 8-bit Status Registers to be read. The instruction by driving /CS low and shifting the instruction code "05h" for Status Register-1, "35h" for Status Register-2 or "15h" for Status Register-3 into the DI pin on the rising edge of CLK. The status register bit shifted out on the DO pin at the falling edge of CLK with most significant bit (MSB) first as shown in Figure 8. Refer to section 7.1 for Status Register descriptions.

8.2.5 Write Status Register-1 (01h) Status Register-2 (31h) & Status Register-3 (11h)
The Write Status Register instruction allows the Status Registers to be written. The writable Status Register bits include: SEC, TB, BP[2:0] in Status Register-1; CMP, LB[3:1], QE, SRL in Status Register-2; DRV0, WPS in Status Register-3. All other Status Register bit locations are read-only and are not affected by the Write Status Register instruction. LB[3:1] are non-volatile OTP bits, once it is programmed, it cannot be cleared to 0.

SR1 SR2寄存器分开写模式

图 2.9 flash 分开写 SR1 和 SR2 寄存器指令

若上述分开写 SR1、SR2 模式在芯片运行中添加写保护不成功，则使用 01H 指令连续写 SR1、SR2 模式：

	A	B	C	D	E	F	G	H
1	addr	cmp	BP4	BP3	BP2	BP1	BP0	
222								
223	W25Q32	0xef4016	06H					
224	128	0	0	1	0	1	0	
225	256	0	0	1	0	1	1	
226	512	0	0	1	1	0	0	
227	1024	0	0	1	1	0	1	
228	2048	0						
229	3072	1						
230	3584	1						
231	3968	1						
232	4032	1						
233	RDSR1	05H						
234	RDSR2	35H						
235	WRSR	01H	sr1	sr2				
236								
237								
238								
239								
240								
241								
242								

8.2.4 Read Status Register-1 (05h), Status Register-2 (35h)
The Read Status Register instructions allow the 8-bit Status Registers to be read. The instruction by driving /CS low and shifting the instruction code "05h" for Status Register-1, "35h" for Status Register-2 or "15h" for Status Register-3 into the DI pin on the rising edge of CLK. The status register bit shifted out on the DO pin at the falling edge of CLK with most significant bit (MSB) first as shown in Figure 8. Refer to section 7.1 for Status Register descriptions.

8.2.5 Write Status Register-1 (01h), Status Register-2 (31h)
The Write Status Register instruction allows the Status Registers to be written. The writable Status Register bits include: SEC, TB, BP[2:0] in Status Register-1; CMP, LB[3:1], QE, SRL in Status Register-2; DRV0, WPS in Status Register-3. All other Status Register bit locations are read-only and are not affected by the Write Status Register instruction. LB[3:1] are non-volatile OTP bits, once it is programmed, it cannot be cleared to 0.

图 2.10 flash 连续写 SR1 和 SR2 寄存器指令

部分 Flash 不存在 SR2 寄存器，则无需写 SR2 相关指令。

	A	B	C	D	E	F	G
1	addr	cmp	BP4	BP3	BP2	BP1	BP0
106							
107	4_ZE ZB25	0x5E3213	06H				
108	256	0	0	0	1	1	0
109	384	0	0	0	1	0	1
110	448	0	0	0	1	0	0
111	480	0	0	0	0	1	1
112	496	0	0	0	0	1	0
113	504	0	0	0	0	0	1
114	RDSR1	05H					
115	WRSR	01H	srl				

图 2.11 flash 只读写 SR1 寄存器指令

11.1.3 写保护参数下载说明

添加好写保护参数后，根据 2.1 的说明执行脚本生成 dir_sys_info 文件，并将其添加到 SDK 资源列表中，小机通过路径搜索找到该文件并进行解析。

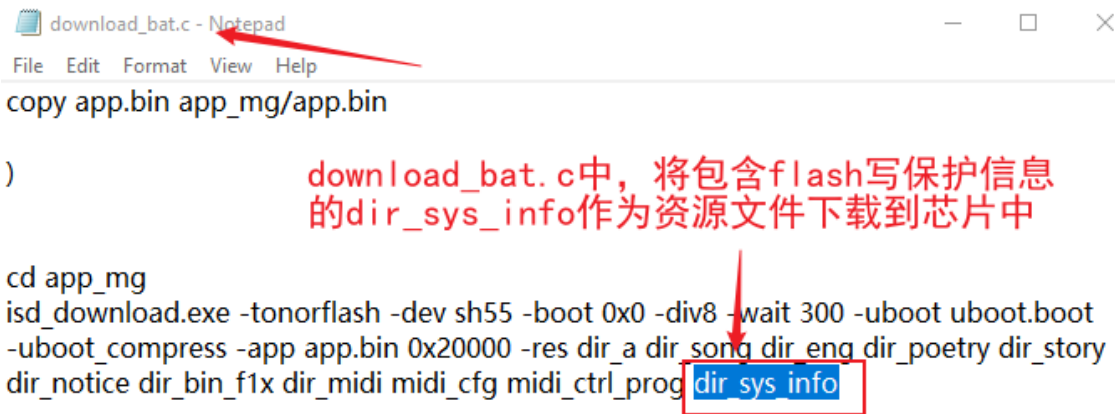


图 2.12 download_bat.c 中将 dir_sys_info 添加到资源列表中

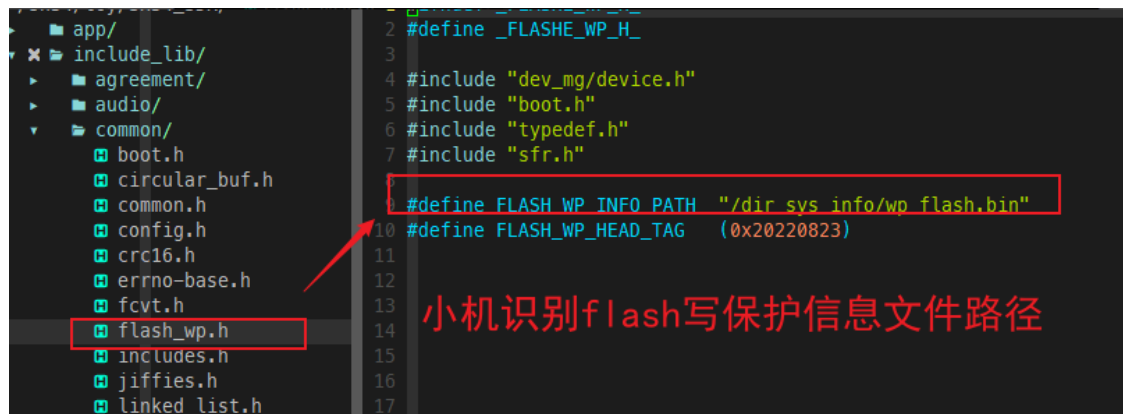


图 2.13 小机识别 flash 写保护信息文件路径

11.1.4 第一点相关函数说明

函数 `struct flash_wp_arg *find_flash_wp_file_arg(void)`

该函数实现解析 `dir_sys_info` 中的 `wp_flash.bin` 文件，逐个查找与正在运行的 flash 匹配的写保护参数，其中：

1. 返回值：

- 成功，返回匹配成功的写保护结构体 `flash_wp_arg` 指针；
- 失败，返回 `NULL`；

11.2 内封 flash 芯片如何添加写保护参数（旧）

上文通过表格的方式保存 flash 写保护参数，存在更新不及时的问题。对于杰理非 A0 的芯片，内封的 flash 型号可能会发生变动，不适合用表格形式记录。

我司烧写器可联网更新 flash 写保护参数，烧写器可在烧写阶段将内封 flash 对应的写保护参数记录在 flash 最尾部，由程序去查找该文件并配置写保护。

本文将介绍如何配置 ini 文件实现该功能：

支持旧的内封 flash 写保护策略的芯片：

支持写保护策略的芯片	AD14N / AC104N	AD15N	AD17N
------------	----------------	-------	-------

11.2.1 配置烧写器获取内封 flash 写保护参数

在 SDK 的下载目录（路径：`app\post_build\sh5x\app_mg`（或其他工程）\）中打开 `isd_config.ini` 文件，并在其中添加烧写器 flash 写保护配置项。

‘ [BURNER_PASSTHROUGH_CFG] FLASH_WRITE_PROTECT=YES; ‘

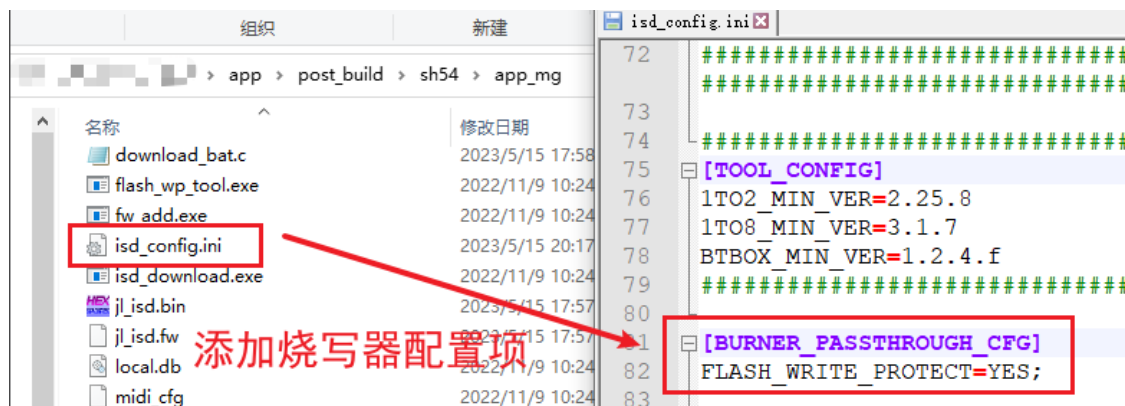
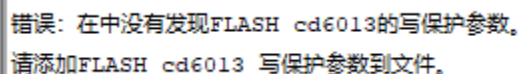


图 3.1 isd_config.ini 文件添加烧写器写保护配置项

11.2.2 配置强制下载工具获取内封 flash 写保护参数

isd_config.ini 文件添加了 FLASH_WRITE_PROTECT=YES 配置后, 使用强制升级工具更新程序会报如下错误:



```
错误: 在中没有发现FLASH cd6013的写保护参数。
请添加FLASH cd6013 写保护参数到文件。
```

图 3.2 强制升级工具报错

解决办法是将 FLASH_WRITE_PROTECT 注释掉或者配置为 NO。

11.2.3 第三点相关函数说明

函数 `int norflash_set_write_protect(u8 enable_write_protect)`

该函数实现开启或关闭内置 flash 写保护功能, 其中参数:

1. Enable_write_protect

- 1: 开启写保护
- 0: 关闭写保护

2. 返回值

- 开启写保护成功: 写保护的最大地址 (即 flash 的 0~ 返回值这块区域保护);
- 关闭写保护成功/开启写保护失败:

失败问题原因分析步骤:

1. 开写保护打印 (在 app_config.c 里 EEPROM 区的打印打开);
2. 检查打印, 如果没有任何打印直接返回 0, 则用户直接跳到步骤 3; 如果只有写保护失败打印, 则跳到步骤 4; 如果打印相关寄存器值出来和失败打印, 则跳到步骤 4
3. 写保护参数没有放进 flash 当中, 用户可以按照文档逐步操作:

“AO 芯片如何添加写保护参数”

“内封 flash 芯片如何添加写保护参数 (旧)”

“内封 flash 芯片如何添加写保护参数 (新)”

4. 确认自己芯片的 flash 是否在 csv 烧写文件里; 确认 app.bin + 资源文件少于对应 flash 的最小保护区域, 用户可以在下载目录里加一些无用的资源文件。
5. 把 flash 手册和打印信息保存联系深圳办或者钉钉开源群联系我们。

详细可以看 “写保护失败解决办法”

11.3 内封 flash 芯片如何添加写保护参数（新）

部分芯片支持新的内封 flash 写保护策略，固件 API 接口没有改变，isd_config.ini、download.bat 配置发生变化，具体请参考文档：[OTP_CFG\(VOTP\) 区域配置说明](#)

支持新的内封 flash 写保护策略的芯片：

支持写保护策略的芯片	AD18N
------------	-------

11.4 程序中开启与关闭写保护功能

SDK 中默认会在内置 flash 初始化的最后，配置 flash 写保护状态。

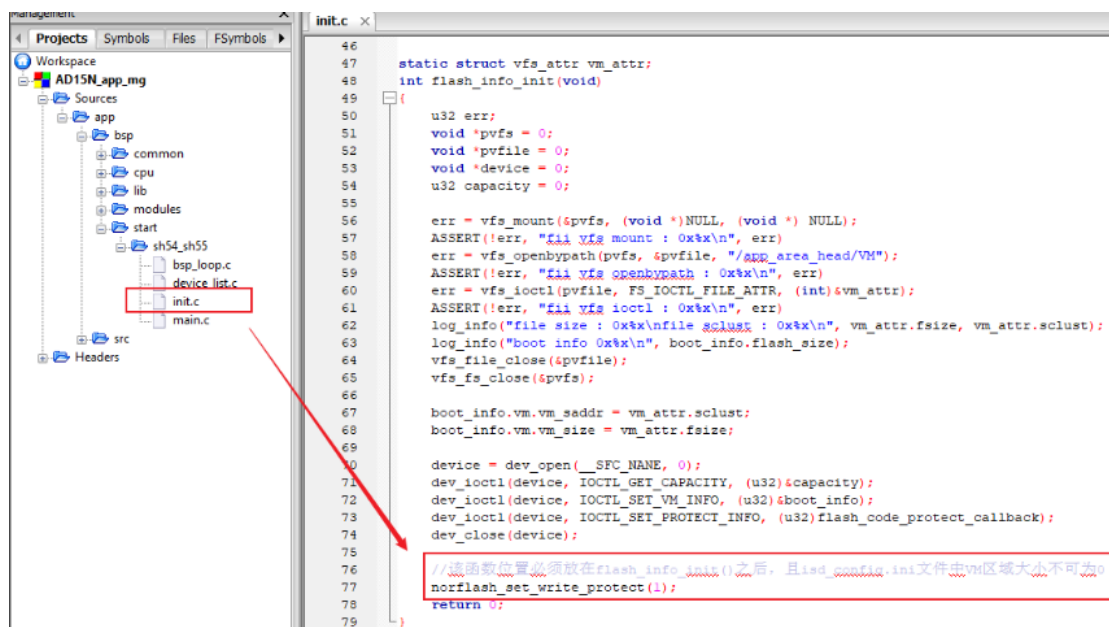


图 4.1 SDK 中添加写保护位置

11.4.1 第五点相关函数说明

函数 `int norflash_set_write_protect(u8 enable_write_protect)`

该函数实现开启或关闭内置 flash 写保护功能，其中参数：

1. Enable_write_protect

- 1: 开启写保护
- 0: 关闭写保护

2. 返回值

- 开启写保护成功：写保护最大地址；
- 关闭写保护成功/开启写保护失败：0

函数 `int norflash_write_protect_config(struct device *device, u32 addr, struct flash_wp_arg *p)`

该函数实现配置 flash 写保护范围，其中参数：

1. device: 设备句柄，传 NULL 即可；
2. addr: 写保护截止范围，传代码资源区后的地址；
3. p: flash 写保护参数；
4. 返回值：
 - 开启写保护成功：写保护最大地址；
 - 关闭写保护成功/开启写保护失败：0

函数 `u16 norflash_read_sr1_sr2(void)`

该函数实现获取内置 flash 的 Status Register 1 和 Status Register 2 寄存器，其返回值高 8 位为 SR2，低 8 位为 SR1；

函数 `u32 flash_code_protect_callback(u32 offset, u32 len)`

该函数实现软件上限制驱动操作内置 flash（存放代码的 flash）。

内置 flash 进行写或擦除操作前会回调该函数，判断会操作到代码与资源区域，则不进行相应操作；

可减低程序跑飞导致程序意外擦写 flash 的概率。其中参数：

1. offset: 设备进行写或擦除操作的地址
2. len: 设备进行写或擦除操作的长度
3. 返回值：
 - 开启写保护成功：写保护最大地址；
 - 关闭写保护成功/开启写保护失败：0

11.4.2 常见问题说明

flash 保护 csv 文件说明

用户需要了解写保护参数文件的一些相关意思：

1. 芯片 flash 型号
2. 芯片 flash_id
3. flash 的最小保护区域

14	WRSR	01H	sr1	sr2					
15									
16	flash型号	flash id							举例
17	8_TL TH25Q-80UA	0xeb6014	06H						
18		128	0	0	1	0	1	0	
19		256	0	0	1	0	1	1	
20		512	0	0	1	1	0	0	
21		768	1	0	0	0	1	1	
22		896	1	0	0	0	1	0	
23		960	1	0	0	0	0	1	
24		992	1	1	0	1	0	0	
25		1008	1	1	0	0	1	1	
26		1016	1	1	0	0	1	0	
27	RDSR1	05H							
28	RDSR2	35H							
29	WRSR	01H	sr1	sr2					
30									

csv 文件简单了解

写保护失败解决办法

写保护失败现象有三：

1. 打印直接返回 0，没有写保护失败打印；
2. 打印返回 0，有写保护失败（set write protect fail）；
3. 打印返回 0，有相关寄存器配置和写保护失败打印（set write protect fail）；

写保护失败原因有三（一一对应上面失败现象）：

1. 写保护参数没有烧写到 flash 当中；

“AO 芯片如何添加写保护参数”

“内封 flash 芯片如何添加写保护参数（旧）”

“内封 flash 芯片如何添加写保护参数（新）”

2. flash 的最小写保护区域大于 app.bin + 资源文件，涉及到了 VM 区域就会写保护失败；” flash 保护 csv 文件说明”
3. 某些 flash 配置特殊，需要找开发人员协助；

AD 系列支持众多硬件中断，拥有 8 级中断优先级，支持中断嵌套；除了硬件中断，还支持软件中断；中断数量如下表所示：

CPU	中断号数量	中断优先级
AD14N / AC104N	32 个（包含 4 个软中断）	0~7，共 8 级
AD15N	32 个（包含 4 个软中断）	0~7，共 8 级
AD17N	32 个（包含 4 个软中断）	0~7，共 8 级
AD18N	64 个（包含 4 个软中断）	0~7，共 8 级
AD24N	48 个（包含 8 个软中断）	0~7，共 8 级
AD23N	128 个（包含 8 个软中断）	0~7，共 8 级

注：中断优先级 0~7，共 8 级，数值越大，优先级越高

12.1 中断的注册与注销

AD 系列注册一个中断，需要中断号、中断响应函数以及中断优先级；

- 中断号：每一个中断会分配一个唯一的中断号，可在 hwi.h 文件中查看；

```

1 #ifndef HWI_H
2 #define HWI_H
3
4
5 #define IRQ_EMUEXCPT_IDX    0
6 #define IRQ_EXCEPTION_IDX  1
7 #define IRQ_SYSCALL_IDX    2
8 #define IRQ_TICKTMR_IDX    3
9 #define IRQ_TIME0_IDX      4
10 #define IRQ_TIME1_IDX      5
11 #define IRQ_TIME2_IDX      6
12 #define IRQ_P33_IDX        8
13 #define IRQ_AUDIO_IDX      9
14
15 #define IRQ_SD0_IDX         12
16 #define IRQ_PORT_IDX       13
17 #define IRQ_IIC_IDX         14
18 #define IRQ_ADC_IDX         15
19 #define IRQ_SPI0_IDX        16
20 #define IRQ_SPI1_IDX        17
21
22 #define IRQ_USB_SOF_IDX     21
23 #define IRQ_USB_CTRL_IDX    22
24 #define IRQ_ALINK0_IDX      25
25
26 #define IRQ_UART0_IDX       10
27 #define IRQ_UART1_IDX       11
28 #define IRQ_MCTMRX_IDX      20
29 #define IRQ_SOFT0_IDX       28
30 #define IRQ_SOFT1_IDX       29
31 #define IRQ_SOFT2_IDX       30
32 #define IRQ_SOFT3_IDX       31
33
34 #define MAX_IRQ_ENTRY_NUM 32
35
36
37 //系统使用到的
38 extern const int IRQ_IRTMR_IP;
39 extern const int IRQ_AUDIO_IP;
40 extern const int IRQ_DECODER_IP;
41 #include lib/cpu/sh54/hwi.h

```

中断号

AD14N 中断号示意图

```

5
6 #define IRQ_EMUEXCPT_IDX 0
7 #define IRQ_EXCEPTION_IDX 1
8 #define IRQ_SYSCALL_IDX 2
9 #define IRQ_TICKTMR_IDX 3
10
11 #define IRQ_SOFT0_IDX 4
12 #define IRQ_SOFT1_IDX 5
13 #define IRQ_SOFT2_IDX 6
14 #define IRQ_SOFT3_IDX 7
15 #define IRQ_SOFT4_IDX 8
16 #define IRQ_SOFT5_IDX 9
17 #define IRQ_SOFT6_IDX 10
18 #define IRQ_SOFT7_IDX 11
19
20 #define IRQ_TIME0_IDX 12
21 #define IRQ_TIME1_IDX 13
22 #define IRQ_TIME2_IDX 14
23 #define IRQ_UART0_IDX 15
24 #define IRQ_UART1_IDX 16
25 #define IRQ_SPI1_IDX 17
26 #define IRQ_SPI2_IDX 18
27 #define IRQ_IIC0_IDX 19
28 #define IRQ_PORT_IDX 20
29 #define IRQ_GPADC_IDX 21
30 #define IRQ_LRCT_IDX 22
31 #define IRQ_GPCNT_IDX 23
32 #define IRQ_SD0_IDX >--- 24
33 #define IRQ_USB_SOF_IDX 25
34 #define IRQ_USB_CTRL_IDX 26
35 #define IRQ_SD0_BRK_IDX 27
36 #define IRQ_MCPWM_TMR_IDX 28
37 #define IRQ_MCPWM_CHX_IDX 29
38
39 #define IRQ_PMU_SOFT0_IDX 30
40 #define IRQ_PMU_SOFT1_IDX 31
41 #define IRQ_PMU_SOFT2_IDX 32
42 #define IRQ_PMU_SOFT3_IDX 33
43
44 #define IRQ_PMU0_IDX 34
45 #define IRQ_PMU1_IDX 35
46 #define IRQ_PMU2_IDX 36
47 #define IRQ_PMU3_IDX 37
48 #define IRQ_PMU4_IDX 38
49
50 #define IRQ_SRC_HW_IDX 41
51 #define IRQ_SPI0_IDX 42
52
53 #define IRQ_ALNK_SOFT 44
54 #define IRQ_APA_SOFT 45
55 #define IRQ_ADC_SOFT 46
56 #define IRQ_DAC_SOFT 47
57
58 #define MAX_IRQ_ENTRY_NUM 48
59
include_lib/cpu/sh58/hwi.h

```

AD24N 中断号示意图

- 中断响应函数：中断响应函数类型必须为无返回值的 void 类型，且中断函数定义时需添加中断标志；

```

17  */
18 #define _timer_init(ch,us) \
19     HWI_Install(IRQ_TIME##ch##_IDX, (u32)timer##ch##_isr, 0); \
20     TIMER_SFR(ch)->PRD = clk_get("lsb")/1000000 * us; \
21     TIMER_SFR(ch)->CON = BIT(0)|BIT(6);
22
23 SET(interrupt(""))
24 static void timer0_isr(void)
25 {
26     TIMER_SFR(0)->CON |= BIT(6);
27     log_char('0');
28     JL_PORTA->DIR &= ~BIT(4);
29     JL_PORTA->DIE |= BIT(4);
30     JL_PORTA->OUT ^= BIT(4);
31 }
32 SET(interrupt(""))
33 static void timer1_isr(void)
34 {
35     TIMER_SFR(1)->CON |= BIT(6);
36     log_char('1');
37 }
38 SET(interrupt(""))
39 static void timer2_isr(void)
40 {
41     TIMER_SFR(2)->CON |= BIT(6);
42     log_char('2');
43 }
44
45 void timer_init(u8 timer_ch, u32 us)
46 {
47     switch (timer_ch) {
48         case 0: timer0_isr(); break;
49         case 1: timer1_isr(); break;
50         case 2: timer2_isr(); break;
51     }
52 }
53
54 #endif
55
56 app/bsp/cpu/sh54/timer_drv.c

```

中断响应函数需要添加标号

中断响应函数添加中断标志示意图

- 中断优先级：中断响应优先级，共 8 级，分别为 0~7，数值越大，优先级越高；

12.1.1 函数 void HWI_Install(unsigned char index, unsigned int isr, unsigned char priority)

该函数实现注册一个中断，其中参数：

- 1、index：需要注册的中断号；
- 2、isr：中断响应函数；
- 3、priority：中断优先级；

12.1.2 函数 void HWI_Uninstall(unsigned char index)

该函数实现注销一个中断，其中参数：

1、index：需要注销的中断号；

12.2 软中断说明

AD 系列支持软件中断；软件中断的注册、注销方式与硬件中断一致；SDK 中可在 soft_isr.c 文件中查看示例：


```

19 SET(interrupt(""))
20 void demo_soft0_isr()
21 {
22     bit_clr_swi(0);
23     si_putchar('0');
24 }
25
26 SET(interrupt(""))
27 void demo_soft1_isr()
28 {
29     bit_clr_swi(1);
30     si_putchar('1');
31 }
32
33
34 SET(interrupt(""))
35 void demo_soft2_isr()
36 {
37     bit_clr_swi(2);
38     si_putchar('2');
39 }
40
41
42 SET(interrupt(""))
43 void demo_soft3_isr()
44 {
45     bit_clr_swi(3);
46     si_putchar('3');
47 }
48
49
50 void demo_soft_init(void)
51 {
52     HWI_Install(IRQ_SOFT0_IDX, (u32)demo_soft0_isr, 0);
53     HWI_Install(IRQ_SOFT1_IDX, (u32)demo_soft1_isr, 0);
54     HWI_Install(IRQ_SOFT2_IDX, (u32)demo_soft2_isr, 0);
55     HWI_Install(IRQ_SOFT3_IDX, (u32)demo_soft3_isr, 0);
56
57     bit_set_swi(0);
58     bit_set_swi(1);
59     bit_set_swi(2);
60     bit_set_swi(3);
61 }
62

```

3、软中断清除pending

1、软中断注册

2、软中断触发

app/bsp/modules/soft_isr/soft_isr.c

软件中断使用示例

12.2.1 函数 void bit_set_swi(unsigned char index)

该函数实现触发一个软件中断，其中参数：

1、index：需要触发的软件中断序号（软中断0~软中断3分别对应0~3）；

注：需要注意此处 *index* 与中断号不同；软中断2的中断号在 *AD14N* 上是 30，需要触发该中断时，该接口传参的 *index* 传 2 即可！

12.2.2 函数 void bit_clr_swi(unsigned char index)

该函数实现清除一个软中断的 Pending，其中参数：

1、index：需要清除pending的软件中断序号（软中断0~软中断3分别对应0~3）；

注：需要注意此处 *index* 与中断号不同；软中断2的中断号在 *AD14N* 上是 30，需要清除该中断 *pending* 时，该接口传参的 *index* 传 2 即可！

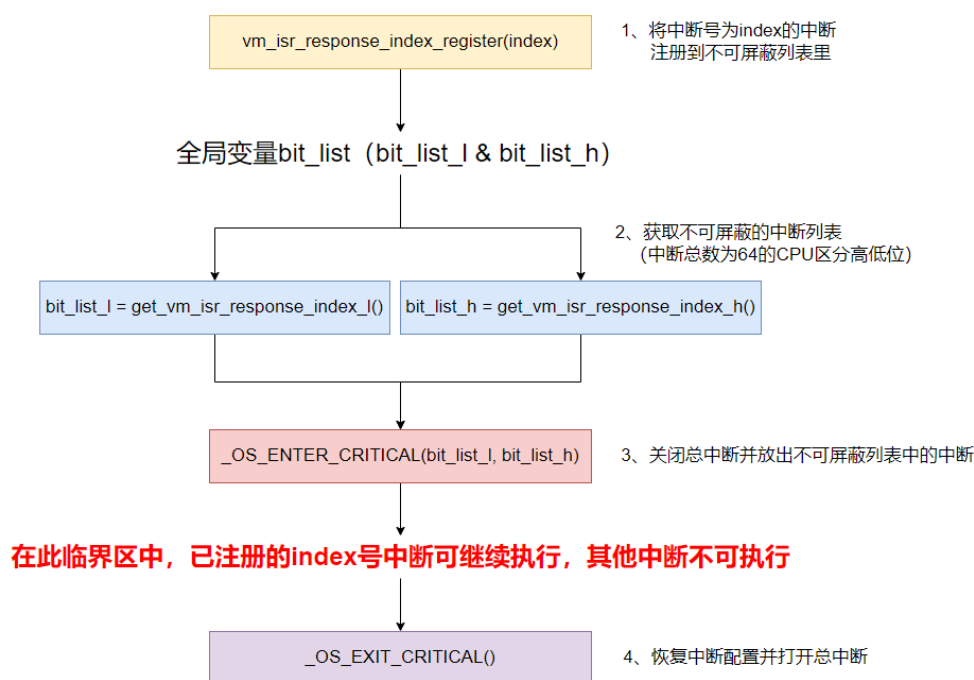
12.3 软件不可屏蔽中断

在进行 VM 擦写时系统会关闭总中断并操作系统 flash，该过程耗时较长，可能会导致播歌卡顿或其他外设中断无法及时响应；

SDK 提供了软件不可屏蔽中断，在关闭总中断的同时允许放出部分中断继续执行（例如放出 audio 中断优化操作 vm 卡音问题），该部分中断函数以及依赖的变量都需要放在 ram 里；

注：AD23N SDK 在读写 flash 时关闭总中断，此时不能使用软件不可屏蔽中断。

本小节介绍软件不可屏蔽中断相关的接口与使用，流程如下：



软件不可屏蔽中断运行流程图

12.3.1 函数 void _OS_ENTER_CRITICAL(u32 bit_list)

该函数适用于中断总数为 32 的芯片 (AD14N / AD15N / AD17N 等)，实现关闭总中断的同时放出 bit_list 参数中置 1 的中断，其中参数：

1、bit_list：需要单独放出的中断list，bit0~bit31分别对应0号~31号中断；

例如 AD14N 需要同时放出 audio（中断号为 9）和 ticktimer 中断（中断号为 3）；那么 bit_list 传入 0x208 即可；

12.3.2 函数 void _OS_ENTER_CRITICAL(u32 bit_list_l, u32 bit_list_h)

该函数适用于中断总数超过 32，小于 64 的芯片 (AD16N / AD18N 等)，实现关闭总中断的同时放出 bit_list_l 和 bit_list_h 参数中置 1 的中断，其中参数：

1、bit_list_l：需要单独放出的中断list的低32位，bit0~bit31分别对应0号~31号中断；
 2、bit_list_h：需要单独放出的中断list的高32位，bit0~bit31分别对应32号~63号中断；

例如 AD18N 需要同时放出音频 APA 中断（中断号为 37）和 ticktimer（中断号为 3），那么 bit_list_h 传入 0x20，bit_list_l 传入 0x8 即可；

12.3.3 函数 void _OS_ENTER_CRITICAL(IRQ_IDX_BITMAP *p_irq_bitmap)

该函数适用于中断号总数超过 64 的芯片 (AD23N)，实现关闭总中断的同时放出 p_irq_bitmap 参数中置 1 的中断，其中参数：

- 1、p_irq_bitmap->irq_0_31：需要单独放出的 0 号~31 号中断，bit0~bit31 分别对应 0 号~31 号中断；
- 2、p_irq_bitmap->irq_32_63：需要单独放出的 32 号~63 号中断，bit0~bit31 分别对应 32 号~63 号中断；

例如 AD23N 需要同时放出 timer2 中断 (中断号为 6) 和 ticktimer (中断号为 3)，那么 p_irq_bitmap->irq_0_31=(BIT(IRQ_TICK_TMR_IDX)|BIT(IRQ_TIME2_IDX)) 即可；

12.3.4 函数 void _OS_EXIT_CRITICAL(void)

该函数实现恢复中断配置并退出关中断状态；

12.3.5 函数 void vm_isr_response_index_register(u8 index)

注：该函数 AD23N 不适用

SDK 中存在一个全局变量用于存储需要放出来的中断 list，该函数实现往该变量中添加中断 bit，其中参数：

- 1、index：需要添加的不屏蔽中断号；例如 AD14N 的 audio 中断号为 9，则传入 9 即可；

12.3.6 函数 void vm_isr_response_index_unregister(u8 index)

注：该函数 AD23N 不适用

该函数实现恢复中断配置并退出关中断状态，其中参数：

- 1、index：删除已添加的不屏蔽中断号；

12.3.7 函数 u32 get_vm_isr_response_index_l(void)

该函数实现获取全局的不可屏蔽中断 list 的低 32 位，其中：

- 1、对于中断总数为 32 的芯片，获取到完整的 bit_list；
- 2、对于中断总数超过 32 且少于 64 的芯片，获取到低 32 位中断的 bit_list；

12.3.8 函数 u32 get_vm_isr_response_index_h(void)

该函数实现获取全局的不可屏蔽中断 list 的高 32 位，仅中断总数超过 32 且少于 64 的芯片有该接口；

12.4 硬件不可屏蔽中断

硬件不可屏蔽中断允许中断在关闭总中断 (即调用了函数 local_irq_disable()) 的情况下继续运行；目前支持硬件不可屏蔽中断的芯片有：AD17N、AD18N、AD24N、AD23N；

注：若需要中断完全不被打断，需要软硬件不可屏蔽中断功能搭配使用；

12.4.1 函数 void irq_unmask_set(u8 index)

该函数实现将传入的 index 中断的优先级设置为最高，并加入硬件不可屏蔽中断列表；其中参数：

1、index：需要添加的不屏蔽中断号；

12.4.2 函数 void irq_unmask_disable(u8 index)

该函数实现关闭硬件不可屏蔽中断功能；

AD14N 与 AD15N 的内核异常管理单元

异常管理单元开启后，会在 MCU 异常时触发中断，并复位或者打印出异常信息并断言；

1. 调用函数 `emu_init` 后异常开始工作。

2. 异常断言

```
1 //内核异常打印,为1时会断言,开发调试时可改为1,量产时必须为0
2 const u8 config_asser = 0;
```

备注:

config_asser = 1 时，异常触发后断言，并在 DEBUG 打印控制开启的情况下会打印出异常信息；

config_asser = 0 时，异常触发后芯片复位。

量产时，请将 config_asser = 0，否则死机后不能复位。

3. 异常断言打印

```
1 const char log_tag_const_i_DEBUG AT(.LOG_TAG_CONST) = CONFIG_DEBUG_LIBS(1);
2 const char log_tag_const_d_DEBUG AT(.LOG_TAG_CONST) = CONFIG_DEBUG_LIBS(1);
3 const char log_tag_const_e_DEBUG AT(.LOG_TAG_CONST) = CONFIG_DEBUG_LIBS(1);
4 const char log_tag_const_c_DEBUG AT(.LOG_TAG_CONST) = CONFIG_DEBUG_LIBS(1);
```

上面四行代码为异常断言打印信息的开关，默认时开启状态。

13.1 内核异常示例（非对齐访问）：

在 `config_asser = 1` 并开启了断言打印的情况下，触发异常会有打印信息，以下以非对齐访问为例解释。

非对齐访问是指 CPU 尝试从非其数据类型自然边界（如 4 字节数据不从 4 的倍数地址开始）的内存地址读取或写入数据，这通常会导致硬件异常或性能损失。

```

1  Error> [DEBUG]
2  -----Err: function:exception_analyze() @line:87
3
4  <Error> [DEBUG]stack_magic[0]:0x5a5a5a5a
5  <Error> [DEBUG]stack_magic0[0]:0x5a5a5a5a
6  <Error> [DEBUG]reti 0x00112434
7  <Error> [DEBUG]rete 0x00000000
8  <Error> [DEBUG]rets 0x00112402
9  <Error> [DEBUG]psr 0x00000000
10 <Error> [DEBUG]ssp 0x00002520
11 <Error> [DEBUG]icfg 0x07010a80
12 <Error> [DEBUG]usp 0x00000800
13 <Error> [DEBUG] sp 0x00001608
14
15 <Error> [DEBUG]---reti---
16 40 AC 11 EA FF FF 40 D4 F8 71 08 80 00 ED 71 AC
17 08 78 20 E8 D1 F8 20 E4 02 92 60 E7 21 00 48 D0
18 60 E8 D0 F8 20 E8 D0 F8 00 80 60 E8 D0 F8 FE E1
19 3F A2 FF 0F 18 2F 21 C6 D8 E1 6F 5C 3F 0F 24 D1
20
21 <Error> [DEBUG]---rets---
22 CF 5B 38 C6 64 E8 06 32 31 80 C1 77 60 E8 C4 1E
23 28 C6 81 C6 03 E1 B0 37 20 E4 00 9D D8 E1 DF 5E
24 20 E8 B0 03 20 E8 B1 01 0F ED 12 FF 52 AC BA 75
25 40 D4 40 AC 11 EA FF FF 40 D4 F8 71 08 80 00 ED
26
27 <Error> [DEBUG]EMU_CON:0x00008007
28 <Error> [DEBUG]EMU_MSG:0x00000001
29 <Error> [DEBUG]-----EMUerr msg0:misalign_err-----
30
31 <Error> [DEBUG]r0:0x000022c6
32 <Error> [DEBUG]r1:0x000022c5
33 <Error> [DEBUG]r2:0x00001d10
34 <Error> [DEBUG]r3:0x00000001
35 <Error> [DEBUG]r4:0x000020e4
36 <Error> [DEBUG]r5:0x003fa000
37 <Error> [DEBUG]r6:0x000022c4
38 <Error> [DEBUG]r7:0x00002020
39 <Error> [DEBUG]r8:0x00006000
40 <Error> [DEBUG]r9:0x000021e0
41 <Error> [DEBUG]r10:0x00001678
42 <Error> [DEBUG]r11:0x00007e8c
43 <Error> [DEBUG]r12:0x00002210
44 <Error> [DEBUG]r13:0x000091420
45 <Error> [DEBUG]r14:0x00000001

```

13.1.1 第 1, 2 行

```
1 Error> [DEBUG]
2 -----Err: function:exception_analyze() @line:87
```

这两行打印表示进入了异常断言。

13.1.2 第 4, 5 行堆栈边界

```
4 <Error> [DEBUG] stack_magic[0]:0x5a5a5a5a
5 <Error> [DEBUG] stack_magic0[0]:0x5a5a5a5a
```

这两行打印堆栈边界信息，边界信息不等于 0x5a5a5a5a 时堆栈已溢出。

备注:

stack_magic != 0x5a5a5a5a 时，爆栈 (Stack Overflow)

这是最常见的情况。当程序调用层次过深、局部变量过大或存在无限递归时，栈顶 (Stack Pointer, SP) 会不断向低地址方向增长 (对于向下生长的栈)，最终超越为栈分配的栈底 (Stack Base/Limit) 边界，侵入到其他内存区域 (如全局数据区或堆区)。

备注:

stack_magic0 != 0x5a5a5a5a 时，爆栈 (Stack Underflow)

这种情况相对少见，但更危险。通常发生在函数返回 (POP 操作) 或调整栈指针时出现了严重错误 (如错误的汇编指令、缓冲区下溢写入破坏了返回地址和栈帧后，又执行了多次返回)。

13.1.3 第 6~13 行 CPU 瞬态

```
06 <Error> [DEBUG] reti 0x00112434
07 <Error> [DEBUG] rete 0x00000000
08 <Error> [DEBUG] rets 0x00112402
09 <Error> [DEBUG] psr 0x00000000
10 <Error> [DEBUG] ssp 0x00002520
11 <Error> [DEBUG] icfg 0x07010a80
12 <Error> [DEBUG] usp 0x00000800
13 <Error> [DEBUG] sp 0x00001608
```

这几行显示的是一个处理器在异常或中断发生时的关键上下文寄存器。它们共同记录了 CPU 被打断那一瞬间的状态，是调试异常、崩溃和中断的核心信息。

备注:

在分析函数调用关系时，需从汇编层面入手，因为 C 语言的函数在编译链接后可能被内联优化，导致源码层面的调用链不完整。

从汇编视角解析寄存器记录的逻辑：

1. 当函数 A 调用函数 B 时，进入 B 函数后，**rets** 寄存器中保存的是 A 函数中“调用指令”的下一条指令地址。此地址是 B 函数执行完毕后需要返回的位置。
2. 若 B 函数在运行过程中触发异常（如非对齐访问），CPU 在跳转至异常中断处理程序后，**reti** 寄存器中保存的是 B 函数中触发异常的那条指令的下一条指令地址。此地址是异常处理结束后理论上应返回的位置（但通常因错误无法直接返回）。

1.reti (0x00112434) - 中断/异常返回地址

作用：记录了被中断的程序下一条即将执行的指令地址（PC 值）。

分析：当 CPU 正在执行 0x00112434 这条指令时（或刚执行完上一条），被异常/中断打断。处理完异常后，CPU 会跳回这个地址继续执行。

调试价值：这是最重要的寄存器之一。通过反汇编这个地址附近的代码，可以精确知道是哪条指令（或前一条指令）触发了异常。

2.rete (0x00000000) - 异常返回地址（可能）

作用：在某些架构中用于记录嵌套异常或特定类型异常（如精确的指令异常）的返回地址。值为 0 可能表示当前是最外层异常，或该寄存器未使用。

分析：如果 **reti** 和 **rets** 都存在，**rete** 有时指向导致异常（如 **misalign_err**）的那条故障指令本身的地址。

3.rets (0x00112402) - 子程序/状态返回地址

作用：可能记录了发生异常时，当前函数的调用者（父函数）的返回地址（即 **LR** 链接寄存器的值），或者是 CPU 在特定模式下的备用返回地址。

分析：0x00112402 这个地址很接近 **reti** (0x00112434)，相差仅 0x32 字节。这表明异常很可能发生在某个刚被调用不久的函数内部。结合这两个地址可以画出函数调用链。

4.psr (0x00000000) - 程序状态寄存器

作用：包含 ALU 状态标志（如零标志 Z、进位 C、负数 N、溢出 V），以及当前处理器模式和中断开关状态。

分析：值为 0 非常可疑。通常至少会有某些状态位被置位（例如 **Thumb** 状态位，在 Cortex-M 中必须为 1）。全 0 可能意味着：

寄存器被意外清零（内存破坏?）。

异常发生在 CPU 复位后的最初状态。

这是某个特定模式下的备份 **PSR**，其值在进入异常时被保存到了其他寄存器。

5.sp (0x00001608) - 当前堆栈指针

作用：最关键的寄存器之一。这是异常发生瞬间的栈指针值。所有局部变量、函数调用帧都保存在这个地址指向的内存区域。

分析：

它是异常现场的“快照”。

其值（0x1608）位于 **ssp**(0x2520) 和 **usp**(0x800) 之间。

13.1.4 第 15~25 行

```
15 <Error> [DEBUG]---reti---
16 40 AC 11 EA FF FF 40 D4 F8 71 08 80 00 ED 71 AC
17 08 78 20 E8 D1 F8 20 E4 02 92 60 E7 21 00 48 D0
18 60 E8 D0 F8 20 E8 D0 F8 00 80 60 E8 D0 F8 FE E1
19 3F A2 FF 0F 18 2F 21 C6 D8 E1 6F 5C 3F 0F 24 D1
20
21 <Error> [DEBUG]---rets---
22 CF 5B 38 C6 64 E8 06 32 31 80 C1 77 60 E8 C4 1E
23 28 C6 81 C6 03 E1 B0 37 20 E4 00 9D D8 E1 DF 5E
24 20 E8 B0 03 20 E8 B1 01 0F ED 12 FF 52 AC BA 75
25 40 D4 40 AC 11 EA FF FF 40 D4 F8 71 08 80 00 ED
```

这几行信息是 reti 与 rets 所在位置附件的机器码。

13.1.5 第 27~28 行

EMU_CON: 异常控制寄存器, bit2 为 1 时, 使能除 0 异常。

```
27 <Error> [DEBUG]EMU_CON:0x00008007
28 <Error> [DEBUG]EMU_MSG:0x00000001
```

EMU_MSG 内核异常信息:

```
bit0 = 1时, 非对齐访问异常
bit1 = 1时, 非法指令异常 (cpu取指出现非法指令)
bit2 = 1时, 除0异常
```

13.1.6 第 29 行异常信息种类

```
29 <Error> [DEBUG]-----EMUerr msg0:misalign_err-----
```

信息显示错误类型, misalign_err 为非对齐访问。

备注: 异常信息种类

- misalign_err : 为非对齐访问
- illeg_err : 非法指令异常
- div0_err : 除 0 异常

13.1.7 第 31~45 行通用寄存器信息

```
31 <Error> [DEBUG] r0:0x000022c6
32 <Error> [DEBUG] r1:0x000022c5
33 <Error> [DEBUG] r2:0x00001d10
34 <Error> [DEBUG] r3:0x00000001
35 <Error> [DEBUG] r4:0x000020e4
36 <Error> [DEBUG] r5:0x003fa000
37 <Error> [DEBUG] r6:0x000022c4
38 <Error> [DEBUG] r7:0x00002020
39 <Error> [DEBUG] r8:0x00006000
40 <Error> [DEBUG] r9:0x000021e0
41 <Error> [DEBUG] r10:0x00001678
42 <Error> [DEBUG] r11:0x00007e8c
43 <Error> [DEBUG] r12:0x00002210
44 <Error> [DEBUG] r13:0x00091420
45 <Error> [DEBUG] r14:0x00000001
```

这些是发生异常时，CPU 通用寄存器在那一刻的“快照”。它们是定位软件错误的最关键线索，相当于程序的“犯罪现场指纹”。

AD17N 的内核异常管理单元

异常管理单元开启后，会在 MCU 异常时触发中断，并复位或者打印出异常信息并断言；

1. 调用函数 `emu_init` 后异常开始工作。

2. 异常断言

```
1 //内核异常打印,为1时会断言,开发调试时可改为1,量产时必须为0
2 const u8 config_asser = 0;
```

备注:

`config_asser = 1` 时，异常触发后断言，并在 `DEBUG` 打印控制开启的情况下会打印出异常信息；

`config_asser = 0` 时，异常触发后芯片复位。

**** 量产时，请将 `config_asser = 0`，否则死机后不能复位。****

3. 异常断言打印

```
1 const char log_tag_const_i_DEBUG AT(.LOG_TAG_CONST) = CONFIG_DEBUG_LIBS(1);
2 const char log_tag_const_d_DEBUG AT(.LOG_TAG_CONST) = CONFIG_DEBUG_LIBS(1);
3 const char log_tag_const_e_DEBUG AT(.LOG_TAG_CONST) = CONFIG_DEBUG_LIBS(1);
4 const char log_tag_const_c_DEBUG AT(.LOG_TAG_CONST) = CONFIG_DEBUG_LIBS(1);
```

上面四行代码为异常断言打印信息的开关，默认时开启状态。

14.1 异常示例（非对齐访问）:

在 config_asser = 1 并开启了断言打印的情况下，触发异常会有打印信息，以下以非对齐访问为例解释。

非对齐访问是指 CPU 尝试从其数据类型自然边界（如 4 字节数据不从 4 的倍数地址开始）的内存地址读取或写入数据，这通常会导致硬件异常或性能损失。

```

1  <Error> [DEBUG]
2  -----Err: function:exception_analyze() @line:478
3
4  <Error> [DEBUG]_EMU_CON : 0xc000001f
5  <Error> [DEBUG]_EMU_MSG : 0x1
6  <Error> [DEBUG]usp : 0x34524b0
7  <Error> [DEBUG]ssp : 0x4a1fbd9b
8  <Error> [DEBUG]sp : 0x3f020fc
9  <Error> [DEBUG]Stack: 0x3f021d0
10 <Error> [DEBUG]rets : 0x400b12e
11 <Error> [DEBUG]reti : 0x400b13e
12 <Error> [DEBUG]rete : 0x0
13 <Error> [DEBUG]psr : 0x8
14 <Error> [DEBUG]icfg : 0x7010a80
15
16 <Error> [DEBUG]--reti---
17 0x0400B11E
18 FC E1 9F B4 01 E1 30 0D 20 E4 20 D6 FC E1 3F B4
19 A0 4F 20 E4 21 D4 60 E7 10 05 40 D0 A0 6F A0 4F
20 60 E8 4A FE C0 80 FF E1 8F 00 18 80 FF E1 5F 00
21 40 E9 0A 19 21 93 38 81 0A 80 FE E1 AF FE 00 E9
22
23 <Error> [DEBUG]--rets---
24 0x0400B10E
25 61 48 20 E4 20 D9 FC E1 EF B4 A1 48 20 E4 20 D8
26 FC E1 9F B4 01 E1 30 0D 20 E4 20 D6 FC E1 3F B4
27 A0 4F 20 E4 21 D4 60 E7 10 05 40 D0 A0 6F A0 4F
28 60 E8 4A FE C0 80 FF E1 8F 00 18 80 FF E1 5F 00
29
30 <Error> [DEBUG]-- sp ---
31 0x03F020FC
32 3B 22 0B 9C E4 23 F0 03 17 0C 01 04 03 C0 FF FF
33 10 08 EE 03 01 00 00 00 18 25 F0 03 60 24 F0 03
34 00 60 00 00 14 30 ED 03 00 00 00 00 AC 24 F0 03
35 24 47 F0 03 14 03 ED 03 F0 24 F0 03 3E B1 00 04
36 00 00 00 00 2E B1 00 04 08 00 00 00 9B BD 1F 4A
37 80 0A 01 07 B0 24 45 03 D5 33 D1 47 A7 41 D1 64
38 E7 8A 0A 6D AC 24 F0 03 30 11 81 81 00 00 00 00
39 7C C1 00 04 C0 14 F0 03 CC 14 F0 03 00 00 00 00
40 00 00 00 00 00 00 00 02 00 00 00 00 00 00 00
41 00 00 00 00 62 E3 26 15 2D DB 69 7C 62 E3 26 15
42 04 44 C7 EB 0F 33 57 7B 9F C3 E7 0B 2F 53 77 9B
43 00 00 00 00 00 00 00 00 D8 DE 00 04 FE FF 00 00
44 FB FF 00 00 00 00 00 00 10 00 00 00 D0 22 00
45 2C 10 F0 03
46
47 <Error> [DEBUG][0-CPU] emu err msg : misalign_err
48
49 <Error> [DEBUG]r0 = 0x9c0b223b

```

(续下页)

(接上页)

```

50 <Error> [DEBUG] r1 = 0x3f023e4
51 <Error> [DEBUG] r2 = 0x4010c17
52 <Error> [DEBUG] r3 = 0xffffc003
53 <Error> [DEBUG] r4 = 0x3ee0810
54 <Error> [DEBUG] r5 = 0x1
55 <Error> [DEBUG] r6 = 0x3f02518
56 <Error> [DEBUG] r7 = 0x3f02460
57 <Error> [DEBUG] r8 = 0x6000
58 <Error> [DEBUG] r9 = 0x3ed3014
59 <Error> [DEBUG] r10 = 0x0
60 <Error> [DEBUG] r11 = 0x3f024ac
61 <Error> [DEBUG] r12 = 0x3f04724
62 <Error> [DEBUG] r13 = 0x3ed0314
63 <Error> [DEBUG] r14 = 0x3f024f0

```

14.1.1 第 1, 2 行

```

1 <Error> [DEBUG]
2 -----Err: function:exception_analyze() @line:478

```

这两行打印表示进入了异常断言。

14.1.2 第 4, 5 行

EMU_CON: 异常控制寄存器:

```

bit2 = 1时,  使能除0异常。
bit3 = 1时,  使能堆栈溢出异常。
bit4 = 1时,  使能PC越界异常 (程序运行到代码限制范围以外)。
bit30 = 1时, 使能CACHE异常。
bit31 = 1时, 使能外部异常。

```

```

4 <Error> [DEBUG] _EMU_CON : 0xc000001f
5 <Error> [DEBUG] _EMU_MSG : 0x1

```

EMU_MSG 内核异常信息:

```

bit0 = 1时,  非对齐访问异常。
bit1 = 1时,  非法指令异常。
bit2 = 1时,  除0异常。
bit3 = 1时,  栈溢出异常。
bit4 = 1时,  PC越界异常 (程序运行到代码限制范围以外)。
bit30 = 1时,  CACHE异常, 程序访问到被保护的CACHE区域。
bit31 = 1时,  外部异常。

```

14.1.3 第 6~14 行 CPU 瞬态

```

06 <Error> [DEBUG]usp   : 0x34524b0
07 <Error> [DEBUG]ssp   : 0x4a1fbd9b
08 <Error> [DEBUG]sp    : 0x3f020fc
09 <Error> [DEBUG]Stack: 0x3f021d0
10 <Error> [DEBUG]rets   : 0x400b12e
11 <Error> [DEBUG]reti   : 0x400b13e
12 <Error> [DEBUG]rete   : 0x0
13 <Error> [DEBUG]psr    : 0x8
14 <Error> [DEBUG]icfg   : 0x7010a80

```

这几行显示的是一个处理器在异常或中断发生时的关键上下文寄存器。它们共同记录了 CPU 被打断那一瞬间的状态，是调试异常、崩溃和中断的核心信息。

备注:

在分析函数调用关系时，需从汇编层面入手，因为 C 语言的函数在编译链接后可能被内联优化，导致源码层面的调用链不完整。

从汇编视角解析寄存器记录的逻辑：

1. 当函数 A 调用函数 B 时，进入 B 函数后，**rets** 寄存器中保存的是 A 函数中“调用指令”的下一条指令地址。此地址是 B 函数执行完毕后需要返回的位置。
2. 若 B 函数在运行过程中触发异常（如非对齐访问），CPU 在跳转至异常中断处理程序后，**reti** 寄存器中保存的是 B 函数中触发异常的那条指令的下一条指令地址。此地址是异常处理结束后理论上应返回的位置（但通常因错误无法直接返回）。

1.reti (0x400b13e) - 中断/异常返回地址

作用：记录了被中断的程序下一条即将执行的指令地址（PC 值）。

分析：当 CPU 正在执行指令时（或刚执行完上一条），被异常/中断打断。处理完异常后，CPU 会跳回这个地址继续执行。

调试价值：这是最重要的寄存器之一。通过反汇编这个地址附近的代码，可以精确知道是哪条指令（或前一条指令）触发了异常。

2.rete (0x400b12e) - 异常返回地址（可能）

作用：在某些架构中用于记录嵌套异常或特定类型异常（如精确的指令异常）的返回地址。值为 0 可能表示当前是最外层异常，或该寄存器未使用。

分析：如果 **reti** 和 **rets** 都存在，**rete** 有时指向导致异常（如 `misalign_err`）的那条故障指令本身的地址。

3.rets (0x00112402) - 子程序/状态返回地址

作用：可能记录了发生异常时，当前函数的调用者（父函数）的返回地址（即 LR 链接寄存器的值），或者是 CPU 在特定模式下的备用返回地址。

分析：0x00112402 这个地址很接近 **reti** (0x00112434)，相差仅 0x32 字节。这表明异常很可能发生在某个刚被调用不久的函数内部。结合这两个地址可以画出函数调用链。

4.psr (0x8) - 程序状态寄存器

作用：包含 ALU 状态标志（如零标志 Z、进位 C、负数 N、溢出 V），以及当前处理器模式和中断开关状态。

分析：值为 0 非常可疑。通常至少会有某些状态位被置位（例如 Thumb 状态位，在 Cortex-M 中必须为 1）。全 0 可能意味着：

寄存器被意外清零（内存破坏?）。

异常发生在 CPU 复位后的最初状态。

这是某个特定模式下的备份 PSR，其值在进入异常时被保存到了其他寄存器。

5.sp (0x3f020fc) - 当前堆栈指针

作用：最关键的寄存器之一。这是异常发生瞬间的栈指针值。所有局部变量、函数调用帧都保存在这个地址指向的内存区域。

分析：

它是异常现场的“快照”。

其值（0x1608）位于 ssp(0x2520) 和 usp(0x800) 之间。

5.Stack (0x3f021d0) - 堆栈起始位置（栈底）

这是堆栈开始的位置（最高地址），堆栈由高向低用。

14.1.4 第 16~28 行 reti 与 rets

```

16 <Error> [DEBUG]--reti---
17 0x0400B11E
18 FC E1 9F B4 01 E1 30 0D 20 E4 20 D6 FC E1 3F B4
19 A0 4F 20 E4 21 D4 60 E7 10 05 40 D0 A0 6F A0 4F
20 60 E8 4A FE C0 80 FF E1 8F 00 18 80 FF E1 5F 00
21 40 E9 0A 19 21 93 38 81 0A 80 FE E1 AF FE 00 E9
22
23 <Error> [DEBUG]--rets---
24 0x0400B10E
25 61 48 20 E4 20 D9 FC E1 EF B4 A1 48 20 E4 20 D8
26 FC E1 9F B4 01 E1 30 0D 20 E4 20 D6 FC E1 3F B4
27 A0 4F 20 E4 21 D4 60 E7 10 05 40 D0 A0 6F A0 4F
28 60 E8 4A FE C0 80 FF E1 8F 00 18 80 FF E1 5F 00

```

这几行信息是 reti 与 rets 所在位置附近的机器码。

第 17 行，表示 reti 指向位置附近，地址 0x0400B11E 开始的机器码。

第 24 行，表示 rets 指向位置附近，地址 0x0400B10E 开始的机器码。

14.1.5 第 30~45 行 sp

```

30 <Error> [DEBUG]-- sp ---
31 0x03F020FC
32 3B 22 0B 9C E4 23 F0 03 17 0C 01 04 03 C0 FF FF
33 10 08 EE 03 01 00 00 00 18 25 F0 03 60 24 F0 03
34 00 60 00 00 14 30 ED 03 00 00 00 00 AC 24 F0 03
35 24 47 F0 03 14 03 ED 03 F0 24 F0 03 3E B1 00 04
36 00 00 00 00 2E B1 00 04 08 00 00 00 9B BD 1F 4A
37 80 0A 01 07 B0 24 45 03 D5 33 D1 47 A7 41 D1 64

```

(续下页)

(接上页)

```

38 E7 8A 0A 6D AC 24 F0 03 30 11 81 81 00 00 00 00
39 7C C1 00 04 C0 14 F0 03 CC 14 F0 03 00 00 00 00
40 00 00 00 00 00 00 00 02 00 00 00 00 00 00 00
41 00 00 00 00 62 E3 26 15 2D DB 69 7C 62 E3 26 15
42 04 44 C7 EB 0F 33 57 7B 9F C3 E7 0B 2F 53 77 9B
43 00 00 00 00 00 00 00 00 D8 DE 00 04 FE FF 00 00
44 FB FF 00 00 00 00 00 00 00 10 00 00 00 D0 22 00
45 2C 10 F0 03

```

这几行信息是 sp 所在位置开始的机器码。

第 31 行，表示 sp 指向位置开始的机器码，起始地址是 0x03F020FC。

14.1.6 第 47 行异常信息种类

```
47 <Error> [DEBUG][0-CPU] emu err msg : misalign_err
```

信息显示错误类型，misalign_err 为非对齐访问。

备注：异常信息种类

misalign_err：为非对齐访问

illeg_err：非法指令异常

div0_err：除 0 异常

stack overflow err：堆栈溢出异常

pc_limit：PC 越界异常（程序运行到代码限制范围以外）

icache excptio：CACHE 异常，程序访问到被保护的 CACHE 区域

sys excption：外部异常

14.1.7 第 49~63 行通用寄存器信息

```

49 <Error> [DEBUG] r0 = 0x9c0b223b
50 <Error> [DEBUG] r1 = 0x3f023e4
51 <Error> [DEBUG] r2 = 0x4010c17
52 <Error> [DEBUG] r3 = 0xffffc003
53 <Error> [DEBUG] r4 = 0x3ee0810
54 <Error> [DEBUG] r5 = 0x1
55 <Error> [DEBUG] r6 = 0x3f02518
56 <Error> [DEBUG] r7 = 0x3f02460
57 <Error> [DEBUG] r8 = 0x6000
58 <Error> [DEBUG] r9 = 0x3ed3014
59 <Error> [DEBUG] r10 = 0x0
60 <Error> [DEBUG] r11 = 0x3f024ac
61 <Error> [DEBUG] r12 = 0x3f04724
62 <Error> [DEBUG] r13 = 0x3ed0314
63 <Error> [DEBUG] r14 = 0x3f024f0

```

这些是发生异常时，CPU 通用寄存器在那一刻的“快照”。它们是定位软件错误的最关键线索，相当于程序的“犯罪现场指纹”。

14.2 外部异常：

在上文中如果是外部系统异常，如下调试信息中的 `misalign_err`，将会是 `sys_excption`。

```
47 <Error> [DEBUG] [0-CPU] emu err msg : sys_excption
```

外部异常出现时，会有如下打印信息产生。

```
<Error> [DEBUG] JL_CEMU->MSG0 : 0XXXXXXXXX
<Error> [DEBUG] JL_CEMU->MSG1 : 0XXXXXXXXX
<Error> [DEBUG] JL_CEMU->MSG2 : 0XXXXXXXXX
```

14.2.1 示例-AUDIO 触发外部异常

以下调试信息为 AUDIO 触发外部系统异常。

```
1 <Error> [1-HCORE] hmem err msg: sys_access_hmem_excption
2 <Error> [DEBUG] at dev
3 <Error> [DEBUG] DBG_AUDIO
```

第 3 行表示，`audio` 模块异常，

备注：部分异常信息种类

```
DBG_ALNK
DBG_AUDIO
DBG_APA
DBG_SPI0
DBG_SPI1
DBG_UART0
DBG_UART1
DBG_GPADC
DBG_IIC
DBG_EQ
DBG_SRC
```

14.2.2 示例-看门狗 WDT 触发异常

```
1  <Error> [2-HSB] emu err msg : watchdog time out
```

AC104/AD14/15/17/18 SDK 提供了操作 gpio 相关功能的函数接口；

其头文件位于 include_lib/cpu/sh5x/gpio.h；

下面分为 4 个部分进行 GPIO 接口函数介绍：

- (1) ” *DACP, DACN* 口设置 (*AC104/AD14/15/17*) ”
- (2) ” 普通 *IO* 口 (*AC104/AD14/15/17/18*) ”
- (3) ” 模块 *IO* 映射 ”
- (4) ” 常见问题和注意事项 ”

15.1 DACP, DACN 口设置 (AC104/AD14/15/17)

(1) void gpio_dacnp_out_mode_init(DACX_MODE_SEL mode, OUTCH_SEL_T outchsel, u8 out)

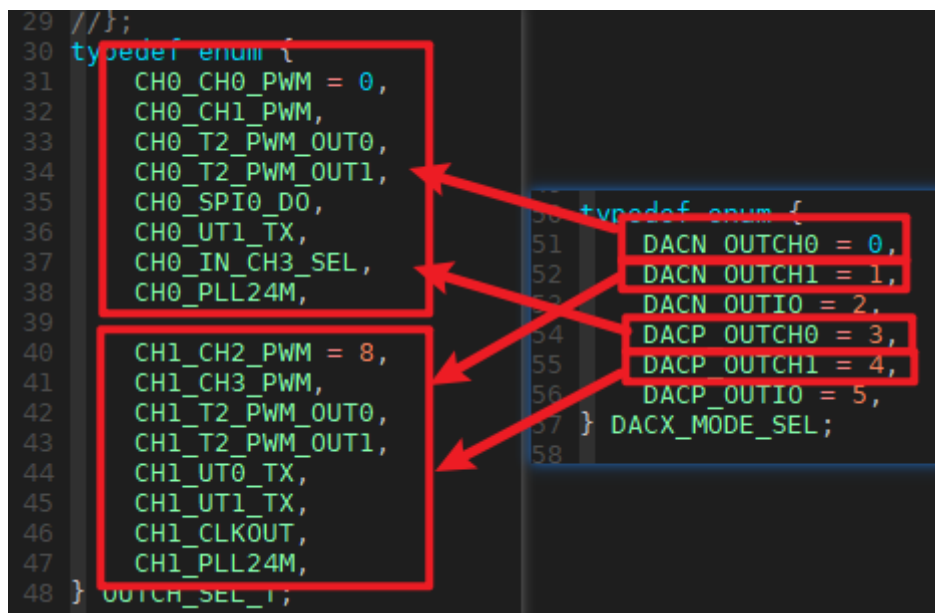
该函数实现将 **DACNO, DACNP** 作为普通 **IO** 输出，或者作为 **outputch** 输出，其中参数：

mode	: DACP 引脚选择做普通 IO 引脚 (DACP_OUTIO) 或者做输出通道 (DACP_OUTCHx)；
outchsel	: 选择 output 源，该参数只在 mode 参数选择做输出方式时有效；
out	: 该参数只在 mode 选择做普通 IO 引脚输出时有效；
	0: 输出 0；
	1: 输出 1；

注意：

当 **mode** 选择输出通道时，**mode** 选择的通道需要与 **outchsel** 通道一样；

如：**mode** 选择了 **DACN_OUTCH0**，则 **outchsel** 必须选择通道 0 的 output 源（即 0~7）；（对应下图）



DACNP_out_mode_init

(2) void gpio_dacnp_out_set(DACX_MODE_SEL mode, u8 out)

该函数实现将 DACNO,DACNP 作为普通 IO 输出, 输出 0 还是 1

```

mode : DACN_OUTIO 或者 DACP_OUTIO;
out   : 0: 输出 0;
       : 1: 输出 1;

```

注意:

使用 DACP,DACN 脚前需注意先关闭 APA 模块

详细步骤可点击下方链接:

[FAQ:DACN/DACP 引脚如何设置为普通 I/O 输出或高阻态?](#)

15.2 普通 IO 口 (AC104/AD14/15/17/18)

寄存器设置示例:

```

示例PA1 IO口常见操作:
JL_PORTA->DIR |= BIT(1);           //PA1口设为输入
JL_PORTA->DIE |= BIT(1);           //PA1口使能数字输入
JL_PORTA->PU  |= BIT(1);           //PA1口打开上拉
JL_PORTA->PD  |= BIT(1);           //PA1口打开下拉
JL_PORTA->OUT |= BIT(1);           //PA1口输出高电平
JL_PORTA->OUT &= ~BIT(1);         //PA1口输出低电平

```

函数使用示例:

```

gpio_set_direction(IO_PORTA_01,1);           //PA1口 设为输入
gpio_set_die(IO_PORTA_01,1);                 //PA1口 使能数字输入
gpio_set_pull_up(IO_PORTA_01,1);             //PA1口 打开上拉
gpio_set_pull_down(IO_PORTA_01,1);           //PA1口 打开下拉
gpio_write(IO_PORTA_01,1);                   //PA1口 输出高电平
gpio_write(IO_PORTA_01,0);                   //PA1口 输出低电平

gpio_set_direction(IO_PORT_DP,1);             //USB_DP口 设为输入
gpio_set_die(IO_PORTA_01,1);                 //USB_DP口 使能数字输入
gpio_set_pull_up(IO_PORTA_01,1);             //USB_DP口 打开上拉
gpio_set_pull_down(IO_PORTA_01,1);           //USB_DP口 打开下拉
gpio_write(IO_PORTA_01,1);                   //USB_DP口 输出高电平
gpio_write(IO_PORTA_01,0);                   //USB_DP口 输出低电平

```

下面介绍相关函数

(1) int gpio_set_direction(u32 gpio, u32 dir)

该函数实现设置 io 口方向, 其中参数:

```

gpio   : 参考宏 IO_PORTx_xx, 如: IO_PORTA_00;
dir    : 0: 输出;
        1: 输入;
返回值 : 0: 设置成功;
        其他值: 设置失败;

```

(2) int gpio_set_pull_up(u32 gpio, int value)

该函数实现设置 gpio 口上拉状态, 其中参数:

```

gpio   : 参考宏 IO_PORTx_xx, 如: IO_PORTA_00;
value  : 0: 不上拉;
        1: 上拉;

```

注意:

该函数只有GPIO设为输入时有效。

(3) int gpio_set_pull_down(u32 gpio, int value)

该函数实现设置 gpio 口下拉状态, 其中参数

```

gpio   : 参考宏 IO_PORTx_xx, 如: IO_PORTA_00;
value  : 0: 不下拉;
        1: 下拉;

```

注意:

该函数只有GPIO设为输入时有效。
AD18部分IO下拉使用寄存器配置无效 (即JL_PORTx->
↪PD), 需要用p33接口函数, 该函数内部已做处理。

详细说明可参考链接[AD18 部分 IO 下拉使用寄存器设置无效](#)

(4) int gpio_set_die(u32 gpio, int value);

该函数实现设置 io 口数字输入状态, 其中参数:

```

gpio   : 参考宏 IO_PORTx_xx, 如: IO_PORTA_00
value  : 0: IO模拟输入;
        1: IO数字输入;

```

(续下页)

(接上页)

返回值：0：设置成功；
其他值：设置失败；

- (5) **u32 gpio_set_dieh(u32 gpio, u32 value)**
该函数实现设置 io 口输入状态，其中参数：

gpio：参考宏 IO_PORTx_xx，如：IO_PORTA_00；
value：0：IO 模拟输入；
1：IO 普通输入；
返回值：0，设置成功；
其他值，设置失败；

- (6) **int gpio_read(u32 gpio)**
该函数实现读取 io 逻辑输入电平，io 口需要设置成数字输入，其中参数：

gpio：参考宏 IO_PORTx_xx，如：IO_PORTA_00；
返回值：0：逻辑0；
1：逻辑1；

- (7) **u32 gpio_write(u32 gpio, u32 value)**
该函数实现设置 io 输出值，其中参数

gpio：参考宏 IO_PORTx_xx，如：IO_PORTA_00；
value：0：输出0；
1：输出1；

15.3 模块 IO 映射

注意：不同芯片映射函数有所区分。

下面分 3 个部分进行介绍：

- (1) ”AC104/AD14/AD15”
- (2) ”AD17”
- (3) ”AD18”

15.3.1 AC104/AD14/AD15

- (1) **u32 gpio_output_channle(u32 gpio, u32 clk)**
该函数实现将 outputchannel 输出到 io 口；其中参数：

gpio：参考宏 IO_PORTx_xx，如：IO_PORTA_00；
clk：参考 gpio.h 文件中的枚举，如：CH0_CH0_PWM；
返回值：0：设置成功；
其他值：设置失败；

注意：

- (1) mcpwm 映射时前两路（即 mcpwm0/1）用通道 0，后两路（即 mcpwm2/3）用通道 1；
 - (2) 如果串口使用了固定引脚，则默认使用了 CH1_UT0_TX；
-

15.3.2 AD17

(1) `int gpio_och_sel_output_signal(u32 gpio, enum OUTPUT_CH_SIGNAL signal)`

该函数实现部分模块 IO 输出映射到任意 IO

gpio : 参考宏 IO_PORTx_xx, 如: IO_PORTA_00;
signal: 参考 gpio.h 文件中 OUTPUT_CH_SIGNAL 的枚举, 如: OUTPUT_CH_SIGNAL_TIMER1_PWM;
返回值 : 无效;

可参考定时器配置 (timer_pwm.c)

(2) `int gpio_och_disable_output_signal(u32 gpio, enum OUTPUT_CH_SIGNAL signal)`

该函数实现关闭对应模块 IO 输出映射

gpio : 参考宏 IO_PORTx_xx, 如: IO_PORTA_00;
signal: 参考 gpio.h 文件中 OUTPUT_CH_SIGNAL 的枚举, 如: OUTPUT_CH_SIGNAL_TIMER1_PWM;
返回值 : 无效;

(3) `int gpio_ich_sel_input_signal(u32 gpio, enum INPUT_CH_SIGNAL signal, enum INPUT_CH_TYPE type)`

该函数实现部分模块 IO 输入映射到任意 IO

gpio : 参考宏 IO_PORTx_xx, 如: IO_PORTA_00;
signal: 参考 gpio.h 文件中 INPUT_CH_SIGNAL 的枚举, 如: INPUT_CH_SIGNAL_TIMER1_CIN;
type : 参考 gpio.h 文件中的 INPUT_CH_TYPE 枚举, 如: INPUT_CH_TYPE_GP_ICH;
返回值 : 通道号;

可参考红外按键设置 (irflt.c)

(4) `int gpio_ich_disable_input_signal(u32 gpio, enum INPUT_CH_SIGNAL signal, enum INPUT_CH_TYPE type)`

该函数实现关闭对应模块 IO 输入映射

gpio : 参考宏 IO_PORTx_xx, 如: IO_PORTA_00;
signal: 参考 gpio.h 文件中 INPUT_CH_SIGNAL 的枚举, 如: INPUT_CH_SIGNAL_TIMER1_CIN;
type : 参考 gpio.h 文件中的 INPUT_CH_TYPE 枚举, 如: INPUT_CH_TYPE_GP_ICH;
返回值 : 通道号;

15.3.3 AD18

(1) **u32 gpio_mux_out(u32 gpio, enum GPIO_OUTPUT_FUN fun)**

该函数实现部分模块输出映射到任意 io 口；其中参数：

```
gpio   : 参考宏 IO_PORTx_xx, 如: IO_PORTA_00;
fun    : 参考 gpio.h 文件中 GPIO_OUTPUT_FUN 的枚举, 如: GPIO_OUT_COMP0_MC_PWM0_H;
```

可参考定时器模块配置 (timer_drv.c)

(2) **u32 gpio_mux_out_close(u32 gpio, u32 fd)**

该函数实现关闭对应模块输出映射；其中参数：

```
gpio   : 参考宏 IO_PORTx_xx, 如: IO_PORTA_00;
fd     : 参考 gpio.h 文件中 GPIO_OUTPUT_FUN 的枚举, 如: GPIO_OUT_COMP0_MC_PWM0_H;
```

(3) **int gpio_mux_in(u32 gpio, enum GPIO_INPUT_FUN fun)**

该函数实现部分模块输入映射到任意 io 口；其中参数：

```
gpio   : 参考宏 IO_PORTx_xx, 如: IO_PORTA_00;
fun    : 参考 gpio.h 文件中 GPIO_INPUT_FUN 的枚举, 如: GPIO_INPUT_ICH0_TIMER0_CLK;
```

可参考红外按键模块配置 (irflt.c)

(4) **u32 gpio_mux_in_close(u32 fd)**

该函数实现关闭对应模块的输入映射；其中参数：

```
fd     : 参考 gpio.h 文件中 GPIO_INPUT_FUN 的枚举, 如: GPIO_INPUT_ICH0_TIMER0_CLK;
```

15.4 常见问题和注意事项

15.4.1 AC104/AD14/AD15/AD17 生产不良率高

[20230626] SDK 同步补丁：修改 GPIO HD 档位默认配置

15.4.2 AD18 部分 IO 下拉使用寄存器设置无效

AD18 PC 口和 PD 口使用寄存器设置下拉无效，建议使用 gpio_set_pull_down 函数来设置 IO 口下拉状态。

各芯片支持的设备类型如下：

芯片支持的设备	内置 flash	外置 flash	SDMMC	usb
AD14N / AC104N	√	√	√	√
AD15N	√	√	√	
AD17N	√	√		
AD18N	√	√		√
AD24N	√	√	√	√
AD23N	√	暂不支持	√	暂不支持

设备管理库的目录位于 `include_lib/liba/dev_mg_lib.a`。其中主要的函数有：

```
int  devices_init();
bool dev_online(const char* name);
void* dev_open(const char* name, void* arg);
int  dev_ioctl(void* device, int cmd, u32 arg);
int  dev_close(void* device);
int  dev_bulk_read(void* _device, void* buf, u32 offset, u32 len);
int  dev_bulk_write(void* _device, void* buf, u32 offset, u32 len);
int  dev_byte_read(void* _device, void* buf, u32 offset, u32 len);
int  dev_byte_write(void* _device, void* buf, u32 offset, u32 len);
```

一般设备使用流程：

1. 定义并赋值设备表；
2. 开机调用 `devices_init()` 完成对所有设备的初始化；
3. 调用 `dev_open` 打开参数中名字为 `name` 的设备；
4. 对设备进行读写（`dev_bulk_read/dev_bulk_write/dev_byte_read/dev_byte_write`）等操作；

16.1 设备注册表

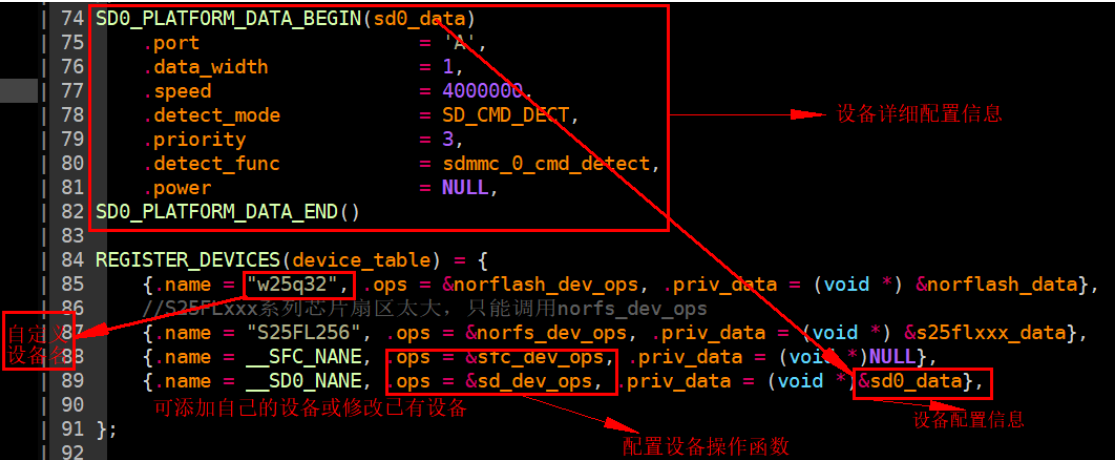
设备注册表 device_tab 中注册了全部需要使用到的设备，其中成员如下：

- 1. name：设备名称的字符串（自定义名称）
- 2. ops：设备内部接口函数集合（不可更改）
- 3. priv_data：设备使用的底层硬件资源及相关模式配置（可根据需要自行配置）

如果要添加设备，直接在下图注册表中接着添加。开机时初始化程序会自动执行初始化。

注：内置 FLASH 只读时不需要挂设备驱动，因为内部硬件已经实现了支持，在使用时向文件系统传递 NULL 的设备句柄就能实现读取内部的 sydf 文件系统！

设备注册表定义及初始化如图 3.1 所示。



```
74 SD0_PLATFORM_DATA_BEGIN(sd0_data)
75     .port                = 'A',
76     .data_width          = 1,
77     .speed               = 4000000,
78     .detect_mode         = SD_CMD_DECT,
79     .priority            = 3,
80     .detect_func         = sdmmc_0_cmd_detect,
81     .power               = NULL,
82 SD0_PLATFORM_DATA_END()
83
84 REGISTER_DEVICES(device_table) = {
85     { .name = "w25q32", .ops = &norflash_dev_ops, .priv_data = (void *) &norflash_data},
86     //S25FLxxx系列芯片扇区太大，只能调用norfs_dev_ops
87     { .name = "S25FL256", .ops = &norfs_dev_ops, .priv_data = (void *) &s25flxxx_data},
88     { .name = __SFC_NAME, .ops = &sfc_dev_ops, .priv_data = (void *) NULL},
89     { .name = __SD0_NAME, .ops = &sd_dev_ops, .priv_data = (void *) &sd0_data},
90 };
91
92
```

Annotations in the image:

- Red box around lines 74-82: 设备详细配置信息
- Red box around line 87: 自定义设备
- Red box around line 88: 可添加自己的设备或修改已有设备
- Red box around line 89: 配置设备操作函数
- Red box around line 89: 设备配置信息

图 1 设备注册表

注意 flash 注册时有两组接口：norflash_dev_ops、norfs_dev_ops（位置见图 2/图 3）。

注册 norflash_dev_ops 接口时，起始地址及长度以 512 字节为单位，比如读取 (dev_bulk_read) 时带地址 1 实际是从 1*512=512 开始读取，且写函数内部自带擦除操作，可以直接写入。

注册 norfs_dev_ops 接口时，起始地址及长度以 1 字节为单位，写函数内部没有擦除，调写函数前需确定要写入的内存地址中没有存入数据，如果有数据要自己写擦除操作。

（注意：读写 flash 设备操作不是调用图 2 图 3 中函数，而是调用 dev_bulk_read/dev_bulk_write 执行读写）

```

748 * 1. 外部调用时以512字节为单位的地址和长度, 且需要驱动write自己处理擦除,
749 * 请使用norflash_dev_ops接口, 否则使用本文件内的其他ops
750 *
751 * 2. 如果不需要驱动自己处理擦除, 可以把宏FLASH_CACHE_ENABLE清零, 或者把
752 * norflash_dev_read()里面调用的_norflash_read()的实参cache填0,
753 * norflash_dev_write()同理
754 *
755 * 3. norflash_dev_ops可以被多个设备名注册, 每个设备名被认为是不同分区, 所以
756 * 需要填不同的分区起始地址和大小, 若分区地址有重叠或者最大分区结束地址大于
757 * flash容量, 会触发ASSERT()
758 *
759 * 4. 关于IOCTL_GET_CAPACITY, 有多个分区注册时返回分区的大小, 如果只注册了1
760 * 个分区, 分区起始地址 == 0时返回flash容量, 起始地址 != 0时返回分区大小,
761 * norflash_dev_ops返回的长度以512字节为单位
762 *
763 * 5. 本文件内的各个ops可以同时使用
764 */
765 const struct device_operations norflash_dev_ops = {
766     .init = norflash_dev_init,
767     .online = norflash_dev_online,
768     .open = norflash_dev_open,
769     .read = norflash_dev_read,
770     .write = norflash_dev_write,
771     .ioctl = norflash_dev_ioctl,
772     .close = norflash_dev_close,
773 };

```

图 2 flash 设备 norflash_dev_ops 接口

```

852 * 1. 外部调用时以1字节为单位的地址和长度, 且驱动write自己不处理擦除,
853 * 请使用norfs_dev_ops接口, 否则使用本文件内的其他ops。注意: 有些文件系
854 * 统需要满足这个条件的驱动, 如果期望修改成驱动内部处理擦除, 需要测试所
855 * 有关联的文件系统是否支持, 或者新建一个符合需求的ops
856 *
857 * 2. 如果需要驱动自己处理擦除, 需要把宏FLASH_CACHE_ENABLE置1, 且
858 * norfs_dev_read()里面调用的_norflash_read()的实参cache填1,
859 * norfs_dev_write()同理
860 *
861 * 3. norfs_dev_ops可以被多个设备名注册, 每个设备名被认为是不同分区, 所以
862 * 需要填不同的分区起始地址和大小, 若分区地址有重叠或者最大分区结束地址大于
863 * flash容量, 会触发ASSERT()
864 *
865 * 4. 关于IOCTL_GET_CAPACITY, 有多个分区注册时返回分区的大小, 如果只注册了1
866 * 个分区, 分区起始地址 == 0时返回flash容量, 起始地址 != 0时返回分区大小
867 * norfs_dev_ops返回的长度以1字节为单位
868 *
869 * 5. 本文件内的各个ops可以同时使用
870 */
871 const struct device_operations norfs_dev_ops = {
872     .init = norfs_dev_init,
873     .online = norfs_dev_online,
874     .open = norfs_dev_open,
875     .read = norfs_dev_read,
876     .write = norfs_dev_write,
877     .ioctl = norfs_dev_ioctl,
878     .close = norfs_dev_close,
879 };

```

图 3 flasflash 设备 norfs_dev_ops 接口 h 配置信息

图 1 中设备详细数据配置信息介绍见下图, SD 接口配置信息见图 4, flash 配置信息图 5。

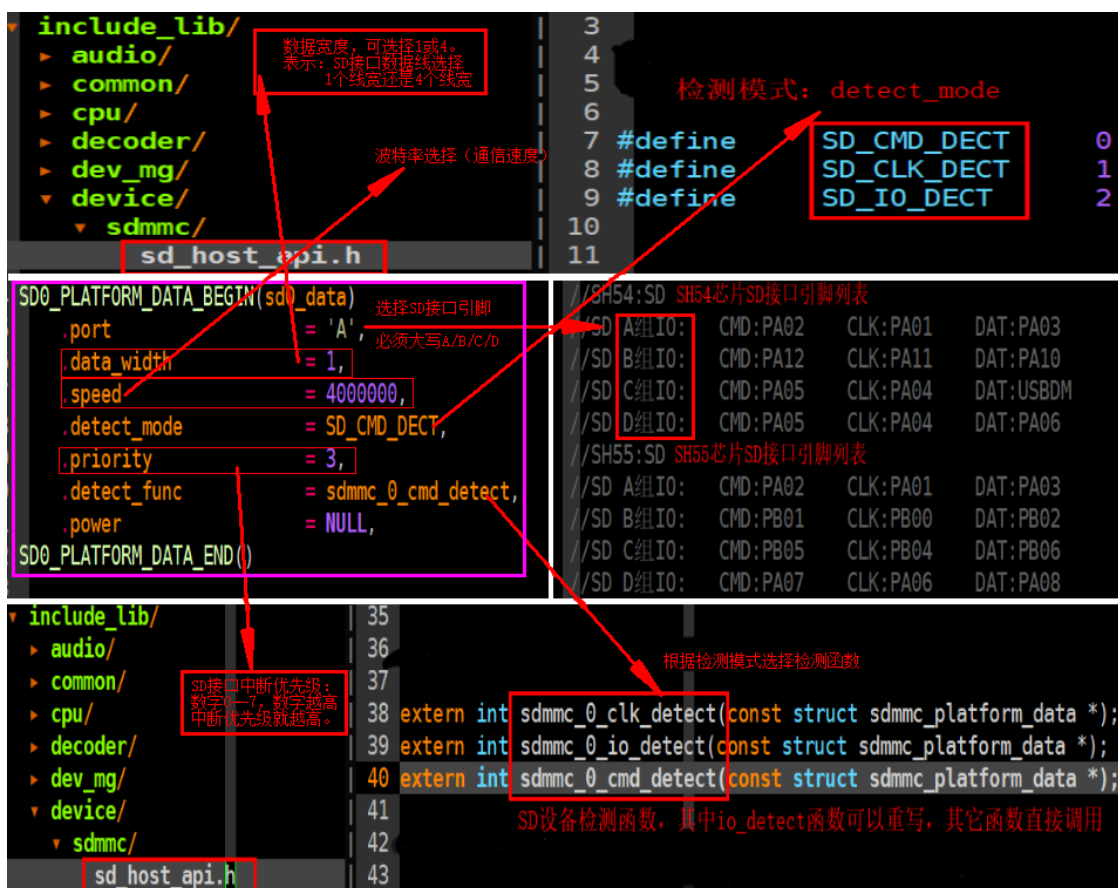


图 4 SD 接口配置信息

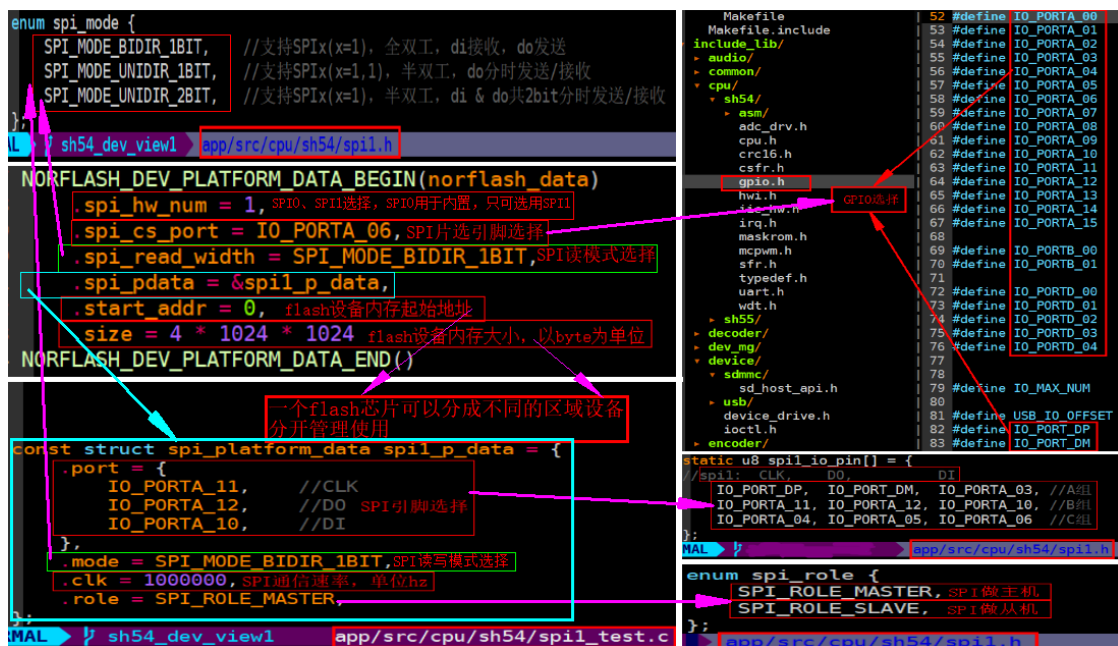


图 5 flash 配置信息

16.2 设备函数

16.2.1 函数 int devices_init()

此函数会将 device_tab 中注册的设备全部初始化，这一步一般在开机时进行。返回值未使用。

16.2.2 函数 bool dev_online(const char* name)

此函数会根据传入的设备名称返回设备在线情况。

返回值: 1 表示设备在线;0 表示不在线;

16.2.3 函数 void* dev_open(const char* name, void* arg)

此函数可以打开设备，函数会返回设备句柄供读写等函数调用。只有打开设备，才能使用读写等函数。函数调用前定义好设备句柄变量：

1. name: 需要打开的设备名称;
2. arg: 设备参数（多数时候为 0，不使用）;
3. 返回值：设备句柄，用于读写等操作函数;

设备名称可自定义，名称信息见下图 6。

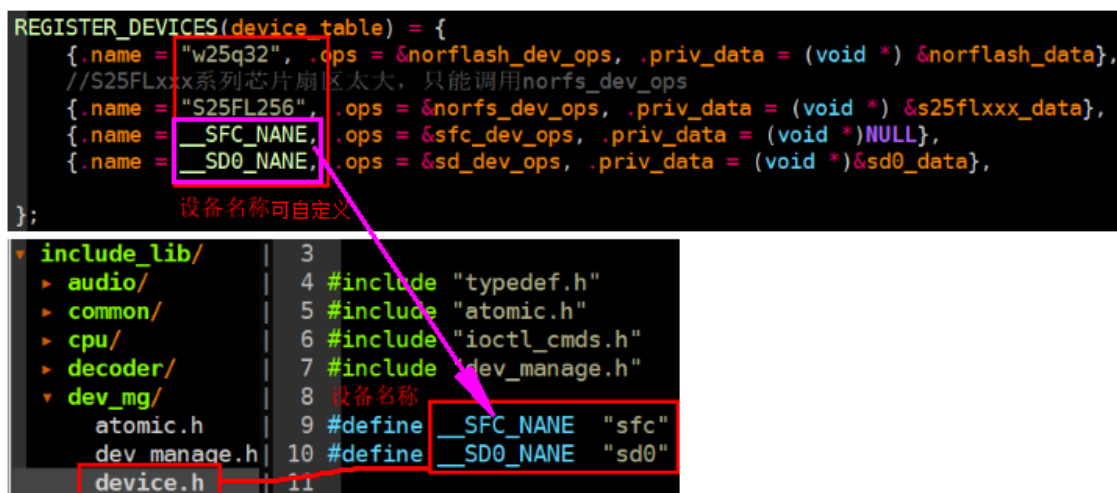


图 6 注册表名称信息

16.2.4 函数 `int dev_bulk_read(void* _device, void* buf, u32 offset, u32 len)`

本函数实现从设备某个地址 `offset` 读取长度为 `len * 512` 字节的数据到 `buf` 的功能:

1. `_device`: `dev_open` 函数返回的设备句柄;
2. `buf`: 缓存, 保存读出的数据, 需自定义;
3. `offset`: 读取起始设备地址;
4. `len`: 需要读取的块数, 通常为 `len*512` 字节长度;
5. 返回值: 正常情况返回长度即形参 `Len`, 错误时返回 0

注意: 形参 `offset`、`len` 范围要在图 7 定义的起始地址和大小范围内才能正常操作设备。

16.2.5 函数 `int dev_bulk_write(void* _device, void* buf, u32 offset, u32 len)`

本函数实现把长度为 `len * 512` 字节的 `buf` 数据写入设备的功能, 写入起始地址为 `offset`:

1. `_device`: `dev_open` 函数返回的设备句柄;
2. `Buf`: 缓存, 保存着需写入的数据;
3. `Offset`: 写入起始设备地址;
4. `Len`: 需要写入的块数, 通常为 `len*512` 字节长度。
5. 返回值: 正常情况返回长度即形参 `Len`, 错误时返回 0

注意: 形参 `offset`、`len` 范围要在图 7 定义的起始地址和大小范围内才能正常操作设备。

16.2.6 函数 `int dev_byte_read(void* _device, void* buf, u32 offset, u32 len)`

本函数实现从设备某个地址 `offset` 读取长度为 `len` 字节的数据到 `buf` 的功能:

1. `_device`: `dev_open` 函数返回的设备句柄;
2. `buf`: 缓存, 保存读出的数据, 需自定义;
3. `offset`: 读取起始设备地址;
4. `len`: 需要读取的字节长度;
5. 返回值: 正常情况返回长度即形参 `Len`, 错误时返回 0

注意: 形参 `offset`、`len` 范围要在图 7 定义的起始地址和大小范围内才能正常操作设备。

16.2.7 函数 `int dev_byte_write(void* _device, void* buf, u32 offset, u32 len)`

本函数实现把长度为 len 字节的 buf 数据写入设备的功能，写入起始地址为 offset:

1. _device: dev_open 函数返回的设备句柄;
2. Buf : 缓存, 保存着需写入的数据;
3. Offset : 写入起始设备地址;
4. Len : 需要写入的字节长度。
5. 返回值: 正常情况返回长度即形参 Len, 错误时返回 0

注意: 形参 offset、len 范围要在图 7 定义的起始地址和大小范围内才能正常操作设备。

16.2.8 函数 `int dev_ioctl(void* device, int cmd, u32 arg)`

本函数实现获取设备信息或设置设备信息, 擦除设备等功能:

1. device: 设备句柄;
2. cmd: 命令, include_lib/dev_mg/ioctl_cmds.h 内能查到定义; 详细命令表参见图 8。常用命令有: (注: 不同设备支持命令会有差别, 并不是支持所以命令)
 - a. IOCTL_CMD_SUSPEND, 设备挂起
 - b. IOCTL_CMD_RESUME, 设备解挂
 - c. IOCTL_GET_ID, 获取设备 ID
 - d. IOCTL_GET_SECTOR_SIZE, 获取设备扇区大小
 - e. IOCTL_GET_BLOCK_SIZE, 获取设备块大小
 - f. IOCTL_GET_CAPACITY, 获取设备容量
 - g. IOCTL_GET_STATUS, 获取设备状态
 - h. IOCTL_ERASE_SECTOR, 擦除设备扇区
 - i. IOCTL_ERASE_BLOCK, 擦除设备块
 - j. IOCTL_ERASE_CHIP, 对设备整片擦除
3. arg: 获取信息时: 用作保存返回的设备信息如容量、ID 等; 擦出时: 用作带入要擦除的地址。
4. 返回值: 反应函数运行状态。0: 表示正常其他值: 出错


```

include_lib/dev_mg/ioctl_cmds.h

#define IOCTL_SET_IRQ_NUM 1
#define IOCTL_SET_PRIORITY 2
#define IOCTL_SET_DATA_WIDTH 3
#define IOCTL_SET_SPEED 4
#define IOCTL_SET_DETECT_MODE 5
#define IOCTL_SET_DETECT_FUNC 6
#define IOCTL_SET_DETECT_TIME_INTERVAL 7
#define IOCTL_SET_PORT 8
#define IOCTL_SET_PORT_FUNC 9
#define IOCTL_SET_CS_PORT_FUNC 10
#define IOCTL_SET_READ_MODE 11
#define IOCTL_SET_WRITE_MODE 12
#define IOCTL_SET_WRITE_PROTECT 13
#define IOCTL_SET_START_BIT 14
#define IOCTL_SET_STOP_BIT 15
#define IOCTL_FLUSH 16
#define IOCTL_REGISTER_IRQ_HANDLER 17
#define IOCTL_UNREGISTER_IRQ_HANDLER 18
#define IOCTL_GET_SYS_TIME 19
#define IOCTL_SET_SYS_TIME 20
#define IOCTL_GET_ALARM 21
#define IOCTL_SET_ALARM 22
#define IOCTL_SET_CAP_LOWSPEED_CARD 23
#define IOCTL_SET_VDD50_EN 30
#define IOCTL_GET_WEEKDAY 32
#define IOCTL_CLR_READ_MODE 33
#define IOCTL_SET_READ_CRC 34
#define IOCTL_GET_READ_CRC 35
#define IOCTL_GET_VOLUME 36
#define IOCTL_SET_VOLUME 37
#define IOCTL_SET_ALARM_ENABLE 38

#define IOCTL_CMD_RESUME 39
#define IOCTL_CMD_SUSPEND 40
#define IOCTL_SET_BASE_ADDR 41
#define IOCTL_SET_ASYNC_MODE 42

#define IOCTL_GET_ID 100
#define IOCTL_GET_SECTOR_SIZE 101
#define IOCTL_GET_BLOCK_SIZE 102
#define IOCTL_GET_CAPACITY 103
#define IOCTL_GET_WIDTH 104
#define IOCTL_GET_HEIGHT 105
#define IOCTL_GET_BLOCK_NUMBER 106
#define IOCTL_CHECK_WRITE_PROTECT 107
#define IOCTL_GET_STATUS 108
#define IOCTL_GET_TYPE 109
#define IOCTL_GET_MAX_LUN 110
#define IOCTL_GET_CUR_LUN 111
#define IOCTL_SET_CUR_LUN 112
#define IOCTL_SET_FORCE_RESET 113
#define IOCTL_SET_CAPACITY 114
#define IOCTL_GET_NORFS_INFO 115

#define IOCTL_ERASE_SECTOR 200
#define IOCTL_ERASE_BLOCK 201
#define IOCTL_ERASE_CHIP 202
#define IOCTL_SET_ENC_END 203
#define IOCTL_ERASE_PAGE 204

#define IOCTL_SET_DATA_CALLBACK 301

```

图 8 cmd 可选择的命令

16.2.9 函数 int dev_close(void* device)

本函数实现关闭设备的功能，对于一些设备调用此函数能省电：

1. device: dev_open 返回的设备句柄；
2. 返回值：暂无意义

支持的文件系统列表：

文件系 统	详情
sydfs	只读文件系统，用于存放资源文件到 norflash；
norfs	循环覆盖文件系统，可读写；针对 norflash 设计；
sim- ple_fat	简单 fat 文件系统驱动，占用资源少，不支持搜索和多文件同时打开；支持 FAT12/16/32, EXFAT；
fatfs	完整的 fat 文件系统驱动，占用资源多，支持搜索；支持 FAT12/16/32, EXFAT；

各芯片 SDK 支持的文件系统类型如下：

芯片	sydfs	norfs	simple_fat	fatfs
AD14N	√	√	√	
AC104N	√	√		√
AD15N	√			
AD17N	√			
AD18N	√	√		√
AD24N (toy)	√	√		√
AD24N (enhanced)	√	√		√
AD23N	√	√		√

SDK 中通过 vfs 来集中管理各类文件系统，涉及的文件目录包括：

- app\bsp\common\fs 目录下的所有 c 文件；
- include_lib\fs 目录下的所有 h 文件；
- errno-base.h 文件；

备注：

vfs 有以下主要函数:

1. void vfs_init(void);
2. u32 vfs_mount(void **ppvfs, void *device, void *type);
3. u32 vfs_openbypath(void *pvfs, void **ppvfile, const char *path);
4. u32 vfs_openbyindex(void *pvfs, void **ppvfile, u32 index);
5. u32 vfs_createfile(void *pvfs, void **ppvfile, u32 *pindex);
6. u32 vfs_read(void *pvfile, void *buf, u32 len);
7. u32 vfs_write(void *pvfile, void *buf, u32 len);
8. u32 vfs_seek(void *pvfile, u32 offset, u32 mode);
9. u32 vfs_file_close(void **ppvfile);
10. u32 vfs_fs_close(void **ppvfs);
11. u32 vfs_file_name(void *pvfile, void *name, u32 len);
12. int vfs_get_attr(void *pvfile, void *pvfs_attr);
13. int vfs_ioctl(void *pvfile, int cmd, int arg);

备注: 其中, **fatfs** 中独有的函数:

1、获取文件读写指针

```
int vfs_ftell(void *pvfile, void *parm);
```

2、获取文件大小

```
int vfs_get_fsize(void *pvfile, void *parm);
```

3、fat 文件系统文件扫描

```
struct vfscan *vfs_fscan_new(void *pvfs, const char *path, const char *arg, u8 max_depth, int (*callback)(void),
struct vfscan *fsn, struct vfscan_reset_info *info);
```

4、释放 fat 文件系统扫描句柄

```
int vfs_fscan_release(void *pvfs, struct vfscan *fs);
```

5、实现 fat 文件系统的文件选取

```
int vfs_select(void *pvfs, void **ppvfile, struct vfscan *fs, int sel_mode, int arg);
```

6、实获取 at 文件系统扫描后的相关文件号

```
int vfs_get_folderinfo(void **pvfile, struct vfscan *fs, int *start_num, int *end_num);
```

7、实现 fat 文件系统创建文件夹

```
int vfs_mk_dir(void *pvfs, char *folder, u8 mode);
```

8、获取录音文件的相关信息

```
int vfs_get_encfolder_info(void *pvfs, char *folder, char *ext, u32 *last_num, u32 *total_num);
```

9、删除文件

```
int vfs_file_delete(void *pvfile);
```

17.1 文件系统介绍

Sdk 中默认支持的文件系统有: sydfs、norfs、fatfs、simple_fatfs、freefs;
可以在 app\bsp\common\fs 目录下查看各文件系统的文件以及支持的接口;
本小节将介绍各个文件系统以及支持的 ioctl 对应的 cmd 命令:

17.2 文件系统使用一般流程

如图, fs 运行大致流程如下;

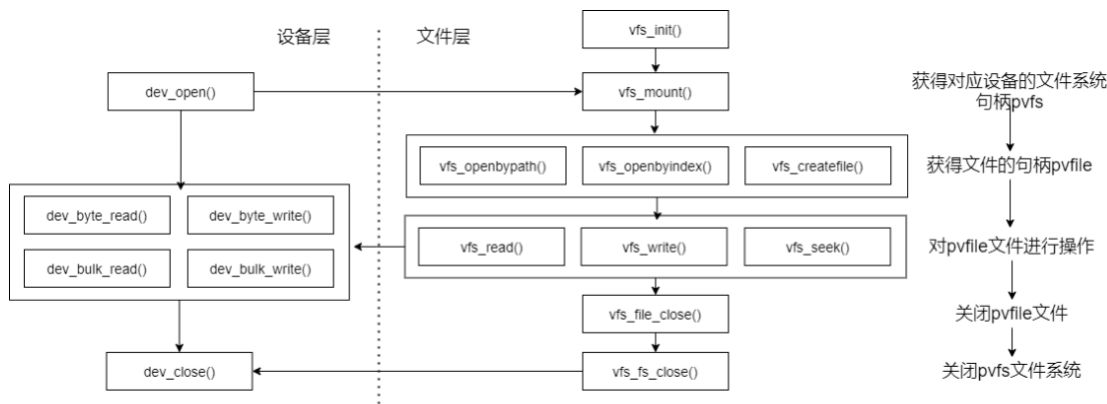


图 2.1 fs 运行大致流程

通用的文件系统使用流程

- 1、文件系统初始化：
调用函数 `void fs_init(void)`，完成对所有文件系统驱动的初始化；
- 2、mount 挂载：
调用 `u32 fs_mount(void **ppvfs, void *device, void *type)`，传递指定设备句柄，并完成对具体文件系统的挂载，并获取文件系统的句柄；
- 3、打开文件：
调用 `u32 fs_openbypath(void *pvfs, void **ppvfile, const char *path)` 或 `u32 fs_select(void *fsn, void *pfs, int sel_mode, void **ppvfile, int arg)` 打开文件，并获取文件句柄；
- 4、读取文件数据：
调用 `u32 fs_read(void *pvfile, void *buf, u32 len)` 对文件进行读操作；
- 5、对文件进行不同的命令操作：
调用 `int fs_ioctl(void *pvfile, int cmd, int arg)` 使用不同的命令进行不同的操作，如获取文件号、获取文件总数、获取文件地址、获取文件名字等；
- 6、关闭文件：
调用 `u32 fs_file_close(void **ppvfile)` 关闭文件，并释放文件句柄。
- 7、关闭文件系统：
调用 `u32 fs_fs_close(void **ppvfs)` 关闭文件系统，并释放文件系统句柄。

(续下页)

(接上页)

可以通过调用接口得到的返回值查看是否调用成功，以及失败的原因；失败原因可在include_→lib/common/errno-base.h中查看。

17.3 通用文件系统管理函数

通用文件系统管理函数将介绍 sydfs、norfs、fatfs 通用的接口，各文件系统具体能使用哪些接口请查看上面的几张图片，下一小节将介绍 fatfs 独有的相关文件系统接口；

17.3.1 函数 void vfs_init(void)

此函数实现对所有文件系统驱动的初始化；在开机第一次调用即可。

17.3.2 函数 u32 vfs_mount(void **ppvfs, void *device, void *type)

此函数实现对指定设备挂在指定类型的文件系统，所挂载的系统必须设备自身支持；其中参数：

1. ppvfs：文件系统句柄指针，若句柄指向为NULL，函数会自行申请空间；
2. device：设备句柄；
 - ① 需要打开sydfs时，若句柄为空，则会使用内置flash；
 - ② 需要打开norfs等其他文件系统时，句柄不能为空；
3. type：文件系统类型的字符串；
 - ① 可选择"sydfile"、"norfs"、"fat"或0，
 - ② 此值为0时，则跳过norfs，在其他文件系统中循环尝试挂载，成功即止；
 - ③ 注：norfs文件系统仅为录音到内置FLASH的应用使用！！
4. 返回值：
 - ① 成功 0；
 - ② 失败 错误请查看errno-base.h。

示例 1, 在内置 flash 挂载 syd 文件系统:

```
1 void *device = dev_open(__SFC_NANE, 0); //获取内置flash设备句柄
2 void *pfs = NULL;
3 u32 err = vfs_mount(&pfs, device, "sydfile"); //内置flash挂载syd文件系统
4 if (pfs != NULL && err == 0) {
5     //内置flash挂载sydfs成功
6 }
```

示例 2, 在外挂 flash 上挂载 norfs 文件系统:

```
1 void *device = dev_open(__EXT_FLASH_NANE, 0); //获取外挂flash设备句柄
2 void *pfs = NULL;
3 u32 err = vfs_mount(&pfs, device, "norfs"); //外置flash挂载norfs文件系统
4 if (pfs != NULL && err == 0) {
5     //外置flash挂载norfs成功
6 }
```

示例 3, 在 SD 卡上挂载 fat 文件系统:

```

1 void *device = dev_open(__SD0_NANE, 0); //获取sd卡设备句柄
2 void *pfs = NULL;
3 u32 err = vfs_mount(&pfs, device, "fat"); //sd卡挂载fat文件系统
4 if (pfs != NULL && err == 0) {
5     //sd卡挂载fat成功
6 }

```

17.3.3 函数 u32 vfs_openbypath(void *pvfs, void **ppvfile, const char *path)

此函数实现打开指定路径的文件，norfs 文件系统无法使用此功能。其中参数：

1. pvfs: 已挂载的文件系统句柄；
2. ppvfile: 文件句柄指针，若句柄指向为NULL，函数会自行申请空间；
3. path: 路径字符串；
 - ① 字符串仅为文件夹路径，那么打开文件夹，打开后仅可用于校验CRC16；
 - ② 字符串包含了完整的文件名，那么会同时包含文件名！
 - ③ 如果字符串为空，则返回错误；
4. 返回值：
 - ① 成功 0；
 - ② 失败 错误请查看errno-base.h。

示例, 在挂载 (vfs_mount) 成功的前提下, 打开 dir_song 里的 so001.f1a 歌曲:

```

1 void *pfile = NULL;
2 u32 err = vfs_openbypath(pfs, &pfile, "dir_song/so001.f1a");
3 if (pfile != NULL && err == 0) {
4     //成功打开dir_song文件夹里的so001.f1a文件
5 }

```

17.3.4 函数 u32 vfs_openbyindex(void *pvfs, void **ppvfile, u32 index)

此函数实现打开指定文件号的文件，其中参数：

1. pvfs: 已挂载的文件系统句柄；
2. ppvfile: 文件句柄指针，若句柄指向为NULL，函数会自行申请空间；
3. index: 文件序号；
4. 返回值：
 - ① 成功 0；
 - ② 失败 错误请查看errno-base.h。

17.3.5 函数 u32 vfs_createfile(void *pvfs, void **ppvfile, u32 *pindex)

此函数实现 norfs、fatfs 创建一个新的文件，syd_fs 无法使用此功能。文件创建和操作结束后，需要调用 vfs_file_close() 关闭文件，其中参数：

1. pvfs: 已挂载的文件系统句柄；
2. ppvfile: 文件句柄指针，若句柄指向为 NULL，函数会自行申请空间；
3. pindex: 文件序号；
 - ① norfs: 传入的 pindex 会被赋值为最新的文件序号；
 - ② fatfs: 传入文件存放位置，如 "/test.txt" 即在根目录创建 test.txt 文件；；
注意：
 - ③ sydfs: 不支持此功能；
4. 返回值：
 - ① 0；
 - ② 失败 错误请查看 errno-base.h。

备注：

1. **norfs 文件系统**：文件创建和操作结束后，需要先调用 vfs_ioctl() 接口传入 FS_IOCTL_FILE_SYNC 命令更新文件信息，**然后再关闭文件**。
2. **fatfs 文件路径**：文件创建时，其所在路径必须完整存在。若文件夹不存在，需预先调用 vfs_mk_dir() 创建，否则会失败。
3. **fatfs 长文件名配置**：
 - 需传入 **Unicode 编码** 的路径
 - 开启 FOPEN_LONG 宏定义
 - 链接相应的函数库
 - 具体实现请参考 Demo 代码。

示例 1-挂载 norfs 文件系统成功前提下，创建一个新的文件：

```

1 void *pfile = NULL;
2 int index;
3 u32 err = vfs_createfile(pfs, &pfile, (u32)&index);
4 if (pfile != NULL && err == 0) {
5     //norfs 文件系统下文件创建成功，index 会被赋值为所创建文件的序号
6 }

```

示例 2-挂载 fat 文件系统成功前提下，创建一个新的文件：

```

1 void *pfile = NULL;
2 char *path = "/abcd.txt";
3 u32 err = vfs_createfile(pfs, &pfile, (u32)path);
4 if (pfile != NULL && err == 0) {
5     //fat 文件系统下文件 abcd.txt 创建成功
6 }

```

17.3.6 函数 u32 vfs_read(void *pvfile, void *buf, u32 len)

此函数实现对指定文件的读操作。其中参数：

1. pvfile: 已打开的文件句柄；
2. buf: 数据buff；
3. len: 需要读取的字节长度；
4. 返回值：
 - ① 成功 读取的字节数；
 - ② 失败 0。

示例, 打开文件成功前提下, 读取文件数据：

```
1 u8 read_buf[32];
2 u32 read_len = vfs_read(pfile, (void *)read_buf, 32);
3 if (read_len != 0) {
4     //成功读取pfile中read_len长度的数据到read_buf里
5 }
```

17.3.7 函数 u32 vfs_write(void *pvfile, void *buf, u32 len)

此函数实现对指定文件的写操作，syd_fs 无法使用此功能，其中参数：

1. pvfile: 已打开的文件句柄；
2. buf: 数据buf；
3. len: 需要写入的字节长度；
4. 返回值：
 - 成功 写入的字节数；
 - 失败 错误请查看errno-base.h。

示例, 打开文件成功前提下, 往文件写入数据：

```
1 u8 write_buf[32] = {0, 1, 2, 3 ... 31};
2 u32 write_len = vfs_write(pfile, (void *)write_buf, 32);
3 if (write_len != 0) {
4     //成功将write_buf中write_len长度的数据写入到pfile
5 }
```

17.3.8 函数 u32 vfs_seek(void *pvfile, u32 offset, u32 mode)

此函数实现改变指定文件中读写指针的偏移，其中参数：

1. pvfile: 已打开的文件句柄；
2. offset: 偏移量；
3. mode: seek模式选择。如图4.2，seek模式有3种，分别为：
 - ① SEEK_SET: 从文件起始位置开始向后偏移
 - ② SEEK_CIR: 从文件读写指针位置开始向后偏移
 - ③ SEEK_END: 从文件末尾位置向前偏移
4. 返回值: 0。

备注:

mode 参数,seek 方式详解:

1. SEEK_SET: 从文件起始位置开始向后偏移, 首先会判断偏移量是否大于文件长度, 若小于则使文件读写指针 $rw_p = offset$;
2. SEEK_CUR: 从文件读写指针位置开始向后偏移, 使 $rw_p = rw_p + offset$;
3. SEEK_END: 从文件末尾位置向前偏移, 首先会判断偏移量是否大于文件长度, 若小于则使 $rw_p = \text{文件长度} - offset$;

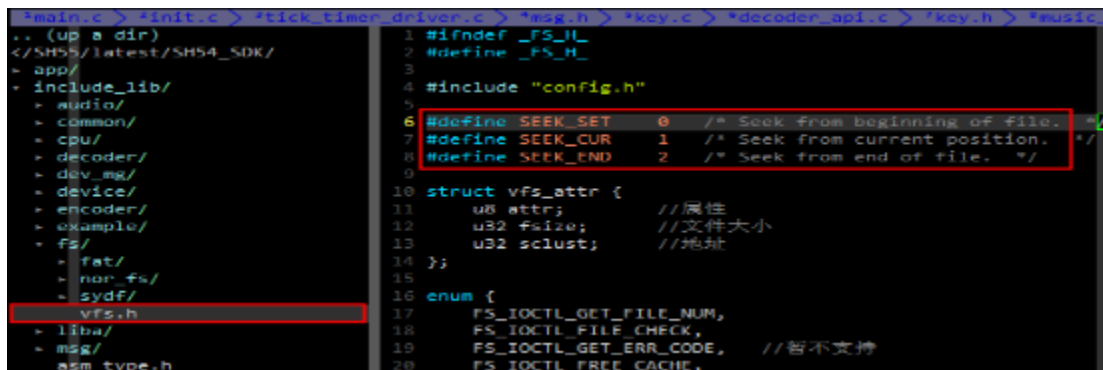
备注:

fat 文件系统默认支持的也是小于等于 16 字符的文件名, v1.9.0 版本过后 fat 文件系统才支持超过 16 字符的长名功能。

若需要在 v1.9.0 版本前 fat 文件系统使用长名功能, 可查看下方链接:

[FAQ: FAT 文件系统下如何获取函数文件名\(短文件名 / 长文件名\)](#)

下图为 seek 模式的选择:



17.3.9 函数 u32 vfs_file_close(void **ppvfile)

此函数实现关闭文件并释放句柄, 其中参数:

1. ppvfile: 已打开的文件句柄指针;
2. 返回值: 0。

示例:

```
1  vfs_file_close(&pfile);
2  if (pfile == NULL) {
3      //成功关闭pfile文件
4  }
```

17.3.10 函数 u32 vfs_fs_close(void **ppvfs)

此函数实现关闭文件系统并释放相关句柄，nor_fs 无法使用此功能，其中参数：

1. ppvfs: 已挂载的文件系统句柄指针；
2. 返回值: NULL。

示例：

```
1 vfs_fs_close(&pfs);
2 if (pfs == NULL) {
3     //成功关闭pfs文件系统
4 }
```

17.3.11 函数 u32 vfs_file_name(void *pvfile, void *name, u32 len)

此函数实现获取指定文件句柄的文件名，其中参数：

1. pvfile: 已打开的文件句柄；
2. name: 存放文件名的buf；
3. len: 获取文件名的长度；
4. 返回值: 成功 返回获取文件名的长度；

备注：

syd 文件系统和 norfs 文件系统都只支持小于等于 16 字符的文件名。

fat 文件系统默认支持的也是小于等于 16 字符的文件名，v1.9.0 版本过后 fat 文件系统才支持超过 16 字符的长名功能。

若需要在 v.1.9.0 版本前 fat 文件系统使用长名功能，可查看下方链接：

[FAQ: FAT 文件系统下如何获取函数文件名\(短文件名 / 长文件名\)](#)

17.3.12 函数 int vfs_get_attr(void *pvfile, void *pvfs_attr)

此函数实现记录文件的属性，包括文件大小和地址，nor_fs 无法使用此功能，其中参数：::

1. pvfile: 已打开的文件句柄；
2. pvfs_attr: 文件系统属性，需要传入 vfs_attr 结构体类型的指针；
3. 返回值: 0。

示例：

```
1 struct vfs_attr file_attr;
2 vfs_get_attr(pfile, &file_attr);
3 //文件地址和大小分别保存在file_attr.sclust和file_attr.fsize中
```

17.3.13 函数 int vfs_ioctl(void *pvfile, int cmd, int arg)

此函数实现对指定文件的相关操作，各文件系统具体的 cmd 命令请查看《文件系统介绍》小节，其中参数：

1. pvfile: 已打开的文件句柄；

2. cmd: 命令；

3. arg: 参数；

4. 返回值：

成功 0；

失败 错误请查看errno-base.h。

cmd: 命令主要有 7 种：

vfs_ioctl cmd 命令	命令含义
FS_IOCTL_DIR	获取文件所在文件系统下的文件数。该命令会将 arg 的值作为地址，并把文件句柄所在文件系统下的所有文件数赋给 arg 指向的值；若 arg 参数为 0，返回错误 E_PAGE_NULL；
FS_IOCTL_FIL	同上；
FS_IOCTL_FS_	获取文件系统下的文件数。该命令会将 arg 的值作为地址，并将特定情况下传入的文件系统句柄下的所有文件数赋值给 arg 指向的值；若 arg 参数为 0，返回错误 E_PAGE_NULL；
FS_IOCTL_FIL	获取文件地址参数。该命令会将 arg 的值作为地址并指向 vfs 地址参数，并把文件长度以及地址赋值给 vfs 地址参数；若 arg 参数为 0 或文件句柄为 NULL，返回错误 E_PAGE_NULL 或 E_PFILE_NULL；
FS_IOCTL_FIL	同步文件。该命令会将录音文件同步并将 arg 的值作为末尾裁切的字节长度；若 arg 参数为 0 则不进行裁切；
FS_IOCTL_FIL	通过文件指针获取文件索引号。该命令会将 arg 的值作为地址，并把文件的索引号赋值给 arg 指向的值。
FS_IOCTL_FS_	通过文件系统指针获取文件索引号。该命令会将 arg 的值作为地址，并把文件的索引号赋值给 arg 指向的值。
FS_IOCTL_GE'	获取文件系统总容量
FS_IOCTL_GE'	获取文件系统剩余容量

函数 vfs_ioctl 获取总容量命令 FS_IOCTL_GET_PARTITION_INFO

获取文件系统总容量, 仅 fat 有效, 参考代码：

```
1 FS_PARTITION_INFO f_info;
2 u32 err = vfs_ioctl(pvfile,FS_IOCTL_GET_PARTITION_INFO, (u32)&(f_info))
3 if(0 != err)
4 {
5     // succeed
6 }
```

函数 `vfs_ioctl` 获取剩余容量命令 `FS_IOCTL_GET_FREE_SPACE`

获取文件系统剩余容量, 仅 fat 有效, 参考代码:

```
1  u32 free_apacity_Kbyte; //
2  u32 err = vfs_ioctl(pvfile, FS_IOCTL_GET_PARTITION_INFO, (u32) &free_apacity_
   ↳ Kbyte);
3  if (0 != err)
4  {
5      // succeed
6  }
```

Sydfs 是系统数据运行的 只读文件系统，可以通过 sdk 里下载目录里的 download_bat.c 工具加上 -res 命令来下载存放资源文件，并于系统运行时读取；



```
22 copy /b %ELF_NAME%.bin+data.bin+cache_ram.bin app.bin
23 #else
24 copy /b %ELF_NAME%.bin+data.bin app.bin
25 #endif
26
27
28 @echo *****
29 @echo BD49 mbox_flash
30 @echo *****
31 @echo % date %
32 cd / d % ~dp0
33
34 ::isd_download.exe -tonorflash -dev bd49 -boot 0x3f01400 -div8 -wait 300 -uboot uboot.boot -app app.bin 0x20000 -res
35 dir_song dir_eng dir_poetry dir_story
36 isd_download.exe -tonorflash -dev bd49 -boot 0x3f01400 -div8 -wait 300 -uboot uboot.boot -app app.bin 0x80000 -res
37 dir_a dir_song dir_eng dir_poetry dir_story dir_midi midi_cfg dir_notice dir_bin_flx
38 :-format all
39 @REM 常用命令说明
40 @rem - format vm // 擦除VM 区域
```

-res 后面跟随的即为下载到内置flash的资源文件

图 1.1 下载目录下内置 flash 资源

该文件系统能使用的接口图 1.1 所示：

```

177 const struct vfs_operations sydf_vfs_ops sec_used(.vfs_operations) = {
178     .fs_type = "sydf",
179     .init      = sydf_init_api,
180     /* .init    = NULL, */
181     .mount     = sydf_monut_api,
182     .openbypath = sydf_open_api,
183     .openbyindex = sydf_openbyindex_api,
184     .openbyfile = sydf_openbyfile_api,
185     .createfile = NULL,
186     .read       = sydf_read_api,
187     .write      = NULL,
188     .seek       = sydf_seek_api,
189     .close_fs   = sydf_fs_close_api,
190     .close_file = sydf_file_close_api,
191     .fget_attr  = sydf_attr_api,
192     .name       = sydf_file_name_api,
193     .ioctl      = sydf_ioctl_api,
194 };

```

图 1.2 sydf 文件系统接口

其中 `ioctl` 支持的 `cmd` 命令有：

① `FS_IOCTL_FILE_TOTAL`, 示例:

```

1  int file_total;
2  vfs_ioctl(pfile, FS_IOCTL_FILE_TOTAL, (int)&file_total);
3  // file_total即为pfile文件所在文件系统的总文件数；

```

② `FS_IOCTL_FS_TOTAL`, 示例:

```

1  int file_total;
2  vfs_ioctl(pfs, FS_IOCTL_FILE_TOTAL, (int)&file_total);
3  // file_total即为pfs文件系统的总文件数；

```

③ `FS_IOCTL_FILE_ATTR`, 示例:

```

1  struct vfs_attr attr;
2  vfs_ioctl(pfile, FS_IOCTL_FILE_ATTR, (int)&attr);
3  // attr即可获得pfile文件的位置和大小；

```

备注:

- 注意：通过 `ioctl` 获取的文件位置仅为该 `pfile` 文件在其上层目录下的相对偏移；若要获取该文件相对于 `flash` 的偏移地址，需要调用 `vfs_get_fsize()`；

18.1 函数 int vfs_file_crc(void *pvfile)

此函数实现计算文件的 CRC16 校验值，目前仅 SYD 可使用此功能，其中参数：

1. pvfile：已打开的文件句柄；
2. 返回值：CRC校验值，计算错误则返回0；

备注：注意：V1.4.0 以及之后的 SDK，可以通过 openbypath 方式打开资源文件进行校验，也可打开 dir 文件用于校验；打开 dir 文件方式仅用于校验！

示例 1:

```
1  vfs_openbypath(pfs, &pfile, "/dir_song");           //校验打包后的dir文件
2  u16 crc16 = vfs_file_crc(pfile);
```

示例 2:

```
1  vfs_openbypath(pfs, &pfile, "/dir_song/so001.f1a"); // 校验资源文件
2  u16 crc16 = vfs_file_crc(pfile);
```

Norfs 是一个循环管理的可读写文件系统，内部空间环形管理，空间不足时新写入的文件会覆盖之前的文件；

norfs 文件系统只通过文件号进行管理，创建新文件后文件号会累加；

该文件系统一般录音在内置 flash 或外置 flash 时使用；

该文件系统能使用的接口如图所示：

```
104 const struct vfs_operations norfs_vfs_ops sec_used(.vfs_operations) = {
105     .fs_type = "norfs",
106     /* .init      = norfs_init_api, */
107     .init      = NULL,
108     .mount      = norfs_monut_api,
109     .openbypath = NULL,
110     .openbyindex = norfs_openbyindex_api,
111     .createfile = norfs_createfile_api,
112     .read       = norfs_read_api,
113     .write      = norfs_write_api,
114     .seek       = norfs_seek_api,
115     .close_fs   = NULL,
116     .close_file = norfs_file_close_api,
117     .fget_attr  = NULL,
118     .name       = norfs_name_api,
119     .ioctl      = norfs_ioctl_api,
120 };
```

图 1.2 norfs 文件系统支持的接口

其中 ioctl 支持的 cmd 命令有：

- ①FS_IOCTL_FILE_TOTAL，同上
- ②FS_IOCTL_FS_TOTAL，同上
- ③FS_IOCTL_FILE_ATTR，同上
- ④FS_IOCTL_FILE_SYNC:

示例：

```
u32 flen = 0;
vfs_ioctl(pfile, FS_IOCTL_FILE_SYNC, (int)&flen);
```

→该命令一般在录音结束时调用，用于更新norfs录音文件头，flen传入时为需要舍弃的录音末尾字节数，

④FS_IOCTL_FILE_INDEX:

示例：

```
int file_index;
vfs_ioctl(pfile, FS_IOCTL_FILE_INDEX, (int)&file_index);
file_index即为pfile文件所在norfs文件系统的最新文件序号；
```

⑤FS_IOCTL_FS_INDEX:

示例：

```
int file_index;
vfs_ioctl(pfs, FS_IOCTL_FS_INDEX, (int)&file_index);
file_index即为pfs所指的norfs文件系统的最新文件序号；
```

FatFs 是一个通用的文件系统，SDK 的 FAT 文件系统驱动包含完整 FAT 驱动、简易 FAT 驱动 (Simple_FAT) 两种，区别如表格所示” *FAT 文件系统简易驱动 & FAT 文件系统完整驱动区别*”；两种 FAT 驱动均不支持对 sector 大小大于 512byte 的 FAT 文件系统进行写操作，读操作正常；Simple_FAT 为 FAT 裁切版，只能通过路径打开文件和读文件接口；完整 FAT 文件系统驱动能使用的接口如下图所示：

```
352 const struct vfs_operations fat_vfs_ops sec_used(.vfs_operations) = {
353     .fs_type = "fat",
354     /* .init = fat_init_api, */
355     .mount = fat_mount_api,
356     .openbypath = fat_openR_api,
357     .createfile = fat_openW_api,
358     .read = fat_read_api,
359     .write = fat_write_api,
360     .seek = fat_seek_api,
361     .close_fs = fat_fs_close_api,
362     .close_file = fat_file_close_api,
363     .fget_attr = fat_attr_api,
364     .name = fat_file_name_api,
365     .flen = fat_get_fsize_api,
366     .ftell = fat_get_pos_api,
367     .fdelete = fat_delete_file_api,
368     /* .openbyindex = fat_openbyindex_api, */
369     /* .openbyclust = fat_openbyclust_api, */
370     .openbyfile = fat_openbyfile_api,
371     .fscan_interrupt = fat_fscan_interrupt_api,
372     .fscan_release = fat_fscan_release_api,
373     .fsel = fat_sel_api,
374     .ioctl = fat_ioctl_api,
375 };
376
```

NORMAL ▶ dev_2.0 app/bsp/common/fs/fat/fat_resource.c

图 1.3 fat 文件系统支持的接口

20.1 FAT 文件系统管理函数

下面的函数为仅 FAT 使用的接口，在 app/bsp/common/fs/fat/fat_resource.c 中的 fat_demo() 接口，可以找到下列函数的使用示例；

20.1.1 函数 int fs_ftell(void *pvfile, void *parm)

此函数实现获取文件读写指针，仅 FAT 可使用该功能。其中参数：

- 1、pvfile：已打开的文件句柄；
- 2、param：用于保存文件系统读写指针；
- 3、返回值：
 - 成功 0；
 - 失败 错误请查看errno-base.h。

20.1.2 函数 int vfs_get_fsize(void *pvfile, void *parm)

此函数实现获取文件大小，仅 FAT 可使用该功能。其中参数：

- 1、pvfile：已打开的文件句柄；
- 2、param：用于保存文件大小的指针；
- 3、返回值：
 - 成功 0；
 - 失败 错误请查看errno-base.h。

20.1.3 函数 struct vfscan *vfs_fscan_new(void *pvfs, const char *path, const char *arg, u8 max_depth, int (*callback)(void), struct vfscan *fsn, struct vfscan_reset_info *info)

此函数实现 FAT 文件系统的文件扫描配置，会返回文件扫描句柄供其他扫描接口使用，与 vfs_fscan_release() 搭配调用，可在 ac104n sdk 中的 play_file.c 中查看该函数具体使用示例。其中参数：

- 1、pvfs：已挂载的文件系统句柄；
- 2、path：文件扫描的起始路径；
- 3、arg：文件扫描配置参数；
 - 例：static const char scan_parm_test[] = “-t”
“MP1MP2MP3WAVTXT”
“-sn -r”
;

- ① 首行“-t”表示需要扫描的文件类型，后面“MP1MP2MP3WAVTXT”即为扫描的后缀名，三个字符为一种类型，最多不
- ② 第三行“-sn”表示扫描文件的顺序是按照文件号排序，另一种组合“-st”表示按照时间排序；
- ③ 第三行“-r”表示搜索会包含文件夹子目录；
- ④ 各种扫描参数配置间，需要以空格隔开，其他配置有：
 - a) 扫描是否将文件夹算入总数，可配置“-d”；

(续下页)

(接上页)

```

        b)扫描只读文件配置” -ar” , 扫描非读文件配置” -a/
→r” , 不配置默认扫描所有文件;
4、max_deep: 扫描深度, 最大为8;
5、callback: 扫描回调;
6、fsn: 文件扫描句柄;
7、info: 扫描结果信息, 包含文件总数和文件夹总数等信息;
8、返回值:
    成功 返回文件扫描句柄;
    失败 错误请查看errno-base.h。

```

20.1.4 函数 void fs_fscan_release(void *pvfs, struct vfscan *fs)

此函数实现释放文件扫描句柄, 仅 FAT 可使用该功能, 与 vfs_fscan() 搭配调用。其中参数:

```

1、pvfs: 已挂载的文件系统句柄;
2、fs: 文件扫描句柄;
3、返回值: 无。

```

20.1.5 函数 int fs_get_folderinfo(void **pvfile, struct vfscan *fs, int *start_num, int *end_num)

此函数实现文件夹扫描, 需使用 vfs_fscan() 返回的文件扫描句柄, 仅 FAT 可使用该功能。其中参数:

```

1、pvfile: 需要扫描的设备里的文件;
2、fs: 文件扫描句柄;
3、Start_num: 用于保存第一个文件夹的第一个文件号;
4、End_num: 用于保存最后一个文件夹的最后一个文件号;
5、返回值:
    成功 0;
    失败 错误请查看errno-base.h。

```

20.1.6 函数 int fs_get_encfolder_info(void *pvfs, char *folder, char *ext, u32 *last_num, u32 *total_num)

此函数实现获取录音文件夹信息, 包括录音文件总数, 仅 FAT 可使用该功能。其中参数:

```

1、pvfs: 已挂载的文件系统句柄;
2、folder: 文件夹路径, 如” /JL_REC” ;
3、ext: 录音文件的后缀名, 如” MP3” ;
4、last_num: 第一个可以使用的录音文件的文件序号;
5、total_num: 总录音文件数;
6、返回值:
    成功 0;
    失败 错误请查看errno-base.h。

```

20.1.7 函数 u32 vfs_file_name(void *pvfile, void *name, u32 len)

此函数实现获取文件的名字。

该函数在获取文件名小于 16byte(带后缀名) 时, 返回的为 u8 字符;

在获取大于 16byte (带后缀名) 时, 返回的是 unicode 码, 且不会返回完整的文件名。

具体需要获取文件长名并最终转化为 u8 字符进行打印出来, 可参考该 issue 步骤:

FAQ: FAT 文件系统下如何获取函数文件名(短文件名 / 长文件名)?

- 1、pvfile: 已打开的文件句柄;
- 2、name: 获取文件名指针。
- 3、len: 获取文件名长度。
- 4、返回值: 文件名长度;

20.1.8 函数 int fs_select(void *pvfs, void **ppvfile, struct vfscan *fs, int sel_mode, int arg)

此函数实现 FAT 文件的文件查找, 支持多种查找方式, 具体示例可查看 fat_demo(void) 接口。其中参数:

- 1、pvfs: 已挂载的文件系统句柄;
- 2、ppvfile: 文件句柄指针, 若句柄指向为 NULL, 函数会自行申请空间;
- 3、fs: 文件扫描句柄;
- 4、sel_mode: 文件查找方式:
 - ① FSEL_BY_NUMBER: 按文件号查找;
 - ② FSEL_BY_SCLUST: 按簇号查找;
 - ③ FSEL_BY_PATH: 按路径查找;
- 5、arg: 文件查找参数;
 - ① 按文件号查找时传入文件号;
 - ② 按簇号查找时传入簇号;
 - ③ 按路径查找时传入相应路径, 支持通配符查找;
- 6、返回值:
 - 成功 0;
 - 失败 错误请查看 errno-base.h。

20.1.9 函数 int vfs_mk_dir(void *pvfs, char *folder, u8 mode)

此函数实现获取创建文件夹, 仅 FAT 可使用该功能。其中参数:

- 1、pvfs: 已挂载的文件系统句柄;
- 2、folder: 文件夹路径, 如 " /JL_REC" ;
- 3、mode: 目录属性;
 - 1: 设置为隐藏属性;
 - 0: 不设置;
- 4、返回值:
 - 成功 0;
 - 失败 错误请查看 errno-base.h。

20.1.10 函数 int vfs_delete_dir(void *pvfs, char *path)

此函数实现获取删除文件夹，仅 FAT 可使用该功能。其中参数：

- 1、pvfs：已挂载的文件系统句柄；
- 2、path：文件夹路径，如” /JL_REC” ；
- 3、返回值：
 - 成功 0；
 - 失败 错误请查看errno-base.h。

使用 demo：

```
log_info("vfs_delete_dir\n");
err = vfs_delete_dir(pvfs, "/test_dir/adir");
if (err != 0) {
    log_error("vfs_delet_dir 0x%x\n", err);
    goto __vfs_delete_fs_close;
}
```

20.1.11 函数 int vfs_get_encfolder_info(void *pvfs, char *folder, char *ext, u32 *last_num, u32 *total_num)

此函数实现获取录音文件夹信息，包括录音文件总数，仅 FAT 可使用该功能。其中参数：

- 1、pvfs：已挂载的文件系统句柄；
- 2、folder：文件夹路径，如” /JL_REC” ；
- 3、ext：录音文件的后缀名，如” MP3” ；
- 4、last_num：第一个可以使用的录音文件的文件序号；
- 5、total_num：总录音文件数；
- 6、返回值：
 - 成功 0；
 - 失败 错误请查看errno-base.h。

20.1.12 函数 u32 vfs_file_delete(void *pvfile)

此函数实现文件的删除，仅 FAT 可使用该功能。其中参数：

- 1、pvfile：已打开的文件句柄；
 - 2、返回值：
 - 成功 0；
 - 失败 错误请查看errno-base.h。
-

20.1.13 函数 `int vfs_select(void *pvfs, void **ppvfile, struct vfscan *fs, int sel_mode, int arg)`

此函数实现 FAT 文件系统的文件查找，支持多种查找方式，具体示例可查看 `fat_demo(void)` 接口。其中参数：

- 1、pvfs：已挂载的文件系统句柄；
- 2、ppvfile：文件句柄指针，若句柄指向为 NULL，函数会自行申请空间；
- 3、fs：文件扫描句柄；
- 4、sel_mode：文件查找方式：
 - ① FSEL_BY_NUMBER：按文件号查找；
 - ② FSEL_BY_SCLUST：按簇号查找；
 - ③ FSEL_BY_PATH：按路径查找；
- 5、arg：文件查找参数；
 - ① 按文件号查找时传入文件号；
 - ② 按簇号查找时传入簇号；
 - ③ 按路径查找时传入相应路径，支持通配符查找；
- 6、返回值：
 - 成功 0；
 - 失败 错误请查看 `errno-base.h`。

20.1.14 函数 `int vfs_ioctl(void *pfile, int cmd, int arg)`

fat 文件系统支持多种命令操作，CMD 命令如下：

- | | |
|---------------------|-----------------------------|
| 1. 创建文件夹 | FS_IOCTL_MK_DIR |
| 2. 获取文件夹序号和文件夹内文件数目 | FS_IOCTL_GET_FOLDER_INFO_3 |
| 3. 获取录音文件信息 | FS_IOCTL_GET_ENCFOLDER_INFO |
| 4. 重置扫描句柄 | FS_IOCTL_RESET_VFSCAN |
| 5. 设置卷标 | FS_IOCTL_SET_VOL |
| 6. 获取分区信息，簇大小，容量 | FS_IOCTL_GET_PARTITION_INFO |
| 7. 获取剩余空间 | FS_IOCTL_GET_FREE_SPACE |
| 8. 获取当前文件的相对路径和绝对路径 | FS_IOCTL_GET_PATH |
| 9. 获取文件系统总容量 | FS_IOCTL_GET_PARTITION_INFO |
| 10. 获取文件系统剩余容量 | FS_IOCTL_GET_FREE_SPACE |

获取当前文件的相对路径和绝对路径 demo：

```
len = vfs_fget_path(pfile, fsn, (u8 *)name, sizeof(name), 0);    //
↪ 获取绝对路径
len = vfs_fget_path(pfile, fsn, (u8 *)name, sizeof(name), 1);    //
↪ 获取相对路径
/*如果路径里的文件夹大于16byte，获取出来的是Unicode码，小于16byte的则为utf8*/
/*返回值为路径长度，name即为获取的路径*/
```

获取剩余空间 demo：

```
u32 cap = 0;
res = vfs_ioctl(pvfs, (int)FS_IOCTL_GET_FREE_SPACE, (int)&cap);
log_info("cap 0x%x\n", res, cap);    //
↪ cap为获取到fat文件系统的剩余容量，单位为k
```

获取分区信息，簇大小，容量 demo：

```

u32 res = 0;
FS_PARTITION_INFO part_info = {0};
res = vfs_ioctl(pvfs, (int)FS_IOCTL_GET_PARTITION_INFO, (int)&part_info);
log_info("0x%x 0x%x 0x%x \n", part_info.clust_size, part_info.total_size, part_
↪info.fs_type);
/*分别对应簇大小，文件系统总容量大小，文件系统类型*/

```

函数 `vfs_ioctl` 获取总容量命令 `FS_IOCTL_GET_PARTITION_INFO`

获取文件系统总容量, 仅 fat 有效, 参考代码:

```

1 FS_PARTITION_INFO f_info;
2 u32 err = vfs_ioctl(pvfile, FS_IOCTL_GET_PARTITION_INFO, (u32) &(f_info))
3 if(0 != err)
4 {
5     // succeed
6 }

```

函数 `vfs_ioctl` 获取剩余容量命令 `FS_IOCTL_GET_FREE_SPACE`

获取文件系统剩余容量, 仅 fat 有效, 参考代码:

```

1 u32 free_apacity_Kbyte; //
2 u32 err = vfs_ioctl(pvfile, FS_IOCTL_GET_PARTITION_INFO, (u32) &free_apacity_Kbyte);
3 if(0 != err)
4 {
5     // succeed
6 }

```

20.1.15 文件夹删除函数 `int vfs_delete_dir(void *pvfs, char *path)`

此函数实现 FAT 文件系统的文件夹删除处理。其中参数:

- 1、pvfs: 已挂载的文件系统句柄;
 - 2、path: 需要删除的文件夹的路径;
- 成功 0;
失败 错误请查看 `errno-base.h`。

20.1.16 格式化接口 `int vfs_format(void **ppvfs, const char *dev_name, const char *type, u32 clust_size, u8 create_new)`

```

int vfs_format(void **ppvfs,
               const char *dev_name,
               const char *type,
               u32 clust_size,
               u8 create_new);

```


此函数用于格式化存储设备并创建文件系统。

重要：重要警告

格式化操作会 ** 永久删除 ** 设备上的所有数据！请确保已备份重要数据后再调用此函数。

参数说明

ppvfs

备注：特殊行为

类型：void **

描述：指向 VFS 指针的指针

格式化操作会释放当前的 VFS 指针：

1. 格式化后，*ppvfs 会被设置为 NULL
 2. 需要重新调用 vfs_mount() 获取新的 VFS 指针
 3. 如果传入 ppvfs == NULL，必须指定有效的 clust_size
-

示例：

```
void *pvfs = NULL;
// 格式化前获取 VFS 指针
vfs_mount(&pvfs, "sd0", "fat");
// 格式化操作
vfs_format(&pvfs, "sd0", "fat", 4*1024, 0);
// 此时 pvfs == NULL，需要重新挂载
```

dev_name

类型：const char *

描述：设备名称

取值示例：

- "sd0" - SD 卡设备
 - "udisk0" - USB 存储设备
 - "ext_flash" - 外部数据 Flash 设备
-

备注：设备可用性

调用前需确保设备已正确连接并识别。

type

类型：const char *

描述：文件系统类型

取值范围：

- "fat" - FAT 文件系统（自动识别为 FAT12、FAT16、FAT32）

小心: 兼容性
不支持 exfat。

clust_size

类型: u32

描述: 簇大小 (字节)

取值范围:

值	含义	备注
0	使用原簇大小	create_new = 0 时使用
1 * 512	512B 簇	适合小文件 (flash 设备优选)
2 * 512	1KB 簇	适合小文件 (flash 设备优选)
4 * 1024	4KB 簇	适合小文件 (flash 设备优选)
8 * 1024	8KB 簇	通用选择
16 * 1024	16KB 簇	适合大文件
32 * 1024	32KB 簇	适合大容量存储
64 * 1024	64KB 簇	适合超大容量存储

重要: 约束条件

- 当 create_new = 1 时, **不能**填 0
- 一般设备建议值: 4KB、8KB、16KB、32KB、64KB
- flash 建议值: 512B、1KB、4KB
- 簇大小影响存储效率和性能

create_new

类型: u8

描述: 创建模式

取值:

值	含义	使用场景
0	维持原文件系统类型	设备已有文件系统, 仅重新格式化
1	新建文件系统类型	设备无文件系统或需要更换类型

备注: 使用场景判断

何时使用 create_new = 1:

1. 新设备首次使用
2. 完全重新组织设备内文件系统的数据结构
3. 设备文件系统损坏无法识别

何时使用 create_new = 0:

1. 设备已有文件系统, 仅需要清空数据

2. 定期维护性格式化
3. 修复文件系统错误

返回值

返回值	含义
0	成功
非 0(FRESULT or -1)	失败

注意事项

警告：数据安全

格式化会永久删除所有数据！

建议的操作流程：

1. 备份重要数据
2. 确认设备名称正确
3. 选择合适的簇大小
4. 执行格式化
5. 重新挂载设备

备注：VFS 指针管理

1. **输入时：***ppvfs 指向当前的 VFS 结构
2. **过程中：**函数会释放该 VFS 结构
3. **输出时：***ppvfs 被设置为 NULL
4. **后续：**必须调用 vfs_mount() 重新获取

小心：参数约束

create_new 和 clust_size 的组合规则：

create_new	clust_size	是否有效
0	0	✓ 有效（使用原簇大小）
0	>0	△ 可能无效（忽略新大小？）
1	0	✗ 无效（必须指定大小）
1	>0	✓ 有效

常见问题

Q: 格式化后为什么需要重新挂载?

A: 格式化会创建新的文件系统结构, 原有的 VFS 上下文已失效。
必须重新挂载以建立新的文件系统连接。

Q: 可以边使用设备边格式化吗?

A: **绝对不行!** 格式化前必须确保:

1. 所有文件已关闭
2. 设备未在被其他进程使用
3. 已卸载设备 (如果之前已挂载)

Q: 如何确定合适的簇大小?

A: 考虑以下因素:

1. **存储设备容量:** 大容量设备可用较大簇
2. **文件平均大小:** 小文件多用小簇
3. **性能需求:** 大簇读写性能更好
4. **空间效率:** 小簇空间浪费少

20.2 FAT 文件系统简易驱动 & FAT 文件系统完整驱动区别

	简易 FAT 文件系统驱动	完整 FAT 文件系统驱动
FAT12 读	×	√
FAT16 读	×	√
FAT32 读	√	√
EXFAT 读	×	√
FAT12 写	×	√
FAT16 写	×	√
FAT32 写	×	√
EXFAT 写	×	√
盘扫描	×	√
长文件名	×	√
短文件名	√	√

CHAPTER 21

freefs 文件系统

freefs 文件系统是用于自定义数据的文件系统，近乎于空的文件系统，可用于在设备中自行管理文件数据。**freefs** 文件系统支持的文件个数以及大小情况如下图所示，如果需要调整文件个数和大小可以直接修改 `free_file_addr_tab`

```

17 #define LOG_TAG_CONST      NORM
18 #define LOG_TAG            "[free_fs]"
19 #include "log.h"
20
21
22
23 #if defined(FREE_FS_EN) && (FREE_FS_EN)
24
25 static const u32 free_file_addr_tab[] = {
26     0,
27     1024 * 4,
28     1024 * 8
29 };
30
31 #define D_FREE_FILE_MAX (sizeof(free_file_addr_tab) / sizeof(free_file_addr_tab[0]))
32
33
34 void freefs_mutex_enter(void)    ///<申请信号量
35 {
36
37 }
38
39 void freefs_mutex_exit(void)    ///<释放信号量
40 {
41
42 }
43
44 /*-----*/
45 /**@brief  提供给FREE文件系统的物理擦除函数，4K单位
46  * @param  hdev: 设备句柄
47  * @param  addr: 设备地址。
48  * @author  liujie
49  * @note    void phy_freefs_eraser(void *hdev, u32 addr)
50  */
51 /*-----*/
52 static void phy_freefs_eraser(void *hdev, u32 addr)
53 {
54     /* log_info(" nor eraser : 0x%x\n", addr); */
55     dev_ioctl(hdev, IOCTL_ERASE_SECTOR, addr);
56 }
57

```

freefs文件系统中的文件个数以及各个文件起始地址

app/bsp/common/fs/free_fs/free_fs.c

图 1.1 freefs 文件系统的文件格式以及各个文件对于设备的起始地址

freefs 在 SDK 中以及注册到文件系统的链表，由于用户可以通过 vfs 接口来使用 freefs 文件系统

```

97 //REGISTER_VFS_OPERATIONS(sydfvfs_ops) = {
98 const struct vfs_operations freefs_vfs_ops sec_used(.vfs_operations) = {
99     .fs_type = "freefs",
100     /* .init      = freefs_init_api, */
101     .init      = NULL,
102     .mount      = freefs_monut_api,
103     .openbypath = NULL,
104     .openbyindex = freefs_openbyindex_api,
105     .createfile = freefs_createfile_api,
106     .read       = freefs_read_api,
107     .write      = freefs_write_api,
108     .seek       = freefs_seek_api,
109     .close_fs   = freefs_close_api,
110     .close_file = freefs_file_close_api,
111     .fget_attr  = NULL,
112     .name       = freefs_name_api,
113     .ioctl      = freefs_ioctl_api,
114 };
115
116 #endif

```

app/bsp/common/fs/free_fs/free_fs_resource.c

图 1.2 freefs 文件系统接口

涉及的代码文件如下：

```
app/bsp/common/fs/free_fs/free_fs.c  
app/bsp/common/fs/free_fs/free_fs_resource.c
```

与过往玩具系列芯片的解码差异比较：

- 1、支持 MMU 的芯片（如 AD23）去掉了 ld 文件中解码使用的代码段，解码算法使用的资源通过 malloc 获取
- 2、本章节为支持 MMU 的芯片（如 AD23）添加了解码资源申请和释放说明。[1 a. 解码功能控制函数](#) [1 b. 各个格式的解码函数](#)

目前 DAC 一直工作在 32K 的采样率，所有解码通路经过重采样之后输出到 DAC。
各种芯片与其支持音频格式如下所示：

格式	后缀名	AD14N	AD15N	AC104N	AD17N	AD18N	AD24N	AD23N
fla	.fla	支持	支持	支持	支持	支持	支持	支持
	.flb .flc .flx							
ump3	.ump3	支持	不支持	支持	不支持	支持	支持	支持
midi	.mid .mfa	支持	支持	支持	支持	支持	支持	支持
a	.a .b .e	支持	支持	支持	支持	支持	支持	支持
MP3	.mp3 .mp2 .mp1	不支持	不支持	支持	不支持	支持	不支持	支持
WAV	.wav	不支持	不支持	支持	不支持	支持	不支持	支持

其中，midi 文件后缀名详情：

工具版本	乐谱后缀名	音色库后缀名
旧版无压缩	.mid	.mid
旧版有压缩	.mid	.mda
新版	.mfa	.mda

目前支持最多支持的解码通路如下：

CPU	两路解码	三路解码
AD14N	f1a / ump3 / midi + a 解码	f1a1 + f1a2 + a 解码
AC104N	f1a / ump3 / midi / 标准 mp3 / wav + a 解码	f1a1 + f1a2 + a 解码
AD15N	f1a / midi + a 解码	不支持
AD17N	f1a / midi + a 解码	不支持
AD18N	f1a / ump3 / midi + a 解码	f1a1 + f1a2 + a 解码
AD24N	f1a / ump3 / midi + a 解码	f1a1 + f1a2 + a 解码

AD23N 的解码使用的 ram 已经全部使用 mmu 进行申请，因此解码可叠加的链路数量仅受解码句柄和算力限制。

解码管理的接口函数：

```
void decoder_init(void);
dec_obj *decoder_io(void *pfile, u32 dec_ctl, dp_buff * dbuff,u8 loop);
void irq_decoder_ret(dec_obj *obj,u32 ret);
```

解码功能控制函数：

```
void decoder_stop(dec_obj *obj, DEC_STOP_WAIT wait);
void decoder_pause(dec_obj *obj);
```

各个格式的解码接口函数：

```
u32 a_decode_api(void *p_file,void **p_dec,void *p_dp_buf)
u32 f1a_decode_api_1(void *p_file,void **p_pdec,void *t_dp_buf)
u32 f1a_decode_api_2(void *p_file,void **p_pdec,void *t_dp_buf)
u32 midi_decode_api(void *p_file,void **ppdec,void *p_dp_buf)
u32 ump3_decode_api(void *p_file, void **p_dec,void *p_dp_buf)
u32 mp3_st_decode_api(void *p_file, void **p_dec, void *p_dp_buf)
u32 mp3_st_decode_api(void *p_file, void **p_dec, void *p_dp_buf)
```

22.1 解码使用一般流程

在开机时需要调用函数 void decoder_init(void)，此函数实现对解码管理器进行初始化，会初始化解码控制器里面的一些变量。

在解码之前要确保已经完成了需要解码的设备，以及设备中的文件进行了初始化。获取到了文件句柄，初始化 DAC 之后便可以开始正常的解码流程：

1、调用 decoder_io 实现对解码器的控制，此函数的传参可以控制：

- a. 对文件在指定的几种解码格式中解析，自动选取正确的格式进行解码；
- b. 控制解码输出的PCM数据结果是否经过音效处理，目前阶段主要是变速变调；
- c. 输入一断点buff，有了此buff才能实现续播功能；
- d. 控制续播的次数；

2、经过上面的一步已经完成了对解码的启动，此时可以调用 decoder_pause、decoder_stop 实现暂停解码与停止解码的功能；

22.2 数据流获取

所有解码格式初始化时，会注册数据输入、输出接口给解码器进行回调。用户可根据需求自行修改数据输入、输出接口。

涉及文件 apps/app/bsp/common/decoder/mp_io.c，涉及的结构体如下：

```
struct if_decoder_io {
    void *priv,
    int(*input)(void *priv, u32 addr, void *buf, int len);
    int(*output)(void *priv, void *data, int len);
};
```

结构体中注册的函数原型：

```
int mp_input(void *priv, u32 addr, void *buf, int len)
{
    dec_obj *obj = priv;
    vfs_seek(obj->p_file, addr, SEEK_SET);
    int rlen = vfs_read(obj->p_file, buf, len);
    return rlen;
}
```

/*

→ 解码开始时调用input接口读取音频文件数据，其中参数addr为文件内的偏移地址，len为读取长度，返回
→ */

```
int mp_output(void *priv, void *data, int len)
```

```
{
    dec_obj *obj = priv;
    return sound_output(&obj->sound, data, len);
}
```

/*

→ 数据解码完成后解码器会调用output接口将解码后的pcm数据写入dac的buff，其中参数data为数据输出的起
→ */

```
26
27
28 const struct if_decoder_io ump3_dec_io0 = {
29     &dec_ump3_hld, //input跟output函数的第一个参数，解码器不做处理，直接回传，可以为NULL
30     mp_input,
31     mp_output,
32 };
33
34 u32 ump3_decode_api(void *p_file, void **p_dec, void *p_dp_buf)
35 {
36     u32 buff_len, i;
37     /* void *name; */
38     /* char name[VFS_FILE_NAME_LEN] = {0}; */
39     decoder_ops_t *ops;
40     log_info("ump3_decode_api\n");
NORMAL +0 ~0 -0 / dev x app/bsp/common/decoder/list/ump3_api.c
```

数据流输入输出结构

22.3 解码管理接口函数

22.3.1 函数 `dec_obj *decoder_io(void *pfile, u32 dec_ctl, dp_buff * dbuff, u8 loop)`

此函数实现对以及获取到的文件进行解码，其中参数：

- 1、void *pfile，已经获取到的文件句柄；
- 2、u32 dec_
 - ctl，解码控制参数，通过对不BIT的置一操作实现对相应功能的控制：
 - ① BIT_UMP3，置上此位为可以将UMP3解码加入到解码轮询中；
 - ② BIT_F1A1/，置上此位为可以将第一路f1a/b/c解码加入到解码轮询中；
 - ③ BIT_F1A2，置上此位为可以将第二路f1a/b/c解码加入到解码轮询中；
 - ④ BIT_MIDI，置上此位为可以将midi解码加入到解码轮询中；
 - ⑤ BIT_A，置上此位为可以将a/b/e解码加入到解码轮询中；
 - ⑥ BIT_MP3_ST，置上此位为可以将标准MP3解码加入到解码轮询中；
 - ⑦ BIT_WAV，置上此位为可以将标准WAV解码加入到解码轮询中；
 - ⑧ BIT_
 - SPEED，置上此位之后，当前这一路解码输出的PCM数据会经过变数变调之后再输出给DAC。
- 3、dp_buff * dbuff，断点缓存，用于续播功能：
 - ① NULL，不能实现续播功能，
 - ② 非NULL，会使用给定的buff实现续播功能
- 4、u8 loop，续播次数控制(0~255)：
 - ① 0，播放完成后不续播；
 - ② 1~244，续播次数；
 - ③ 255，一直持续续播；
- 5、返回值，解码句柄，利用解码句柄可以实现对当前解码通道进行控制（主要是暂停和播放）。

```

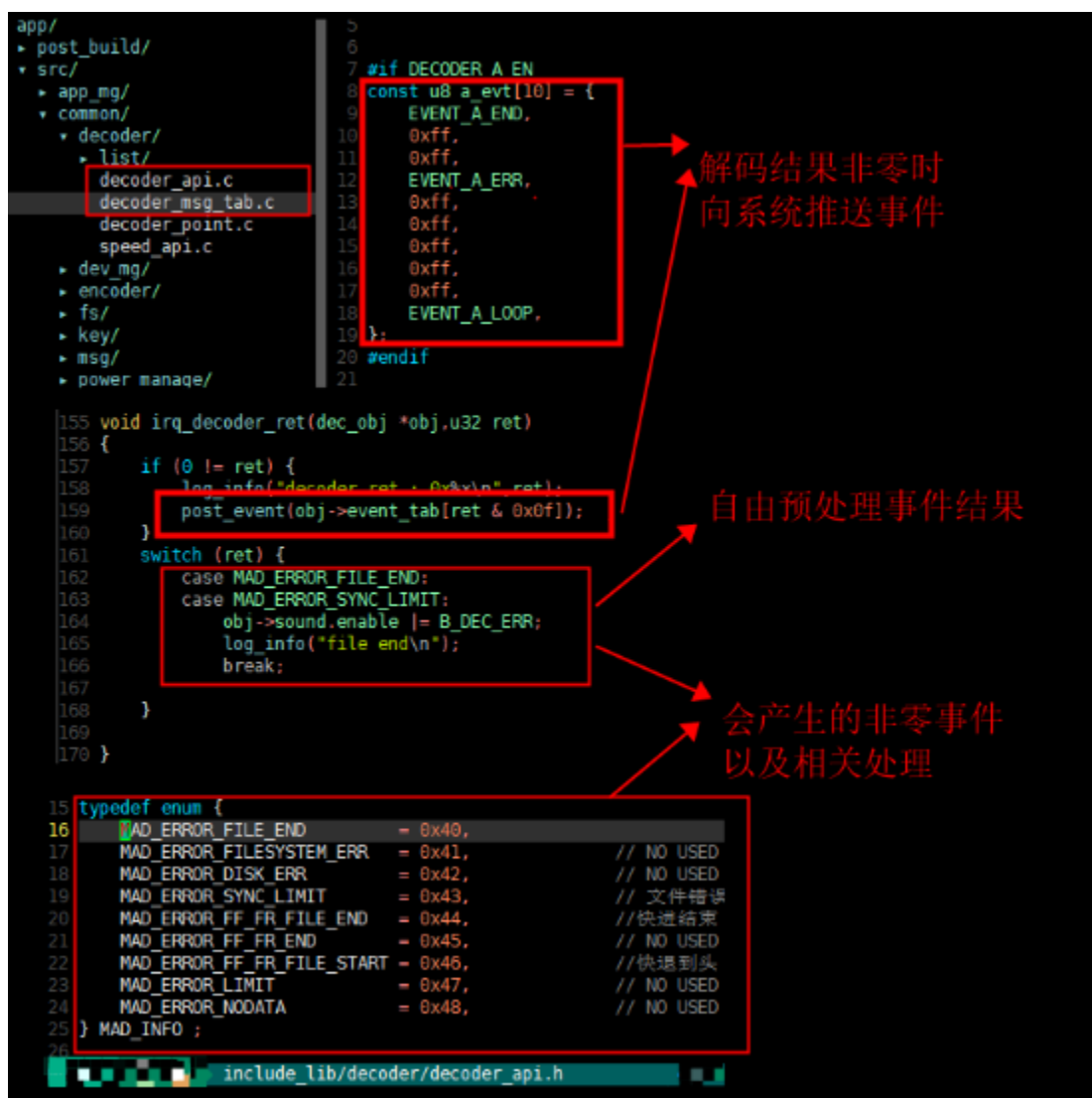
1 #include "typedef.h"
2
3 enum {
4     #if DECODER_F1A_EN
5         INDEX_F1A1 = 0,
6         INDEX_F1A2,
7     #endif
8     #if DECODER_UMP3_EN
9         INDEX_MP3,
10    #endif
11    #if DECODER_A_EN
12        INDEX_A,
13    #endif
14    #if DECODER_MIDI_EN
15        INDEX_MIDI,
16    #endif
17    INDEX_E_SPEED = 12,
18 };
19
20 #define BIT_SPEED BIT(INDEX_E_SPEED)
21
22 #if DECODER_UMP3_EN
23 #define UMP3_HLD (u32)&dec_mp3_hld
24 #define UMP3_LST UMP3_HLD,
25 #define UMP3_API (u32)&mp3_decode_api,
26 #define UMP3_MUI TAB {F1A1_HLD, F1A2_HLD, MIDI_HLD},
27 #define BIT_MP3 BIT(INDEX_MP3)
28 #else
29 #define UMP3_HLD (u32)NULL
30 #define UMP3_LST
31 #define UMP3_API
32 #define UMP3_MUI TAB
33 #define BIT_MP3 0
34 #endif

```

22.3.2 函数 void irq_decoder_ret(dec_obj *obj,u32 ret)

此函数是解码器在解码过程中的回调函数，不能再这里做长时间的操作，因为这样会卡慢解码的速度，其中参数：

- 1、dec_obj *obj，解码句柄
- 2、ret，解码运行的结果，
 - 0为成功，
 - 其他如图
- 3、返回值，无。



22.4 解码功能控制函数

22.4.1 函数 void decoder_pause(dec_obj *obj)

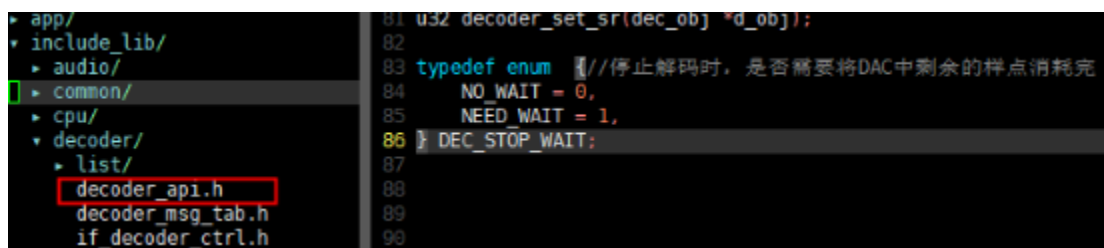
此函数实现在解码过程中对当前解码进行暂停播放与继续播放的控制，其中参数：

1、dec_obj *obj，解码句柄，启动解码时从函数decoder_io的返回值获取得到。

22.4.2 函数 void decoder_stop(dec_obj *obj, DEC_STOP_WAIT wait)

此函数实现停止当前解码的功能，如果使用的是支持 MMU 的芯片（如 AD23），则会释放算法中请到的资源。其中参数：

- 1、dec_obj *obj，解码句柄，启动解码时从函数decoder_io的返回值获取得到。
- 2、DEC_STOP_WAIT wait，是否等待DAC消耗完解码输出的PCM数据后再停止解码
 - ① NO_WAIT，直接停止解码
 - ② NO_WAIT，等待DAC消耗完全部的解码输出数据后再停止解码



22.5 各个格式的解码函数

各种格式的解码启动函数统一被 decoder_io 所调用，其一般名称为：XXX_decode_api。XXX 为其格式名称。

```
u32 a_decode_api(void *p_file, void **p_dec, void *p_dp_buf)
u32 flac_decode_api_1(void *p_file, void **p_pdec, void *t_dp_buf)
u32 flac_decode_api_2(void *p_file, void **p_pdec, void *t_dp_buf)
u32 midi_decode_api(void *p_file, void **ppdec, void *p_dp_buf)
u32 mp3_decode_api(void *p_file, void **p_dec, void *p_dp_buf)
```

22.5.1 统一介绍 decode_api

函数 u32 XXX_decode_api(void *p_file, void **p_dec, void *p_dp_buf):

- ① void *p_file，文件句柄
- ② void **p_dec，存放解码句柄指针的指针
- ③ void *p_dp_buf，断点buff，提供给续播使用，注意MIDI不支持

AD23N 所有格式的 decode_api 在调用时都会申请解码所需要的资源，并且将资源释放函数统一注册到解码句柄，decode_stop 的时候直接通过句柄释放资源。

F1X 格式解码流程说明

23.1 简介

f1x 音频，隶属于 fla 系列格式，能够做到自定义拼接多段音频达到重复播放以及无缝衔接的效果。其中，使用“音频转换工具”可以调整参数改变音频间衔接的效果；代码中可以指定音频的播放循序。

SDK 中默认按照 f1X 内的文件顺序选择文件，再根据 `g_loop_tab[]` 数组的值指定各段音频的循环次数。

注：本文需要使用到“音频文件转换工具 *_1.2.12*”及以上版本，需要 *ADI4N / ADI5N v1.0.7* 和 *AC104N v1.0.5* 及其之后的 *sdk* 支持。

23.2 结构体 / 函数说明

23.2.1 结构体 f1x_ctl

该结构体有以下参数：

- a. Index：当前解码的音频在 f1x 文件的序号，与工具制作 f1x 时输入的文件顺序有关；
- b. Cnt：当前音频的循环次数；
- c. `loop_tab[F1X_MAX_UNIT]`：用户自定义的音频循环次数，默认为 `u8 g_loop_tab[10]`；
- d. `buff[F1X_MAX_UNIT]`：各个音频段的偏移地址；

23.2.2 函数 `void *f1x_play_api(void *pfile, u8 *loop_tab, u32 size, u8 index, u32 addr)`

该函数是 f1x 音频的初始化函数，通过该函数可以获得 f1x 文件各段音频的偏移地址；如有有断点，还会找到断点对应的音频率号；其中参数：

- a. Pfile：文件句柄；
- b. Loop_tab：用户传入的音频循环表的指针，sdk 中传入的是 g_loop_tab[] 数组指针；
- c. Size：用户传入的音频循环表的大小；
注意：g_loop_tab 默认是 10 个成员，与 F1X_MAX_UNIT 对应；若用户音频数量较多，需要同时修改 g_loop_tab[] 数组大小以及 F1X_MAX_UNIT 的值；
- d. Index：f1x 解码通道，类似 f1a 可以同时解 2 路，由解码框架自行分配；
- e. Addr：有断点时，传入的断点信息；
- f. 返回值：成功 f1x 管理句柄指针 &f1x_io；失败 0

23.2.3 函数 `int f1x_parsing(void *pfile, u32 *buff, u32 sum)`

该函数用于解析 f1x 音频文件，获取各段音频的偏移地址；用户可以根据各段音频的偏移地址在函数 set_play_file() 中自定义播放；其中参数：

- a. Pfile：文件句柄；
- b. Buff：用于保存各段音频偏移地址的 buff；
- c. Sum：f1x 文件中的音频总数；
- d. 返回值：成功 0，失败错误请查看 errno-base.h

23.2.4 函数 `u32 set_play_file(void *priv, u32 *startpos, u32 *endpos)`

f1x 经过函数 f1x_play_api() 初始化并播放完第一段音频后，解码器会调用本函数选择下一段要播放的音频；sdk 中是按照 f1x 文件中默认的音频顺序进行播放的，用户可修改该函数自定义播放顺序；其中参数：

- a. Priv：该私有参数传入的是正在解码的 f1x 文件的管理结构体 f1x_ctl；
- b. Startpos：下一段准备解码的音频起始偏移地址，由 f1x_ctl 的 buff 成员提供；
- c. Endpos：下一段准备解码的音频结束偏移地址；
- d. 返回值：播放下一段音频 0，退出播放 -1

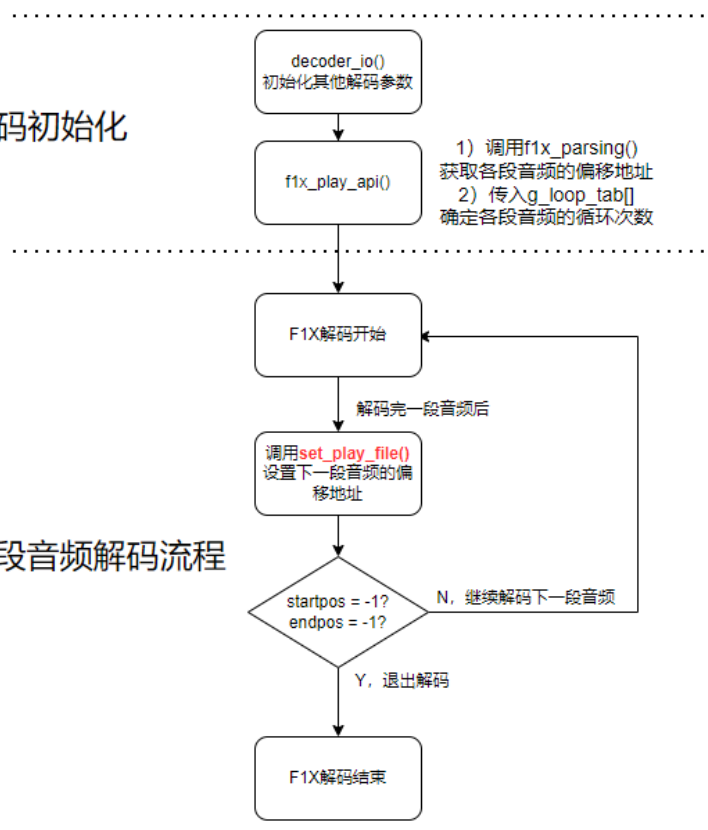
23.3 解码流程

如下图，函数 f1x_play_api() 完成相应初始化操作，包括获取各段音频在 f1x 文件的偏移地址，以及确定各段音频的循环次数。

初始化结束开始播放第一段音频，第一段音频解码结束后会调用函数 set_play_file() 设置接下来要解码的音频偏移地址，以选择要播放的音频。

F1X解码初始化

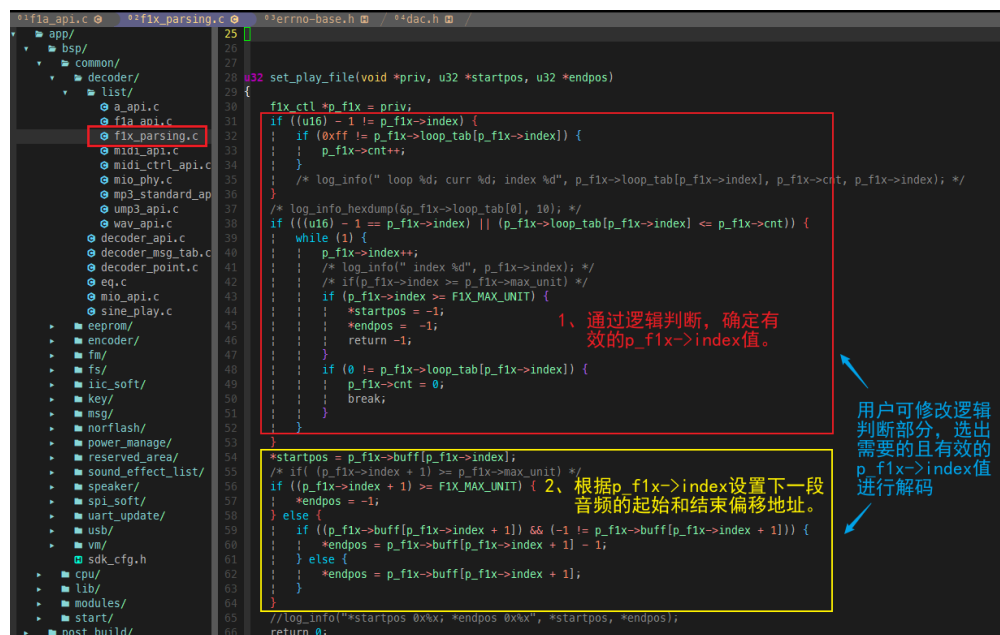
F1X各段音频解码流程



函数 set_play_file() 是 f1x 功能的核心。

SDK 中该函数默认流程是按照制作 f1x 文件时的文件顺序，根据 g_loop_tab[] 中定义的循环次数进行循环播放，不支持乱序播放。

如下图，用户可以修改函数 set_play_file() 中的逻辑判断部分，自定义播放顺序。



各个芯片的编码差异比较:

1、支持 MMU 的芯片（如 AD23）去掉了 ld 文件中编码使用的代码段，编码算法使用的资源通过 malloc 获取 2、本章节为支持 MMU 的芯片（如 AD23）添加了编码资源申请和释放说明。编码功能控制接口函数”

各芯片支持的编码情况如下:

格式	AD14N	AC104N	AD15N	AD17N	AD18N	AD24N	AD23N
支持最高 ADC 采样率	24000	24000	不支持 ADC	不支持 ADC	不支持 ADC	48000	96000
标准 MP2 编码	不支持	支持				不支持	支持
ump2 编码	支持	支持				支持	支持
a 编码	支持	支持				支持	支持

编码前需要配置编码采样率以及码率，各编码格式支持的采样率以及码率如下:

标准 MP2 编码码率 (BR) / 采样率 (SR) 对应关系								
采样率 (sr)	8000	12000	16000	22050	24000	32000	44100	48000
码率 (BR)	+8kbps					32kbps		
	16kbps					48kbps		
	24kbps					56kbps		
	32kbps					64kbps		
	40kbps					80kbps		
	48kbps					96kbps		
	56kbps					112kbps		
	64kbps					128kbps		
	80kbps					160kbps		
	88kbps					160kbps		
	96kbps					224kbps		
	112kbps					256kbps		
	128kbps					320kbps		
	160kbps					384kbps		

ump2 编码采样率 (sr) / 压缩比 (cr) 对应关系								
采样率 (SR)	8000	12000	16000	22050	24000	32000	44100	48000
压缩比 (CR)	3							
	4							
	5							
	6							
	7							
	8							
注: ump2 码率 (br) 的范围为: 采样率 (sr) * 16 / 压缩比 (CR), 其中压缩比的范围为 3 ~ 8; 例如 SR = 24K, 那么 BR 的范围为: (24 * 16 / 8) ~ (24 * 16 / 3)								

A 格式编码码率 (BR) / 采样率 (SR) 对应关系					
采样率 (SR)	8000	12000	16000	24000	32000
码率 (BR)	32kbps	48kbps	64kbps	96kbps	128kbps

所有编码被统一管理。

编码用到的接口函数:

```
void encoder_io(u32(* fun)(void*), void* pfile);
void stop_encode(void* pfile, u32 dlen);
u32 a_encode_api(void* p_file);
u32 ump3_encode_api(void* p_file);
u32 mp3_encode_api(void* p_file);
```

24.1 编码应用流程

在启动录音前需要先调用函数 `rec_phy_init`，此函数实现对编码管理器进行初始化，会初始化编码控制器里面的一些变量资源。

在编码之前要确保已经完成了设备以及文件系统的初始化，以及在设备中创建好了录音文件，并获取到了相应的文件句柄，对于 `NOR_FS` 而言创建文件时能直接返回文件号：

1. 调用 `encoder_io` 实现对编码的启动，此函数会自动适应当前 ADC 工作的频率；
2. 当录音到一定时间后，需要关闭录音可以调用 `stop_encode` 实现停止录音；

注：录音只是支持一路录音，所以不需要获取管理编码句柄，这里会和解码有区别！

24.2 编码功能控制接口函数

24.2.1 void encoder_io(u32(* fun)(void*), void* pfile)

此函数实现注册编码相关中断和启动编码：

1. `u32(* fun)(void* )`，传递不同的编码函数只恨可以实现编写不同格式的音频文件：
 - a. `a_encode_api`，A 格式的编码启动接口函数，如果使用的是支持 MMU 的芯片（如 AD23），则会申请算法需要的资源；
 - b. `mp3_encode_api`，标准 mp2 编码启动接口函数，如果使用的是支持 MMU 的芯片（如 AD23），则会申请算法需要的资源；
 - c. `ump3_encode_api`，ump2 编码启动接口函数，如果使用的是支持 MMU 的芯片（如 AD23），则会申请算法需要的资源；
2. `void* p_file`，创建文件时，获取到的文件句柄。

24.2.2 void stop_encode(void* pfile, u32 dlen)

此函数可以停止编码，如果使用的是支持 MMU 的芯片（如 AD23），则会释放算法申请到的资源：

1. `void* p_file`，创建文件时，获取到的文件句柄；
2. `u32 dlen`，停止解码时，抛弃文件结尾部分数据的长度；

JL_MIO 功能

JL_MIO 用于在音频文件中添加一些波形信息，我司芯片环境中可以使用这些波形信息进行相应的控制。

mio 文件制作详情见 JLMio 使用说明文档，本章介绍 mio 在 SDK 中的应用。

25.1 JLMio 原理

mio 原理是通过 JLMio.exe 工具将对应歌曲自定义对应时刻生成对应 pwm 通道或者 IO 通道的波形信息文件。

小机通过播放该歌曲的时候，读取 mio 文件，获取出歌曲对应位置的波形信息（如 pwm 通道对应时刻的周期、IO 通道对应时刻的电平）

获取出对应的波形信息后，通过相应的 run 接口去设置对应 timer 的 pwm 周期或者 io 的电平。

备注：

mio 的 pwm 通道默认使用 timer0，且在 SDK 里默认限制了 PWM 通道（MIO_MAX_CHL_PWM）的个数，用户可以自行调整 pwm 通道个数和修改初始化、run 接口。

同理 IO 通道（MIO_MAX_CHL_IO）

整个 mio 驱动均已开放，用户可根据自身实际情况修改初始化或 run 函数。

25.2 JLMio 一般流程

25.2.1 使用 JLMio.exe 创建 mio 文件

详情见 JLMio 使用说明。生成的 mio 文件会在工具目录的 Data 文件夹下；

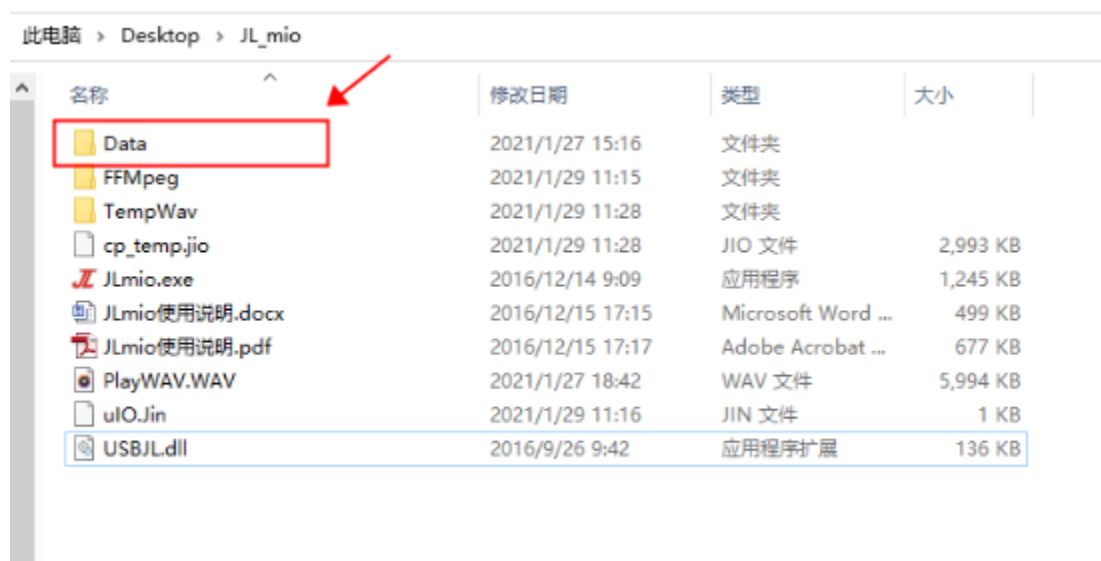


图 1 mio 文件生成位置

25.2.2 使用打包工具将音频文件与所制成的 mio 文件放于同一目录下

打包工具请参考 2.3 打包工具使用，注意：音频文件需要与生成的 mio 文件放于同一目录下；生成目录文件时，不能将 mio 放在目录的第一个文件位置处，否则音乐播放可能会失败！

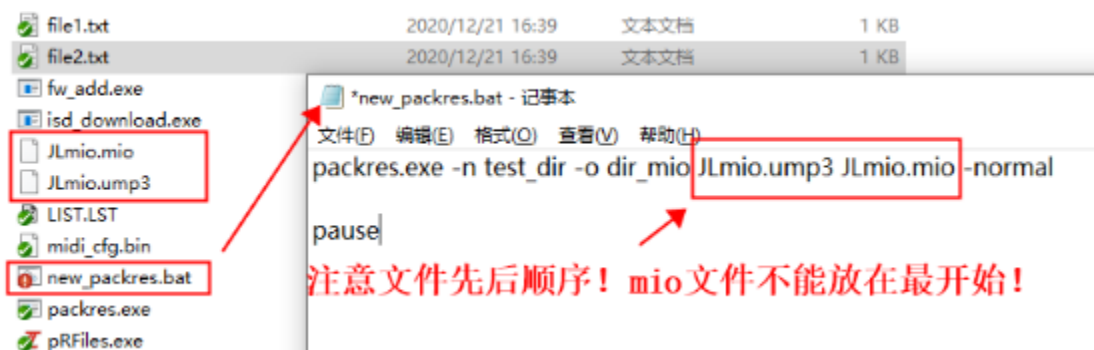


图 2 mio 文件打包位置顺序注意事项

dir_mio																ANSI ASCII	
Offset	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	
0	40	41	4A	99	20	00	00	00	10	48	22	00	03	FF	00	00	@AJ" H" y
16	74	65	73	74	5F	64	69	72	00	FF	FF	FF	FF	FF	FF	FF	test_dir yyyyyyy
32	77	C3	A2	59	60	00	00	00	B0	8D	19	00	02	FF	00	00	wAeY" " y
48	4A	4C	6D	69	6F	2E	75	6D	70	33	00	FF	FF	FF	FF	FF	JLmio.ump3 yyyyyy
64	CD	5C	A8	15	10	8E	19	00	F8	B9	08	00	02	FF	01	00	I\ " Z " " y
80	4A	4C	6D	69	6F	2E	6D	69	6F	00	FF	FF	FF	FF	FF	FF	JLmio.mio yyyyyyy
96	6D	54	53	02	08	00	00	00	00	00	00	00	00	80	CD		mIS ei
112	DC	92	00	00	C0	66	6E	49	00	00	E0	4B	03	00	00	80	U' ÅfnI aK e
128	D2	00	00	00	20	6E	0A	62	02	00	00	37	05	31	01	00	ô n b 7 1
144	80	2F	0D	00	00	00	4A	03	00	00	80	B8	29	88	09	00	€/ J e,) ^
160	00	DC	14	C4	04	00	00	BE	34	00	00	00	28	0D	00	00	Ü Å ¼ (

保证mio文件不会放于文件夹最开头即可！

图 3 mio 文件正确打包位置顺序示意图

25.2.3 在 download.bat 中添加相应 mio 文件夹

在 download.bat 中添加所打包的包含音频文件与 mio 文件的文件夹目录 dir_mio，放在 -res 参数后面即可；



图 4 将 dir_mio 文件夹添加进下载目录中

25.2.4 在 app_music.c 中添加相应 mio 文件夹

在 app_music 中添加所打包的包含音频文件与 mio 文件的文件夹目录，可放在任意位置并用按键进行文件夹切换；

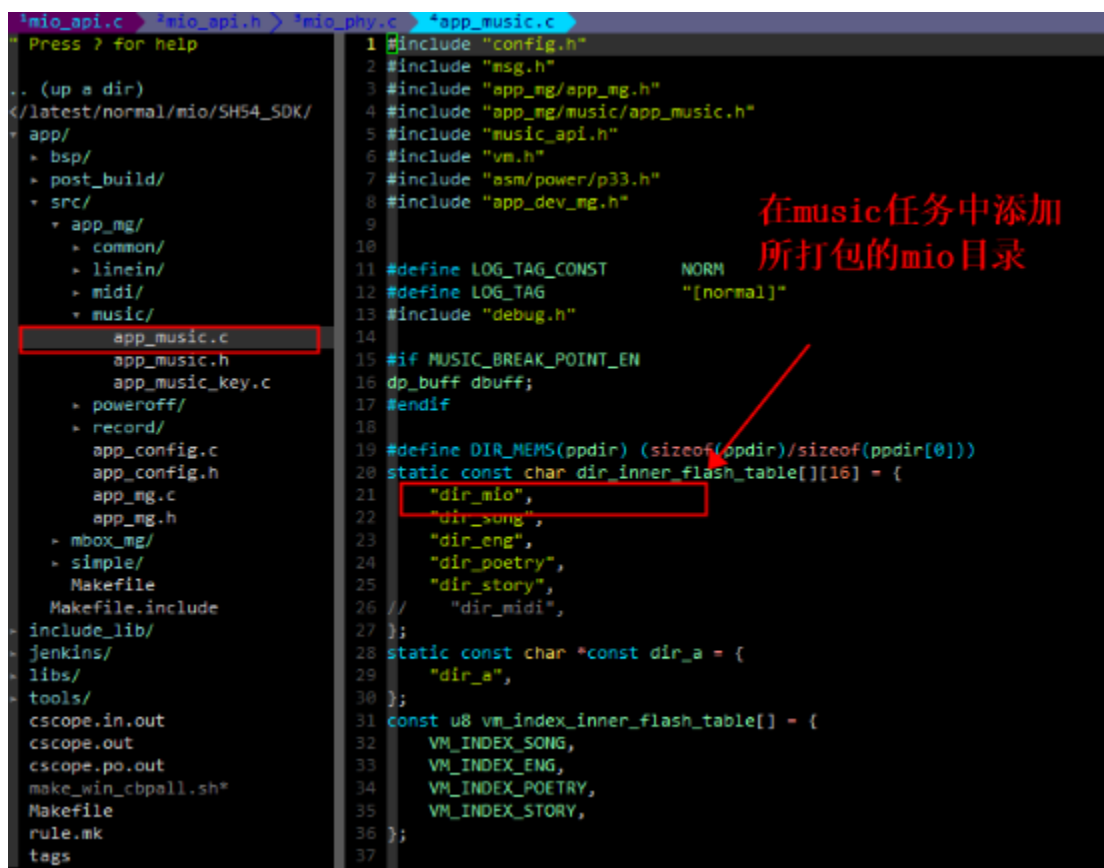


图 5 在 app_music 中添加 mio 文件夹

25.2.5 在 mio_api.c 中将相应宏开关打开

在 mio_api 中将 MIO_EN 宏定义打开即可，SDK 默认使用 1 路 PWM (PA12) 以及 2 路 IO 口 (PA1 与 PA2) 作为示例，用户可自行添加并完善功能；

注意：

1. mio 功能会占用相应引脚，若不用 mio 功能需关闭改宏开关。
2. MIO_IO 引脚与偏移配置如下图意思为 PA1 开始，若 mio 文件有 3 个 IO 通道，则从 PA1 开始往下设置，PA1、PA2、PA3 作为 MIO 的 IO 通道输出。

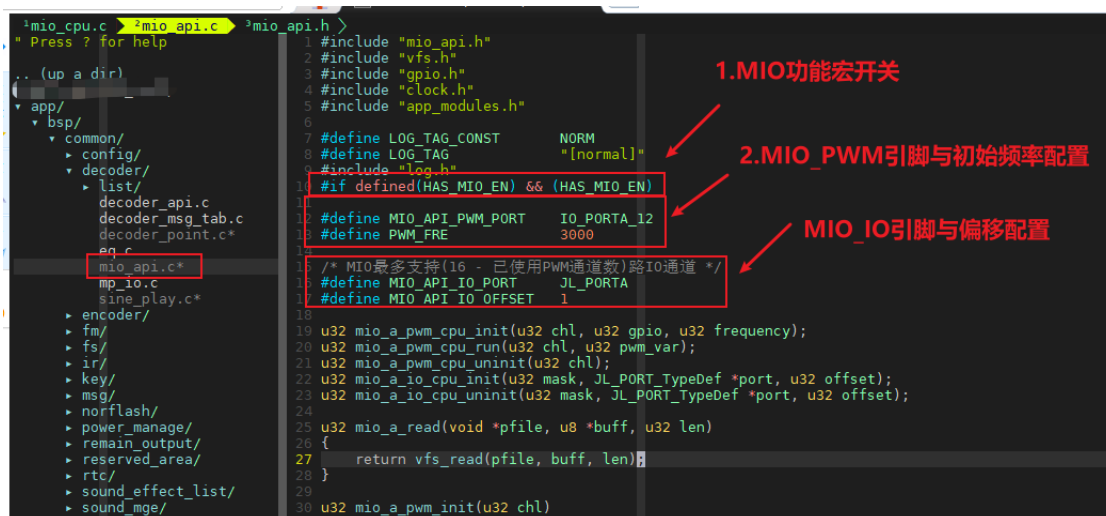


图 6 mio_pwm 与 mio_io 宏定义配置

25.3 JLMio 接口介绍

25.3.1 u32 mio_a_pwm_init(u32 chl)

此函数实现 mio_pwm 通道输出的初始化，其中参数：

- 1、chl：需要进行初始化的pwm通道编号；
- 2、返回值：0：成功；其他值：errno-base.h查看

25.3.2 u32 mio_a_pwm_run(u32 chl, u32 pwm_var)

此函数实现 mio_pwm 通道输出，其中参数：

- 1、chl：正在输出的pwm通道编号；
- 2、pwm_var：mio文件中读取到的pwm数值，可用于配置pwm输出占空比；
- 3、返回值：0：成功；其他值：errno-base.h查看

25.3.3 u32 mio_a_pwm_uninit(u32 chl)

此函数实现 mio_pwm 通道输出的注销，其中参数：

- 1、chl：需要进行注销的pwm通道编号；
- 2、返回值：0：成功；其他值：errno-base.h查看

25.3.4 u32 mio_a_io_init(u32 mask)

此函数实现 mio_io 通道输出的初始化，其中参数：

- 1、mask：需要进行初始化的io通道（mio最多支持（16-pwm通道）路io通道）；
 - ① mio文件中设置的每一个io通道依次对应mask中每个位；
 - ② 如单io通道mask会传入0x01，双io通道mask会传入0x3，以此类推；
- 2、返回值：0：成功；其他值：errno-base.h查看

25.3.5 u32 mio_a_io_run(u32 mask, u32 io_ver)

此函数实现 mio_io 通道输出，其中参数：

- 1、mask：需要进行初始化的io通道；
- 2、io_ver：mio文件中读取到的io数值，可用于配置IO口的电平；
- 3、返回值：0：成功；其他值：errno-base.h查看

25.3.6 u32 mio_a_io_uninit(u32 mask)

此函数实现 mio_io 通道输出的注销，其中参数：

- 1、mask：需要进行注销的io通道（mio最多支持（16-pwm通道）路io通道）；
 - ① mio文件中设置的每一个io通道依次对应mask中每个位；
 - ② 如单io通道mask会传入0x01，双io通道mask会传入0x3，以此类推；
- 2、返回值：0：成功；其他值：errno-base.h查看

玩具应用篇 (voice_toy)

26.1 简介

2023 年 7 月 11 日发布的 AD1x-457_AC104-release_v1.7.0 SDK，新增了 voice_toy 工程；

该工程在旧版玩具工程 app_mg 与示例工程 simple 基础上进行优化，使应用示例更加简单、直观，便于用户上手开发；

各芯片支持的应用模式如下：

应用模式	AD24N	AD14N	AD15N	AD17N	AD18N
toy_music	√	√	√	√	√
toy_midi	√	√	√	√	√
toy_midi_keyboard	√	√	√	√	√
toy_record	√	√			
toy_speaker	√	√			
toy_linein	√	√			
toy_softoff	√	√	√	√	√
toy_idle(demo)	√	√	√	√	√

不同的模式间会有资源和按键互斥的关系。应用上面做了一个简单的模式切换管理来管理不同模式的资源和按键。

模式管理的整体管理思路是，进入该模式的时候初始化模式需要使用的资源和按键处理函数，退出该模式的时候关闭该模式需要使用的资源和释放按键处理函数。

模式初始化以及模式切换函数：

1. void app(void);
2. void app_next_mode(void)

模式管理接口的文件摆布：

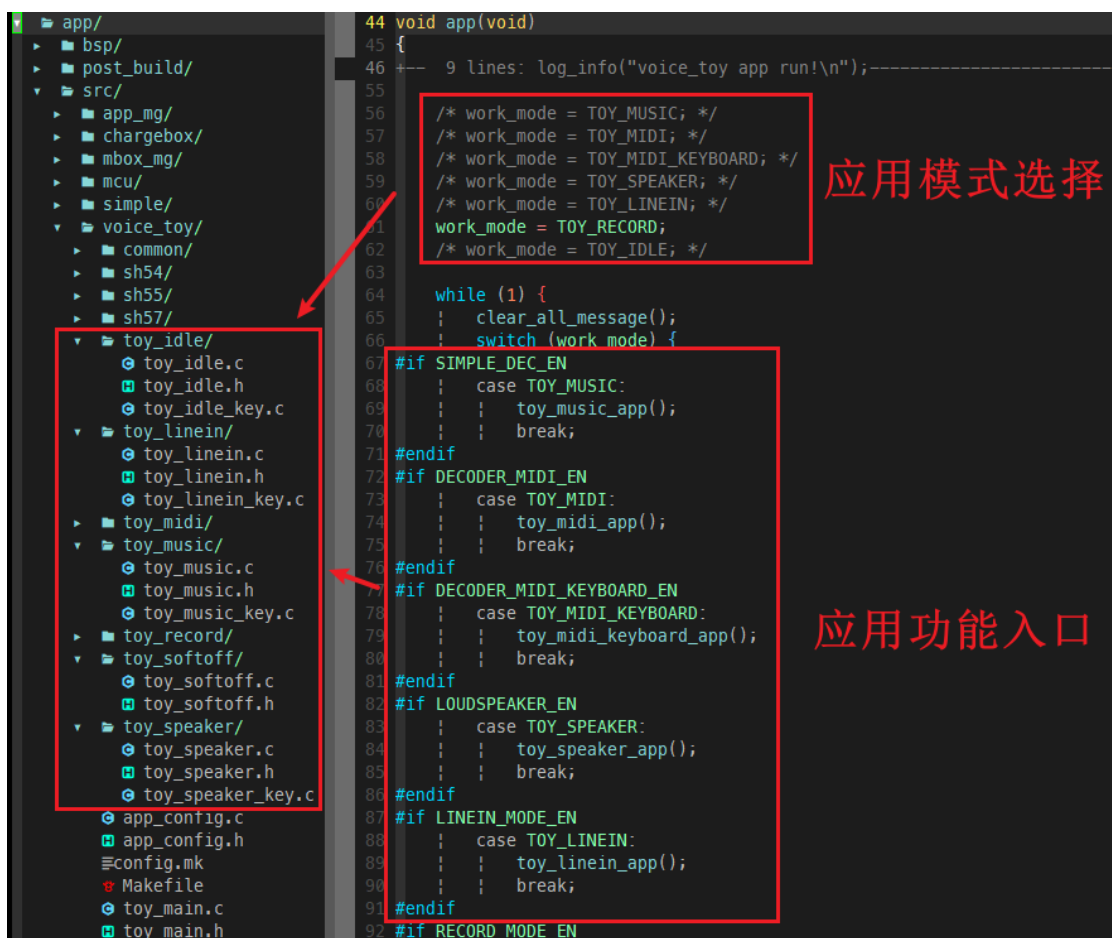


图 1 voice_toy 应用模式管理

26.1.1 模式相关函数

函数 `void app_next_mode(void)`

此函数切换下一个应用模式，上一个应用切换前需要释放申请的资源：

26.2 toy_music 多路解码模式

26.2.1 模式说明

- 多路解码模式使用了 SYD 文件系统，能够播放打包成 dir 文件的 F1A/B/C/X、UMP3，并叠加 A 文件解码，同时支持外挂 FLASH 解码。
- 该模式还具备 VM 记忆，可记录解码文件序号与断点。
- 多路解码使用 `play_control` 结构体，其成员如下：

```
typedef struct __play_control {
void *pfs;           //文件系统句柄
void *pfile;         //文件句柄
dec_obj *p_dec_obj;  //解码器句柄
void *pdp;           //断点指针
u32  findex;         //当前播放的文件序号
u32  ftotal;         //当前文件夹中的文件总数
u16  dec_type;       //解码器支持的解码类型, 如BIT_F1A1
u8   dev_index;      //当前挂载设备的序号, 如INNER_FLASH_RO
u8   play_mode;      //播放模式 (Music模式使用)
void *device;        //当前挂载设备的句柄
void *pdir;          //文件夹指针, 用于切换文件夹
u8   *p_vm_tab;      //存放vm项指针, 用于记忆不同文件夹的掉电数据
u8   dir_total;      //文件夹总数
u8   dir_index;      //当前播放的文件夹序号
u8   loop;           //播放循环次数
// u8   flag;        //标志位 (Music模式使用)
} play_control;
```

- 涉及的文件目录: app/src/voice_toy/toy_music, 涉及的应用函数如下:

26.2.2 解码播放函数

函数 u32 simple_dev_fs_mount(play_control *ppctl)

此函数是多路解码挂载设备和 SYD 文件系统, 其中参数:

1. ppctl: 音乐播放器句柄指针;
2. 返回值:
 - 成功 返回0;
 - 失败 错误值可查看errno-base.h;

函数 u32 simple_dev_fs_close(play_control *ppctl)

此函数实现多路解码释放文件、文件系统句柄, 并关闭设备; 释放的句柄分别是结构体 play_control 中的成员 pfile, pfs 以及 device, 其中参数:

1. ppctl: 音乐播放器句柄指针;
2. 返回值: 0;

函数 u32 get_decoder_status(dec_obj *obj)

此函数实现获取多路解码当前的状态, 分别有播放停止 (MUSIC_STOP)、播放暂停 (MUSIC_PAUSE) 以及播放中 (MUSIC_PLAY) 三种状态, 其中参数:

1. obj: 解码句柄;
2. 返回值: 返回当前解码器播放状态;

函数 u32 simple_play_file_bypath(play_control *ppctl, const char *path)

此函数实现通过传入的路径播放文件，其中参数：

1. ppctl: 音乐播放器句柄指针；
2. path: 音频文件路径（对于外挂flash，忽略第一级目录）；
3. 返回值：
 - 成功 返回0；
 - 失败 错误值可查看errno-base.h；

函数 u32 simple_play_file_byindex(play_control *ppctl, u32 index)

此函数实现通过传入的序号播放文件，ppctl 中的 pdir 为需要播放的文件夹路径，其中参数：

1. ppctl: 音乐播放器句柄指针；
2. index: 音频文件序号；
3. 返回值：
 - 成功 返回0；
 - 失败 错误值可查看errno-base.h；

函数 u32 play_one_file(play_control *ppctl)

此函数实现通过 ppctl 中的 findex 参数播放文件，播放成功后记录 VM 数据，该接口由应用层调用，其中参数

1. ppctl: 音乐播放器句柄指针；
2. 返回值：
 - 成功 返回0；
 - 失败 错误值可查看errno-base.h；

函数 u32 play_prev_file(play_control *ppctl)

此函数实现播放上一首音频，播放成功后记录 VM 数据，该接口由应用层调用，其中参数

1. ppctl: 音乐播放器句柄指针；
2. 返回值：
 - 成功 返回0；
 - 失败 错误值可查看errno-base.h；

函数 u32 play_next_file(play_control *ppctl)

此函数实现播放下一首音频，播放成功后记录 VM 数据，该接口由应用层调用，其中参数

1. ppctl: 音乐播放器句柄指针；
2. 返回值：
 - 成功 返回0；
 - 失败 错误值可查看errno-base.h；

函数 u32 simple_switch_device(play_control *ppctl)

此函数实现切换播放设备（内置-外挂 flash 互相切换），该接口由应用层调用，其中参数

1. ppctl: 音乐播放器句柄指针；
2. 返回值：
成功 返回0；
失败 错误值可查看errno-base.h；

函数 u32 simple_next_dir(play_control *ppctl)

此函数实现切换下一个文件夹，该接口由应用层调用，其中参数

1. ppctl: 音乐播放器句柄指针；
2. 返回值：
成功 返回0；
失败 错误值可查看errno-base.h；

函数 int simple_play_speed_set(play_control *ppctl, SPEED_PITCH_PARA_STRUCT *sp_parm)

此函数实现在播放过程中动态调整变速变调参数，该接口需要在链接变速变调音效后才可在应用层使用，其中参数:

1. ppctl: 音乐播放器句柄指针；
2. 变速变调参数：其结构体成员具体可查看音效变速变调章节；
3. 返回值：
成功 返回0；
失败 返回-1；

26.3 toy_midi midi 解码模式

26.3.1 模式说明

- midi 解码模式使用了 SYD 文件系统，能够播放打包生成 dir 的 midi 文件，并叠加 A 文件解码。
- midi 解码同样使用了 play_control 结构体作为播放句柄。

26.3.2 相关函数

函数 void midi_decode_init(void)

该函数实现查找 midi 解码的音色库并记录音色库信息，在系统初始化中调用；

函数 void midi_dec_config(dec_obj *p_dec_pbj, u32 cmd, void *parm)

此函数实现查找 midi 解码的功能参数，cmd 的具体介绍请看 midi 文件第 2 节 MIDI 乐谱解码介绍，其中参数码的音色库并记录音色库信息，在系统初始化中调用；

函数 void midi_mode_set(dec_obj *p_dec_pbj, u8 mode)

该函数实现配置 midi 解码的模式，其中参数：

- 1、p_dec_obj: midi 解码句柄；
- 2、mode: midi 解码模式，可用 mode 如下：
 - ① CMD_MIDI_CTRL_MODE_0，正常解码模式；
 - ② CMD_MIDI_CTRL_MODE_1，one key one note 模式；
 - ③ CMD_MIDI_CTRL_MODE_2，只推消息不出声模式；
 - ④ CMD_MIDI_CTRL_MODE_W2S，外部音源模式；

函数 void midi_okon_goon(dec_obj *p_dec_pbj)

该函数实现 midi 解码 okon 模式启动下一个音符，其中参数：

- 1、p_dec_obj: midi 解码句柄

函数 void midi_init_info(MIDI_INIT_STRCUT *init_info)

该函数实现 midi 解码配置初始化，会给 midi 解码器主动调用，其中参数：

- 1、init_info: midi 解码初始化配置结构体，具体请参照代码中的配置信息

26.4 toy_midi_keyboard midi 琴模式

26.4.1 模式说明

- MIDI 琴功能支持通过函数驱动 midi 解码。该应用依赖我司自定义的 mid 音色库，用户可通过我司 mid 工具进行转换，详情请看《JLmidi 工具使用说明》。

26.4.2 相关函数

函数 void midi_ctrl_decode_init(void)

该函数实现查找 midi 琴的音色库并记录音色库信息；

函数 u32 toy_midi_ctrl_set_prog(dec_obj *obj, u8 prog, u8 chn)

该函数实现查找设置 midi 琴的乐器，0 其中参数：

- 1.obj: midi 琴解码句柄；
- 2.prog: 乐器号，与midi音色库制作工具中的乐器号一致。详情请看《杰理AD14 AD15N_→MIDI应用说明文档》说明；
- 3.chn: 发声通道，该值范围在0~15；
- 4.返回值：成功 0；
失败 其他值；

函数 u32 toy_midi_ctrl_pitch_bend(dec_obj *obj, u16 pitch_val, u8 chn)

该函数实现配置 midi 琴弯音轮参数，其中参数：

- 1.obj: midi 琴解码句柄；
- 2.pitch_val: 弯音值，默认值是256，可根据该值调整；
- 3.chn: 发声通道，该值范围在0~15；
- 4.返回值：成功 0；
失败 其他值；

函数 u32 toy_midi_ctrl_dec_config_api(dec_obj *obj, u32 cmd, void *parm)

该函数实现配置 midi 琴的其他功能，其功能参数和 midi 解码中的一致，其中参数：

- 1.obj: midi 琴解码句柄；
- 2.cmd: 可用命令如下：
CMD_MIDI_CTRL_TEMPO;
CMD_MIDI_SET_EX_VOL;
CMD_INIT_CONFIGS;
CMD_MIDI_SET_SEMITONE;
CMD_MIDI_SET_SWITCH;
- 3.parm: 命令参数，可查看midi解码中的介绍；
- 4.返回值：成功 0；
失败 其他值

函数 u32 toy_midi_ctrl_note_on(dec_obj *obj, u8 nkey, u8 nvel, u8 chn)

该函数实现弹奏指定通道的音符，其中参数：

- 1.obj: midi 琴解码句柄；
- 2.nkey: 音符，与midi音色库制作工具中的音符号一致，范围在0~127；
- 3.nvel: 琴键按下力度，范围在0~→127；部分乐器（如钢琴）有效，不同力度会使按下后的余音长短不同；
- 4.chn: 发声通道，该值范围在0~15；
- 5.返回值：成功 0；
失败 其他值；

函数 u32 toy_midi_ctrl_note_off(dec_obj *obj, u8 nkey, u8 chn, u16 decay_time)

该函数实现查找设置 midi 琴的乐器。对于声音不会衰弱的乐器（如笛子），调用 Note_on 函数后需要调用该函数以结束音符，对于声音会衰弱的乐器（如钢琴）可不用。其中参数：

- 1.obj: midi 琴解码句柄；
- 2.nkey: 音符，与midi音色库制作工具中的音符号一致，范围在0~127；
- 3.chn: 发声通道，该值范围在0~15；
- 4.decay_time: 衰减时间，单位ms，若为0则使用音色自带的衰减；
- 5.返回值：成功 0；
失败 其他值；

函数 u32 toy_midi_ctrl_vel_vibrate(dec_obj *obj, u8 nkey, u8 vel_step, u8 vel_rate, u8 chn)

该函数实现配置音符的颤音效果，其中参数：

- 1.obj: midi 琴解码句柄；
- 2.nkey: 音符，与midi音色库制作工具中的音符号一致，范围在0~127；
- 3.vel_step: 抖动步长，该值范围在0~7；
- 4.vel_rate: 抖动幅度，该值范围在0~15；
- 5.chn: 发声通道，该值范围在0~15；
- 6.返回值：成功 0；
失败 其他值；

函数 u32 midi_ctrl_melody_stop_trigger(void *priv, u8 key)

该函数实现停止 midi 琴解码，在所有音符停止后会被调用，其中参数：

- 1.obj: midi 琴解码句柄；
- 2.返回值：无；

函数 void midi_ctrl_callback_init(dec_obj *obj, u32(*melody_callback)(void *, u8, u8), u32(*melody_stop_callback)(void *, u8))

该函数实现配置音符开始与结束的回调函数，其中参数：

- 1.obj: midi 琴解码句柄；
- 2.melody_callback: 音符开始回调函数，回调函数的原型参照《杰理AD14 AD15N_↪MIDI应用说明文档》1.2.6.1主旋律的音符回调功能中的说明；
- 1.melody_stop_callback: 音符结束回调函数，回调函数的原型参照《杰理AD14 AD15N_↪MIDI应用说明文档》1.2.6.2主旋律的音符结束回调功能中的说明；
- 2.返回值：无；

函数 void toy_midi_ctrl_get_play_key(dec_obj *obj, u8 chn)

该函数实现获取指定通道正在播放的所有 key 值，其中参数：

1.obj: midi 琴解码句柄；
2.chn: 需要查询的通道；
返回值：无；

26.5 toy_speaker 扩音模式

26.5.1 模式说明

- 扩音模式实现将 mic 采集到的数据直通或者串联算法输出到 dac。
- 扩音模式算法支持的算法详细见音效算法章节。

26.5.2 相关函数

函数 void audio_adc_speaker_start(void)

该函数实现扩音模式初始化 adc 和 dac，并且串联算法的整体流程，初始化扩音模式时使用；

函数 void audio_adc_speaker_reless(void)

该函数实现扩音模式断开 adc 和 dac 数据流链接流程，并且释放 src 变采样申请使用的资源，在关闭扩音模式时使用；

26.6 toy_linein linein 模式

26.6.1 模式说明

- 该模式实现数字 linein，该模式下需要将 AUDIO_ADC 和 AUDIO_DAC 配置到相同的采样率，可调节音量
-

26.7 toy_record 录音模式

26.7.1 模式说明

- 录音模式实现将 mic 采集到的数据进行编码，并通过 norfs 文件系统方式保存至内置 flash(放置代码的 flash)
 - 支持 A 格式和 ump3 格式编码
-

- 录音模式使用 Encode_Control 结构体，其成员如下：

```
typedef struct _Encode_Control {
char fs_name[8];           //编码文件系统名
void *device;              //编码设备句柄
void *pfs;                 //编码文件系统句柄
void *pfile;               //编码文件句柄
union {
    u32 rec_file_index;    //nor_fs最新文件号
};
u8 dev_index;              //编码设备序号
ENC_STATUS enc_status;     //编码器工作状态
} Encode_Control;
```

26.7.2 相关函数

函数 static int encode_start(Encode_Control *obj)

该函数实现录音功能启动。内部会开启 ADC 模块和整个录音的设备和文件流程；

函数 int norfs_enc_file_create(Encode_Control *obj)

该函数实现录音模式创建录音文件，内部包含开设备、挂载 norfs 文件系统和创建文件流程，其参数：

- 1、obj：录音句柄；
- 2、返回值：
 - 成功 返回0；
 - 失败 其他值；

函数 void encoder_stop(Encode_Control *obj)

该函数实现录音模式停止编码，内部会关闭编码、写文件流程和 ADC 模块；

函数 dec_obj *norfs_enc_file_decode(Encode_Control *obj, u16 dec_type)

该函数实现录音模式播放录音文件，内部包含开设备、挂载 norfs 文件系统和创建文件流程，其参数：

- 1、obj：录音句柄；
- 2、dec_type：解码格式；
- 3、返回值：
 - 成功 返回0；
 - 失败 其他值；

玩具应用篇（旧工程 - app_mg）

27.1 简介

该工程在 2023 年 7 月 11 日发布的 AD1x-457_AC104-release_v1.7.0 SDK 后停止维护，后续仅维护 voice_toy 工程；

该工程从 2025 年 4 月发布的 AD24N-release_v1.1.0 SDK 版本开始，将应用核心代码移到 voice_func 文件夹，仅将工程相关配置文件保留在 voice_toy 目录；

AD140，AD150 的应用模式有 MIDI 模式、音乐模式、录音模式和关机模式，不同的模式间会有资源和按键互斥的关系。应用上面做了一个简单的模式切换管理来管理不同模式的资源和按键。

模式管理的整体管理思路是，进入该模式的时候初始化模式需要使用的资源和按键处理函数，退出该模式的时候关闭该模式需要使用的资源和释放按键处理函数。

模式初始化函数：

1. int app_mg_init(APP_T app,void* param);
2. void app(void);

模式切换函数：

1. int app_switch(APP_T app,void* param);
2. int app_switch_next(void);
3. int app_switch_priv(void);
4. u8 app_get(void);

模式管理接口的文件摆布：

```
src/  
  ▼ app_mg/  
    ▼ common/  
      ▶ music_api/  
        common_msg.c  
        idle_check.c  
    ▼ midi/  
      app_midi.c  
      app_midi.h  
      app_midi_key.c  
    ▼ music/  
      app_music.c  
      app_music.h  
      app_music_key.c  
    ▼ poweroff/  
      app_poweroff.c  
      app_poweroff.h  
      app_poweroff_key.c  
    ▼ record/  
      app_record.c  
      app_record.h  
      app_record_key.c  
  app_mg.c  
  app_mg.h
```

图 1 模式管理文件摆放

27.2 模式初始化函数

27.2.1 函数 int app_mg_init(APP_T app,void* param)

此函数初始化了第一个模式: 1. app : 初始化的模式; 2. param: 初始化的模式参数;

27.2.2 函数 void app(void)

此函数为模式管理的入口函数, 并且模式间的切换由该函数完成。

27.3 模式切换函数

27.3.1 函数 `int app_switch(APP_T app,void* param)`

此函数可以切换到指定的模式:

1. `app`: 指定的 `app` 模式;
2. `param`: 切换模式参数;

27.3.2 函数 `int app_switch_next(void)`

此函数用来自动切换 `app` 到下一个模式, 模式列表看 `APP_T`。

27.3.3 函数 `int app_switch_priv(void)`

此函数用来自动切换 `app` 到上一个模式, 模式列表看 `APP_T`。

27.3.4 函数 `u8 app_get(void)`

此函数用来获取当前的 `app` 是在哪个模式。

27.4 常用工程配置文件

27.4.1 “`app_config.c`” 文件

“`app_config.c`” 可以配置中断优先级, `AUDIO` 参数, 打印等。

详细请看章节 “1.2.1 应用配置文件 《`app_config.c`》 介绍”。

27.4.2 函数 `u8 app_get(void)`

“`app_config.h`” 配置了串口, 按键选项, 按键音, `SD` 卡使能, `USB` 模块配置项。

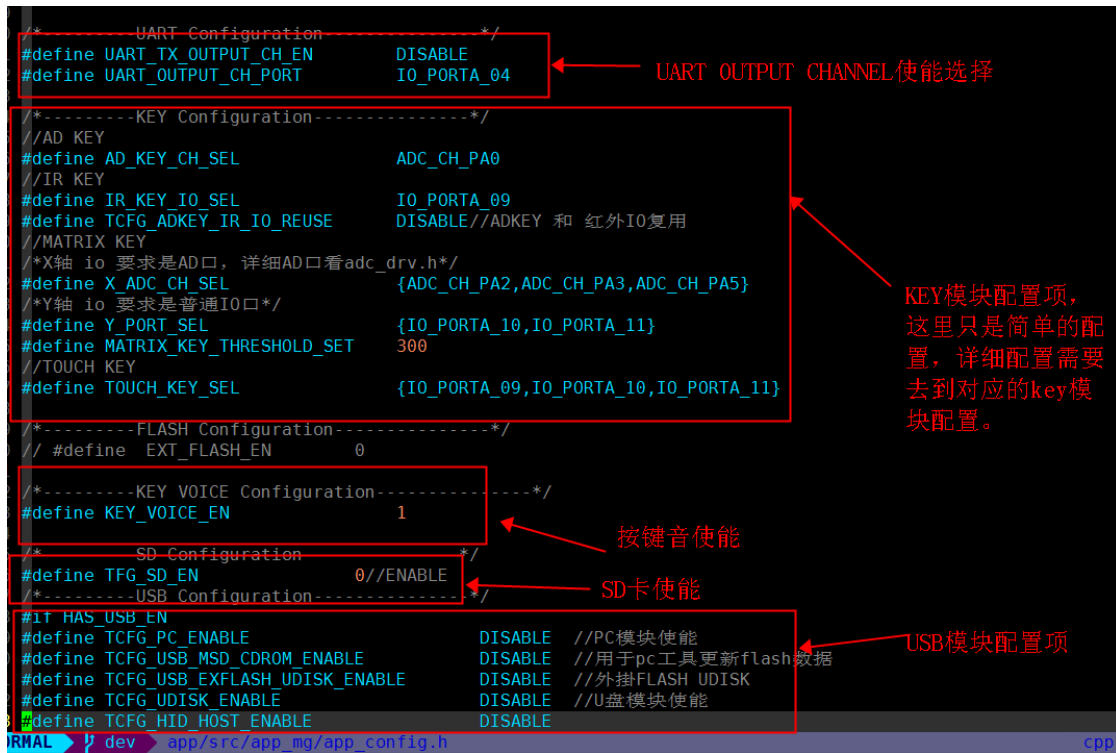


图 2 “app_config.h” 文件介绍

27.4.3 code block 工程配置

code block 工程配置配置了一些模块使能，用户可以在这里选择开启和关闭相关的模块。

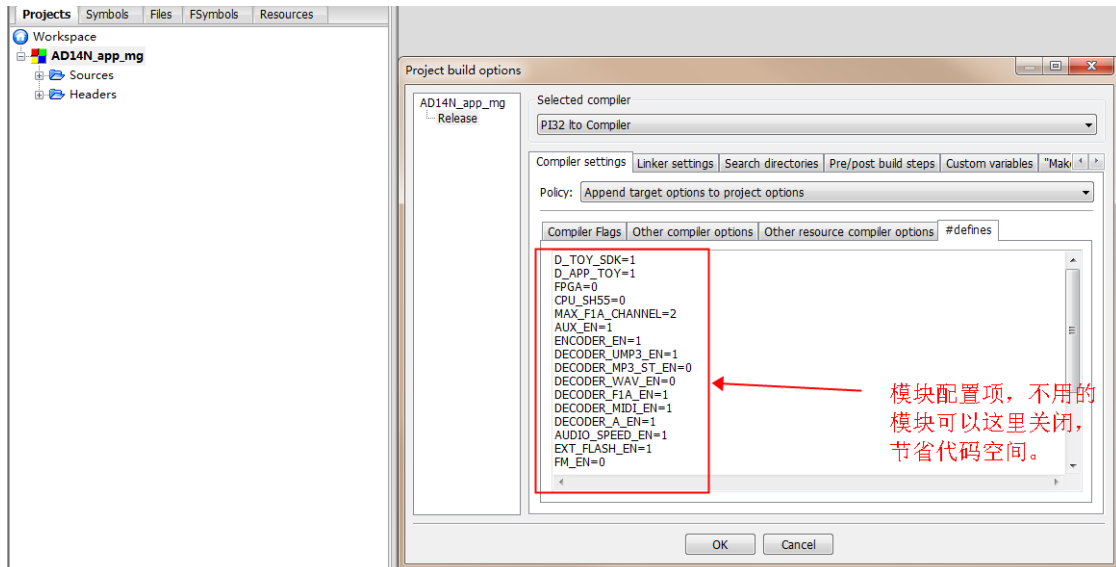


图 3 code block 工程配置

小音箱应用

AC104N 的应用模式有 MIDI 模式、音乐模式、录音模式和关机模式，不同的模式间会有资源和按键互斥的关系。应用上面做了一个简单的模式切换管理来管理不同模式的资源和按键。

模式管理的整体管理思路是，进入该模式的时候初始化模式需要使用的资源和按键处理函数，退出该模式的时候关闭该模式需要使用的资源和释放按键处理函数。

模式初始化函数:

```
1. int app_mg_init (APP_T app, void* param);  
2. void app (void);
```

模式切换函数:

```
1. int app_switch (APP_T app, void* param);  
2. int app_switch_next (void);  
3. int app_switch_priv (void);  
4. u8 app_get (void);
```

模式管理接口的文件摆布 (见下图):


```
src/  
  ▼ app_mg/  
    ▼ common/  
      ▶ music_api/  
        common_msg.c  
        idle_check.c  
    ▼ midi/  
      app_midi.c  
      app_midi.h  
      app_midi_key.c  
    ▼ music/  
      app_music.c  
      app_music.h  
      app_music_key.c  
    ▼ poweroff/  
      app_poweroff.c  
      app_poweroff.h  
      app_poweroff_key.c  
    ▼ record/  
      app_record.c  
      app_record.h  
      app_record_key.c  
  app_mg.c  
  app_mg.h
```

图 4 模式管理文件摆放

AUDIO DAC

AD14N / AD15N / AC104N / AD17N / AD18N 内置应用于语音场合的 DAC，此 DAC 支持 16bit 精度，单声道。

同时 AD14N / AC104N 支持最多 3 路 DAC 解码输出，AD15N / AD17N 最多支持 2 路 DAC 解码输出。

各芯片支持的 AUDIO DAC 采样率如下：

DAC 采样率	AD14N/AC104N	AD15N	AD17N	AD18N(APA)	AD18N(RDAC)
8000	√	√	不支持	不支持	√
12000	√	√	不支持	不支持	√
16000	√	√	不支持	不支持	√
24000	√	√	不支持	不支持	√
32000	√	√	√	√	√
44100	不支持	不支持	√	√	√
48000	不支持	不支持	√	√	√

本章介绍 DAC 相关结构体以及接口。

Audio DAC 涉及相关文件，分别是 app/lib/dac_api.c 和 include_lib/audio/dac_api.h 以及 include_lib/audio/dac.h。

Audio DAC 有以下结构体：

```
typedef struct _DAC_MANAGE {
    sound_out_obj *sound[DAC_CHANNEL_NUMBER];
    void (*kick[DAC_CHANNEL_NUMBER])(void *);    //kick 函数
    u8 ch;                                         //解码通道数
```

(续下页)

(接上页)

```
u16 vol_phy;           //模拟音量
u8 vol;                //数字音量
u8 flag;               //标志位
} DAC_MANAGE;
```

Audio DAC 主要有以下函数接口：

```
void dac_mode_init(void);
void dac_init_api(u32 sr);
void dac_power_on(u32 sr, bool delay_flag);
void dac_off_api(void);
bool dac_mute(bool mute);
bool regist_dac_channel(void *psound, void *kick);
bool unregist_dac_channel(void *psound);
u8 dac_vol(char set, u8 vol);
void fill_dac_fill(u8 *buf, u32 len);
```

29.1 AUDIO DAC 各芯片的音频性能指标

各芯片音频性能对比如下：

芯片 APA 差分性能对比	AD14N(PWM	AD15N(PWM	AD17N(MODE2	AD18N(MODE2)
Output Swing(空载)	2.3Vrms	2.3Vrms	2.2Vrms	2.2Vrms
THD	-33dB	-33dB	-70+dB	-66+dB

芯片 RDAC 性能对比	AC104N	AD16N	AD18N
Output Swing(空载)	430mVrms	760mVrms	807mVrms
SNR	83dB	97dB	84dB
DR	82dB	96dB	84dB
THD	-70dB	-82dB	-75dB
Noise Floor	30uVrms	11uVrms	50uVrms

备注：AC104 / AD14 详细 DAC 性能指标可参考 “AD14/AC104N Audio_AD2DA(RDAC 输出) 性能指标测试说明” 章节；

29.2 AUDIO DAC 配置参数

下面介绍 AC104N / AD14N / AD15N / AD17N / AD18N 的 AUDIO_DAC 配置驱动；

- 1. “AD14N / AD15N / AC104N DAC 驱动模式介绍”
- 2. “AD17N APA 驱动模式介绍”
- 3. “AD18N AUDIO 驱动模式介绍”

29.2.1 AD14N / AD15N / AC104N DAC 驱动模式介绍

AD14N / AC104N 支持的 DAC 驱动模式模式在 dac_cpu.h 文件中可以进行选择，通过设置 DAC_CURR_MODE 定义来决定使用哪种 AUDIO_DAC 驱动模式，其中：

芯 片	DAC 工作模式	DAC_PWM 工作模式特点	DAC_DSM2 差分/单端推，音质好，功耗较高	DAC_DSM3 & DAC_DSM11 差分推，音质好、功耗高	DAC_DSM7 输出模拟信号，音质好
AD14N / AC104N	/	支持	仅 C 版芯片支持	仅 C 版芯片支持	支持单端推
AD15N		支持	支持	仅 C 版芯片支持	不支持

```
> src/
  Makefile.include
  include lib/
  > agreement/
  > audio/
    audio.h
    audio_adc.h
    audio_analog.h
    audio_link.h
    dac.h
    dac_api.h
    echo_api.h
    howling_api.h
    howling_pitchshifter_api.h
    notch_howling_api.h
    reverb_api.h
    src.h
  > common/
  > cpu/
  > decoder/
  > dev_err/
```

```
32 #define DAC_SRBITS (7<<1)
33
34 #define DAC_ENABLE BIT(0)
35
36
37 #define DAC_MODE_1_A ( DAC_PWM )
38 // 以下DSM模式暂时皆不可用
39 // #define DAC_MODE_3_C ( DAC_DSM | DAC_DSM2 )
40 // #define DAC_MODE_3_D ( DAC_DSM | DAC_DSM3 )
41 #define DAC_MODE_1_B ( DAC_DSM | DAC_DSM11 )
42
43 #define DAC_MODE_5_A ( DAC_DSM | DAC_DSM7) //AD150不支持
44
45 #ifdef D_APP_MBOX
46 #define DAC_CURR_MODE DAC_MODE_5_A
47 #else
48 #define DAC_CURR_MODE DAC_MODE_1_A
49 #endif
```

只能选择1_A或5_A模式，其他不用

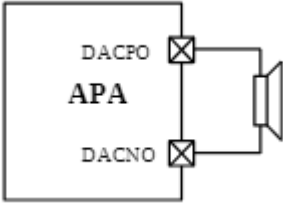
ac104默认使用5_A
ad140和ad150默认使用1_A

dac 工作模式设置示意图

DAC 的驱动方式介绍：

1) Class-D 直接驱动喇叭 (DAC_PWM、DAC_DSM2/3/11 模式)

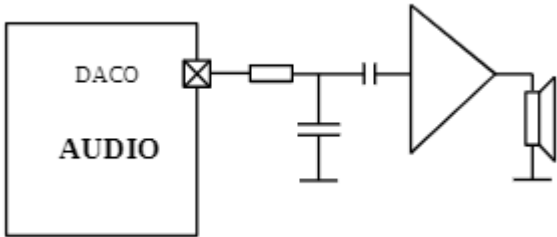
该驱动方式通过 DACP 和 DACN 引脚输出，内置功放。



ClassD 直接驱动喇叭示意图

2) 单端输出驱动单端功放 (DAC_DSM7 模式)

该驱动方式在 AD14N / AC104N 通过 PB0 输出，需要外接功放芯片。(AD15N 不支持该驱动方式)



dac 单端输出模式示意图

29.2.2 AD17N APA 驱动模式介绍

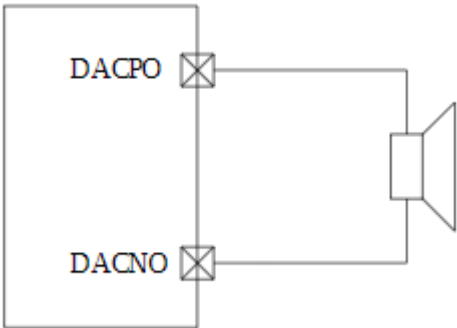
AD17N 支持的 APA 工作模式（宏 APA_MODE_DEFAULT 配置）在 dac_cpu.h 文件中可以进行选择，其中：

AD17N APA 工作模式	工作模式特点
MODE1	支持单端，单端输出性能好
MODE2	支持差分，差分输出性能好
MODE3	暂不支持

APA 的驱动方式有：

1. 差分直推喇叭（推荐使用 MODE2）

该驱动方式对应 MODE1、MODE2 模式，通过 APAP 和 APAN 引脚输出，内置功放。



差分直推喇叭示意图

```
11 #define APA_MODE_BTL      (0 << 11)
12 #define APA_MODE_PBTl    (1 << 11)
13
14 #define APA_MODE1_NOR     (0 << 9)
15 #define APA_MODE2_SNR    (1 << 9)
16 #define APA_MODE3_THD    (2 << 9)
17 #define APA_MODE_BITS    (3 << 9)
18
19 #define APA_MUTE           (1 << 8)
20
21 #define APA_PND            (1 << 7)
22 #define APA_CPND          (1 << 6)
23 #define APA_IE_EN         (1 << 5)
24 #define APA_PWM_EN        (1 << 4)
25 #define APA_DSM_EN        (1 << 3)
26
27 //apa 输出采样率
28 #define APA_SR48K         (0<<0)
29 #define APA_SR44K1        (1<<0)
30 #define APA_SR32K         (2<<0)
31 #define APA_SRBITS        (7<<0)
32 #define DAC_SRBITS        APA_SRBITS
33
34
35 #define APA_CLK_DEFAULT    APA_PLL240M
36 #define APA_MODE_DEFAULT  APA_MODE2_SNR
37 #define DAC_DEFAULT        (APA_IE_EN | APA_MODE_DEFAULT | APA_MODE_BTL)
38
39
40 #define audio_clk_init()  SFR(JL_CLOCK->CLK_CON2, 0, 2, APA_CLK_DEFAULT)
41
42 #define SR_DEFAULT 32000
43
44
45 #endif
```

APA差分推配置

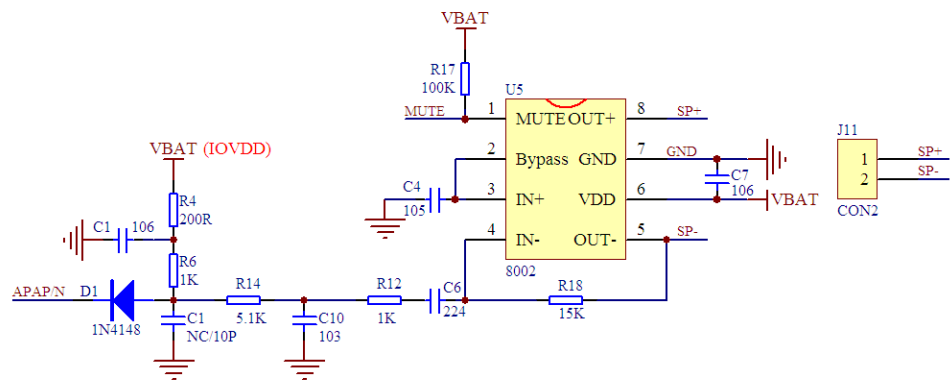
APA默认采样率配置

include_lib/cpu/sh57/dac_cpu.h

差分直推模式配置

2. 单端隔直推功放（推荐使用 MODE1）

该驱动方式对应 MODE1 模式，通过 APAP、APAN 引脚输出推功放（不支持立体声）。



单端隔直推功放原理图

```

11 #define APA_MODE_BTL      (0 << 11)
12 #define APA_MODE_PBTl    (1 << 11)
13
14 #define APA_MODE1_NOR     (0 << 9)
15 #define APA_MODE2_SNR    (1 << 9)
16 #define APA_MODE3_THD    (2 << 9)
17 #define APA_MODE_BITS    (3 << 9)
18
19 #define APA_MUTE          (1 << 8)
20
21 #define APA_PND           (1 << 7)
22 #define APA_CPND         (1 << 6)
23 #define APA_IE_EN        (1 << 5)
24 #define APA_PWM_EN       (1 << 4)
25 #define APA_DSM_EN       (1 << 3)
26
27 //apa 输出采样率
28 #define APA_SR48K         (0<<0)
29 #define APA_SR44K1        (1<<0)
30 #define APA_SR32K         (2<<0)
31 #define APA_SRBITS        (7<<0)
32 #define DAC_SRBITS        APA_SRBITS
33
34
35 #define APA_CLK_DEFAULT   APA_PLL240M
36 #define APA_MODE_DEFAULT  APA_MODE1_NOR
37 #define DAC_DEFAULT       (APA_IE_EN | APA_MODE_DEFAULT | APA_MODE_BTL)
38
39
40 #define audio_clk_init()  SFR(JL_CLOCK->CLK_CON2, 0, 2, APA_CLK_DEFAULT)
41
42 #define SR_DEFAULT        32000
43
44
45 #endif

```

APA单端隔直推配置

APA默认采样率配置

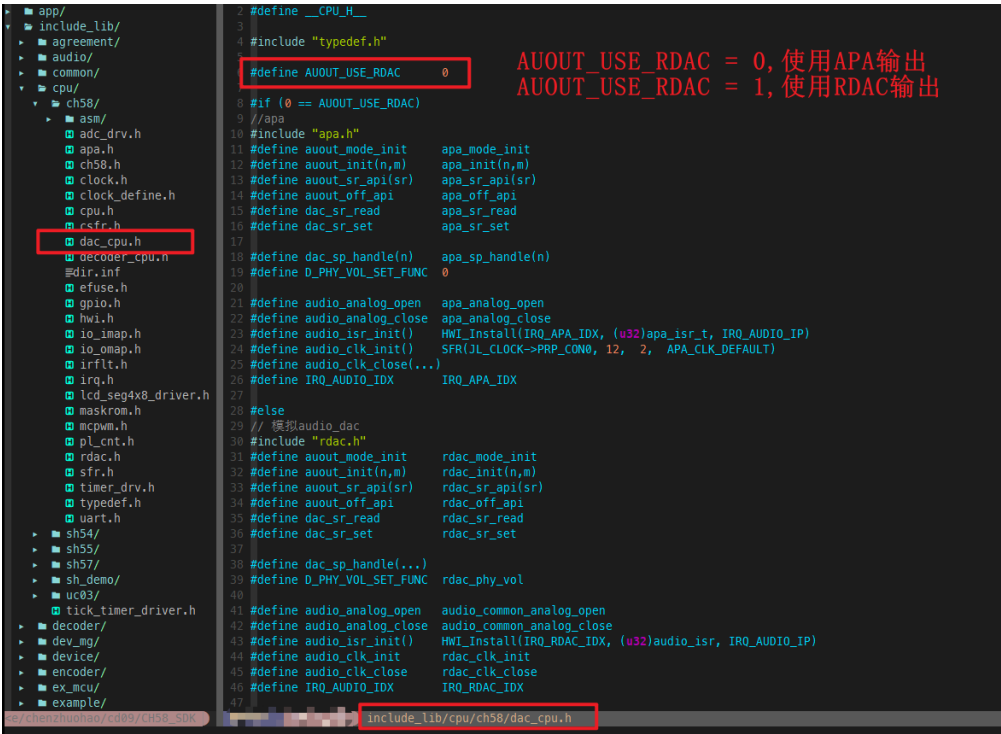
include_lib/cpu/sh57/dac_cpu.h

单端隔直推模式配置

29.2.3 AD18N AUDIO 驱动模式介绍

AD18N AUDIO 支持 Class-D 功放的 APA 工作模式，以及 16bit 单声道模拟 DAC 输出的 RDAC 模式；

可在 dac_cpu.h 文件中配置 AUOUT_USE_RDAC 宏来进行选择，其中：



AD18N 音频输出方式选择

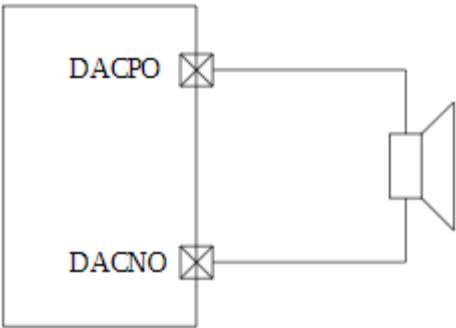
其中 APA 配置如下：

AD18N APA 工作模式	工作模式特点
MODE1	支持单端，单端输出性能好
MODE2	支持差分，差分输出性能好
MODE3	暂不支持

APA 的驱动方式有：

1. 差分直推喇叭（推荐使用 MODE2）

该驱动方式对应 MODE1、MODE2 模式，通过 APAP 和 APAN 引脚输出，内置功放。



差分直推喇叭示意图


```

11 //APA 模式
12 #define APA_PLL320M      1
13 #define APA_PLL240M_LOW  2
14 #define APA_PLL240M_HIGH 3
15 // #define APA_HSBCLK      3
16
17 #define APA_MODE_BTL      (0 << 11)
18 #define APA_MODE_PBTl    (1 << 11)
19
20 #define APA_MODE1_NOR     (0 << 9)
21 #define APA_MODE2_SNR     (1 << 9)
22 #define APA_MODE3_THD     (2 << 9) //暂不支持
23 #define APA_MODE_BITS     (3 << 9)
24
25 #define APA_MUTE          (1 << 8)
26
27 #define APA_PND           (1 << 7)
28 #define APA_CPND         (1 << 6)
29 #define APA_IE_EN        (1 << 5)
30 #define APA_PWM_EN       (1 << 4)
31 #define APA_DSM_EN       (1 << 3)
32
33 //apa 输出采样率
34 #define APA_SR48K         (0<<0)
35 #define APA_SR44K1        (1<<0)
36 #define APA_SR32K         (2<<0)
37 #define APA_SRBITS        (7<<0)
38
39
40 #define APA_CLK_DEFAULT    APA_PLL240M_LOW
41 #define APA_MODE_DEFAULT   APA_MODE2_SNR
42 #define APA_DEFAULT        (APA_IE_EN | APA_MODE_DEFAULT | APA_MODE_BTL)
43
44 #define apa_sp_handle(n) do {
45     static signed char ii = 1;
46     if (ii > 0) {
47         ii = -1;
48     } else {
49         ii = 1;
50     }
51     (n) += ii;
52 } while(0);
53
54 #include _lib/cpu/ch58/apa.h

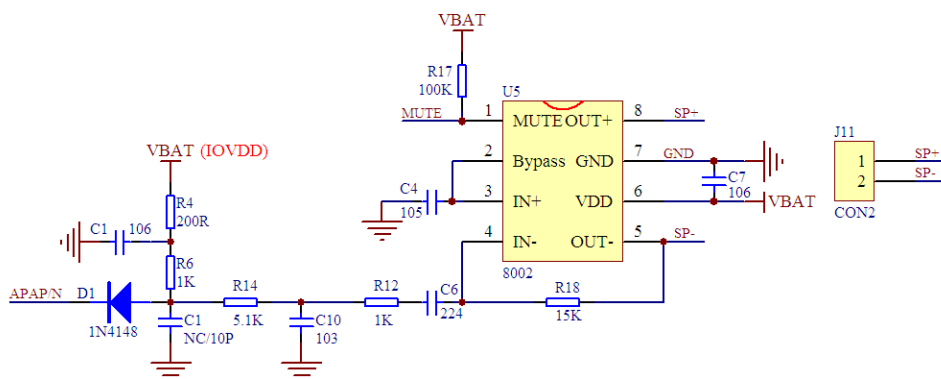
```

APA差分输出配置

差分直推模式配置

2. 单端隔直推功放（推荐使用 MODE1）

该驱动方式对应 MODE1 模式，通过 APAP、APAN 引脚输出推功放（不支持立体声）。



单端隔直推功放原理图

```

11 //APA 模式
12 #define APA_PLL320M      1
13 #define APA_PLL240M_LOW  2
14 #define APA_PLL240M_HIGH 3
15 // #define APA_HSBCLK      3
16
17 #define APA_MODE_BTL      (0 << 11)
18 #define APA_MODE_PBTTL   (1 << 11)
19
20 #define APA_MODE1_NOR      (0 << 9)
21 #define APA_MODE2_SNR      (1 << 9)
22 #define APA_MODE3_THD      (2 << 9) //暂不支持
23 #define APA_MODE_BITS      (3 << 9)
24
25 #define APA_MUTE           (1 << 8)
26
27 #define APA_PND            (1 << 7)
28 #define APA_CPND          (1 << 6)
29 #define APA_IE_EN         (1 << 5)
30 #define APA_PWM_EN        (1 << 4)
31 #define APA_DSM_EN        (1 << 3)
32
33 //apa 输出采样率
34 #define APA_SR48K         (0<<0)
35 #define APA_SR44K1        (1<<0)
36 #define APA_SR32K         (2<<0)
37 #define APA_SRBITS        (7<<0)
38
39
40 #define APA_CIK_DEFAULT    APA_PLL240M_LOW
41 #define APA_MODE_DEFAULT  APA_MODE1_NOR
42 #define APA_DEFAULT        (APA_IE_EN | APA_MODE_DEFAULT | APA_MODE_BTL)
43
44 #define apa_sp_handle(n) do {
45     static signed char ii = 1;
46     if (ii > 0) {
47         ii = -1;
48     } else {
49         ii = 1;
50     }
51     (n) += ii;
52     while(0);
53 } while(0)
54
55 #include_lib/cpu/ch58/apa.h

```

APA单端隔直推配置

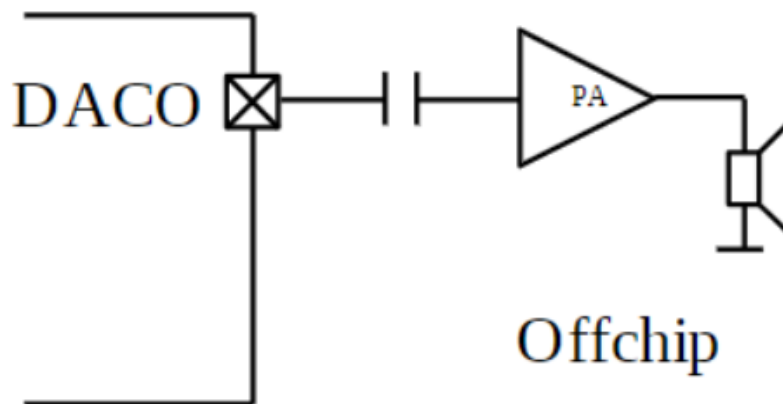
单端隔直推模式配置

RDAC 的驱动方式有：

1. 单端隔直推功放

该驱动方式在 AD18N 通过 DAC 脚输出，需要外接功放芯片。

注：部分封装没有引出 DAC 引脚，不可使用 RDAC 功能！



29.3 AUDIO DAC 接口介绍

29.3.1 函数 void dac_mode_init(void)

此函数实现对 DAC 管理结构体和相关解码参数进行初始化，开机初始化调用一次即可。

29.3.2 函数 void dac_init_api(u32 sr)

此函数实现对 DAC 硬件资源和相关寄存器进行初始化，此处会根据 `dac.h` 文件中宏定义 `DAC_CURR_MODE` 的值来选择开启 DAC 工作模式，开机初始化调用一次即可，其中参数：

```
1、sr：采样率，默认选择为32000；  
① 8000；  
② 12000；  
③ 16000；  
④ 24000；  
⑤ 32000；  
⑥ 44100；  
⑦ 48000；
```

29.3.3 函数 void dac_power_on(u32 sr, bool delay_flag)

此函数实现所有 DAC 模块的初始化，其中参数：

```
1、sr：采样率，默认选择为32000；  
① 8000；  
② 12000；  
③ 16000；  
④ 24000；  
⑤ 32000；  
⑥ 44100；  
⑦ 48000；  
2、delay_flag：延迟标志，上电时初始化dac需要延迟至少1.2ms，运行后不需要；  
① 0，不延迟；  
② 1，延迟；
```

29.3.4 函数 void dac_off_api(void)

此函数实现关闭所有 DAC 模块。

29.3.5 函数 bool dac_mute(bool mute)

此函数实现对 DAC 的软件 mute，其中参数：

- 1、mute: mute 值；
- ① 1，设置 mute 状态；
- ② 0，解除 mute 状态；

29.3.6 函数 u8 dac_vol(char set, u8 vol)

此函数实现设置 DAC 的数字音量，其中参数：

- 1、set:
 - ① ‘+’，增大一格数字音量；
 - ② ‘-’，减小一个数字音量；
 - ③ ‘r’，返回当前数字音量；
 - ④ 0，传入 0，直接将数字音量设置另一参数 vol 的值；
- 2、vol: 0-31，当 set 为 0 时，直接设置为此音量，最大值为 31；
- 3、返回值：当前设置的数字音量值；

29.3.7 函数 bool regist_dac_channel(void *psound, void *kick)

此函数实现将 sound_out_obj 注册到 DAC 中，AD140 中一共有 3 路 DAC，其中参数：

- 1、psound: sound_out_obj 指向的 sound 结构体；
- 2、kick: kick 函数；
- 3、返回值：
 - 成功 true；
 - 失败 false；

29.3.8 函数 bool unregist_dac_channel(void *psound)

此函数实现将 DAC 注册列表中指定的 sound_out_obj 删除，其中参数：

- 1、psound: sound_out_obj 指向的 sound 结构体；
- 2、返回值：
 - 成功 true；
 - 失败 false；

29.3.9 函数 void fill_dac_fill(u8 *buf, u32 len)

此函数实现将解码后的音频样点数据经过处理后推给 DAC 的缓存 buf 使 DAC 进行转换，其中参数：

- 1、buf: DAC 即将要进行转换的解码样点数据缓存 buf；
- 2、len: 该 buf 的大小，默认为 1k 个 byte，存有 512 个样点；

该接口实现将解码后的样点数据（存放于 dac_mge.sound[i]->p_obuf 中）经过处理后输出至 DAC。DAC 支持 16bit 精度，一个样点为 2 个 byte。其具体操作如下：

- ① DAC转换一次后，会在转换结束后调用该接口将新的数据样点传给DAC；
- ② 该接口首先会找到正在解码播放的通道，并读取该解码器解码出来的样点buf；
- ③ rptr[i]是解码读出的一个原始样点数据，可以读取该数组获得数据；
- ④ sp_buf[sp_cnt]是经过音量处理后的样点数据，可以读取该数组获得数据；
- ⑤ sp_buf[sp_cnt]中的数据为最终DAC进行转换的样点数据；
- ⑥ 待512个样点数据全部处理完后，启动kick继续进行下一轮DAC转换；

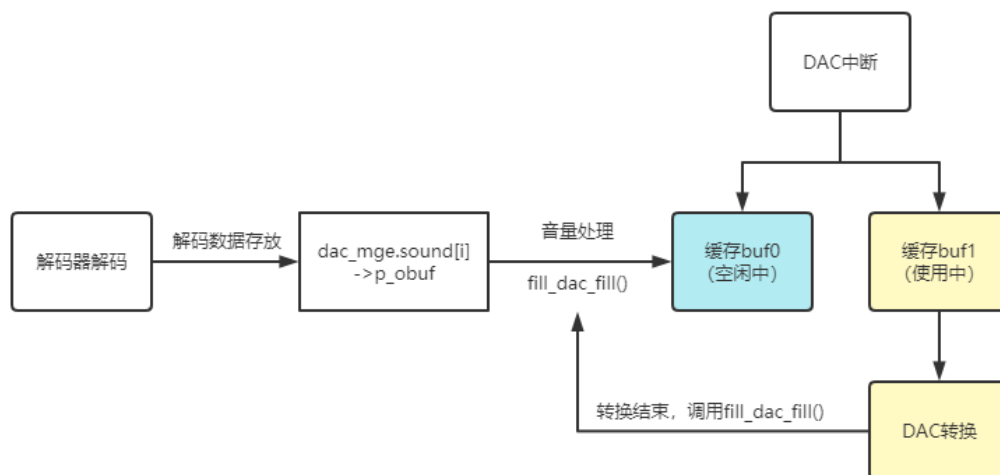


图 4 fill_dac_fill() 调用示意图

29.3.10 函数 void single_apu_startup_mute_cb(void)

此函数在 audio 初始化时由 audio 库回调，使用 APA 引脚 MUTE 住功放防止 po 声。仅 AD15N/AD17N 单端输出时使用。

29.3.11 函数 void single_apu_mute(u8 mute)

此函数控制 APA 作功放 MUTE 的 IO 引脚。仅 AD15N/AD17N 单端输出时使用，其中参数：

- 1、mute:
- 1: mute住功放；
 - 0: 解mute；

AUDIO ADC (AD14N / AC104N)

本章介绍 AUDIO_ADC 的相关内容。

AD14N / AC104N 内置应用于语音场合的 AUDIO_ADC，可外接驻极体 MIC 来录制外界声音或者 linein 输入。

此 AUDIO_ADC 支持 16bit 精度，单声道，支持 8k/12k/16k/24k 四种采样率；

AUDIO_ADC 只支持 1 路输入，在 SDK 可设置为采集 MIC 或 LINEIN 的信号，此外在 SDK 软件里实现了最多两路分发（用户可修改）。

Audio ADC 涉及两个文件，分别是

1. app/lib/adc_api.c
2. include_lib/audio/audio_adc.h。

Audio ADC 有以下结构体：

```
typedef struct _AUDIO_ADC_MANAGE {  
    sound_out_obj *sound[ADC_CHANNEL_NUMBER];  
    void (*kick[ADC_CHANNEL_NUMBER])(void *);    //kick函数  
    u8 flag;                                       //标志位  
} AUDIO_ADC_MANAGE;
```

Audio ADC 主要有以下函数接口：

```
void audio_adc_mode_init(void);  
u32 audio_adc_init_api(u32 sr, ADC_MODE mode, u32 ch);  
void audio_adc_off_api(void);  
bool regist_audio_adc_channel(void *psound, void *kick);  
bool unregist_audio_adc_channel(void *psound);
```

30.1 AUDIO ADC 性能指标

详细芯片性能指标可参考“*AD14/AC104N Audio_AD2DA(RDAC 输出) 性能指标测试说明*”章节

30.2 AUDIO ADC 接口介绍

30.2.1 函数 void audio_adc_mode_init(void)

此函数实现对 AUDIO_ADC 管理结构体和相关参数进行初始化，开机初始化调用一次即可。

30.2.2 函数 u32 audio_adc_init_api(u32 sr, ADC_MODE mode, u32 ch)

此函数实现对 AUDIO_ADC 硬件资源和相关寄存器进行初始化，此处会根据 mode 的值来选择 AUDIO_ADC 数据源，其中参数：

```
1、sr：采样率，默认选择为24000；
    ① 8000；
    ② 12000；
    ③ 16000；
    ④ 24000；

2、mode：ADC模式，默认选择为ADC_MIC；
    ① ADC_MIC：ADUIO_ADC数据来源为MIC采集回来
    ② ADC_LINE_IN：AUDIO_ADC数据来源为LINEIN输入

3、ch：选择的ADC通道；
    该参数仅对LINE_IN模式有效，MIC模式下传入0即可：
    BIT(0)：LINEIN 输入为 PA13；
    BIT(1)：LINEIN 输入为 PA14；

4、返回值：成功 0；
```

30.2.3 函数 void audio_adc_off_api(void)

此函数实现关闭所有 AUDIO_ADC 模块。

30.2.4 函数 bool regist_audio_adc_channel(void *psound, void *kick)

此函数实现将 sound_out_obj 注册到 AUDIO_ADC 中，用户可将采集到的数据最多分发两路处理，其中参数：

```
1、psound：sound_out_obj指向的sound结构体；
2、kick：kick函数；
3、返回值：
    成功 true；
    失败 false；
```

30.2.5 函数 `bool unregister_audio_adc_channel(void *psound)`

此函数实现将 AUDIO_ADC 注册列表中指定的 `sound_out_obj` 移除，其中参数：

- 1、`psound`： `sound_out_obj` 指向的 `sound` 结构体；
- 2、返回值：
 - 成功 `true`；
 - 失败 `false`；

30.2.6 函数 `void fill_audio_adc_fill(u8 *buf, u32 len)`

此函数在 AUDIO_ADC 中断中被调用，用于将 AUDIO_ADC 转换出来放在 `buf` 里的缓存数据推到指定的空间保存，其中参数：

- 1、`buf`： AUDIO_ADC 转换后样点存放的缓存 `buf`；
- 2、`len`： 该 `buf` 的大小，默认为 64 个 `byte`，存有 32 个样点；

该接口实现将 AUDIO_ADC 转换完成的数据样点写到相应的空间（存放于 `audio_adc_mge.sound[i]->p_obuf`）中，其具体操作如下：

- ① `AUDIO_ADC`
→ 转换一次后，会在中断结束后调用该接口将转换的数据样点取走存放于相应的空间；
- ② 该接口首先会选择正在编码播放的通道，并找到存放样点数据的 `buf`；
- ③ 一次性将 `buf` 中的数据写入注册了该 `AUDIO_ADC` 通道的 `audio_adc_mge.sound[i]->p_obuf` 里；
- ④ 若 `kick` 函数存在则运行 `kick` 函数，为空则该 `AUDIO_ADC` 填数函数运行完毕。

Audio 指标测试数据 (AD14N / AC104N)

本文主要列举 AD14/AC104N 的 audio 指标测试参考数据，主要有以下数据可供参考

1、Audio_ADC 音频性能指标测试

“AD14/AC104N Audio_ADC 音频性能指标测试说明”

2、Audio_DAC (RDAC 输出) 音频性能指标测试

“AD14/AC104N Audio_DAC(RDAC 输出) 性能指标测试说明”

3、Audio_DAC (APA 输出) 音频性能指标测试

“AD14/AC104N Audio_DAC(APA 输出) 性能指标测试说明”

4、Audio_AD2DA (RDAC 输出) 音频性能指标测试

“AD14/AC104N Audio_AD2DA(RDAC 输出) 性能指标测试说明”

5、Audio_AD2DA (APA 输出) 音频性能指标测试

“AD14/AC104N Audio_AD2DA(APA 输出) 性能指标测试说明”

用户排查有关音频指标问题时，可以参考对应场景的测试条件和测试数据。

31.1 测试机指标名词说明

本章节测试的 Audio 性能数据主要涉及以下指标数据，开发者可以通过下列说明了解每个指标数据的意义。

AD14 Audio 音频性能指标相关名词说明	
相关名词	名词对应说明
input swing	AP 机向测试样机输入的最大不失真输入幅度，该值需要幅度扫描测试得出，其他指标的数据以该输入幅度为标准
Output Swing	输出幅度
SNR	信噪比
DR	动态范围
THD+N	总谐波失真加噪声
THD	总谐波失真
Noise Ratio	噪声比
Noise floor	噪底
前级增益	AD14 的 MIC_0dB 控制增益，只有 0dB 和 6dB 可选
后级增益	AD14 的 MIC_PGA 增益控制档位

31.2 AD14/AC104N Audio_ADC 音频性能指标测试说明

AD14/AC104N Audio_ADC 的性能指标数据具体如下，用户可根据表中的测试条件和测试方式，参考自己的 ADC 硬件是否存在指标或者其他问题。

31.2.1 AD14/AC104N Audio_ADC 性能测试条件

AD14/AC104N Audio_ADC 性能测试条件如下：

- 1、该测试使用USB_Audio方式测试Audio_ADC性能。
 - 2、测试仪器为APX525参数测试音频分析仪
 - 3、空载测试
 - 4、AP 机输出测试音频1KHz_0dB_sine、1KHz_-60dB_sine
 - 5、ADC增益：前级增益0dB，后级增益28dB（即 MIC_0DB == 1；MIC_PGA_G == 14）
 - 6、ADC_SR == 16K
 - 7、数字音量调到最大（音量档位为31），即不对样点做任何衰减处理
 - 8、带电容方案（即MIC_CAPLESS_EN == FALSE），开发板VCOM带105电容
 - 9、app_config.c中const变量AUDIO_VCMCAP置为FALSE
 - 10、测试样机使用电池供电，样机环境为开发板

样机供电与接线方式：电池给测试样机供电，测试样机用 USB 线连接带有 AP 机上位机软件的 PC 端，APX525 仪器输出端用单端线连接芯片 micin（需要隔直电容）和模拟地。

31.2.2 AD14/AC104N Audio_ADC 性能测试数据

基于上述条件，测试得出 AD14/AC104N Audio_ADC 性能数据如下表。

AD14 V1.8.0 SDK Audio_ADC 音频性能测试数据			
测试参数	value/UNIT	AP 机配置条件	测试方法
input swing	7 mVrms	输入 1KHz_0dB, dc_offset 1.2V • dc_offset 1.2V • with butterworth (HP: 20Hz、LP: 20KHz) • with A Weighted Filter	AP 机幅度扫描测试 出最大不失真输入幅 度
Output Swing	-2.881 dBFS		AP 机向样机输入 1KHz_0dB 正弦波, 最 大不失真输入幅度
SNR	60.571 dB		AP 机向样机输入 1KHz_0dB 正弦波测 试
DR	60.265 dB		AP 机向样机输入 1KHz-60dB 正弦波测 试
THD+N	-59.854 dB		AP 机向样机输入 1KHz_0dB 正弦波测 试
THD	-62.954 dB		AP 机向样机输入 1KHz_0dB 正弦波测 试
Noise Ratio	-62.57 dB		AP 机向样机输入 1KHz_0dB 正弦波测 试
Noise floor	-62.536 dBFS	• with butterworth (HP: 20Hz、LP: 20KHz) • with A-Weighted Filter	AP 机不输入波形测 试

31.3 AD14/AC104N Audio_DAC 音频性能指标测试说明

AD14/AC104N Audio_DAC 的性能指标数据具体如下，用户可参考下列测试条件和测试数据，排查自己的 DAC 硬件是否存在指标或者其他问题，需要了解的是，AD14/AC104N Audio_DAC 有 RDAC 和 APA 两种输出方式。

31.3.1 AD14/AC104N Audio_DAC(RDAC 输出) 性能指标测试说明

AD14/AC104N Audio_DAC(RDAC 输出) 性能指标测试说明内容包含测试条件和测试数据。

AD14/AC104N Audio_DAC(RDAC 输出) 性能指标测试条件

AD14/AC104N Audio_DAC(RDAC 输出) 性能测试条件如下：

- 1、测试仪器为APX525参数测试音频分析仪，AP机输入配置终端选择100K欧。
 - 2、空载测试
 - 3、AP机输出测试音频1KHz_0dB_sine、1KHz-60dB_sine
 - 4、DAC配置：DAC_SR == 32K
 - 5、数字音量调到最大（音量档位为31），即不对样点做任何衰减处理
 - 6、DAC模式为RADAC输出模式（即DAC_CURR_MODE == DAC_MODE_5_A）
 - 7、带电容方案（即MIC_CAPLESS_EN == FALSE），开发板VCOM带105电容；
 - 8、app_config.c中const变量AUDIO_VCMCAP置为FALSE
 - 9、测试样机使用电池供电，样机环境为开发板；

样机供电与接线方式：电池给测试样机供电，AP机用单端线接芯片 DAC 口和模拟地。

AD14/AC104N Audio_DAC(RDAC 输出) 性能数据

基于上述条件，测试得出数据结果如下表。

AD14 v1.8.0 SDK Audio_RDAC 音频性能测试数据			
测试参数	value/UNIT	AP 机配置条件	测试方法
Output Swing	424.2 mVrms	• 1KHz_0dB_sine • with butterworth (HP: 20Hz、LP: 20KHz) • with A-Weighted Filter	播放 1KHz_0dB 正弦波文件, 最大音量
SNR	96 dB	• with butterworth (HP: 20Hz、LP: 20KHz) • with A Weighted Filter	按键切换, 先播静音文件, 再播 1KHz_0dB 正弦波
DR	90 dB		按键切换, 先播 1KHz-60dB 正弦波文件, 再播 1KHz_0dB 正弦波
THD+N	68 dB		样机播放 1KHz_0dB 正弦波测试
THD	68 dB		样机播放 1KHz_0dB 正弦波测试
Noise Ratio	84 dB		样机播放 1KHz_0dB 正弦波测试
Noise floor	5 uVrms	with A-Weighted Filter	样机播放静音文件或者 DAC 样点填 0

31.3.2 AD14/AC104N Audio_DAC(APA 输出) 性能指标测试说明

AD14/AC104N Audio_DAC(APA 输出) 性能指标测试说明内容包含测试条件和测试数据, 用户可以参考以下测试条件和测试数据, 排查芯片的音频硬件是否存在明显指标或者其他问题。

AD14/AC104N Audio_DAC(APA 输出) 性能测试条件

AD14/AC104N Audio_DAC 的性能测试条件如下:

- 1、测试仪器为 APX525 参数测试音频分析仪, AP 机输入配置终端选择 200K 欧
- 2、空载与带 8 欧负载测试
- 3、AP 机输出测试音频 1KHz_0dB_sine、1KHz_-60dB_sine
- 4、DAC 配置: APA_SR == 32K
- 5、数字音量调到最大 (音量档位为 31), 即不对样点做任何衰减处理
- 6、DAC 模式为 APA_PWM 输出模式 (即 DAC_CURR_MODE == DAC_MODE_1_A)
- 7、app_config.c 中 const 变量 AUDIO_VCMCAP 置为 FALSE
- 8、带电容方案 (即 MIC_CAPLESS_EN == FALSE), 开发板 VCOM 带 105 电容;
- 9、测试样机使用电池供电, 电池电压 3.88V, 样机环境为开发板

样机供电与接线方式: 使用电池给测试样机供电, 芯片 DACPO、DACNO 和模拟地用差分线接无源滤波器输入端, 无源滤波器输出端接 AP 机输入端。

注: APA 的性能受芯片供电电压影响, 对比时需要对齐上述测试条件的电池电压!!!

AD14/AC104N Audio_DAC(APA 输出) 性能测试数据

基于上述条件，测试得出数据结果如下表。

AD14 v1.8.0 SDK Audio_APA 音频性能测试				
测试参数	value/UNIT (APA 空载)	value/UNIT (APA 带 8 欧负载)	AP 机配置条件	测试方法
Output Swing	2.392 Vrms	1.752 Vrms	• 1KHz_0dB_si • with butterworth (HP: 20Hz、LP: 20KHz) • with A-Weighted Filter	播放 1KHz_0dB 正弦波文件，最大音量
DR	75.650 dB	74.646 dB	• with butterworth (HP: 20Hz、LP: 20KHz) • with A-Weighted Filter	样机按键切换，先播 1KHz-60dB 正弦波文件，再播 1KHz_0dB 正弦波
THD+N	-30.399 dB	-27.107 dB		样机播放 1KHz_0dB 正弦波测试
THD	-30.393 dB	-27.039 dB		样机播放 1KHz_0dB 正弦波测试
Noise Ratio	-75.227 dB	-74.511 dB		样机播放 1KHz_0dB 正弦波测试
Noise floor	898.7 nVrms	893.6 nVrms	with A-Weighted Filter	样机播放静音文件或者 DAC 样点填 0

31.4 AD14/AC104N Audio_AD2DA 音频性能指标测试说明

如果用户的使用场景是 ADC 直通 DAC，可将 ADC 和 DAC 之间的算法流程去除，将两者采样率均调整为 24K，对照以下数据排查芯片是否存在明显指标或者其他问题，下面会分别给出 RDAC 和 APA 输出方式的音频性能指标测试说明，每个说明内部会包含对应的测试条件和测试数据。

31.4.1 AD14/AC104N Audio_AD2DA(RDAC 输出) 性能指标测试说明

AD14/AC104N Audio_DAC(RDAC 输出) 性能指标测试说明内容包含测试条件和测试数据，用户可以参考以下测试条件和测试数据，排查芯片的音频硬件是否存在明显指标或者其他问题。

AD14/AC104N Audio_AD2DA(RDAC 输出) 性能测试条件

AD14/AC104N Audio_AD2DA(RDAC 输出) 性能测试条件如下：

- 1、测试仪器为APX525参数测试音频分析仪，AP机输入配置终端选择100K欧。
- 2、空载测试
- 3、AP机输出测试音频1KHz_0dB_sine、1KHz-60dB_sine
- 4、ADC与DAC采样率配置：ADC_SR == APA_SR == 24K
- 5、ADC增益：前级增益0dB，后级增益28dB（即 MIC_0DB == 1；MIC_PGA_G == 14）
- 6、DAC模式为RADAC输出模式（即DAC_CURR_MODE == DAC_MODE_5_A）
- 7、数字音量调到最大（音量档位为31），即不对样点做任何衰减处理
- 8、运行SDK的扩音模式（不加其他任何算法连接）；
- 9、带电容方案（即MIC_CAPLESS_EN == FALSE），开发板VCOM带105电容；
- 10、测试样机使用电池供电，样机环境为开发板；
- 11、app_config.c中const变量AUDIO_VCMCAP置为FALSE

样机供电与接线方式：使用电池给测试样机供电，APX525 仪器输入端用单端线接芯片 DAC 口和模拟地，APX525 仪器输出端用单端线连接芯片 micin（需要隔直电容）和模拟地。

AD14/AC104N Audio_AD2DA(RDAC 输出) 性能测试数据

基于上述条件，测试得出 AD14/AC104N Audio_AD2DA(RDAC 输出) 性能数据如下表。

AD14 Audio_AD2DA (RDAC) 直通音频性能测试			
测试参数	value/UNIT	AP 机配置条件	测试方法
input Swing	7 uVrms	输入 1KHz_0dB_sine, dc_offset 1.2V	AP 机幅度扫描测试出最大不失真输入幅度
Output Swing	331.1 mVrms		AP 机向样机输入 1KHz_0dB 正弦波, 最大不失真输入幅度
Output Swing(AGND)	294.6 uVrms	• with butterworth (HP: 20Hz、LP: 20KHz) • with A-Weighted Filter	mic_in 电容不接仪器, 接到 AGND
SNR	61.03 dB	• dc_offset 1.2V • with butterworth (HP: 20Hz、LP: 20KHz) • with A Weighted Filter	AP 机向样机输入 1KHz_0dB 正弦波测试
DR	61.438 dB		AP 机向样机输入 1KHz-60dB 正弦波测试
THD+N	-56.018 dB		AP 机向样机输入 1KHz_0dB 正弦波测试
THD	-57.023 dB		AP 机向样机输入 1KHz_0dB 正弦波测试
Noise Ratio	-61.447 dB	• with butterworth (HP: 20Hz、LP: 20KHz) • with A-Weighted Filter	AP 机向样机输入 1KHz_0dB 正弦波测试
Noise floor	298.4 uVrms		AP 机不输入波形测试

31.4.2 AD14/AC104N Audio_AD2DA(APA 输出) 性能指标测试说明

AD14/AC104N Audio_AD2DA(APA 输出) 性能指标测试说明内容包含测试条件和测试数据, 用户可以参考以下测试条件和测试数据, 排查芯片的音频硬件是否存在明显指标或者其他问题。

AD14/AC104N Audio_AD2DA(APA 输出) 性能测试条件

AD14/AC104N Audio_AD2DA(APA 输出) 性能测试条件如下:

- 1、测试仪器为APX525参数测试音频分析仪，AP机输入配置终端选择200K欧。
- 2、空载测试
- 3、AP机输出测试音频1KHz_0dB_sine、1KHz_-60dB_sine
- 4、ADC与APA采样率配置：ADC_SR == APA_SR == 24K
- 5、ADC增益：前级增益0dB，后级增益28dB（即 MIC_0DB == 1；MIC_PGA_G == 14）
- 6、DAC模式为APA_PWM输出模式（即DAC_CURR_MODE == DAC_MODE_1_A）
- 7、数字音量调到最大（音量档位为31），即不对样点做任何衰减处理
- 8、带电容方案（即MIC_CAPLESS_EN == FALSE），开发板VCOM带105电容；
- 9、测试样机使用电池供电，电池电压3.88V，样机环境为开发板；
- 10、app_config.c中const变量AUDIO_VCMCAP置为FALSE

样机供电与接线方式：使用电池给测试样机供电，芯片 DACPO、DACNO 和模拟地用差分线接无源滤波器输入端，无源滤波器输出端接 AP 机输入端，AP 机输出端用单端线连接芯片 micin（需要隔直电容）和模拟地。

注：APA 的性能受芯片供电电压影响，对比时需要对齐上述测试条件的电池电压!!!

AD14/AC104N Audio_AD2DA(APA 输出) 性能测试数据

基于上述条件，测试得出 AD14/AC104N Audio_AD2DA(APA) 性能数据如下表。

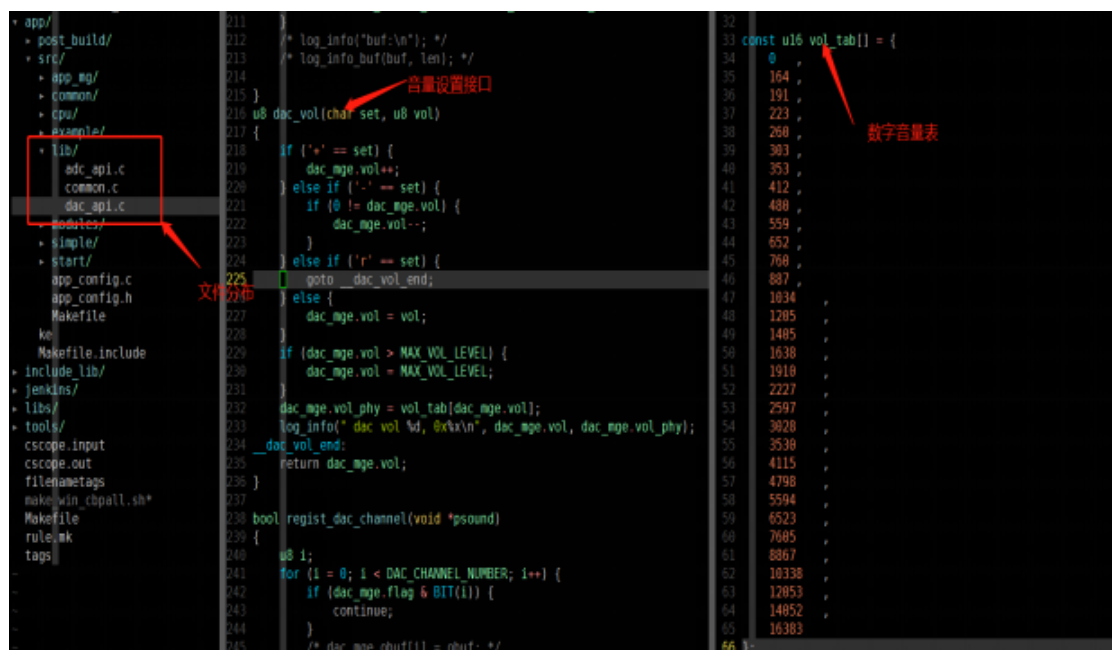
AD14 Audio_AD2DA (APA) 直通音频性能测试			
测试参数	value/UNIT (APA 空载)	AP 机配置条件	测试方法
input Swing	7 uVrms	输入 1KHz_0dB_sine, dc_offset 1.2V	AP 机幅度扫描测试出最大不失真输入幅度
Output Swing	1.96 Vrms		AP 机向样机输入 1KHz_0dB 正弦波, 最大不失真输入幅度
Output Swing(AGND)	4.46 mVrms	• with butterworth (HP: 20Hz, LP: 20KHz) • with A-Weighted Filter	mic_in 电容不接仪器, 接到 AGND
SNR	51.92 dB	• dc_offset 1.2V • with butterworth (HP: 20Hz, LP: 20KHz) • with A Weighted Filter	AP 机向样机输入 1KHz_0dB 正弦波测试
DR	54.877 dB		AP 机向样机输入 1KHz-60dB 正弦波测试
THD+N	-29.622 dB		AP 机向样机输入 1KHz_0dB 正弦波测试
THD	-29.63 dB		AP 机向样机输入 1KHz_0dB 正弦波测试
Noise Ratio	-60.211 dB	• With butterworth (HP: 20Hz, LP: 20KHz) • with A-Weighted Filter	AP 机向样机输入 1KHz_0dB 正弦波测试
Noise floor	5.068 mVrms		AP 机不输入波形测试

本章节主要介绍音量控制。

32.1 底层音量接口

AD14/15/17/18 提供了数字音量设置接口，可以实现音量加 1，音量减 1，获取音量和设置音量到特定值操作。

音量设置接口的文件分布如下所示：



32.1.1 函数 void dac_mode_init(u16 vol, void *phy_vol_set_func)

此函数实现了音量初始化，其中参数：

- 1、vol：初始化设置的音量值。
- 2、phy_vol_set_func：设置模拟音量的函数指针，用于AD18的RDAC输出模式配置模拟音量，设置后通过传入的回调函数调节

32.1.2 函数 u8 dac_vol(char set,u8 vol)

此函数实现了播放音频的音量设置，其中参数：

- 1、set：'+' 为音量加1， '-' 为音量减1， 'r' 为获取音量， 's' 为设置音量为vol的值。
- 2、vol：set参数为's' 时候需要设置的音量值。

32.2 音量应用介绍

上电时候默认设置音量为固定值 16：

```

174
175     d_key_voice_init();
176
177     audio_init();
178     dac_mode_init(16, D_PHY_VOL_SET_FUNC);
179     dac_init_api(SR_DEFAULT, 1);
180
181     d_mio_module_init();

```

关机时候默认设置音量为固定值 16：

```

static int app_poweroff_init(void *param)
{
    decoder_init();
    dac_vol(0, 16);
    key_table_sel((void *)poweroff_msg_filter);
}

```

设置音量值的时候，会把音量写入 vm，可以上电时候读取 vm 值来做音量记忆。音量值写入 vm 动作放在半秒消息做，防止写 vm 频率过高导致播放卡音：

```
void music_vol_update(void)
{
    u8 vol = 0;
    vm_read(VM_INDEX_VOL, &vol, sizeof(vol));
    if (vol != dac_vol('r', 0)) {
        vol = dac_vol('r', 0);
        vm_write(VM_INDEX_VOL, &vol, sizeof(vol));
        //log_info("vol update to vm \n");
    }
}

int common_msg_deal(int *msg)
{
    switch (*msg) {
    case MSG_500MS:
        wdt_clear();
        music_vol_update();
        break;
    case MSG_POWER_OFF:
        app_switch(APP_POWEROFF, NULL);
        break;
    case MSG_LOW_POWER:
        app_switch(APP_POWEROFF, NULL);
        break;
    case MSG_ENTER_IDLE:
        app_switch(APP_POWEROFF, NULL);
        break;
    case MSG_VOL_UP:
        dac_vol('+', 255);
        log_info("vol:%d \n", dac_vol('r', 0));
        break;
    case MSG_VOL_DOWN:
        dac_vol('-', 255);
        log_info("vol:%d \n", dac_vol('r', 0));
        break;
    }
```

把音量值通过半秒消息写入VM

设置音量

混响 (echo)

混响 (echo) 音效一般用于普通扩音, 可配置干湿音、回声等效果; 一般需要与啸叫抑制算法一起使用。

33.1 1 配置参数说明

- **DOWN_S_FLAG:** 下采参数
1 代表下采, 0 代表不下采; 下采申请的 buf 会较小, 但是回声部分高频会丢失;
- **echo_work_buf_len:** echo 运算需要的 buf 大小
根据是否下采以及输入音频采样率决定, 可通过 ops->need_buf() 接口获取大小;

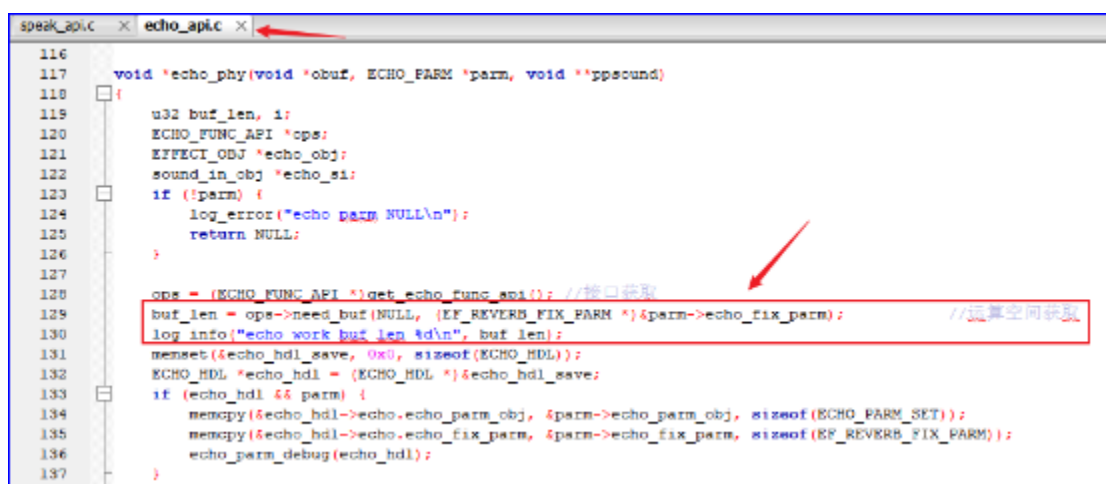


图 1 echo buff 大小获取

33.2 相关函数

33.2.1 函数 `void *link_echo_sound(void *p_sound_out, void *p_dac_cbuf, void **pp_effect, u32 sr)`

该函数为串联音频链路的标准接口，实现响应算法的启动，申请算法需要的资源，并添加到音频链路上，其中参数：

- 1、p_sound_out：当前算法的前级音频通道句柄；
- 2、p_dac_cbuf：输出的音频buff；
- 3、pp_effect：存放当前音效算法控制句柄指针的指针；
- 4、sr：当前输入音频的采样率；
- 5、返回值：启动之后最新的音频通道句柄；

33.2.2 函数 `void *echo_api(void *obuf, u32 sr, void **ppsound)`

该函数用于初始化混响（echo）的相关参数，由函数 link_echo_sound() 调用即可，需要关注的其内部参数的初始化：

- 1、decayval：回升衰减比，范围：0~70；
- 2、delay：回声延时，范围：0~max_ms，超过max_ms会当场max_ms；
- 3、energy_vad_threshold：mute阈值，默认值为512；低于该能量值的语音输出静音；
→扩音方案中可以用该方式降低底噪；
- 4、direct_sound_enable：干声叠加使能位；
- 5、max_ms：最大回声延时时间，最大值为100；
- 6、sr：输入音频的采样率，支持16K / 24K，会影响混响算法运算buff的大小；
- 7、wetgain：湿声增益，默认为3000；
- 8、drygain：干声增益，默认为4096，干声叠加使能为0时不起作用；

33.2.3 函数 `void echo_release(void **ppeffect)`

该函数用于释放音效申请的资源。适用于带有 MMU 的芯片（如 AD23N）

- 1、ppeffect：存放当前音效算法控制句柄指针的指针，即 link 时传入的指针；

移频啸叫抑制 (pitchshifter_howling)

如果在一个环路系统的 AD->...->DA 中存在某个频点明显的响应峰值, 该位置容易形成啸叫。因此, AD->...->移频->...->DA, 在一个 AD 到 DA 的环路中, 移频模块将频率不断移动, 加入非线性运算, 来改变整个环路的频响, 防止同一个频率被反复放大。

移频防啸叫模块, 可以配合 EQ 模块, 自适应陷波模块一起处理。具体根据实际样机的资源效果做选择。

34.1 1 配置参数说明

- **FRESHIFT_SPEED_MODE_QUALITY: 滤波器音阶数**
滤波器音阶数配置范围 [2, 8], 音阶数越大, 运算量越大;
- **FRESHIFT_USE_LONG_FILTER: 是否使用加长滤波来减少镜像频率**
- **FRESHIFT_USE_BAND_RESTRICT: 移频较多的时候, 选择是否加入抗混叠滤波, 减少混叠的使能标志**
- **howling_work_buf: 移频啸叫抑制运算需要的 buff**
根据移频方式和滤波器音阶数决定 buff 大小, 可通过 ops->need_buf() 接口获取大小;

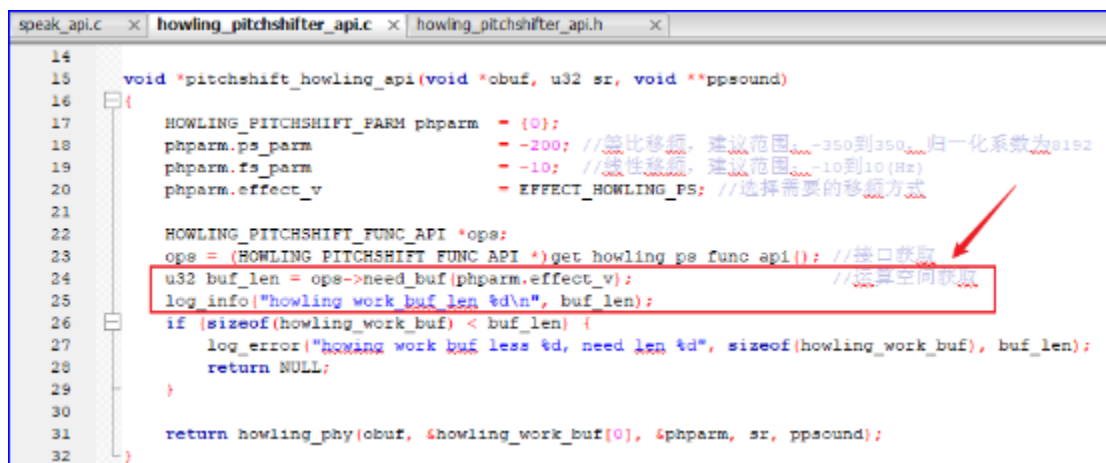


图 1 pitch_shifting buff 大小获取

34.2 2 相关函数

34.2.1 2.1 函数 void *link_pitchshift_howling_sound(void *p_sound_out, void *p_dac_cbuf, void **pp_effect, u32 sr)

该函数为串联音频链路的标准接口，实现响应算法的启动，申请算法需要的资源，并添加到音频链路上，其中参数：

- 1、p_sound_out：当前算法的前级音频通道句柄；
- 2、② p_dac_cbuf：输出的音频buff；
- 3、pp_effect：存放当前音效算法控制句柄指针的指针；
- 4、sr：当前输入音频的采样率，线性移频时采样率要和音源一致，否则会出现移频和配置频率不一致；
- 5、返回值：启动之后最新的音频通道句柄；

34.2.2 2.2 函数 void *pitchshift_howling_api(void *obuf, u32 sr, void **ppsound)

该函数用于初始化移频抑制啸叫（pitchshifter_howling）的相关参数，由函数 link_pitchshift_howling_sound() 调用即可，需要关注的其内部参数的初始化：

- 1、effect_v：移频方式，可选EFFECT_HOWLING_PS或EFFECT_HOWLING_FS；
- 2、ps_parm：等比移频，建议范围[-350, 350]，归一化系数为8192，effect_v置上EFFECT_HOWLING_PS有效；
- 3、fs_parm：线性移频，单位Hz，向高频移取正数，向低频移取负数，effect_v置上EFFECT_HOWLING_FS有效；

34.2.3 2.3 函数 void update_howling_parm_fs_api(u32 sr, s16 new_fs)

该函数用于动态更新线性移频参数

- 1、sr：采样率；
- 2、new_fs：需要更新的线性移频参数；

备注：使用该函数前，在初始化需要移频方式选中线性移频，否则该函数调用无效。

```
16 const int FRESHIFT_USE_BAND_RESTRICT = 1;
17 u32 howling_work_buf[(1980 + 3) / 4] AT(.howling_data);
18
19 void *pitchshift_howling_api(void *obuf, u32 sr, void **ppsound) 初始化时使用线性移频方式
20 {
21     HOWLING_PITCHSHIFT_PARM phparm = {0};
22     phparm.ps_parm = -200; //等比移频, 建议范围: -350到350, 归一化系数为8192
23     phparm.fs_parm = -10; //线性移频, 单位Hz, 移频方式要选择FS, 移动多少Hz就配
24     phparm.effect_v = EFFECT_HOWLING_FS; //选择需要的移频方式
25
26     HOWLING_PITCHSHIFT_FUNC_API *ops;
27     ops = (HOWLING_PITCHSHIFT_FUNC_API *)get_howling_ps_func_api(); //接口获取
28     u32 buf_len = ops->need_buf(phparm.effect_v); //运算空间获取
29     log_info("howling work_buf len %d\n", buf_len);
30     if (sizeof(howling_work_buf) < buf_len) {
31         log_error("howling work buf less %d, need len %d", sizeof(howling_work_buf), buf_len);
32         return NULL;
33     }
34
35     return howling_phy(obuf, &howling_work_buf[0], &phparm, sr, ppsound);
36 }
```

图 1 更新线性移频参数前初始化需要采用线性移频方式

自适应陷波啸叫抑制 (notch_howling)

使用扩音时，当麦克风到喇叭整体环路反馈大于 1 的时候，就会发生啸叫。只要改变整个环路的反馈，就可以抑制啸叫。自适应陷波抑制啸叫是其中一种算法，可以抑制改变环路反馈声音的频率，环路反馈一直改变，起到了平衡磨损的作用，防止某个频率一直放大。

35.1 配置参数说明

- `notch_howling_work_buf`: 自适应陷波啸叫抑制运算需要的 buff

根据配置参数决定 buff 大小，可通过 `ops->need_buf()` 接口获取大小；

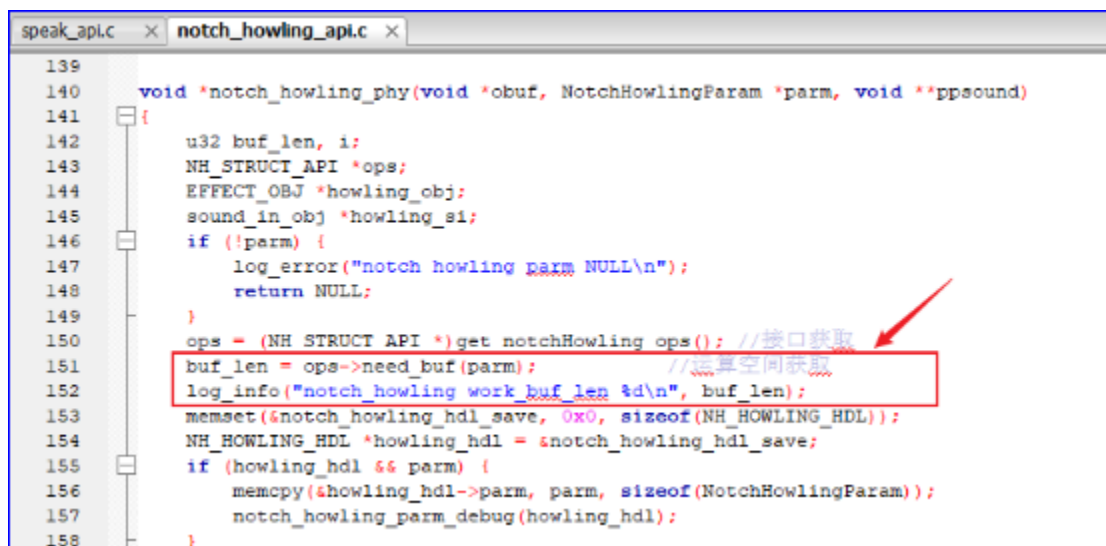


图 1 notch_howling buff 大小获取

35.2 相关函数

35.2.1 函数 void *link_notch_howling_sound(void *p_sound_out, void *p_dac_cbuf, void **pp_effect, u32 sr)

该函数为串联音频链路的标准接口，实现响应算法的启动，申请算法需要的资源，并添加到音频链路上，其中参数：

- 1、p_sound_out：当前算法的前级音频通道句柄；
- 2、② p_dac_cbuf：输出的音频buff；
- 3、pp_effect：存放当前音效算法控制句柄指针的指针；
- 4、sr：当前输入音频的采样率；
- 5、返回值：启动之后最新的音频通道句柄；

35.2.2 函数 void *notch_howling_api(void *obuf, u32 sr, void **ppsound)

该函数用于初始化自适应陷波抑制啸叫（notch_howling）的相关参数，由函数 link_notch_howling_sound() 调用即可，需要关注的其内部参数的初始化：

- 1、gain：陷波器压制程度，该值越大防啸叫效果越好，但发声啸叫频点误检时音质会更差；
- 2、Q：陷波器带宽，该值越小防啸叫效果越好，但发声啸叫频点误检时音质会更差；
- 3、fade_time：陷波器启动与释放时间，该值越小启动与释放越快，但可能导致杂音出现和音质变差；
- 4、threshold：频点啸叫判定阈值，该值越小防啸叫效果越好，但越容易误检；
- 5、SampleRate：输入音频的采样率；

35.2.3 函数 void notchhowling_release(void **ppeffect)

该函数用于释放音效申请的资源。适用于带有 MMU 的芯片（如 AD23N）

- 1、ppeffect：存放当前音效算法控制句柄指针的指针，即 link 时传入的指针；

35.3 资源使用说明

陷波啸叫抑制使用到的运算 buff 由于要经过硬件 fft 访问，所以不能 malloc 运算 buff，否则可能出现一些边界问题。因此该算法的运算 buff 保留在 overlay。

```

1 //*****phy*****/
2 #include "circular_buf.h"
3 typedef struct NH_HOWLING_HDL {
4     EFFECT_OBJ obj; // 必须在第一个
5     sound_in obj s1;
6     NH_IO_CONTEXT io;
7     NotchHowlingParam param;
8     /* u32 update; */
9     /* cbuffer t cbuf; */
10 } NH_HOWLING_HDL;
11
12 u32 notch_howling_work_buf[(3280 * 3) / 4] AT(.notch_howling_data); // 由于该算法硬件fft会操作该buf，所以该buf地址使用物理地址，notchhowling算法的fft输入输出都在work_buf
13 /* NH_HOWLING_HDL notch_howling_hdl save; */
14 /* u8 notch_howling_in_buf[160 * 2] AT(.notch_howling_data); // 陷波器啸叫抑制，固定160点运算一次 */

```

图 2 陷波啸叫抑制运算 buff

陷波啸叫抑制除了该 buff 以外，其他句柄所使用到的 ram 都在 link 时通过 malloc 获取。

均衡器 (PCM_EQ)

AD14N / AC104N / AD24N 可以使用杰理 EQ 工具进行离线 EQ 调试，将需要的频段增益抬高或者压低，使音质感受更佳。

36.1 1 配置参数说明

- **eq_filt_32000: PCM_EQ 处理 32K 采样率音频的运算数据**
该数据由 EQ 工具生成，最多可以有 10 段 EQ 数据 (seg0~seg9)，详情请见 EQ 工具使用说明章节，不同采样率拥有各自的 eq_filt 表；
- **g_work_buf: PCM_EQ 运算需要的 buff**
根据 EQ 使用的频点数 (段数) 决定 buff 大小，可通过 pcm_eq_buf_len() 接口获取大小；

```

1 pcm_eq_api.c
2 #define LOG_TAG CONST NORM
3 #define LOG_TAG "[normal]"
4 #include "log.h"
5
6 #if PCM_EQ_EN
7
8 static u32 g_work_buf[660 / 4] AT(.pcm_eq_data);
9
10 const int *get_pcm_eq_tab(u32 sr);
11
12 PCM_EQ_PARM parm;
13 void *pcm_eq_api(void *obuf, u32 sr, u32 channel, void **ppsound)
14 {
15     memset(&parm, 0, sizeof(PCM_EQ_PARM));
16     parm.nSection = 10;
17
18     u32 buf_size = pcm_eq_buf_len(parm.nSection, channel);
19
20     if (sizeof(g_work_buf) < buf_size) {
21         log_info("PCM EQ Work Data space is not big enough:%d : %d", sizeof(g_work_buf), buf_size);
22         return NULL;
23     }
24     log_info("PCM EQ Work Data space is enough:%d : %d", sizeof(g_work_buf), buf_size);
25
26     parm.LCoeff_OnChip = get_pcm_eq_tab(sr);
27     if (1 < channel) {
28         parm.RCoeff_OnChip = get_pcm_eq_tab(sr);
29     }
30     parm.LGain = 1 << 20;
31     parm.RGain = 1 << 20;
32     parm.SHI = 10;
33     parm.SHO = 8;
34     parm.channel = channel;
35     return pcm_eq_phy(obuf, &g_work_buf[0], &parm, ppsound);
36 }

```

1.pcm_eq运算buf的大小

2.buf_size为库需要的运算buf大小

3.需要给库运算buf足够的大小

图 1 pcm_eq buff 大小获取

36.2 2 相关函数

36.2.1 2.1 函数 void *link_pcm_eq_sound(void *p_sound_out, void *p_dac_cbuf, void **pp_effect, u32 sr, u32 channel)

该函数为串联音频链路的标准接口，实现响应算法的启动，并添加到音频链路上，其中参数：

- 1、p_sound_out：当前算法的前级音频通道句柄；
- 2、p_dac_cbuf：输出的音频buff；
- 3、pp_effect：存放当前音效算法控制句柄指针的指针；
- 4、sr：当前输入音频的采样率；
- 5、channel：声道数，ad14n和ac104n只有单声道，传1即可；
- 6、返回值：启动之后最新的音频通道句柄；

36.2.2 2.2 函数 void *pcm_eq_api(void *obuf, u32 sr, u32 channel, void **ppsound)

该函数用于初始化 PCM_EQ 的相关参数，由函数 link_pcm_eq_sound() 调用即可，需要关注的其内部参数的初始化：

- 1、nSection：EQ频点数（段数），不可超过各采样率的eq_filt表中的段数，范围[1, 10]；
- 2、channel：陷波器带宽，该值越小防啸叫效果越好，但发声啸叫频点误检时音质会更差；

36.3 3 EQ 工具使用

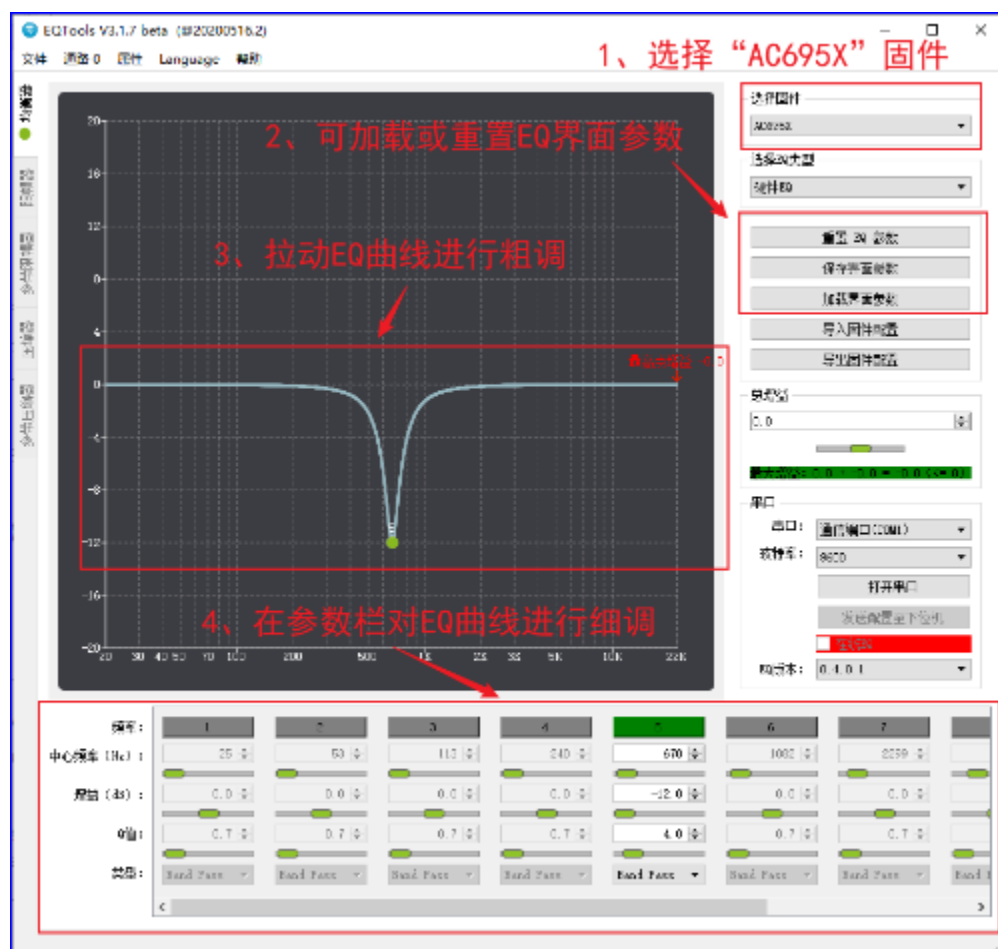
AD14N / AC104N / AD24N 可以使用杰理 EQ 工具进行离线 EQ 调试，调试完成后可以导出当前的 EQ 配置，并加载到程序中验证效果。

工具下载地址：



36.3.1 3.1 EQ 工具使用介绍

- 1、打开“EQ-3.1.7-beta-20200516.2.exe”工具并选择固件“AC695X”；
- 2、选择需要的频点数，对于不需要的频点可以点击下方频率行对应跑按钮使其变灰；
- 3、拉动 EQ 曲线进行粗调；
- 4、粗调结束后，可在下方参数框对 EQ 曲线进行细调，可设置准确的中心频率、增益、Q 值以及滤波器类型；

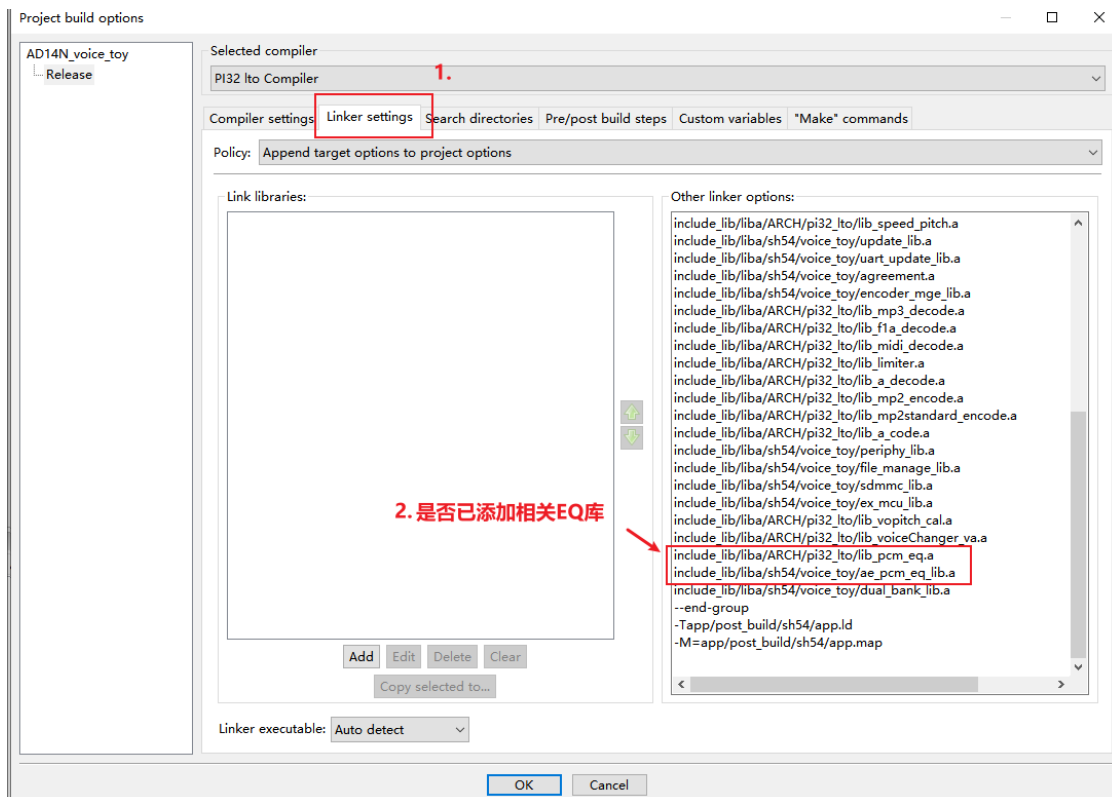


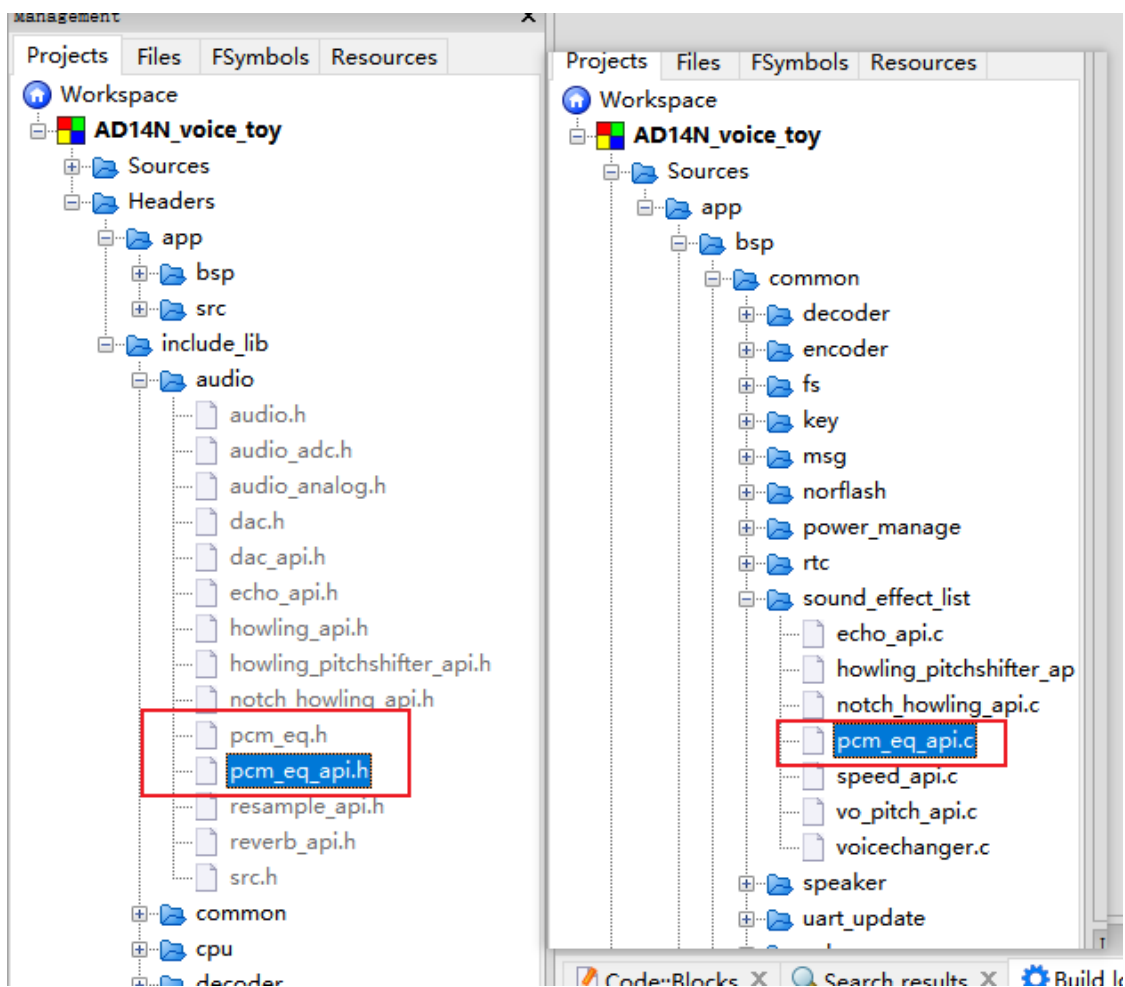
- 5、EQ 调节结束后，点击右边“保存界面参数”按钮对编辑的 EQ 界面进行保存；
- 6、保存后，再点击“导出固件配置”按钮，工具根目录会生成新的“hw_eq_table.h”文件；

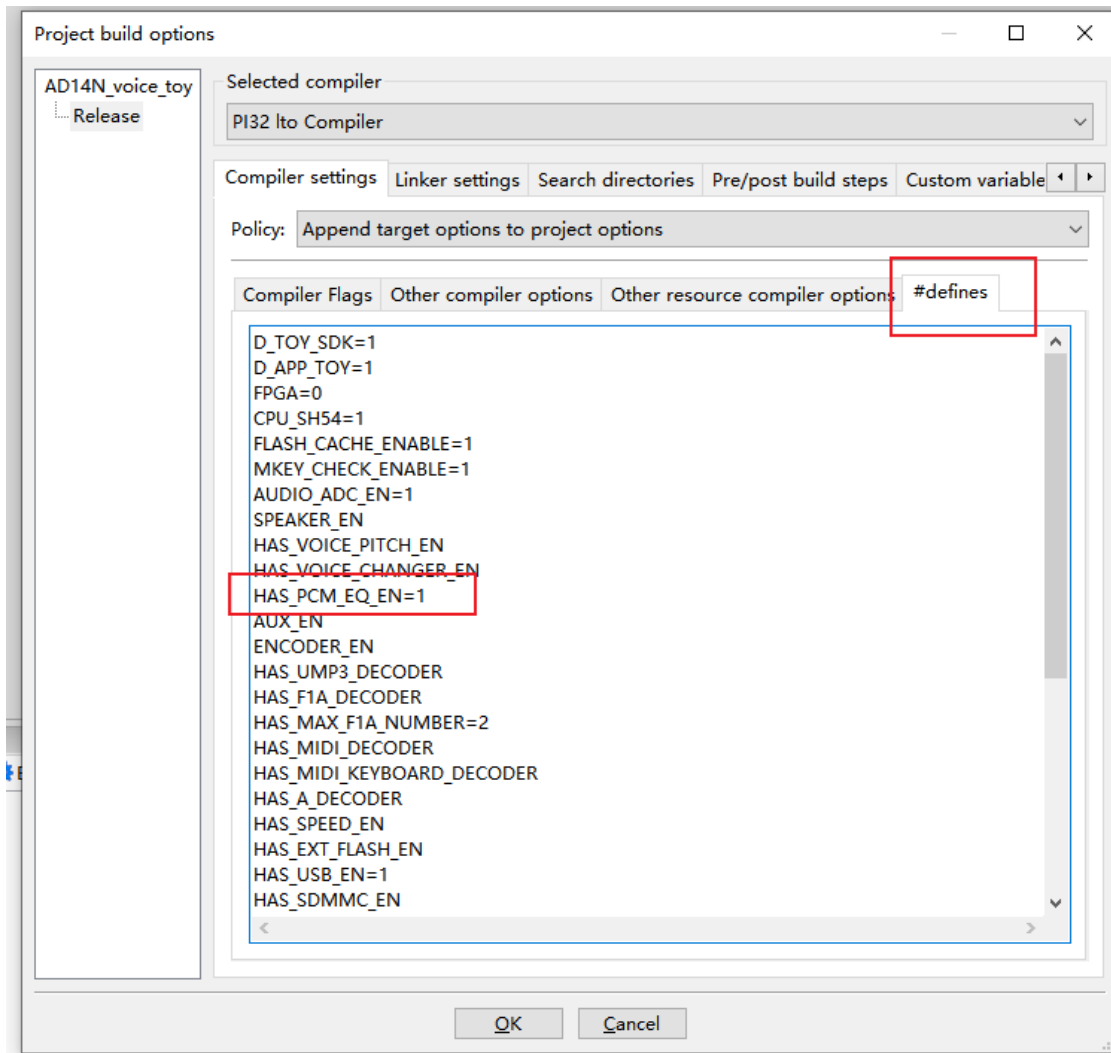


36.3.2 3.2 SDK 配置说明

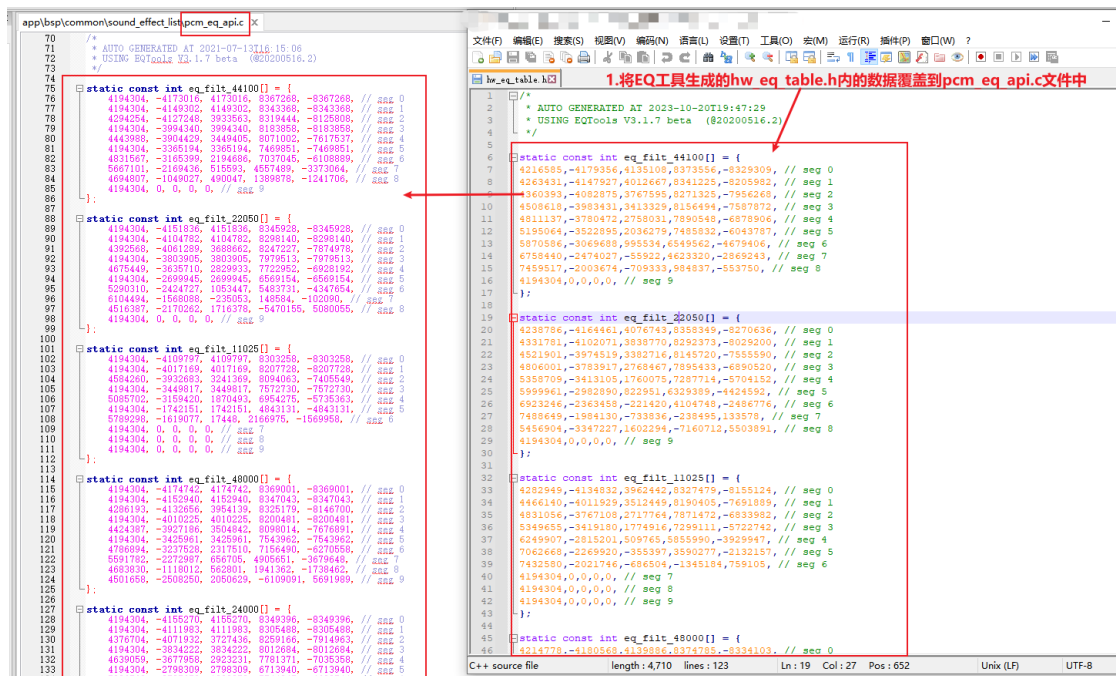
1、SDK 工程导入 pcm_eq 相关文件；



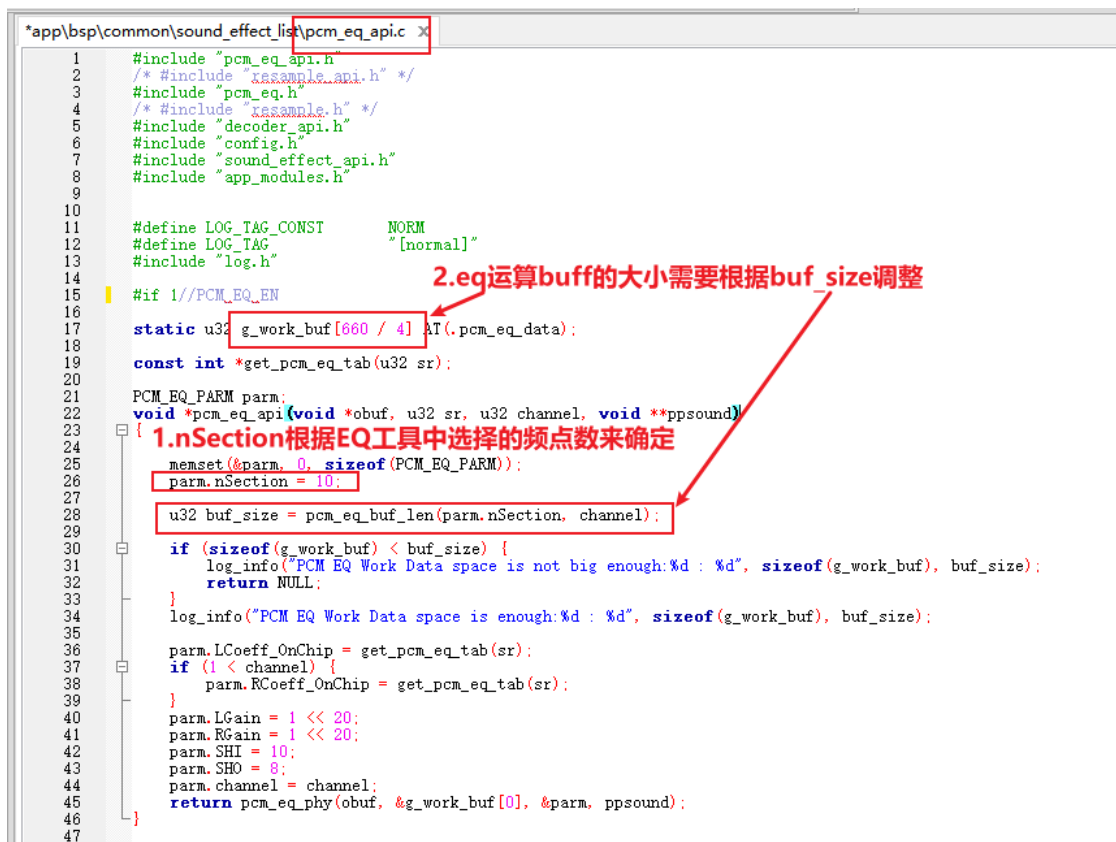




2、打开 pcm_eq_api.c 文件，并将 EQ 工具生成的 hw_eq_table.h 文件中的 eq 数据覆盖原本的 eq 数据；



3、打开 pcm_eq_api.c 文件，根据 EQ 工具选择的频点数调整 nSection 值，并根据 buf_size 打印调整 g_work_buf[] 大小；



4、打开 decoder_api.c 文件，将 EQ 串入音频解码链路中，运行测试效果；需要注意 eq 所用的 ram 是否与其他资源复用了，建议不要同时开启多种音效；

```

*app\bsp\common\sound_effect_list\pcm_eq_api.c X app\bsp\common\decoder\decoder_api.c X app\bsp\common\speaker\speak_api.c X
151
152     if (0 == res) {
153         // 设置解码参数
154         struct vifs_attr fattr = {0};
155         vifs_get_attr(pfile, &fattr);
156         /* log_info("file %d \n", fattr.fsize); */
157         if (fattr.fsize) {
158             decoder_set_file_size(p_dec, fattr.fsize);
159         }
160         p_curr_sound = &p_dec->sound;
161         p_curr_sound->enable = 0;
162         sound_out_obj *first_sound = p_curr_sound;
163         void *cbuf_o = p_dec->sound.p_obuf;
164         #if defined(AUDIO_SPEED_EN) && (AUDIO_SPEED_EN)
165             // 设置速度
166             if (dec_ctl & BII_SPEED) {
167                 p_curr_sound->effect = speed_api(cbuf_o, p_dec->sr, (void **) &p_next_sound);
168                 if (NULL != p_curr_sound->effect) {
169                     p_curr_sound->enable |= B_DEC_EFFECT;
170                     p_curr_sound = p_next_sound;
171                     p_curr_sound->p_obuf = cbuf_o;
172                     p_next_sound = 0;
173                     /* log_info("src init succ\n"); */
174                 }
175             }
176         #endif
177
178         #if 1//PCM_EQ_EN
179             p_curr_sound = link_pcm_eq_sound(p_curr_sound, &cbuf_d, 0, 32000, 1);
180         #endif
181
182         // 设置SRC
183         p_curr_sound->enable = 0;
184         #if defined(HAS_HW_SRC_EN) || defined(HAS_SW_SRC_EN)
185             if (SR_DEFAULT != p_dec->sr) {
186                 p_curr_sound = link_src_sound(p_curr_sound,
187                                                 cbuf_o,
188                                                 (void **) &p_next_sound);
189             }
190         #endif
191     }

```

将EQ串入解码链路中，其中sr参数需要根据需要的输出的采样率确定，channel默认为1

变音音效 (vo_pitch)

AD14N / AC104N SDK 中自带变音音效，可自行调整参数做到不同的效果。

37.1 可选音效

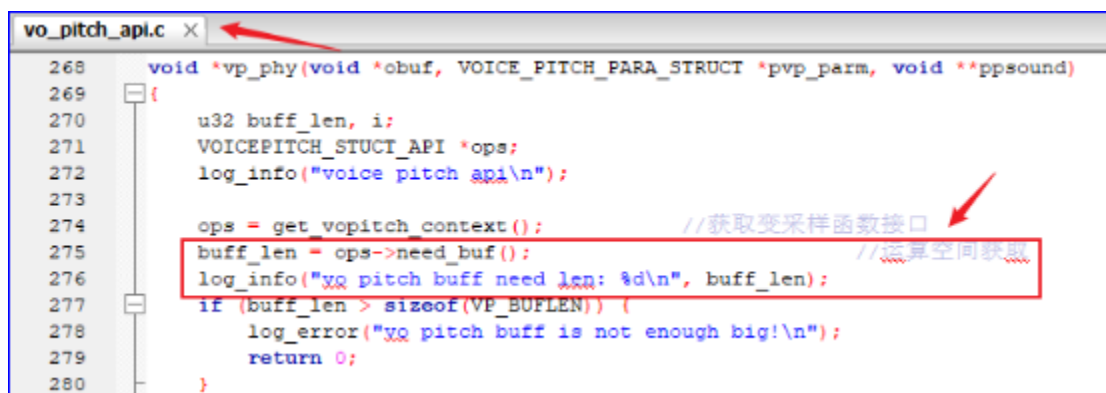
- VP_CMD_ROBOT: 平调机器音模式
- VP_CMD_ROBOT2: 变调机器音模式
- VP_CMD_PITCHSHIFT: 变调模式
- VP_CMD_RAP: RAP 模式, 将 ADC 输入的声音按照 midi 文件的节奏进行播放
- VP_CMD_PITCHSHIFT2: 变声模式 2
- VP_CMD_RAP_REALTIME: 你说我唱模式, 将 ADC 输入的声音按照 midi 文件的节奏进行实时播放
- VP_CMD_CARTOON: 卡通模式

各音效的参数配置详见 SDK;

37.2 配置参数说明

- VC_ENABLE_FLAG: 变声模块使能, 该变量为 1 时, 使能变声模块;
- EXTRA_DATA_SIZE: RAP 模式可以包含的音源长度, 单位是 short;
- VO_RAP_LOOPEN: RAP 模式是否 repeat 音源
- VO_RAP_COMPRESS_RATE: RAP 压缩比, 该值越大, RAP 音源时间压缩的越多;
- VP_DECAY_VAL: ECHO 模式的 decay 速度
- VP_HIS_LEN: ECHO 模式的 delay: 复用其他模式的 Buf
- VP_BUFLen: 变音音效运算需要的 buff

根据所选模式以及参数决定 buff 大小, 可通过 ops->need_buf() 接口获取大小;



37.3 相关结构体及函数

37.3.1 结构体 VOICE_PITCH_PARA_STRUCT

该结构体用于配置各类变音音效, 该结构体中部分成员的值只对特定的变音效果有效, 其中成员:

- 1、do_flag: 选择的音效, 参考VP_CMD枚举;
- 2、samplerate; 输入音频的采样率, 支持16K或24K;
- 3、noise_
 - dc: 能量阈值, 默认为2048, 一些持续存在的噪声经过变声有可能听起来会更明显, 所以该阈值是用来过滤
- 4、pitchrate: 音调高低, 默认值128代表原始音调, 小于128音调升高, 大于128音调降低。建议范围[40, 256];
- 5、midi_file: RAP模式的输入音源, 仅RAP模式有效;
- 6、midifile_len: RAP模式的输入音源大小, 仅RAP模式有效;
- 7、callback: RAP模式的状态回调;
- 8、priv: 私有参数, 一般传NULL;

37.3.2 函数 void *link_voice_pitch_sound(sound_out_obj *p_curr_sound, void *p_dac_cbuf, void **pp_effect, u32 cmd)

该函数为串联音频链路的标准接口，实现响应算法的启动，申请算法需要的资源，并添加到音频链路上，其中参数：

- 1、p_sound_out：当前算法的前级音频通道句柄；
- 2、p_dac_cbuf：输出的音频buff；
- 3、pp_effect：存放当前音效算法控制句柄指针的指针；
- 4、cmd：选择的变音类型，参考VP_CMD枚举；
- 5、返回值：启动之后最新的音频通道句柄；

37.3.3 函数 void voicepitch_release(void **ppeffect)

该函数用于释放音效申请的资源。适用于带有 MMU 的芯片（如 AD23N）

- 1、ppeffect：存放当前音效算法控制句柄指针的指针，即 link 时传入的指针；

37.3.4 函数 void *voice_pitch_api(void *obuf, u32 cmd, void **ppsound)

该函数用于初始化变音音效相关参数，由函数 link_voice_pitch_sound() 调用即可；该函数内部会调用函数 vp_cmd_case()，用于配置所选的变音音效以及其参数。

37.3.5 函数 void rap_callback(void *priv, int pos)

该函数为 RAP 模式的回调函数，由解码器调用。其中的参数 pos 表示 RAP 模式的状态；RAP 模式初始化后，pos 会处于 RAP_PREPARE 状态等待输入音源；当输入音源大于所设置的 noise_dc 能量阈值后，会触发 RAP 模式，此时 pos 会在 PAR_START 状态；当 RAP 播放结束后，pos 会在 RAP_END 状态循环。若需要重置 RAP 状态，需调用函数 rap_reopen()。

37.3.6 函数 void rap_reopen(void)

该函数用于 RAP 模式播放结束后，让 RAP 重回 RAP_PREPARE 状态。

37.4 RAP 和 RAP_REALTIME 模式文件替换说明

用户可以通过替换该音效依赖的 midi 文件表来进行播出不同的音调，midi 文件表位于头文件 midi_file0_h.h。


```

1 #ifndef _MIDIFILE_FILE0_H_
2 #define _MIDIFILE_FILE0_H_
3
4
5 static const unsigned char midifile_file0_tab[] = {
6     3, 76, 74, 116, 80, 81, 74, 117, 26, 27, 76, 117, 52, 106, 74, 117, 53, 107, 79, 117, 54,
7     8, 78, 117, 106, 213, 74, 117, 54,
8     55, 74, 117, 53, 53, 76, 117, 53, 107, 74, 117, 53, 107, 81, 117, 53, 107, 79, 117, 54, 21
9     74, 117, 54,
10    55, 74, 117, 52, 53, 86, 117, 53, 107, 83, 117, 53, 107, 79, 117, 53, 107, 78, 117, 53, 10
11    76, 117, 54,
12    107, 84, 117, 54, 55, 84, 117, 52, 53, 83, 117, 53, 106, 79, 117, 54, 108, 83, 117, 53, 10
13    81, 117, 107,
14    214, 74, 117, 54
15 };
16
17 static const unsigned char midifile_file1_tab[] = {
18     6, 69, 76, 127, 32, 35, 75, 127, 33, 36, 76, 127, 34, 37, 75, 127, 34, 35, 76, 127, 34, 37
19     71, 127, 34, 37, 74, 127, 33,
20     38, 72, 127, 33, 35, 69, 127, 106, 2, 57, 112, 73, 0, 60, 127, 36, 36, 64, 112, 216, 38, 6
21     112, 146,
22     36, 60, 127, 33, 35, 64, 127, 33, 36, 69, 127, 34, 39, 71, 127, 102, 1, 52, 112, 72, 35, 6
23     112, 184,
24     39, 68, 112, 145, 35, 64, 127, 33, 37, 68, 127, 33, 36, 71, 127, 34, 37, 72, 127, 102, 1,
25     , 112, 72,
26     36, 64, 112, 183, 38, 69, 112, 214, 35, 64, 127, 34, 35, 76, 127, 34, 37, 75, 127, 33, 38,
27     6, 127, 34,
28     37, 75, 127, 33, 35, 76, 127, 34, 38, 71, 127, 34, 35, 74, 127, 34, 39, 72, 127, 33, 36, 6
29     127, 110,
30     0, 57, 112, 74, 37, 64, 112, 203, 38, 69, 112, 145, 34, 60, 127, 33, 37, 64, 127, 33, 36,
31     , 127, 34,
32 };
33
34 static const unsigned char midifile_file2_tab[] = {
35     8, 67, 79, 110, 128, 0, 67, 70, 42, 42, 62, 70, 43, 43, 67, 70, 43, 43, 79, 110, 171, 0, 7
36     70, 43, 43, 67, 70, 43,
37     43, 62, 70, 42, 43, 55, 70, 255, 42, 78, 110, 43, 43, 79, 110, 43, 43, 81, 110, 43, 43, 79
38     110, 43,
39     43, 76, 110, 43, 43, 79, 110, 128, 0, 71, 70, 42, 43, 59, 70, 42, 42, 64, 70, 43, 43, 79,
40     0, 171,
41     0, 67, 70, 43, 43, 64, 70, 43, 43, 59, 70, 43, 43, 55, 70, 255, 43, 78, 110, 42, 43, 79, 1
42     , 42,
43     42, 81, 110, 43, 43, 79, 110, 43, 43, 74, 110, 43, 43, 76, 110, 128, 0, 67, 70, 43, 43, 64
44     70, 42,
45     43, 67, 70, 85, 42, 76, 110, 172, 43, 67, 70, 43, 43, 64, 70, 43, 43, 60, 70, 255, 43, 72,
46     10, 43,
47     43, 74, 110, 42, 43, 76, 110, 42, 42, 74, 110, 43, 43, 72, 110, 43, 43, 71, 110, 43, 0, 59
48     70, 255,
49     43, 66, 110, 43, 43, 71, 110, 42, 43, 74, 110, 42, 43, 71, 110, 42, 42, 66, 110, 43, 43, 5
50     70, 43,
51     43, 59, 70, 43, 43, 62, 70, 43, 43, 71, 70, 42, 43, 59, 70, 42, 42, 74, 70, 43, 43, 74, 11
52     255,
53     0, 55, 70, 43, 43, 57, 70, 43, 43, 60, 70, 43, 43, 69, 70, 43, 43, 64, 70, 42, 43, 67, 70,
54     2,
55     42, 72, 110, 86, 0, 69, 70, 129, 86, 74, 110, 43, 43, 76, 110, 85, 0, 60, 70, 128, 86, 78,
56     10, 42,

```

NORMAL | OpenSource_AD1... include_lib/vo_pitch/midi_file0.h.cpp 5% 16/283 :10

用户可以通过 get_miditabs 工具生成 midi 文件表，首先将需要使用的 midi 文件表放入工具目录，然后点击 t.bat 生成头文件

事先放入的midi文件 工具转换生成的头文件，内含midi文件表

名称	修改日期	类型	大小
cut_midlib.exe	2021/6/10 14:10	应用程序	142 KB
daoxiang.mid	2022/9/9 15:39	MIDI 序列	10 KB
midi_file0.h	2023/8/24 9:43	Header file	15 KB
midilist.txt	2023/8/24 9:43	文本文档	1 KB
msvcr100d.dll	2019/4/22 14:47	应用程序扩展	1,464 KB
t.bat	2021/6/9 16:09	Windows 批处理...	1 KB
vcruntime140.dll	2019/4/22 14:47	应用程序扩展	84 KB
说明.txt	2021/6/9 16:51	文本文档	1 KB

打开头文件可以看到生成的 midi 文件表

```

1  #ifndef _MIDIFILE_FILE0_H_
2  #define _MIDIFILE_FILE0_H_
3  static const unsigned char midifile_file3_tab[]={
4      8, 61, 45, 79, 38, 42, 52, 64, 39, 43, 64, 69, 77, 0, 59, 72, 19, 22, 61, 64, 19, 21, 57, 65, 39, 43, 40, 71, 38,
5      43, 52, 61, 38, 43, 64, 69, 77, 0, 56, 67, 77, 85, 42, 76, 39, 43, 52, 62, 39, 43, 64, 70, 77,
6      0, 59, 73, 39, 43, 57, 62, 39, 43, 49, 71, 38, 43, 59, 62, 38, 43, 64, 71, 77, 0, 61, 66, 77,
7      85, 47, 72, 39, 43, 54, 61, 39, 43, 64, 67, 77, 0, 57, 70, 77, 86, 40, 74, 38, 43, 52, 62, 38,
8      43, 64, 69, 77, 0, 57, 71, 77, 85, 45, 71, 39, 43, 52, 58, 39, 43, 64, 69, 154, 0, 59, 71, 39,
9      43, 57, 58, 19, 21, 59, 59, 39, 43, 57, 67, 20, 22, 52, 63, 38, 43, 59, 69, 38, 42, 64, 68, 43,
10     0, 52, 63, 39, 43, 45, 81, 39, 1, 69, 76, 42, 42, 52, 62, 39, 0, 73, 80, 43, 43, 64, 60, 20,
11     0, 76, 87, 129, 0, 59, 60, 19, 22, 61, 63, 19, 21, 57, 66, 39, 43, 40, 74, 38, 43, 52, 60, 38,
12     0, 71, 81, 43, 43, 64, 59, 38, 0, 68, 74, 43, 0, 56, 59, 38, 43, 52, 60, 38, 0, 64, 71, 42,
13     42, 42, 73, 39, 1, 69, 70, 42, 42, 49, 60, 39, 0, 73, 79, 43, 43, 59, 58, 39, 0, 76, 86, 129,
14     0, 64, 58, 39, 43, 57, 62, 39, 43, 49, 70, 38, 43, 56, 62, 38, 0, 71, 83, 43, 43, 61, 58, 38,
15     0, 68, 76, 43, 0, 64, 58, 38, 43, 59, 61, 38, 0, 64, 72, 42, 42, 47, 71, 39, 1, 69, 78, 115,
16     42, 54, 66, 39, 43, 62, 59, 39, 0, 57, 59, 39, 43, 64, 67, 43, 0, 54, 65, 39, 43, 40, 70, 38,
17     0, 69, 75, 116, 43, 52, 63, 38, 43, 64, 59, 38, 0, 57, 59, 38, 43, 64, 58, 42, 0, 52, 61, 38,
18     42, 45, 70, 39, 1, 61, 64, 231, 10, 64, 64, 114, 11, 69, 64, 255, 21, 52, 63, 39, 43, 64, 60, 39,
19     0, 59, 60, 39, 43, 57, 66, 19, 21, 59, 64, 39, 43, 57, 68, 20, 22, 52, 63, 38, 43, 59, 64, 38,
20     21, 69, 81, 22, 22, 57, 58, 38, 0, 69, 78, 21, 21, 66, 73, 21, 21, 45, 79, 39, 1, 69, 77, 42,
21     42, 52, 61, 39, 0, 66, 69, 22, 22, 69, 71, 43, 21, 57, 66, 39, 22, 66, 72, 21, 21, 52, 61, 39,
22     0, 69, 76, 21, 22, 66, 72, 21, 21, 44, 79, 39, 0, 71, 75, 21, 22, 71, 74, 21, 21, 52, 58, 38,
23     0, 71, 70, 21, 21, 71, 66, 22, 22, 56, 66, 38, 0, 73, 74, 43, 43, 52, 58, 38, 0, 69, 73, 38,
24     42, 42, 76, 39, 1, 69, 75, 42, 42, 49, 61, 39, 0, 66, 69, 22, 22, 69, 66, 43, 21, 54, 67, 39,
25     22, 66, 67, 21, 21, 49, 59, 39, 0, 69, 71, 21, 22, 66, 73, 21, 21, 40, 74, 39, 0, 71, 73, 21,
26     22, 71, 71, 21, 21, 49, 58, 38, 0, 71, 69, 21, 21, 71, 67, 22, 22, 52, 65, 38, 0, 73, 72, 43,
27     43, 49, 58, 38, 0, 69, 72, 38, 42, 47, 74, 39, 1, 69, 73, 42, 42, 54, 56, 39, 0, 66, 71, 22

```

最后将生成的文件表复制到 SDK 的头文件 midi_file0.h，然后在函数初始化设置配置文件表，图中以 rap_realtime 模式为例

```

182 void vp_cmd_rap_realtime(VOICE_PITCH_PARA_STRUCT *p_vc_parm)
183 {
184     p_vc_parm->do_flag = HARMO_RAP_REALTIME ;
185     p_vc_parm->noise_dc = 2048;
186     p_vc_parm->samplerate = read_audio_adc_sr();
187     p_vc_parm->hamorrage = 128; //改变音色, 50到250
188     p_vc_parm->pitchrate = 128; //改变音高, 50到250
189     p_vc_parm->midi_file = midifile_file3_tab; //改变旋律
190     p_vc_parm->midifile_len = sizeof(midifile_file3_tab);
191     //无效参数:
192     p_vc_parm->callback = rap_callback;
193     p_vc_parm->priv = NULL;
194 }
195

```

填入使用的文件表

混合变音音效 (voicechanger)

AD14N / AC104N / AD24N SDK 中新增 voicechanger 变音音效，可自行调整参数做到不同的效果，发出不同的人声或者鸟叫声。

38.1 可选音效

- EFFECT_VC_AV_PITCHSHIFT: 人声变音 1
- EFFECT_VC_F0_TD: 人声变音 2
- EFFECT_VC_AV_F0_TD_AUTOFIT: 人声变音 3
- EFFECT_VC_AV_PITCHSHIFT_AUTOFIT: 人声变音 4
- EFFECT_VC_AV_BIRD: 鸟叫声
- EFFECT_VC_AV_BIRD1: 鸟叫声 1
- EFFECT_VC_AV_BIRD2: 鸟叫声 2
- EFFECT_VC_AV_BIRD3: 鸟叫声 3
- EFFECT_VC_AV_BIRD4: 鸟叫声 4
- EFFECT_VC_AV_BIRD5: 鸟叫声 5
- EFFECT_VC_AV_BIRD6: 鸟叫声 6

其中 EFFECT_VC_AV_PITCHSHIFT 跟 EFFECT_VC_AV_F0_TD 是 2 种变声算法

EFFECT_VC_AV_PITCHSHIFT 中 formant_shift 参数无效；EFFECT_VC_AV_F0_TD 中可以通过组合不同的 shiftv 跟 formant_shift 调出不同的效果。

例如变女声可以设置成 formant_shift=80;shiftv=60;

EFFECT_VC_AV_F0_TD_AUTOFIT 跟 EFFECT_VC_AV_PITCHSHIFT_AUTOFIT 是会检测声音。

例如如果是童音即使 pitch 设高，也不要调高那么多。

EFFECT_VC_AV_BIRD 到 EFFECT_VC_AV_BIRD6 是把人声转换成鸟声，例如人声唱一段旋律，

然后转换出来对应的鸟叫声旋律。其中 pitch 会改变鸟声的整体音高，

鸟叫建议用 EFFECT_VC_AV_BIRD5 或 EFFECT_VC_AV_BIRD6

38.2 配置参数说明

- const int VC_NG_THRES: 能量阈值，低于这个能量的声音会被 mute 掉，建议默认设成 0，根据实际底噪水平设置。

voicechanger 变音音效的参数由两个结构体管理

结构体 VOICECHANGER_AV_PARM

- shiftv 调小，音调变高，变声模式建议范围 50 到 200，鸟叫声模式建议范围 30 到 100
- formant 在 EFFECT_VC_AV_F0_TD, EFFECT_VC_AV_F0_TD_AUTOFIT 模式下，调小音色变亮，调小音色变沉。范围 50 到 200
- speedv 变速参数，范围 50 到 200。如果是实时变声，ad 直接变声到 da 的，因为进出数据量需要一致。这时候 speedv 必须设成 100，不然会有杂音。

下面参数是 EFFECT_VC_AV_BIRD5 跟 EFFECT_VC_AV_BIRD6 模式下是调节颤音用的

结构体 VOICESYN_AV_PARM

- vibrate_rate_u 范围 0-100，非 0 代表开启颤音，0 代表关闭
- vibrate_rate_d 范围 0-100，影响颤音最大幅度，越大颤音幅度越大（BIRD6 无效）
- vibrate_lenCtrl 范围 0-100，颤音衰减速度，影响颤音持续时长，100 代表最短，0 代表最长
- randpercent 有效范围 50-100 【或者 0 到 50，镜像对称】，微调音色的（BIRD6 无效）

38.3 相关函数接口

38.3.1 函数 void *link_voice_changer_sound(void *p_sound_out, void p_dac_cbuf, void **pp_effect, u32 in_sr)

该函数为串联音频链路的标准接口，实现响应算法的启动，申请算法需要的资源，并添加到音频链路上，其中参数：

- 1、p_sound_out：当前算法的前级音频通道句柄；
- 2、p_dac_cbuf：输出的音频buff；
- 3、pp_effect：存放当前音效算法控制句柄指针的指针；
- 4、in_sr：采样率；
- 5、返回值：启动之后最新的音频通道句柄；

38.3.2 函数 void *voice_changer_api(void *obuf, u32 sr, void **ppsound)

该函数用于初始化变音音效相关参数，由函数 link_voice_changer_sound() 调用即可；

38.3.3 函数 void update_voice_changer_parm(VOICECHANGER_AV_PARM *new_vc_parm, VOICESYN_AV_PARM *new_vsyn_ctrol)

该函数用于运行 voicechanger 变音音效时，动态调整参数，使用时注意采样率要和之前一致，其中参数：

- 1、new_vc_parm：修改配置后新的参数结构体1
- 2、new_vsyn_ctrol：修改配置后新的参数结构体2

38.3.4 函数 void voicechanger_release(void **ppeffect)

该函数用于释放音效申请的资源。适用于带有 MMU 的芯片（如 AD23N）

- 1、ppeffect：存放当前音效算法控制句柄指针的指针，即 link 时传入的指针；

变速变调 (speed)

speed 算法在 SDK 中一般用于解码变速变调，只要在解码前置上 BIT_SPEED 位就可使用（可参考录音播放流程），该算法实时性较好，也可用于扩音。

39.1 配置参数说明

该算法的参数主要由 SPEED_PITCH_PARA_STRUCT 结构体控制，其说明如下：

- * Insample：输入音频采样率，与 audio_adc 采样率一致即可；
- * Pitchrate：变调比例，用于调节变调效果，需要与 const 变量搭配使用；
 - | 当 const 变量 SPITCH_PARM_PR_HResolution = 0 时，pitchrate =
 ↪ 128 为不变调；小于 128 时音调变高；大于 128 时音调变低；
 - | 当 const 变量 SPITCH_PARM_PR_HResolution = 1 时，pitchrate =
 ↪ 32768 为不变调；小于 32768 时音调变高；大于 32768 时音调变低；
- * Speedin / Speedout：变速比例，扩音应用下默认为 1；
- * Quality：变调运算质量，与运算速度成正比，配置范围 3~8；

```

20 void *speed_api(void *obuf, u32 insample, void **ppsound)
21 {
22     log_info("speed_api\n");
23
24     sp_parm.insample = insample;           //输入音频采样率
25     if (SPITCH_PARM_PR_HResolution) {
26         sp_parm.pitchrate = 32768;         //变调比例，32768:为不变调，<32768:音调变高，>32768:音调变低，声音更沉
27     } else {
28         sp_parm.pitchrate = 100;           //变调比例，128:为不变调，<128:音调变高，>128:音调变低，声音更沉
29     }
30     //speedout/speedin 的结果大于0.6小于1.8.变快变慢最好都不要超过2倍
31     sp_parm.speedin = 80;
32     sp_parm.speedout = 144;                //sp_parm.speedin: sp_parm.speedout为变速比例，例如speedin=1;speedout=2;则为变慢1倍
33     sp_parm.quality = 4;                  //变调运算的运算质量，跟速度成正比，配置范围3-8
34     sp_parm.pitchflag = 1;                //如果pitchflag=1,就是输入输出速度只由speedin: speedout决定,调节音调的时候，速度比例保持不变
35     //如果pitchflag=0,就是变速变调比例分开控制，这样实际变速的比例是 (speedout/speedin)*pitchrate了。
36     return speed_phy(obuf, &sp_parm, ppsound);
37 }
38
NORMAL P app/bsp/common/sound_effect_list/speed_api.c

```

图 1 变速变调参数初始化

变调限制:

1. SPITCH_PARM_PR_HResolution=0 时, pitchrate 以 128 为原始音调;
 - 音调最多降低为原来的 1/6, 即 pitchrate=128*6;
 - 音调可升高原来的 6 倍, 即 pitchrate=128/6; 升高音调没有严格的限制不能超过 6 倍, 但是不建议升高过多, 过多效果会比较差。
2. SPITCH_PARM_PR_HResolution=1 时, pitchrate 以 32768 为原始音调;
 - 音调最多降低为原来的 1/2, 即 pitchrate=65535;
 - 音调升高没有严格限制;

变速限制:

- speedin,speedout: 没有严格限制比例。但是变快太多或者变慢太多效果都会比较差。

39.2 相关函数

39.2.1 函数 void *speed_api(void *obuf, u32 insample, void **ppsound)

该函数用于初始化变速变调 (speed) 的相关参数, 申请算法需要的资源, 由应用层调用即可, 其中参数:

- 1、obuf: 输出的音频buff;
- 2、insample: 当前输入音频的采样率;
- 3、ppsound: 当前算法的前级音频通道句柄;

39.2.2 函数 void sp_release(void **ppeffect)

该函数用于释放音效申请的资源。适用于带有 MMU 的芯片 (如 AD23N)

- 1、ppeffect: 存放当前音效算法控制句柄指针的指针, 即 link 时传入的指针;

39.2.3 函数 int music_play_speed_set_para(music_play_obj *hdl, SPEED_PITCH_PARA_STRCUT *sp_parm)

该函数用于 app_mg 工程, 在解码流程中动态设置变速变调参数, 其中参数:

- 1、*hdl: 音乐播放句柄;
- 2、*sp_parm: 新的变速变调参数指针;
- 3、返回值: 成功, 返回0; 失败, 返回-1;

voice_toy 工程也有类似的相关函数, 具体参考发声玩具应用章节。

分贝检测 (energe_detect)

AD14N / AC104N / AD24N / AD23N 支持分贝检测。

涉及文件：

- 1、energe_detect_api.c
 - 2、energe_api.h
 - 3、lib_energe.a
-

40.1 配置参数 (energe_param) 说明

- sample_rate: 采样率
- channel: 通道 (配置 1 即可)
- detect_time: 检测时间 (单位为 ms), 该参数在当 attFactor 与 relFactor 为 0 时有效
- attFactor: 启动因子 (int)($\exp(-1 / (\text{time} * \text{sample_rate})) * (1 \ll 30)$); time 为 0.001 ~ 0.1; 默认 0.001
time 越小 db 值上升越快
- sample_rate: 释放因子 (int)($\exp(-1 / (\text{time} * \text{sample_rate})) * (1 \ll 30)$); time 为 0.3 ~ 0.9; 默认 0.5
time 越小 db 值下降越快

注意：

1、db 值计算库里分两种模式：

- (1)、attFactor 与 relFactor 等于 0 时。实时计算 detect_time 这一段时间 pcm 数据最大值的 db 值
- (2)、当 attFactor 或者 relFactor 不等 0 时, detect_time 参数无效。库里有做特殊处理, db 值上升快, 下降慢。由于定点运算 db 值大小会比实际 db 值小 1 ~ 2db

2、获取 mdb 值函数，获取到的 mdb 值指针，包含两个通道的 mdb 值，若只有一个通道则取第一个值，如 db_value[0];
获取到的是 mdb 值，转化为 db 值的关系是 $db = mdb / 1000$;

40.2 使用 demo

demo 在 energe_detect_api.c 最下面。:

```
void demo()
{
    /* 分贝检测算法使用流程demo */

    u32 adc_sr = read_audio_adc_sr();
    /* 1. 分贝检测初始化 */
    energe_detect_init(adc_sr);

    /* 2. 把需要检测的数据和长度传入run函数 */
    energe_run_api(data, len);

    /* 3. 获取detect_time 内pcm数据最大值的db值 */
    get_energe_db_api();

    /* 4. 如果停止使用该算法时，释放算法申请的资源 */
    ed_buf_release();
}
```

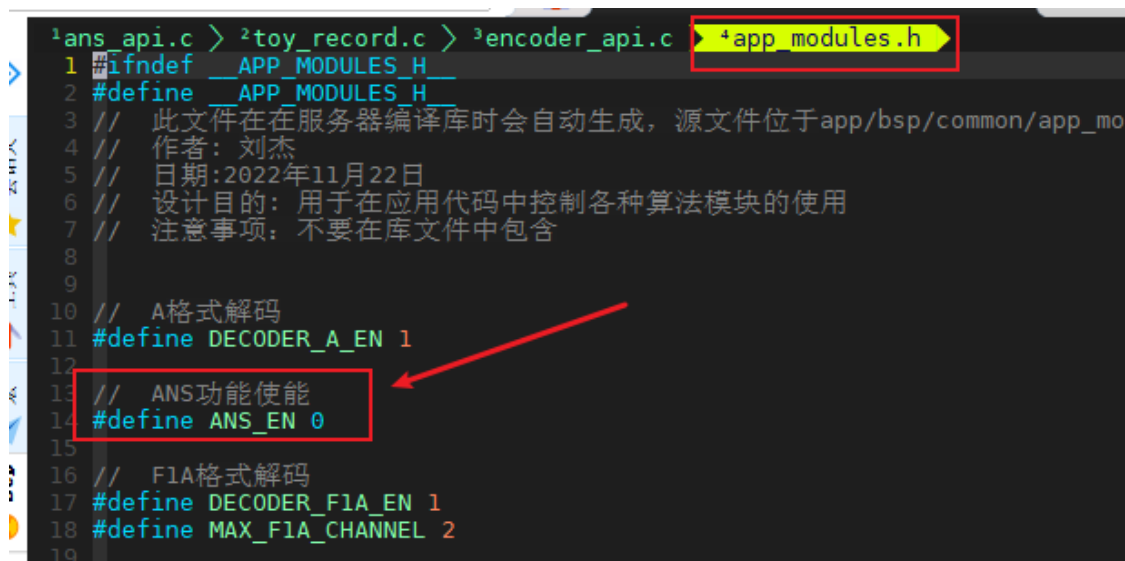
音频降噪算法 (ANS)

AD14 / AD24 / AD23 支持音频降噪算法。

SDK 默认关闭音频降噪算法 (缩写: ANS), 用户需在 `app_modules.h` 打开。

使用音频降噪算法 (缩写: ANS) 需要注意以下配置:

1. 输入给降噪算法数据的采样率 (最好为 16k 采样率)
2. 运行降噪算法时的系统时钟需要在 107M 以上
3. 在链接文件里 (`app_ld.c`) 降噪算法使用的 `ans_data` 段与其他功能的复用, 确认在使用降噪算法时, 该空间不会被其他功能干扰。



```
1 ans_api.c > 2 toy_record.c > 3 encoder_api.c > 4 app_modules.h
1 #ifndef __APP_MODULES_H__
2 #define __APP_MODULES_H__
3 // 此文件在在服务器编译库时会自动生成, 源文件位于app/bsp/common/app_mo
4 // 作者: 刘杰
5 // 日期: 2022年11月22日
6 // 设计目的: 用于在应用代码中控制各种算法模块的使用
7 // 注意事项: 不要在库文件中包含
8
9
10 // A格式解码
11 #define DECODER_A_EN 1
12
13 // ANS功能使能
14 #define ANS_EN 0
15
16 // F1A格式解码
17 #define DECODER_F1A_EN 1
18 #define MAX_F1A_CHANNEL 2
19
```

音频降噪宏

音频降噪算法 (缩写: ANS) 的使用可以参考下方 [gitee](#) 链接:

FAQ: 音频降噪算法的使用 <https://gitee.com/Jieli-Tech/AW30N/issues/IBKTBY>

41.1 涉及文件

源文件:

ans_api.c

头文件:

ans_api.h

NoiseSuppressLib.h

库文件:

libNoiseSuppress_pi32v2_OnChip.a

lib_SW_FFT_pi32v2_OnChip.a

链接文件:

app_ld.C 或 app_ld_overlay_custom.c 文件

```
182     PROVIDE(fla_2_buf_end = .);
183 }
184 .d_rec
185 {
186     *(.rec_data)
187     *(.ans_data)
188     rec_data_end = .;
189 }
190 .d_enc_ima
191 {
192     . = rec_data_end;
193     *(.enc_a_data)
194 }
195 .d_enc_mp3
196 {
197     . = rec_data_end;
198     *(.enc_mp3_data)
199 }
```

降噪算法默认放在复用区域里
并且跟着录音模式后面
注意会与解码、其他音效复用

'app/post_build/sh58/voice_toy/app_ld.c' c

降噪算法在链接文件复用位置

SDK 在录音模式已加入音频降噪算法。用户可作参考。

```

18 #include "log.h"
19
20 #if (defined(ANS_EN) && (ANS_EN))
21 #include "ans_api.h"
22 cbuffer_t cbuf_ans AT(.ans_data);
23 u32 ans_buff[512 / 4] AT(.ans_data);
24 #endif
25
26 cbuffer_t cbuf_adc AT(.rec_data);
27 u8 adc_buff[512 * 10] AT(.rec_data);
28
29 #if (0 == FPGA)
30 sound_out_obj rec_sound;
31 sound_out_obj *enc_in_sound;
32
33 #define START_ADC_RUN rec_sound.enable |= (B_DEC_RUN_EN | B_REC_RUN)
34 #define STOP_ADC_RUN rec_sound.enable &= ~(B_DEC_RUN_EN | B_REC_RUN)
35 void rec_phy_init(void)
36 {
37     memset(&rec_sound, 0, sizeof(rec_sound));
38
39     cbuf_init(&cbuf_adc, &adc_buff[0], sizeof(adc_buff));
40     rec_sound.p_obuf = &cbuf_adc; //cbuf_adc为audio_adc的数据
41
42     sound_out_obj *p_curr_sound = &rec_sound;
43
44     #if (defined(ANS_EN) && (ANS_EN))
45     cbuf_init(&cbuf_ans, &ans_buff[0], sizeof(ans_buff)); //cbuf_ans 为link
46     p_curr_sound = link_ans_sound(p_curr_sound, &cbuf_ans, read_audio_adc_sr());
47     #endif
48
49     if (p_curr_sound != &rec_sound) {
50         stream_sound_init(&rec_sound, kick_encode_api);
51         regist_audio_adc_channel(&rec_sound, (void *)kick_sound);
52     } else {
53         regist_audio_adc_channel(&rec_sound, (void *)kick_encode_api);
54     }
55
56     enc_in_sound = p_curr_sound;
57     rec_sound.enable |= B_DEC_RUN_EN | B_DEC_FIRST;
58
59 }
60 void rec_phy_suspend(void)
61 {

```

1.串联降噪算法到数据流当中

2.若有音效则初始化数据流

/app/bsp/common/encoder/encoder_api.c c utf-8[unix] 25%

录音模式串联音频降噪

41.2 相关参数

1. 采样率:

降噪算法只支持 8k 和 16k 采样率。

建议使用 16k 采样率。

```

1 encoder_api.c 2 ans_api.c
1 #include "circular_buf.h"
2 #include "errno-base.h"
3 #include "NoiseSuppressLib.h"
4 #include "app_modules.h"
5 #include "sound_effect_api.h"
6 #include "remain_output.h"
7
8 #define LOG_TAG_CONST      NORM
9 /* #define LOG_TAG_CONST    OFF */
10 #define LOG_TAG            "[ans]"
11 #include "log.h"
12
13 #if defined(ANS_EN) && (ANS_EN)
14
15 /*
16  * ANS STATE BITMAP
17  */
18
19 #define NS_MODE            0
20 #define NS_IS_WIDEBAND     1//0:8K 1:16K
21
22 #define READSIZE           128    //每次run处理样点数
23 #define ANS_RUN_BUFSIZE   5400
24 #define ANS_TMP_BUFSIZE   3604
25 #define ANS_NEAR_SIZE     (READSIZE * 2)
26
27 static int ans_runbuf[ANS_RUN_BUFSIZE / 4] AT(.ans_data);
28 static int ans_tmpbuf[ANS_TMP_BUFSIZE / 4] AT(.ans_data);
29
30 static short ans_output_buff[READSIZE] AT(.ans_data);
31
32 static remain_ops ans_remain_ops AT(.ans_data);
33 static EFFECT_OBJ ans_effect_obj AT(.ans_data);
34 /* ----- */
35
36

```

音频降噪采样率配置

2. 系统时钟:

降噪算法需要较高的系统时钟。

AD14N 经测试最低需要 96MHz 时钟。

AD24N 经测试最低需要 107MHz 时钟。

41.3 相关函数

41.3.1 函数 void *link_ans_sound(void *p_sound_out, void *p_ans_obuf, void **pp_effect, u32 sr)

该函数为 LINK 音频降噪算法，实现算法的启动，申请算法需要的资源，并添加到音频链路上，其中参数：

- 1、p_sound_out：当前算法的前级音频通道句柄；
- 2、p_ans_obuf：输出的音频buff；
- 3、pp_effect：存放当前音效算法控制句柄指针的指针；
- 4、sr：当前输入音频的采样率；
- 5、返回值：启动之后最新的音频通道句柄；

41.3.2 函数 `int ans_run(void *hld, short *inbuf, int len)`

该函数为降噪算法数据处理流程。

该降噪算法固定输入 128 个样点，经（NoiseSuppress_Process）处理后输出。

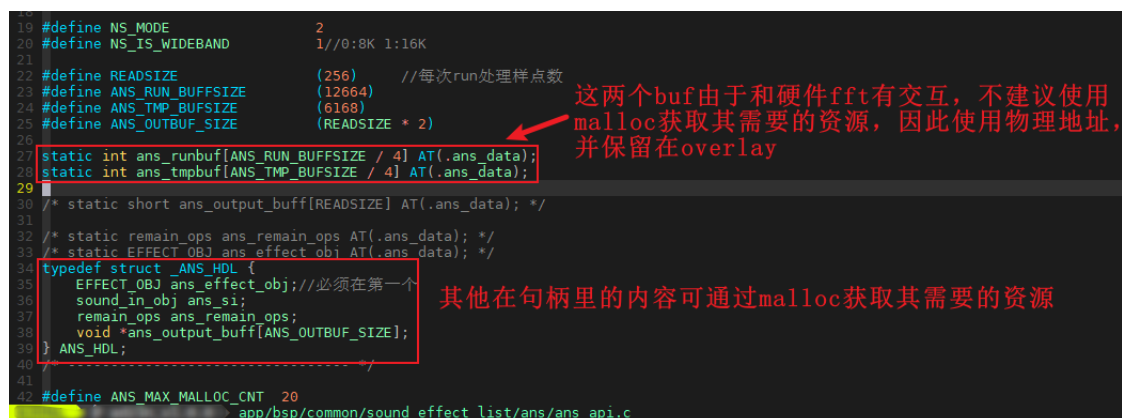
41.3.3 函数 `void ans_release(void **ppeffect)`

该函数用于释放音效申请的资源。适用于带有 MMU 的芯片（如 AD23N）

- 1、ppeffect：存放当前音效算法控制句柄指针的指针，即 link 时传入的指针；

41.4 ans 算法和 MMU 的联动情况

对于支持 MMU 的芯片（如 AD23），ANS 算法有一部分资源可以通过 malloc 获取，另一部分只能保留在 overlay



```
19 #define NS_MODE 2
20 #define NS_IS_WIDEBAND 1//0:8K 1:16K
21
22 #define READSIZE (256) //每次run处理样点数
23 #define ANS_RUN_BUFSIZE (12664)
24 #define ANS_TMP_BUFSIZE (6168)
25 #define ANS_OUTBUF_SIZE (READSIZE * 2)
26
27 static int ans_runbuf[ANS_RUN_BUFSIZE / 4] AT(.ans_data);
28 static int ans_tmpbuf[ANS_TMP_BUFSIZE / 4] AT(.ans_data);
29
30 /* static short ans_output_buff[READSIZE] AT(.ans_data); */
31
32 /* static remain_ops ans_remain_ops AT(.ans_data); */
33 /* static EFFECT_OBJ ans_effect_obj AT(.ans_data); */
34 typedef struct _ANS_HDL {
35     EFFECT_OBJ ans_effect_obj; //必须在第一个
36     sound_in_obj ans_si;
37     remain_ops ans_remain_ops;
38     void *ans_output_buff[ANS_OUTBUF_SIZE];
39 } ANS_HDL;
40
41
42 #define ANS_MAX_MALLOC_CNT 20
43 app/bsp/common/sound_effect_list/ans/ans_api.c
```

这两个buf由于和硬件fft有交互，不建议使用 malloc 获取其需要的资源，因此使用物理地址，并保留在 overlay

其他在句柄里的内容可通过 malloc 获取其需要的资源

ans 降噪算法资源情况介绍

低功耗休眠

AD14N / AD15N / AC104N / AD17N / AD24N 支持多种低功耗模式，同时支持多种唤醒方式。具体每颗芯片对低功耗的支持情况如下：

玩具系列芯片对低功耗模式支持情况			
芯片	powerdown	softoff	poweroff
AD14N/AC104N	√	√	×
AD15N	√	√	×
AD17N	√	√	×
AD18N	√	√	×
AD24N	√	√	√
AD23N	√	√	√

各种低功耗模式的区别如下：

模式	区别
Softoff	Softoff 唤醒后芯片需要从 MASKROM 开始重新跑流程；
Power-down	Powerdown 唤醒后芯片会原地唤醒开始跑。
Poweroff	Poweroff 休眠时，CPU 和寄存器都会掉电，但 ram 会保持。唤醒从 MASKROM 跳转到出 poweroff 的函数

功耗水平：

模式 / CPU	AD14N / AC104N	AD15N	AD17N	AD18N	AD24N	AD23N
Softoff	1.7uA+	1.7uA+	1.4uA+	3.3uA+	2uA+	2uA+
Powerdown	27uA+	27uA+	23uA+	33uA+	50uA+	35uA+
Poweroff	/	/	/	/	约 20uA+	约 15uA+
支持最多唤醒口	8 个	12 个	8 个	8 个	8 个	8 个

启动速度：

- 注 1：启动速度测试方法为上电/触发唤醒到运行 *main.c* 第一行的时间。
- 注 2：软关机快速启动不会运行 *uboot* 流程，可以加快启动时间，但无法进行 *uboot* 升级操作。
- 注 3：冷启动（快速启动）需要使用烧写器烧快速启动功能才能使用，烧写后会无法使用强制下载工具升级，请谨慎使用。

启动方式	AD14N / AC104N	AD15N	AD17N	AD18N	AD24N	AD23N
冷启动（首次上电）	40ms	36ms	31ms	39ms	37ms	597ms
冷启动（快速启动）	/	/	/	/	16ms	/
软关机唤醒（正常启动）	35ms	32ms	23ms	32ms	26ms	/
软关机唤醒（快速启动）	无快速启动	20ms	9ms	15ms	5ms	/

快速启动对 flash 的要求：

- 1.flash 启动时间小于 1ms。
- 2.flash 不需要发 soft reset，就可以启动。

Soft off 及 Power down 有以下相关函数：

```
void sys_softoff(void)
void sys_power_down(u32 usec);
```

唤醒相关函数：

```
int power_wakeup_reason(void);
u8 is_wakeup_source(enum WAKEUP_REASON index)
```

42.1 关机与唤醒相关参数配置

42.1.1 A0 芯片 flash 使用 VDDIO 供电配置

- 在使用 A0 封装的芯片，即存放代码的系统 flash 由 VDDIO 供电时，需要将该宏置 1，否则可能会出现休眠功耗大的问题；

芯片	A0 芯片使用 VDDIO 给系统 flash 供电配置宏
AD14N / AC104N	TCFG_KEEP_FLASH_POWER_GATE
AD15N	TCFG_KEEP_FLASH_POWER_GATE
AD17N	FLASH_POWER_GATE_VDDIO
AD18N	FLASH_POWER_GATE_VDDIO
AD24N	PCONTROL_FLASH_PG_VDDIO
AD23N	PCONTROL_FLASH_PG_VDDIO


```

34 /*****power_param*****/
35
36 #define TCFG_LOWPOWER_POWER_SEL      PWR_LD015          //电源模式设置, 可选DCDC和LD0
37 #define TCFG_LOWPOWER_BTOSC_DISABLE  0                //低功耗模式下BTOSC是否保持
38 #define TCFG_LOWPOWER_LOWPWR_SEL      SLEEP_EN          //SNIFF状态下芯片是否进入powerdown
39 /*mvdidiom: 2.0~3.4*/
40 #define TCFG_LOWPOWER_VDDIOM_LEVEL    VDDIOM_V0L_32V
41 /*wvddiom: 2.0~3.4*/
42 #define TCFG_LOWPOWER_VDDIOW_LEVEL    VDDIOW_V0L_28V    //弱VDDIO等级配置
43 #define TCFG_KEEP_FLASH_POWER_GATE    0                //power_gate引脚由用户控制
44 #define TCFG_LOWPOWER_DAC_OPEN        1                //出低功耗是否开dac电源, 对于频繁进出低功耗应用建议在dac使用前开dac电源
45
46 const struct low_power_param power_param = {
47     .config      = TCFG_LOWPOWER_LOWPWR_SEL,          //0: sniff时芯片不进入低功耗 1: sniff时芯片进入powerdown
48     .btosc_hz     = 16000000,                        //外按晶振频率
49     .delay_us     = 16000000 / 1000000L,             //提供给低功耗模块的延时(不需要需修改)
50     .btosc_disable = TCFG_LOWPOWER_BTOSC_DISABLE,     //进入低功耗时BTOSC是否保持
51     .vddiom_lev    = TCFG_LOWPOWER_VDDIOM_LEVEL,      //强VDDIO等级, 可选: 2.0V 2.2V 2.4V 2.6V 2.8V 3.0V 3.2V 3.6V
52     .vddiow_lev    = TCFG_LOWPOWER_VDDIOW_LEVEL,      //弱VDDIO等级, 可选: 2.1V 2.4V 2.8V 3.2V
53     .osc_type     = OSC_TYPE_LRC,
54     .flash_pg      = TCFG_KEEP_FLASH_POWER_GATE,
55     .vdc13_cap_en  = 0,                              //根据vdc13引脚是否有外部电容来配置, 1.外挂电容 0.无外挂电容
56     .vddio_keep    = 0,
57 };
58
59 /*****wake_up_param*****/
60 const struct port_wakeup port0 = {
61     .pullup_down_enable = ENABLE,                    //配置I/O 内部上下拉是否使能
62     .edge               = FALLING_EDGE,              //唤醒方式选择, 可选: 上升沿\下降沿
63     .attribute          = BLUETOOTH_RESUME,           //保留参数
64 };
65 app/bsp/cpu/sh54/power_api.c

```

A0 芯片 flash 使用 VDDIO 供电配置

42.1.2 VDDIO 电压档位配置

- 芯片的 VDDIO 电压会随着系统状态的变化而不同；正常运行时使用强 VDDIO 档位（MVD-DIO），Powerdown/Softoff 时使用弱 VDDIO 档位（WVDDIO）；
- 修改 power_param 结构体中的 vddiom_lev 成员，可以配置芯片的强 VDDIO 电压；
- 修改 power_param 结构体中的 vddiow_lev 成员，可以配置芯片的弱 VDDIO 电压；
- power_param 的 vddio_keep 参数置为 1 时，进入 powerdown 前后的 VDDIO 电压不会发生变化，仅 AD14N / AD15N / AC104N 有效；
- AD17N 默认保持 VDDIO 电压档位；

芯片	默认强 VDDIO 档位	默认弱 VDDIO 档位
AD14N / AC104N	3.2v	2.8v
AD15N	3.2v	2.8v
AD17N	3.2v	3.2v
AD18N	3.2v	3.2v
AD24N	3.0v	2.8v
AD23N	3.3v	2.8v

```

34 /*****power_param*****/
35
36 #define TCFG_LOWPOWER_POWER_SEL      PWR_LD015          //电源模式设置, 可选DCDC和LD0
37 #define TCFG_LOWPOWER_BTOSC_DISABLE  0                  //低功耗模式下BTOSC是否保持
38 #define TCFG_LOWPOWER_LOWPOWER_SEL   SLEEP_EN           //SNIFF状态下芯片是否进入powerdown
39
40 /*#vddiom: 2.0~3.4*/
41 #define TCFG_LOWPOWER_VDDIOM_LEVEL    VDDIOM_VOL_32V
42 /*#vddiom: 2.0~3.4*/
43 #define TCFG_LOWPOWER_VDDIOW_LEVEL    VDDIOW_VOL_28V    //弱VDDIO等级配置
44 #define TCFG_KEEP_FLASH_POWER_GATE   0                  //power_gate引脚由用户控制
45 #define TCFG_LOWPOWER_DAC_OPEN        1                  //出低功耗是否开dac电源, 对于需要进出低功耗应用建议在dac使用前开dac电源
46
47 const struct low_power_param power_param = {
48     .config      = TCFG_LOWPOWER_LOWPOWER_SEL,          //0: sniff时芯片不进入低功耗 1: sniff时芯片进入powerdown
49     .btosc_hz     = 16000000,                          //外接晶振频率
50     .delay_us     = 1600000 / 1000000L,                //提供给低功耗模块的延时(不需要需修改)
51     .btosc_disable = TCFG_LOWPOWER_BTOSC_DISABLE,       //进入低功耗时BTOSC是否保持
52     .vddiom_lev    = TCFG_LOWPOWER_VDDIOM_LEVEL,        //强VDDIO等级, 可选: 2.0V 2.2V 2.4V 2.6V 2.8V 3.0V 3.2V 3.6V
53     .vddiow_lev    = TCFG_LOWPOWER_VDDIOW_LEVEL,        //弱VDDIO等级, 可选: 2.1V 2.4V 2.8V 3.2V
54     .osc_type     = OSC_TYPE_LRC,
55     .flash_pg     = TCFG_KEEP_FLASH_POWER_GATE,         //根据vdc13引脚是否有外部电容来配置, 1. 外挂电容 0. 无外挂电容
56     .vddio_keep   = 0,
57 };
58
59 /*****wakeup_param*****/
60 const struct port_wakeup port0 = {
61     .pullup_down_enable = ENABLE,                      //配置I/O 内部上下拉是否使能
62     .edge               = FALLING_EDGE,                //唤醒方式选择, 可选: 上升沿\下降沿
63     .attribute           = BLUETOOTH_RESUME,            //保留参数
64 };
65 app/bsp/cpu/sh54/power_api.c

```

VDDIO 电压档位配置示意图

42.2 低功耗休眠相关函数

42.2.1 函数 void sys_softoff()

此函数实现使系统进入软关机, 唤醒后系统复位重新运行;
 内有 gpio_config_soft_poweroff() 函数, 该函数为进入 softoff 前关闭相关影响功耗的功能, 如 audio;

42.2.2 函数 void sys_power_down(u32 usec)

此函数实现进入系统 Powerdown 或者 Poweroff, 可通过定时唤醒或 IO 触发唤醒, 其中参数:

- 1、usec: 唤醒时间 (微秒);
 - ①. -2: 静态睡眠, 需要等待按键唤醒;
 - ②. 非-
- 2: 睡眠时间, 单位us, 如1000000即1s后唤醒, 睡眠时间不可超过看门狗唤醒时间的一半

..note ::

用户若使用到 audio 部分或已初始化 audio 部分, 进入 powerdown 前需要关闭 audio, 避免功耗过高。具体操作可参考 sys_idle_deal() 函数。

注: 调用该函数时进入 powerdown 还是 poweroff, 取决于 TCFG_LOWPOWER_LOWPOWER_SEL。该宏等于 SLEEP_EN 时, 休眠会进入 powerdown; 等于 DEEP_SLEEP_EN 时休眠会进入 poweroff。

42.2.3 函数 void board_set_soft_poweroff(void)

此函数实现在进入软关机之前将 IO 口设置成高阻，若需要保留原来状态可修改该函数。

设置对应引脚之后，需要在 close_gpio() 函数中，将对应的 I/O 不设置为高阻态；

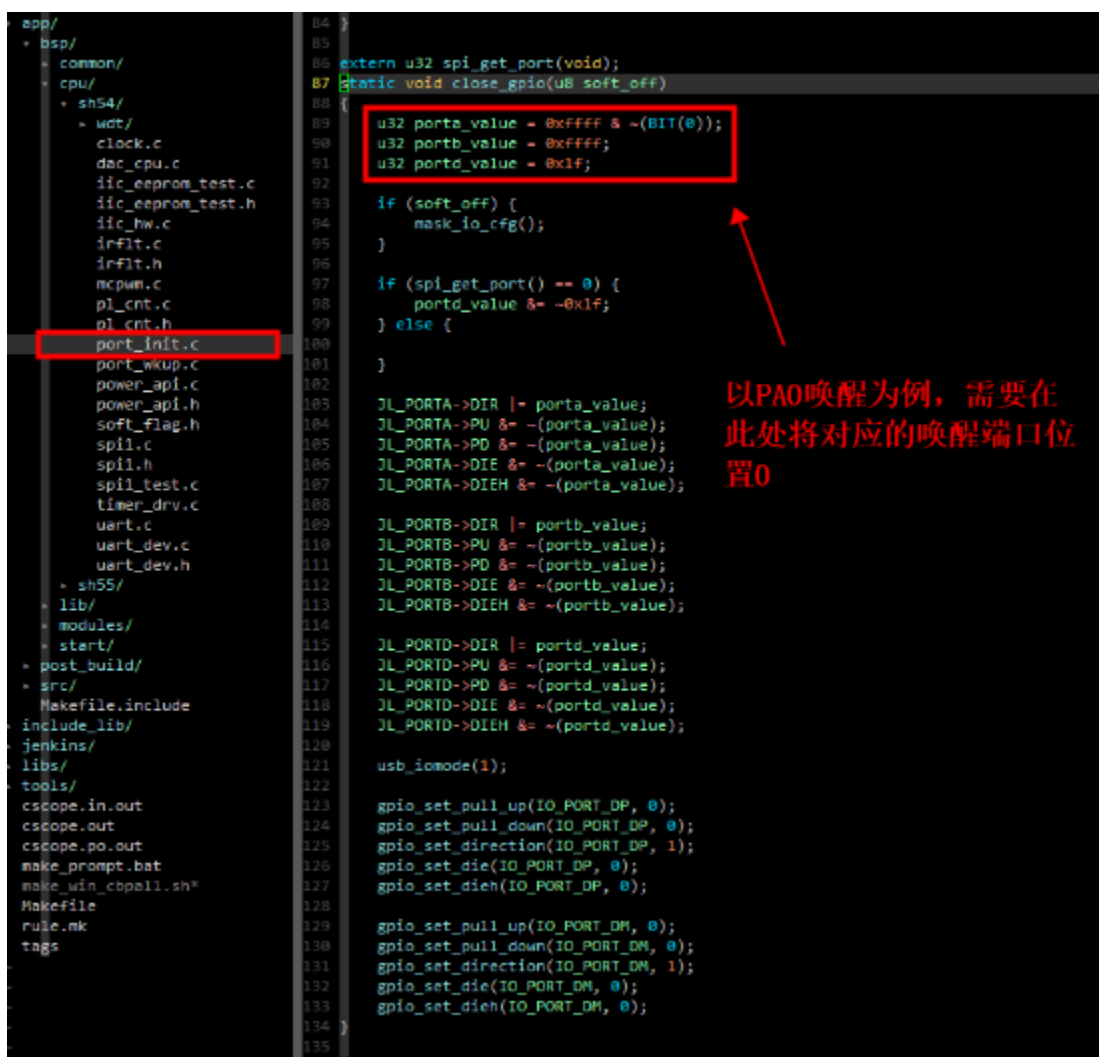


图 2 close_gpio 函数

备注:

用户若使用到 audio 部分或已初始化 audio 部分，进入 powerdown 前需要关闭 audio，避免功耗过高。具体操作可参考 gpio_config_soft_poweroff() 函数。（该函数为进入 softoff 的回调函数）

低功耗唤醒

芯片在进入了低功耗后，可通过预先配置唤醒源进行唤醒。

本章节介绍低功耗唤醒配置步骤。

1. 初始化唤醒口 -> “初始化唤醒口”；
 2. 进入低功耗前把对应唤醒口保护起来 -> “保护唤醒口”；
 3. 唤醒源获取 -> “唤醒源获取”；
-

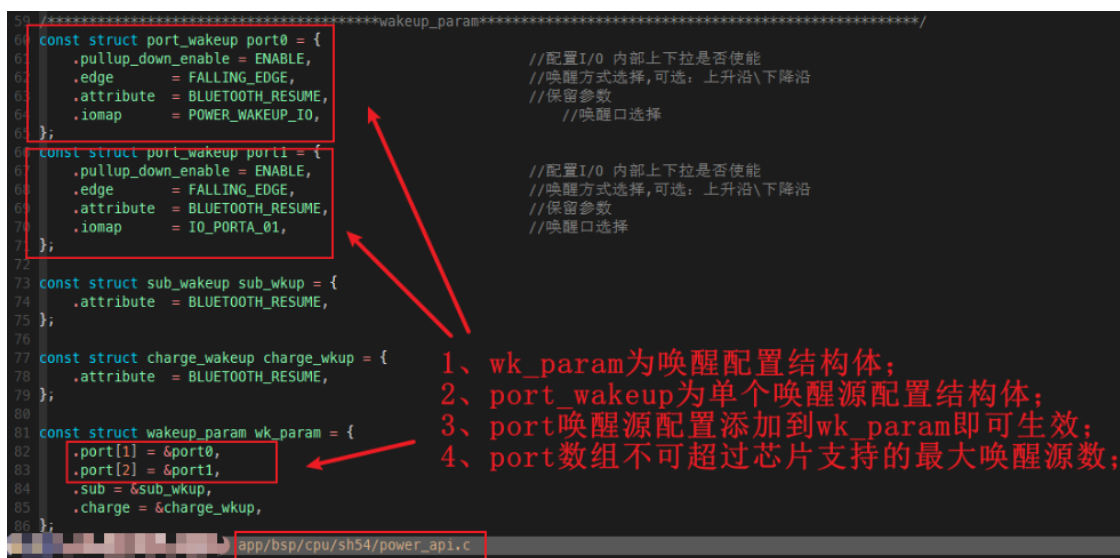
43.1 初始化唤醒口

不同芯片的初始化唤醒口略有不同，但大致流程一致：

1. AD14 / AC104 / AD15 初始化唤醒口配置 -> “AD14 / AC104 / AD15 初始化唤醒口配置”；
2. AD17 / AD18 初始化唤醒口配置 -> “AD17 / AD18 初始化唤醒口配置”；
3. AD23 / AD24 初始化唤醒口配置 -> “AD23 / AD24 初始化唤醒口配置”；

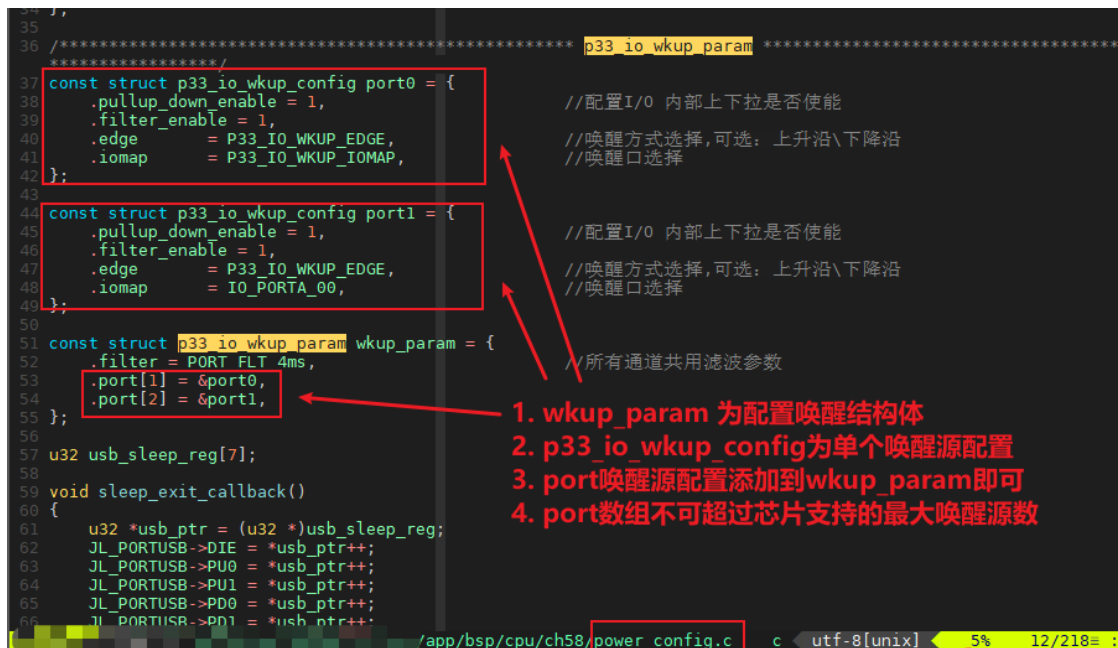
备注：唤醒源配置的滤波参数为所有 port 统一使用，非单个 port 使用。

43.1.1 AD14 / AC104 / AD15 初始化唤醒口配置



AD14 / AC104 / AD15 唤醒源配置

43.1.2 AD17 / AD18 初始化唤醒口配置



AD17_AD18 唤醒源配置

43.1.3 AD23 / AD24 初始化唤醒口配置



AD23 / AD24 唤醒源配置

43.1.4 保护唤醒口

芯片进入低功耗时会将所有 I/O 设置为高阻态；

所以配置好唤醒源后，还需要在进入低功耗流程中释放对应 I/O 引脚，以免对应 IO 状态被设置为高阻。

新添加的唤醒口需要单独释放，否则无法触发唤醒。

不同芯片保护唤醒口位置不一样，但是处理方式一样：

1. AD14 / AC104 / AD15 保护唤醒口 -> “AD14 / AC104 / AD15 保护唤醒口”；
2. AD17 / AD18 保护唤醒口 -> “AD17 / AD18 保护唤醒口”；
3. AD24 保护唤醒口 -> “AD24 保护唤醒口”；

43.1.5 AD14 / AC104 / AD15 保护唤醒口

AD14N / AD15N / AC104N:

```

115 void close_gpio(u8 soft_off)
116 {
117     u32 porta_value = 0xffff;
118     u32 portb_value = 0xffff;
119     u32 portd_value = 0x1f;
120
121     if (!(POWER_WAKEUP_IO & 0x10)) {
122         porta_value &= ~BIT(POWER_WAKEUP_IO);
123     } else {
124         portb_value &= ~BIT(POWER_WAKEUP_IO & (~0xfff0));
125     }
126
127     if (soft_off) {
128         mask_io_cfg();
129     }
130
131     if (spi_get_port() == 0) {
132         portd_value &= ~0x1f;
133         if (power_param.flash_pg) {
134             portd_value |= BIT(4);
135         }
136     } else {
137
138     }
139 }

```

低功耗时释放默认唤醒引脚，
新添加的唤醒口需手动将对应
I/O释放

app/bsp/cpu/sh54/port_init.c

AD14N_AD15N_AC104N 低功耗释放新增唤醒引脚

43.1.6 AD17 / AD18 保护唤醒口

AD17N / AD18:

```

79 void board_set_soft_poweroff()
80 {
81     u32 gpio_config[3] = {0xffff, 0xffff, 0xffff};
82     #define PORT_PROTECT(gpio) gpio_config[gpio/IO_GROUP_NUM] &= ~BIT(gpio%IO_GROUP_NUM)
83
84     PORT_PROTECT(POWER_WAKEUP_IO);
85
86     //MCLR
87     if (is_mclr_en()) {
88         PORT_PROTECT(MCLR_PORT);
89     }
90
91     //长按复位
92     if (is_pinr_en()) {
93         u8 port_sel = get_pinr_port();
94         if ((port_sel >= PA0_IN) && (port_sel <= PA12_IN)) {
95             PORT_PROTECT(IO_PORTA_00 + (port_sel - PA0_IN));
96         } else if ((port_sel >= PB0_IN) && (port_sel <= PB3_IN)) {
97             PORT_PROTECT(IO_PORTB_00 + (port_sel - PB0_IN));
98         }
99     }
100 }

```

AD17N低功耗时释放唤醒引脚，
新增的唤醒引脚需要做相同处理

app/bsp/cpu/sh57/power_config.c

AD17 / AD18 低功耗释放新增唤醒引脚

43.1.7 AD24 保护唤醒口

AD24N:



```

1 power_port.c
2 #include "asm/power_interface.h"
3 #include "app_config.h"
4 #define LOG_TAG_CONST      PMU
5 #define LOG_TAG            "[PMU]"
6 #include "log.h"
7
8
9
10 __attribute__((weak))
11 void board_gpio_config_soft_poweroff(u32 *gpio_config)
12 {
13     //注册在板卡
14 }
15
16 void gpio config soft poweroff(void)
17 {
18     PORT_TABLE(g);
19     log_info("-----%s-----", FUNCTION);
20     PORT_PROTECT(POWER_WAKEUP_IO);
21     board_gpio_config_soft_poweroff(gpio_config);
22     __port_init((u32)gpio_config);
23 }
24
25
26
27
28

```

添加对应需要保护不被设置成高阻的IO即可

AD24N 低功耗释放新增唤醒引脚

43.2 唤醒源获取

唤醒源获取功能只能获取 softoff 的唤醒源。

一、AD14N / AD15N / AC104N 唤醒源获取：

函数 u32 power_wakeup_reason(void)

此函数实现获取 softoff 唤醒源信息，该接口需要在系统初始化 sys_power_init() 之前调用，否则获取的唤醒源不准确，其中参数：

1、返回值：唤醒端口（对应wakeup_param中的port成员）；

二、AD17N / AD18N / AD23 / AD24N 唤醒源获取

函数 u8 is_wakeup_source(enum WAKEUP_REASON index)

此函数实现获取 softoff 唤醒源信息，外部可传参检查是否为本次唤醒的唤醒源；

获取 softoff 唤醒源需要在电源初始化 power_early_flowring() 之后调用，否则获取的唤醒源不准确，其中参数：

1、index：唤醒源；
2、返回值：1：传入的参数是本次的唤醒源，0：传入的参数非本次唤醒源；

PMU 复位功能介绍

本章节介绍 PMU 的复位功能。

芯片复位功能有以下几种：

1. 主动复位 -> “主动复位”；
2. 长按复位 -> “长按复位”；
3. MCLR 短按复位 -> “*MCLR* 短按复位”；

44.1 主动复位

用户可以调取

1. `chip_reset()`
2. `system_reset()`

44.2 长按复位

用户调用 `power_reset_init()` 初始化长按复位功能。

44.2.1 AD14N / AD15N / AC104N 长按复位功能

函数 `void power_reset_init(const struct reset_param *rs_param)` 实现长按复位初始化。

此函数，该接口需要在电源初始化的最后调用（必须要在唤醒口设置后设置）

其中 `rs_param` 结构体为：

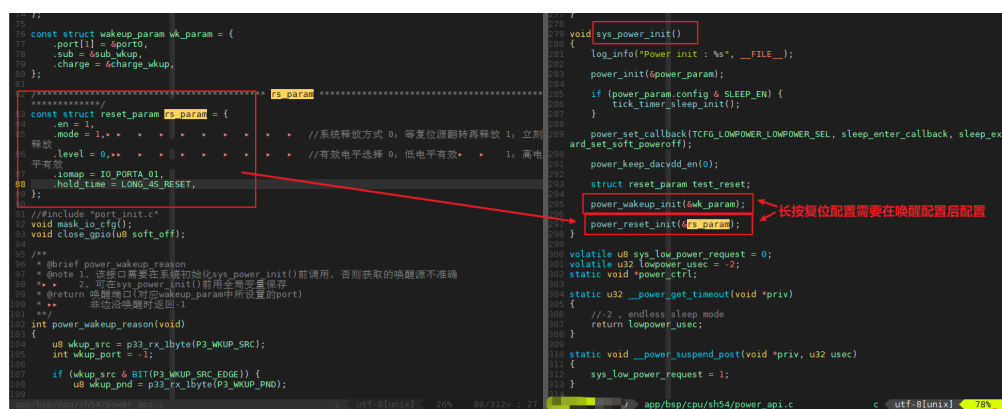
```
1 struct reset_param {
2     u8 en; //是否使能长按复位功能
3     u8 mode; //0:等待io电平翻转后复位, 1:长按时间到了立即复位;
4     u8 level; //长按复位高低电平选择, 0:低电平, 1:高电平;
5     u8 iomap; //长按复位io选择 (如 IO_PORTA_01)
6     int hold_time; //复位最小门槛时间选择
7 };
```

- 参数 `mode`：释放模式参考 AD18N 的复位模式说明” -> “参数：*release* - 释放模式”；

hold_time value	复位最小门槛时间
0	1s
1	2S
2	4S
3	8S
4~7	16S
other	非法

SDK 默认没有打开该功能。

用户若需要该功能，只需在唤醒 IO 设置后配置即可。



长按复位配置

44.2.2 AD17N / AD18N 长按复位功能

函数 `void gpio_longpress_pin0_reset_config(u32 pin, u32 level, u32 time, u8 release, u8 protect)` 长按复位初始化。

该接口需要在电源初始化的最后调用：

```
1 void gpio_longpress_pin0_reset_config(u32 pin, u32 level, u32 time, u8_
   ↪ release, u8 protect)
```

参数：

```
1.pin:      任意GPIO；
2.level:    0(下降沿触发) 1(上升沿触发)；
3.time:     0(disable) 1 2 4 8 16单位为秒；
4.release:  0(等待电平翻转后复位) 1(立即复位)
5.protect:  0(关闭写保护) 1(打开写保护)；
6.返回值:  无
```

示例：

```
1 // 配置：PA5，下降沿触发，长按4秒，等待释放后复位，关闭写保护
2 gpio_longpress_pin0_reset_config(IO_PORTA_05, 0, 4, 0, 0);
```

参数：pin - GPIO 引脚

类型：u32 (32 位无符号整数)

说明：指定用于长按复位的 GPIO 引脚编号

示例：IO_PORTA_05 等

参数：level - 触发电平

类型：u32

取值：

```
0: 下降沿触发 (从高电平变为低电平)
1: 上升沿触发 (从低电平变为高电平)
```

说明：指定什么电平变化会开始计时

参数：time - 长按时间

类型：u32

取值：

time value	复位最小门槛时间
0	disable
1	1S
2	2S
4	4S
8	8S
16	16S
other	非法

参数：release - 释放模式

类型：u8（8 位无符号字符）

取值：

- 0：等待电平翻转后复位（按键释放后才执行复位）
- 1：立即复位（达到时间后立即执行复位，不管按键是否释放）

备注： release = 0：等待电平翻转后复位

用户操作流程：

1. 用户按下复位按键（GPIO 变为低电平）
2. 系统开始计时 4 秒
3. 用户保持按下状态 4 秒以上
4. 系统检测到已满 4 秒，但不立即复位
5. 用户释放按键（GPIO 变回高电平）
6. **此时系统执行复位**

结果：用户必须完成”按下-保持-释放”的完整动作才会复位

备注： release = 1 立即复位模式

用户操作流程：

1. 用户按下复位按键（GPIO 变为低电平）
2. 系统开始计时 4 秒
3. 用户保持按下状态达到 4 秒
4. 系统立即执行复位，不管用户是否还在按住按键
5. 系统复位后，按键状态被忽略

结果：只要按下时间达到 4 秒，系统就立即复位

参数: protect - 写保护 (AD17N 无此功能)

AD18N 默认的长按复位 IO 为 PA0;

类型: u8

取值: :

0: 关闭写保护 (可以直接修改配置)
1: 打开写保护 (防止意外修改配置)

备注: protect = 1, 开机首次调用函数 gpio_longpress_pin0_reset_config 后, 再次调用此函数将无效, 并会锁定默认长按复位 IO 状态。

1. 示例 1, 默认长按复位 IO (PA0)

```
1 // 场景: 配置默认复位引脚 PA0 并开启写保护
2 gpio_longpress_pin0_reset_config(IO_PORTA_00, 0, 4, 1, 1);
```

备注: 配置默认长按复位 IO, 上电首次调用函数后

- ✓ 配置锁定: 无法再次调用此函数修改复位配置
- ✓ IO 状态锁定: PA5 引脚的电平状态也无法被其他代码修改
- ✓ 双重保护: 配置和硬件状态都被锁定

2. 示例 2, 普通 GPIO 引脚并开启写保护:

```
1 // 场景: 配置默认复位引脚 PA5 并开启写保护
2 gpio_longpress_pin0_reset_config(IO_PORTA_05, 0, 4, 1, 1);
```

备注: 配置其他 GPIO 引脚, 上电首次调用函数后

- ✓ 配置锁定: 无法再次调用此函数修改复位配置
- ✗ IO 状态未锁定: PA5 引脚的电平状态可以被其他代码修改
- △ 单层保护: 只锁定配置, 不锁定硬件状态

44.2.3 AD24N / AD23N 长按复位功能

函数 void gpio_longpress_pin0_reset_config(u32 pin, u32 level, u32 time, u32 release, enum gpio_mode pullup_down_mode)

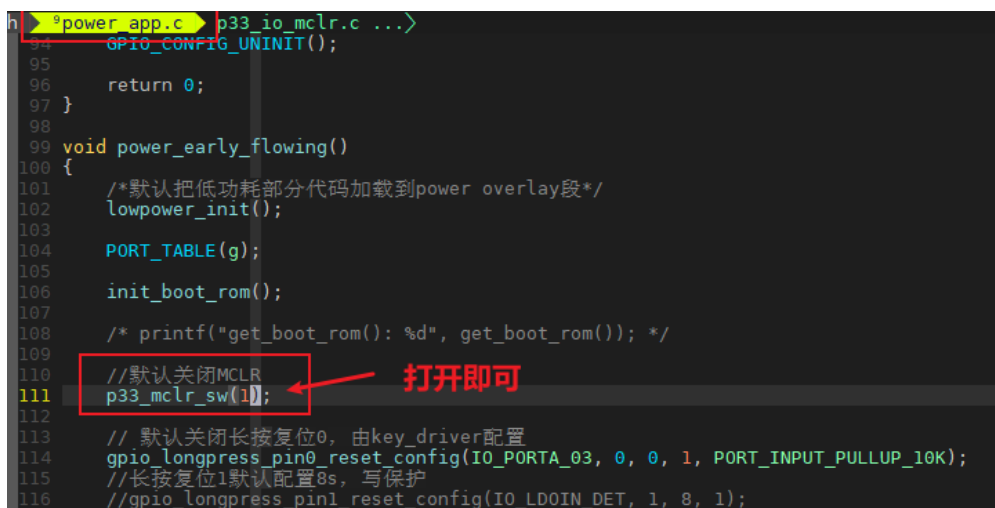
此函数实现长按复位初始化，该接口需要在电源初始化的最后调用，其中参数：

- 1、pin: 复位I/O; (如: IO_PORTA_01)
- 2、level: 触发方式
 - ①、0: 下降沿触发;
 - ②、1: 上升沿触发;
- 3、time: 时间
 - ①、0: disable;
 - ②、1: 1s复位;
 - ③、2: 2s复位;
 - ④、4: 4s复位;
 - ⑤、8: 8s复位;
 - ⑥、16: 16s复位
- 3、release: 系统释放方式
 - ①、0: 等待电平翻转后复位;
 - ②、1: 立即复位;
- 4、pullup_down_mode: 该IO初始状态; f
用户根据gpio_mode枚举选择对应的IO状态即可;
选择PORT_KEEP_STATE 即不操作该IO状态。

44.3 MCLR 短按复位

AD24N / AD23N 支持 MCLR 复位 (MCLR 与 PA8 绑定)。

用户可以在 power_app.c 里打开 p33_mclr_sw(1) 即可。



```

94  GPIO_CONFIG_UNINIT();
95
96  return 0;
97 }
98
99 void power_early_flowng()
100 {
101     /*默认把低功耗部分代码加载到power overlay段*/
102     lowpower_init();
103
104     PORT_TABLE(g);
105
106     init_boot_rom();
107
108     /* printf("get_boot_rom(): %d", get_boot_rom()); */
109
110     //默认关闭MCLR
111     p33_mclr_sw(1);
112
113     // 默认关闭长按复位0. 由key_driver配置
114     gpio_longpress_pin0_reset_config(IO_PORTA_03, 0, 0, 1, PORT_INPUT_PULLUP_10K);
115     //长按复位1默认配置8s, 写保护
116     //gpio_longpress_pin1_reset_config(IO_LDOIN_DET, 1, 8, 1);
  
```

AD24N_MCLR 复位配置

44.4 复位源获取

一、AD24N / AD23N 复位源函数如下：

函数 `bool is_reset_source(enum RST_REASON index);`

获取复位源需要在电源初始化 `power_early_flowng()` 之后调用，否则获取的唤醒源不准确，其中参数：

- 1、`index`：复位源；
- 2、返回值：1：传入的参数是本次的复位源，0：传入的参数非本次复位源；

低功耗复用功能

AD24N 支持低功耗部分代码、变量与其他功能复用，从而节省空间。
默认 SDK 低功耗部分暂无与其他功能进行复用。

下面介绍如何把低功耗部分与解码部分的功能的进行复用，从而节省 ram 空间

45.1 低功耗复用示例

该示例为低功耗与解码格式复用。
经测试，该示例在 AD24N 最多可节省 1.8k 空间。

45.1.1 打开低功耗复用功能宏

在 app_config.h 文件里打开 TCFG_LOWPOWER_OVERLAY 宏

```
131 #endif
132
133 //*****
134 // PMU Configuration
135 //*****
136 #define TCFG_LOWPOWER_POWER_SEL PWR_LD015 //电源模式设置，可选DCDC和LDO
137 #define TCFG_LOWPOWER_BTOSC_DISABLE 0 //低功耗模式下BTOSC是否保持
138 #define TCFG_LOWPOWER_LOWPOWER_SEL 1//DEEP_SLEEP_EN //SNIFF状态下芯片是否进入powerdown
139 #define TCFG_LOWPOWER_PATTERN SOFT_MODE//SOFT_BY_POWER_MODE //选择软关机的方式
140 #define TCFG_LOWPOWER_VDDIOM_LEVEL VDDIOM_VOL_30V
141 #define TCFG_LOWPOWER_VDDIOW_LEVEL VDDIOW_VOL_28V //弱VDDIO等级配置
142 #define TCFG_LOWPOWER_OSC_TYPE OSC_TYPE_LRC
143 #define TCFG_LOWPOWER_SOFT 1
144 #define TCFG_LOWPOWER_OVERLAY 0
145
146 #endif
147
/app/src/voice_toy/app_config.h
```


打开低功耗复用功能宏

45.1.2 修改链接文件

1. 打开 app_ld.c 文件，找到低功耗使用的 ram 空间

如图为低功耗使用的 ram 空间暂时单独出来，并无与其他功能复用。

```

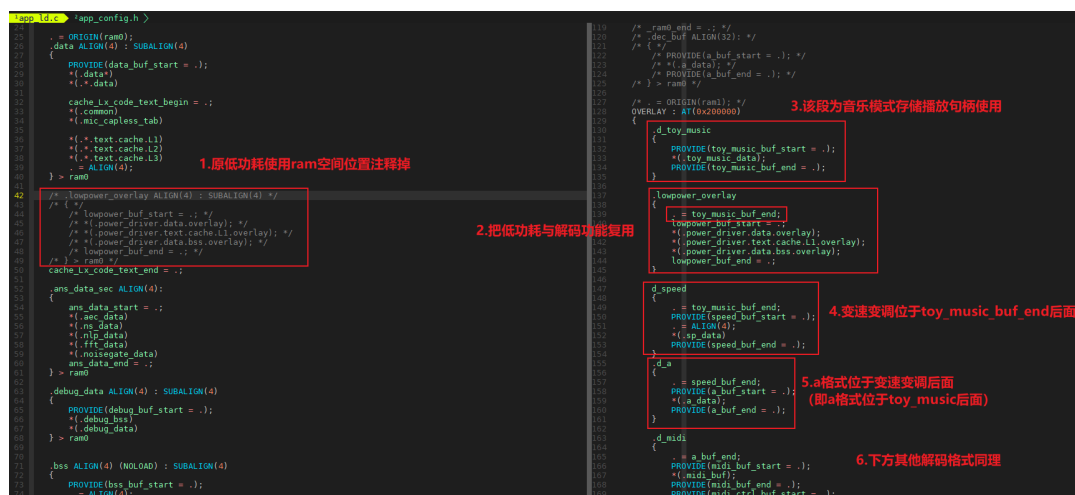
1app_ld.c 2app_config.h >
24
25     . = ORIGIN(ram0);
26     .data ALIGN(4) : SUBALIGN(4)
27     {
28         PROVIDE(data_buf_start = .);
29         *(.data*)
30         *(*.data)
31
32         cache_Lx_code_text_begin = .;
33         *(.common)
34         *(.mic_capless_tab)
35
36         *(*.text.cache.L1)
37         *(*.text.cache.L2)
38         *(*.text.cache.L3)
39         . = ALIGN(4);
40     } > ram0
41
42     .lowpower_overlay ALIGN(4) : SUBALIGN(4)
43     {
44         lowpower_buf_start = .;
45         *(.power_driver.data.overlay);
46         *(.power_driver.text.cache.L1.overlay);
47         *(.power_driver.data.bss.overlay);
48         lowpower_buf_end = .;
49     } > ram0
50     cache_Lx_code_text_end = .;
51
52     .ans_data_sec ALIGN(4):
53     {
54         ans_data_start = .;
55         *(.ans_data)

```

低功耗使用的 ram 空间

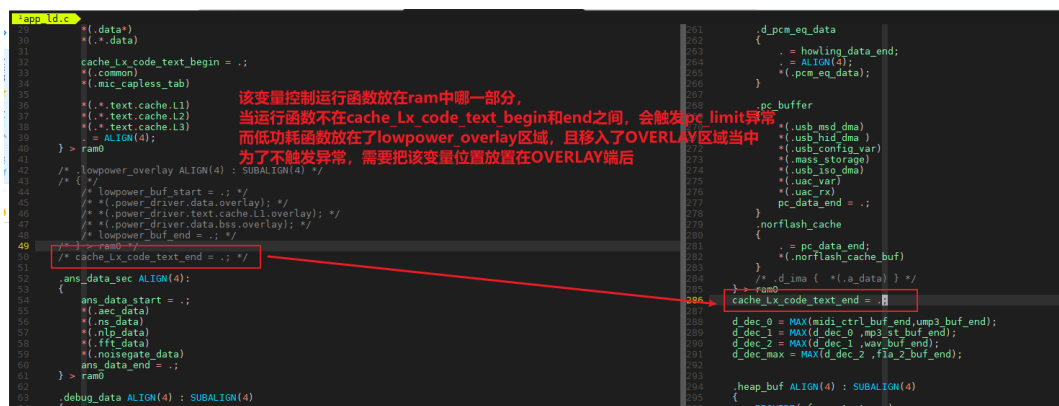
2. 修改 lowpower_overlay 段位置，与目标功能复用

如图为将原本低功耗使用的 ram 空间移入 OVERLAY 区域，并与解码格式进行复用。



低功耗复用解码功能

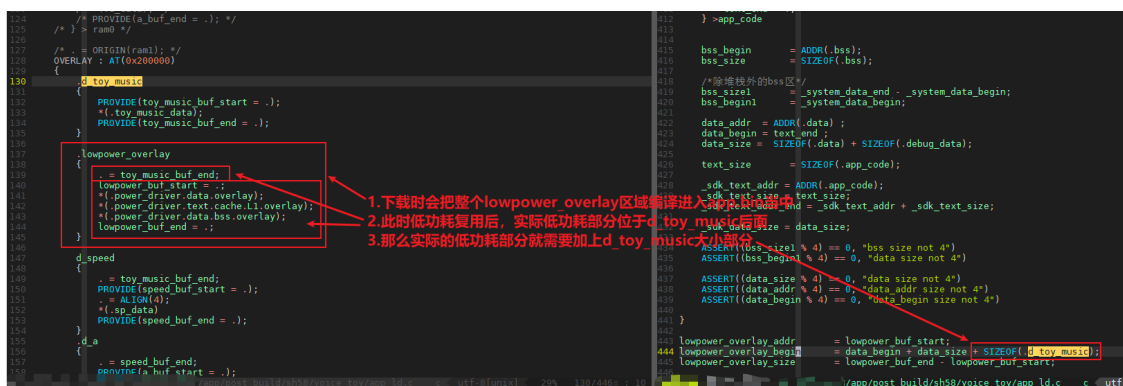
3. 调整 cache_Lx_code_text_end 位置



调整 pc_limit 范围

4. 调整 lowpower_overlay_begin 位置

该变量为低功耗从 app.bin 当中加载回低功耗代码地址



调整 lowpower_overlay_begin 位置

45.1.3 修改应用层与低功耗复用的功能

当前示例中，低功耗与解码功能复用同一块 ram 空间，即两者功能 **不能** 同时执行。

常见的在音乐模式里，默认 SDK 是 500ms 的时候判断到当前没有在播放歌曲，就会进入 powerdown 当中。

当低功耗与解码复用后，如暂停音乐后，500ms 消息判断到当前没有播放音乐，就会进入 powerdown。

此时原本存储解码信息的 ram 空间被破坏用于了低功耗执行，等到退出低功耗后，解码功能需要重新执行初始化才可以使用。

通俗来讲：解码的暂停功能在进入低功耗后已失效，因为重新播放需要获取原本解码位置信息，但存储这些信息的 ram 空间都被破坏用于了低功耗使用。

下图把暂停功能改为关闭解码模块，避免退出低功耗后，继续使用暂停播放功能导致异常。

```

33         break;
34
35     case MSG_PP:
36         log_info("MSG_PP\n");
37         //原有暂停函数为停止解码，但是相关原本的解码信息还是保留的
38         /* decoder_pause(dec_pctl[0].p_dec_obj); */
39         //需要把整个解码模块关闭掉，避免进入低功耗时相关解码动作还在进行
40         decoder_stop(dec_pctl[0].p_dec_obj, NEED_WAIT);
41         break;
42     case MSG_PREV_FILE:
43         log_info("MSG_PREV_FILE\n");
44         play_prev_file(&dec_pctl[0]);
45         break;
46     case MSG_NEXT_FILE:
47         log_info("MSG_NEXT_FILE\n");
48
49 NORMAL ▶ dev_2.0 <bmit_v2/app/src/voice_toy/toy_music/toy_music.c[+] c utf-8[

```

修改音乐模式与低功耗复用的功能

备注：用户若复用其他功能，需要注意复用的功能使用的 RAM 空间是否会被低功耗破坏，做好初始化或者其他规避手段。

45.1.4 最终

编译，查看 heap_buf 空间多出空间大小。

修改前，剩余 RAM 空间为 0x10af;

修改后，剩余 RAM 空间为 0x17cf;

经对比，多出约 1.8K 空间；

(codeblock 版本可以查看 logs & others 里的 build_log 界面)

Sections:					Sections:				
Idx Name	Size	Address	Type		Idx Name	Size	Address	Type	
0	00000000	0000000000000000			0	00000000	0000000000000000		
1 .boot_data	00000024	000000000030beec	DATA		1 .boot_data	00000024	000000000030beec	DATA	
2 .data	0000089c	0000000000304000	TEXT DATA		2 .data	0000089c	0000000000304000	TEXT DATA	
3 .lowpower_overlay	00000708	000000000030489c	TEXT DATA		3 .bss	00002080	000000000030489c	BSS	
4 .bss	00002080	0000000000304fa4	BSS		4 .ans_bss_sec	00000004	000000000030691c	BSS	
5 .ans_bss_sec	0000001c	0000000000307024	BSS		5 .effect_buf	00000000	0000000000306920	DATA	
6 .effect_buf	00000000	0000000000307040	DATA		6 .d_toy_music	000001ac	0000000000306920	DATA	
7 .d_toy_music	000001ac	0000000000307040	DATA		7 .lowpower_overlay	000008c9	0000000000306920	TEXT DATA	
8 .d_speed	00000d98	0000000000307040	DATA		8 .d_speed	00000d98	0000000000306920	DATA	
9 .d_a	000012d0	0000000000307040	DATA		9 .d_a	000012d0	0000000000306920	DATA	
10 .d_midi	00003dfc	0000000000307040	DATA		10 .d_midi	00003dfc	0000000000306920	DATA	
11 .d_ump3	00003208	0000000000307040	DATA		11 .d_ump3	00003208	0000000000306920	DATA	
12 .d_mp3_st	000012d0	0000000000307040	BSS		12 .d_mp3_st	000012d0	0000000000306920	BSS	
13 .d_wav	000012d0	0000000000307040	BSS		13 .d_wav	000012d0	0000000000306920	BSS	
14 .d_fla	00003648	0000000000307040	DATA		14 .d_fla	00003648	0000000000306920	DATA	
15 .d_rec	00001420	0000000000307040	DATA		15 .d_rec	00001420	0000000000306920	DATA	
16 .d_enc_ima	00001420	0000000000307040	BSS		16 .d_enc_ima	00001420	0000000000306920	BSS	
17 .d_enc_mp3	00002538	0000000000307040	DATA		17 .d_enc_mp3	00002538	0000000000306920	DATA	
18 .d_aux	00000434	0000000000307040	DATA		18 .d_aux	00000434	0000000000306920	DATA	
19 .d_vp_data0	00001d84	0000000000307040	DATA		19 .d_vp_data0	00001d84	0000000000306920	DATA	
20 .d_vp_data1	00001d84	0000000000307040	BSS		20 .d_vp_data1	00001d84	0000000000306920	BSS	
21 .d_vc_data	00001d84	0000000000307040	BSS		21 .d_vc_data	00001d84	0000000000306920	BSS	
22 .d_echo_data	00003288	0000000000307040	DATA		22 .d_echo_data	00003288	0000000000306920	DATA	
23 .d_pcm_eq_data	00001d84	0000000000307040	BSS		23 .d_pcm_eq_data	00001d84	0000000000306920	BSS	
24 .pc_buffer	00000d44	0000000000307040	DATA		24 .pc_buffer	00000d44	0000000000306920	DATA	
25 .norflash_cache	00000d44	0000000000307040	BSS		25 .norflash_cache	00000d44	0000000000306920	BSS	
26 .heap_buf	000010af	000000000030ae3c	BSS		26 .heap_buf	000017cf	000000000030a71c	BSS	
27 .app_code	0002ab00	0000000000c000100	TEXT DATA		27 .app_code	0002ab00	0000000000c000100	TEXT DATA	
28 .debug_str	0001831f	00000000000000000			28 .debug_str	0001831f	00000000000000000		
29 .debug_loc	00014ff6	00000000000000000			29 .debug_loc	00014ff6	00000000000000000		
30 .debug_abbrev	00000e82	00000000000000000			30 .debug_abbrev	00000e82	00000000000000000		
31 .debug_info	00043862	00000000000000000			31 .debug_info	00043862	00000000000000000		
32 .debug_ranges	00002218	00000000000000000			32 .debug_ranges	00002218	00000000000000000		
33 .debug_macinfo	00000001	00000000000000000			33 .debug_macinfo	00000001	00000000000000000		
34 .debug_pubnames	0000b3e9	00000000000000000			34 .debug_pubnames	0000b3e9	00000000000000000		
35 .debug_pubtypes	00007c5d	00000000000000000			35 .debug_pubtypes	00007c5d	00000000000000000		
36 .debug_line	00025ffd	00000000000000000			36 .debug_line	00025ffd	00000000000000000		
37 .debug_aranges	00000028	00000000000000000			37 .debug_aranges	00000028	00000000000000000		
38 .shstrtab	000001b8	00000000000000000			38 .shstrtab	000001b8	00000000000000000		
39 .symtab	000591a0	00000000000000000			39 .symtab	000591a0	00000000000000000		
40 .strtab	0000f451	00000000000000000			40 .strtab	0000f451	00000000000000000		
host-client -project sh58_rom -f app.bin sdk.elf isd_config.ini ./voic					host-client -project sh58_rom -f app.bin sdk.elf isd_config.ini .				

修改前后的剩余 RAM 空间大小对比

IVD 低电管理

用户可以在一拖二烧写时进行配置 lvd，也可以在 SDK 里配置 lvd 电压，量产时建议在烧写阶段配置 LVD 电压。

下面分为几个部分来介绍 lvd 功能：

1. “lvd 配置 *lvd_config*”
2. “获取 lvd 电压 *get_lvd_vol*”

46.1 lvd 配置 *lvd_config*

该接口为配置 lvd 电压接口函数。

参数：

- 1.LVD_VOL vol : 配置的 lvd 电压。（用户可查看 *p33_api.h* 支持的 lvd 电压）
- 2.u8 expin_en : 外部 IO 引脚（PA1）检测电压使能
- 3.LVD_MODE mode : LVD 模式。（用户可查看 *p33_api.h* 支持的 lvd 模式）
- 4.void (*callback) : 回调函数

备注：

expin_en 使能

打开该使能后，LVD 只针对外部 IO 引脚（PB0）做 LVD 检测。

该外部 IO 引脚不可更改。（即 PB0）

原型：

```
void lvd_config(LVD_VOL vol, u8 expin_en, LVD_MODE mode, void (*callback));
```

46.2 获取 lvd 电压 get_lvd_vol

该接口函数为获取 lvd 档位对应的电压。

返回值：对应电压。

```
u32 get_lvd_vol(void);
```

部分芯片低功耗调试配置

本文档描述了部分芯片中低功耗相关的调试配置选项。

这些配置分为两部分：打印调试控制和实际运行代码控制, 通过预编译常量控制特定功能的使能状态。

开关这些控制位，可以控制代码大小。

47.1 各个系列芯片支持的配置情况

备注：

表格符号含义：

1. √ 表示已经实现
 2. × 表示在当前项目中未实现
-

47.1.1 修改后禁止用于生产的配置

表一 AD1x-4578_AC104 SDK 低功耗配置支持情况，此表内容修改后禁止用于生产					
序号	配置项	AD14N	AD15N	AD17N	CD09(CH58)
1	debug_is_idle	×	×	√	√

47.1.2 允许修改的配置

表二 AD1x-4578_AC104 SDK 低功耗配置支持情况，此表内容允许修改					
序号	配置项	AD14N	AD15N	AD17N	CD09(CH58)
1	control_soff	√	√	√	×
2	control_pdown	√	√	√	×
3	control_poff	×	×	×	×

47.1.3 严禁修改的配置

表三 AD1x-4578_AC104 SDK 低功耗配置支持情况，此表内容严禁修改					
序号	配置项	AD14N	AD15N	AD17N	CD09(CH58)
1	pdebug_xosc_resume	×	×	×	×
2	pdebug_pdown_info	×	×	×	×
3	pdebug_uart_lowpower	×	×	×	×
4	pdebug_uart_port	×	×	×	×
5	pdebug_putbyte_pdown	×	×	×	×
6	pdebug_putbyte_soff	×	×	×	×
7	pdebug_lp_dump_ram	×	×	×	×
8	pdebug_uart_flowring	×	×	×	×
9	pdebug_reserve_time	×	×	×	×
10	control_power_reset_mark	×	×	×	×
11	control_power_soft_reset	×	×	×	×
12	control_power_pinr_reset	×	×	×	×
13	control_power_pinr_reset	×	×	×	×
14	control_power_latch_reset	×	×	×	×
15	control_power_reset	×	×	×	×
16	control_power_wakeup_mark	×	×	×	×
17	control_power_wakeup	×	×	×	×

47.2 第一部分：打印调试控制

此部分配置用于调试目的，通过串口输出调试信息，不影响实际功能逻辑。

47.2.1 空闲状态调试

1. 功能:

系统调用休眠函数，但还没有完全进入低功耗休眠时的打印信息。

```
1 const char debug_is_idle = 0;
```

2. 输出信息:

- 蓝牙和系统可进入低功耗的时间（毫秒）
- 当前处于 busy 状态的模块
- 若 usr_timer 忙，则打印对应的函数地址

47.3 第二部分：实际运行代码控制

此部分配置控制实际功能模块的编译与运行。

47.3.1 PMU 相关模块开关

1. 使能 pdown 模块:

```
1 #if 1 // 由 TCFG_LOWPOWER_LOWPOWER_SEL 配置决定
2 const bool control_pdown = 1;
3 #else
4 const bool control_pdown = 0;
5 #endif
```

2. 使能 SOFF 模块:

```
1 const bool control_soff = 1; // 通常由 TCFG_LOWPOWER_SOFF 配置
```

3. 使能 poff 模块:

```
1 #if 0 // 由 TCFG_LOWPOWER_LOWPOWER_SEL 配置决定
2 const bool control_poff = 1; // DEEP_SLEEP_EN 时使能
3 #else
4 const bool control_poff = 0;
5 #endif
```

VDDIO 电压配置说明

VDDIO : 芯片引脚出来的电压。

48.1 VDDIO 作用域



AD17 vddio 作用域

上电过程，即电源上电，可能会存在抖动，一般会设置比较高的 VDDIO 档位。

Boot-loader 和 SDK，根据 flash 运行所需的电压，VDDIO&VBAT 负载情况，重新调整 VDDIO 档位。

48.2 如何配置每个阶段的 VDDIO 档位

48.2.1 上电过程：

芯片 VDDIO 会有一个默认电压值；该电压值对应电源上电时刻，由于此时电源可能会存在抖动，所以一般会设置比较高的 VDDIO 档位。

48.2.2 Boot-Loader:

即 uboot 工程, 会从 ini 文件里面读取 VDDIO 配置, 重新调整 VDDIO 档位。

48.2.3 SDK:

AC104, AD14, AD15 在 power_api.c 里配置 power_param 结构体的 vddiom_lev 成员, 即可修改 VDDIO 档位。如下图:

```

1 #include "log.h"
2
3 /*****power_param*****/
4
5 #define TCFG_LOWPPOWER_POWER_SEL      PWR_LD015          //电源模式设置, 可选DCDC和LDO
6 #define TCFG_LOWPPOWER_BTOSC_DISABLE  0                //低功耗模式下BTOSC是否保持
7 #define TCFG_LOWPPOWER_LOWPPOWER_SEL  SLEEP_EN          //SNIFF状态下芯片是否进入powerdown
8 /*mvdidiom: 2.0~3.4*/
9 #define TCFG_LOWPPOWER_VDDIOM_LEVEL    VDDIOM_VOL_32V
10 /*wvddidiom: 2.0~3.4*/
11 #define TCFG_LOWPPOWER_VDDIOW_LEVEL    VDDIOW_VOL_28V    //弱VDDIO等级配置
12 #define TCFG_KEEP_FLASH_POWER_GATE    0                //power_gate引脚由用户控制
13 #define TCFG_LOWPPOWER_DAC_OPEN       1                //出低功耗是否开dac电源, 对于频繁进出低功耗应用建议在dac使用前开dac电源
14
15 const struct low_power_param power_param = {
16     .config      = TCFG_LOWPPOWER_LOWPPOWER_SEL,          //0: sniff时芯片不进入低功耗 1: sniff时芯片进入powerdown
17     .btosc_hz    = 16000000,                               //外接晶振频率
18     .delay_us    = 16000000 / 1000000L,                   //提供给低功耗模块的延时(不需要需修改)
19     .btosc_disable = TCFG_LOWPPOWER_BTOSC_DISABLE,        //进入低功耗时BTOSC是否保持
20     .vddiom_lev   = TCFG_LOWPPOWER_VDDIOM_LEVEL,          //强VDDIO等级, 可选: 2.0V 2.2V 2.4V 2.6V 2.8V 3.0V
21     .vddiow_lev   = TCFG_LOWPPOWER_VDDIOW_LEVEL,          //弱VDDIO等级, 可选: 2.1V 2.4V 2.8V 3.2V
22     .osc_type     = OSC_TYPE_LRC,
23     .flash_pg     = TCFG_KEEP_FLASH_POWER_GATE,
24     .vdc13_cap_en = 0,                                     //根据vdc13引脚是否有外部电容来配置, 1.外挂电容 0.无
25     .vddio_keep   = 0,
26 };

```

AC104, AD14, AD15 配置 vddio 档位

AD17, AD18, AD24, AD23 在 power_config.c 里配置 power_param 结构体的 vddiom_lev 成员, 即可修改 VDDIO 档位。如下图:

```

1 #include "log.h"
2
3 #define P33_IO_WKUP_IOMAP      POWER_WAKEUP_IO
4 #define P33_IO_WKUP_EDGE      FALLING_EDGE
5 #define P33_IO_LONG_PRESS_RESET TO_PORTB_A1
6 #define VDDIOM_VOL_CONFIG      VDDIOM_VOL_32V
7 #define FLASH_POWER_GATE_VDDIO 0 //flash power接vddio时配置为1, 内封flash配置为0
8
9 const struct low_power_param power_param = {
10     .config      = SLEEP_EN,          //0: sniff时芯片不进入低功耗 1: sniff时芯片进入powerdown
11     .btosc_hz    = 16000000,          //外接晶振频率
12     .vddiom_lev   = VDDIOM_VOL_CONFIG, //强VDDIO等级, 可选: 2.0V 2.2V 2.4V 2.6V 2.8V 3.0V 3.2V 3.6V
13     .osc_type     = OSC_TYPE_LRC,     //低功耗使用时钟
14 };
15
16 const struct p33_io_wkup_config port0 = {
17     .pullup_down_enable = ENABLE,     //配置I/O 内部上下拉是否使能
18     .edge               = P33_IO_WKUP_EDGE, //唤醒方式选择, 可选: 上升沿\下降沿
19     .iomap              = P33_IO_WKUP_IOMAP, //唤醒口选择
20 };

```

AD17, AD18 配置 vddio 档位

48.3 常见问题介绍

48.3.1 VDDIO 偏差

实际值与配置值相差 0.2V 为正常现象。

偏差过大请联系助理。

单备份升级说明

AD14 只支持设备升级（即 SD 卡升级和 U 盘升级）
该升级方式为单备份升级。

正常的升级流程：

SDK -> 调用升级接口 -> 校验文件 -> 校验成功 -> 复位 -> 进入 loader 进行升级 -> 升级完毕，
复位 -> 进入升级后的 SDK

在进入 loader 进行升级前，任何错误都会导致升级退出，将错误的返回值返回到主流程。
而单备份升级，在进入 loader 升级后，旧 SDK 程序已经被破坏，无法再回到旧 SDK。
此时若遇到断电等错误操作，可重新上电插卡（U 盘）继续升级，直到升级成功会自行复位进入
升级后的 SDK。

下面分几部分进行介绍：

1. “单备份升级流程”；
 2. “单备份升级相关功能开关”；
 3. “单备份升级相关函数接口”；
-

49.1 单备份升级流程

1. 使用对应的 SD 卡或者 U 盘升级需要先打开对应设备的宏。

sd 卡由在 app_modules.h 里的 TFG_SD_EN 控制

U 盘由在 app_config.h 里的 TCFG_UDISK_ENABLE 控制

2. 设备升级需要使用到 fat 文件系统，默认 AD14 使用的是 simple_fat 文件系统即可。

3. 调用对应的升级入口

try_to_upgrade(对应的设备，升级文件的路径);

在该升级入口里会打开对应的设备，查看对应路径下的升级文件。

(用户可参考 common_msg.c 里的 sd 卡上线消息处理升级流程)

```

63
64 /* 设备升级 */
65 #if defined(TCFG_UDISK_ENABLE) && (TCFG_UDISK_ENABLE)
66     case MSG_USB_DISK_IN:
67         log_info("udisk in\n");
68         usb_host_mount(0, 3, 20, 200);
69 #endif
70 #if defined(TFG_SD_EN) && (TFG_SD_EN)
71     case MSG_SDMC_A_IN:
72 #endif
73 #if defined(TFG_DEV_UPGRADE_SUPPORT) && (1 == TFG_DEV_UPGRADE_SUPPORT)
74     u8 dev_idx = *msg - MSG_USB_DISK_IN;
75     device_update(dev_idx);
76 #endif
77     break;
78
79 #if defined(TCFG_UDISK_ENABLE) && (TCFG_UDISK_ENABLE)
80     case MSG_USB_DISK_OUT:
81         log_info("udisk out\n");
82         usb_host_unmount(0);
83         break;
84 #endif
85 #if defined(TFG_SD_EN) && (TFG_SD_EN)
86     case MSG_SDMC_A_OUT:
87         break;
88 #endif
89 #if TCFG_PC_ENABLE
90     case MSG_USB_PC_IN:
91     case MSG_PC_IN:

```

1. 检测到U盘 / sd卡上线

2. 进入升级

/app/src/voice_toy/common/common_msg.c c utf-8[u

4. 校验文件，成功即进入升级，升级成功会自动复位。

```

[14:20:51.249] [Info] [normal]file_len = 0x27000
[14:20:51.249] [Info] [normal]file name : flash.bin
[14:20:51.249] [Info] [normal]file name : info.log
[14:20:51.249] [Info] [normal]file name : uboot.version
[14:20:51.249] [Info] [normal]file name : isd_config.ini
[14:20:51.249] [Info] [normal]file name : v_ota.bin
[14:20:51.249] [Info] [normal]file name : ota.bin
[14:20:51.249] [Info] [normal]file_addr = 0x280c0
[14:20:51.249] [Info] [normal]file_len = 0x80b3
[14:20:51.249] [Info] [normal]ota file head name : sd0_ota0.bin
[14:20:51.249] [Info] [normal]file_addr = 0x28140
[14:20:51.249] [Info] [normal]file_len = 0x12ed
[14:20:51.249] yzzzzzyzzzzzy[Info] [normal]go to upgrade .....
[14:20:51.479] € [Info] [main]-----sh5x apps-----
[14:20:54.809] [Info] [PMU]--Reset Source : 0x1
[14:20:54.809] [Info] [PMU]P33 RESET
[14:20:54.809] [Info] [PMU]--P3 Reset Source : 0x80
[14:20:54.819] [Info] [PMU]SOFT RESET
[14:20:54.819] [Info] [pwrap]Power init : /home/liuhaokun/
my_document/sh54/newest_sh54/app/bsp/cpu/sh54/power_api.c
[14:20:54.819] [Info] [PMU]VDDIO KEEP : 0
[14:20:54.819] [Info] [PMU]VDC13 KEEP : 0
[14:20:54.819] [Info] [PMU]SPI boot port : 0x0
[14:20:54.819] [Info] [PMU]POWER_OSC_INFO: LRC - 00032000
[14:20:54.819] [Info] [PMU]Delay n * uS: 00000016 / 00000016
[14:20:54.819] [Info] [PMU]Prepare : 31(us) / Recover 3500(us) /
3500(us)
[14:20:54.819] [Info] [PMU]Low power Tprp : 0x1 / Tcke : 0x70
[14:20:54.819] [Info] [PMU]Low power Reserve time: 0x106 (tick)

```

1.校验完文件成功，进入升级
需要等待一段时间

2.升级成功，
自行复位
进入新程序

49.2 单备份升级相关功能开关

单备份升级相关功能是在 app_config.c 里由 const 变量来控制。分别是：

1. const u8 dev_update_use_eeprom

升级使用的区域：

0：VM 区域；
1：eeprom 区域；

2. const u8 dev_update_keep_io_status;

升级时是否保持住 IO 状态：

0: 不保持;
1: 保持;

3. `const u8 dev_update_power_io;`

升级时使用到的电源 IO:

-1: 没有;
任意 IO (如 `IO_PORTA_01`)

4. `const u8 ufw_vid_need_to_be_different;`

升级文件的 vid 要求:

0: 升级文件的 vid 要和原 SDK 相同
1: 升级文件的 vid 与原 SDK 不一样;
2: 升级文件的 vid > 当前 SDK 的 vid
3: 升级文件的 vid < 当前 SDK 的 vid

49.3 单备份升级相关函数接口

下面介绍升级相关函数接口。

49.3.1 `u32 try_to_upgrade(char *dev_name, char *up_file_path)`

升级入口。

会打开对应的设备, 检查对应路径下的升级文件, 校验成功后会进行复位进入 loader 进行升级。

参数:

1. `dev_name`: 升级设备
 __UDISK0: U 盘;
 __SD0_NANE: SD 卡;
2. `up_file_path`: 升级文件路径
 默认为 `"/update.ufw"`
3. 返回值:
 0: 成功
 有值: 错误。

49.3.2 `u32 check_ufw_file(char *dev_name, char *up_file_path);`

检查升级文件, 并不进行升级。

参数:


```
1. dev_name: 升级设备
   __UDISK0: U盘;
   __SD0_NANE: SD卡;
2. up_file_path: 升级文件路径
   默认为 "/update.ufw"
3. 返回值:
   0: 成功
   有值: 错误。
```

49.3.3 u8 *user_get_ufw_vid(u8 *len);

获取升级文件的 vid 指针。
需要搭配 check_ufw_file 使用。

参数:

```
len: 获取到的vid的长度 (4个byte)
返回值: 升级文件的vid。
```

49.3.4 u8 *user_get_flash_vid(u8 *len);

获取自身的 vid。

参数:

```
len: 获取到的vid的长度 (4个byte)
返回值: 自身的vid。
```

双备份升级

双备份升级存储结构有两个 app code 区域，当芯片正在运行 app code1，升级时将新固件写入 app code 2 区域，反之亦然。升级时对 app code N 区域数据校验成功后，复位即可运行 app code N 代码。当升级中断或者新写入固件校验不成功，不会影响原来 app code 的运行

杰理双备份升级有两种方式实现

1. 设备主动升级（由设备端主动向远端获取升级数据，通信协议需要能支持文件内容获取能力）
2. 设备被动升级（由远端主动推送升级数据，设备端被动写入的方式，远端可控制升级流程）

50.1 支持双备份升级的芯片以及对应 SDK 版本

支持双备份升级的芯片：AD14/15/16/18/、AC104 和 AD24

AD14/15/AC104 的 v1.7.0 SDK 和 AD16 v1.3.0SDK 需要添加补丁支持双备份设备被动升级。后续版本 SDK 会直接支持。

50.2 SDK 相关配置

首先需要在 isd_config.ini 加入以下配置：

```
[EXTRA_CFG_PARAM]
BR22_TWS_DB=YES;           //dual bank flash framework enable (only valid for_
↪dual_bank)
FLASH_SIZE=1M;             //flash_size cfg (only valid for dual_bank)
BR22_TWS_VERSION=0;        //default fw version(only valid for dual_bank)
DB_UPDATE_DATA=YES;        //generate db_update_data.bin(only valid for dual_bank)
SPECIAL_OPT=0;             //special cfg : single bin generating(only valid for_
```

(续下页)

(接上页)

```

↪dual_bank)
FORCE_4K_ALIGN=YES; // force align with 4k bytes (only valid for dual_bank)
#NEW_FLASH_FS=YES; // enable single bank flash framework (only valid for_
↪single_bank)

```

另外, AD14/15/16/18 以及 AC104 需要关闭所有写保护, 其中软件写保护位于

```

85
86 AT_SPI_CODE /*该函数放置段不可更改*/
87 u32 flash_code_protect_callback(u32 offset, u32 len)
88 {
89     return 0; 返回0即可
90     u32 limit_addr = vm_attr.sclust;
91     /* log_info("0x%x 0x%x", limit_addr, offset); */
92     if ((offset < limit_addr) || ((offset + len) > boot_info.flash_size)) {
93         /* 超过正常擦写区域, 不进行擦写操作 */
94         return 1;
95     } else {
96         /* 进行擦写操作 */
97         return 0;
98     }
99 }
100

```

图 1 软件写保护

如果升级使用 simple_fat 读取升级文件, 需要去掉框内的限制

```

4
5 u32 smpl_fat_read_api(void *pfile, void *buff, u32 len)
6 {
7     u32 rlen = f_read(&pfi, buff, len);
8     /* if (rlen != len) { */
9     /* return 0; */
10    /* } */
11    return rlen;
12 }
13
14 u32 smpl_fat_seek_api(void *pfile, u32 offset, u32 fromwhere)
15 {
16     return f_seek(&pfi, offset);
17 }
18
19 u32 smpl_fat_file_close_api(void **ppfile)
20 {
21     return (u32) * ppfile;
22 }

```

图 2 simple_fat 读函数

设备被动升级采用 db_update_data.bin 作为升级文件, 升级文件会在编译下载的时候生成, 在 SDK 示例中由于 fat 文件系统有 8+3 的长名限制, 名字被缩短为 db_data.bin

50.3 相关接口说明

50.3.1 函数 u32 jlfs_get_idle_bank_info(u32 *bank_addr, u32 *bank_size)

此函数获取 idle_bank 的信息, 其中参数:

1. bank_addr: 空闲的code区地址, 接口内部会更新到指针;
2. bank_size: 空闲的code区大小, 接口内部会更新到指针
3. 返回值:

(续下页)

(接上页)

成功 返回0;
失败 其他值;

50.3.2 函数 u32 jlfs_updata_dual_bank_info(u32 bank_addr, u16 data_crc)

此函数更新 idle_bank 的信息其中参数:

1. bank_addr: 空闲的code区地址;
2. data_crc: 升级文件校验值;
3. 返回值:
成功 返回0;
失败 其他值;

50.3.3 函数 u32 jlfs_check_dual_bank_info(u32 update_bank_addr)

此函数校验写入升级信息后的 idle_bank 信息, 其中参数:

1. update_bank_addr: 升级的code区地址;
2. 返回值:
成功 返回0;
失败 其他值;

50.4 常见客户问题

1、AD15 双备份升级后，使用强制升级工具升级失败？

AD15 的 download 工具在下载的时候不会去主动擦除 code2 区的代码，所以需要在下载目录的 download_bat.c 加入 -format all 全擦除 flash 指令擦除 code2 区域代码，加入指令后需要编译一次，看到图中的全擦除模式生效即为配置成功。

```

diff_u0 0 [FAVG_IS_NULL]
打开设备 (COM3) 成功
SPI nor flash online.
Online flash id: ef4016
Online flash size: 4M
Online external flash id: ef4017
Online external flash size: 8M
Erase Flash Size is 4096
ISDdownload
-----FLASH INFO-----
| PID : SH54
| FLASH_BIN_SIZE : 0x16000
| FLASH_NEED_SIZE : 0x100000
| FLASH_REAL_SIZE : 0x400000
| VM_REAL_SIZE : 0x6000
| VM_START_ADDR : 0xf4000
| VM_END_ADDR : 0xfa000
| EEPROM_RESERVED_SIZE : 0x6000
| EEPROM_RESERVED_START : 0xfa000
| EEPROM_RESERVED_END : 0x100000
| CODE_BOUNDARY_LINE : 0x7a000
| LAVE_SIZE : 0x64000
| ENTRY_ADDR : 0x100100
| ERASE MODE : all
| UBOOT VERSION : 2023-06-25 17:46:17, UBOOT-$-@20230625-$1242b31
| LOADER VERSION : 2023-06-30 20:11:01, LOADER-v1.0.0-$-@20230630-$a806557
-----
正在擦除FLASH[0x0-0x400000].....
...
.

```

图 3 ad15 编译擦除 flash 示意图

2、双备份升级流程，获取到的版本号出现溢出？

双备份升级的获取到的版本号是使用 u16 变量保存，升级到版本号为 65535 之后，再升级就会出现版本号溢出。但是不影响后续的升级和使用流程。

下面分为 4 个部分来介绍 VM 的基本原理与特点：

1. “VM 记忆的基本原理”
2. “VM 记忆的系统局限性”
3. “NEW VM 的预擦除”
4. “NEW VM 常见 BUG”

51.1 VM 记忆的基本原理

SDK 提供了 vm 掉电记忆存储功能，该功能用于在内置 flash 里保存断电之后还需要保存的信息。

- 1、VM 会将自己所分得的内部 FLASH 区域划为 A,B 两块区域，
- 2、例如 A 区域负责记录 VM 数据，B 区域此时处于空闲状态，当 A 区域数据趋于饱和时，会把 A 区域的数据整合到 B 区域（即格式整理），后续 B 区域记录 VM 数据，A 区域处于空闲状态，以此往复。
- 3、若此时格式整理后，A 区域原有数据没有被擦除（未调用擦除函数）
- 4、用户去读取数据，会读取到原有区域的数据（即旧数据），导致数据错误或者读不到数据。

51.2 VM 记忆的系统局限性

VM 的数据记忆主要记录到系统 flash 中，系统 flash 中存放着整个系统的代码，大量的代码需要在系统 flash 中执行；

VM 的数据记忆过程中有可能需要对系统 flash 进行擦除操作；

系统 flash 在被擦除时不可以被访问，且不同的 flash 擦除时间不一致；

基于以上几点 当 VM 触发到系统 flash 擦除时，这时系统 flash 代码是不能执行的，典型的现象就是 解码卡音。

为了优化上述的问题，编写新的 NEW VM 驱动，在一定程度上改善此问题。

51.3 NEW VM 的预擦除

为了解决解码时，擦除系统 flash 可能导致的卡音，new vm 尽量不在系统繁忙时对系统 flash 进行擦除；

new vm 提供了预擦除函数 `nvm_pre_erase_next`，可以在系统空闲时对系统 flash 当前与空闲区进行 **提前预擦除**、或 **滞后擦除**；

new vm 的写函数在 AB 区格式整理时，也不会对老区域进行擦除，而依赖预擦除在空闲时擦除；（未擦除再次上电会有老数据残留风险）

51.4 NEW VM 常见 BUG

部分客户在使用 NEW VM，自己编写功能中，遗忘或者未能合理调用预擦除功能导致，数据不能有效更新；此问题有两种解决方式：

- 1、在写入数据后，系统掉电或关机前，调用预擦除；（推荐）
- 2、打开 `config_vm_erase_after_format_en` 变量，打开此变量后，new vm 驱动会在需要擦除时实时擦除系统 flash，这样一来 NEW VM 驱动的行为将和老的 VM 一直，不再具备对卡音问题的优化；

下面分为 7 个部分来介绍 VM 功能：

1. “VM 掉电存储使用说明”
2. “VM 掉电存储版本简单说明”
3. “新旧 VM 文件组成及版本选择”
4. “VM 相关参数介绍”
5. “VM 掉电记忆存储接口”
6. “VM 常见问题介绍”

51.5 VM 掉电存储使用说明

VM 记忆存储使用的步骤分为三步：

- 第一步：系统初始化时，调用函数 `vm_init_api()` 初始化 VM 功能。
- 第二步：使用 `vm_write()` 接口把需要保存的数据写入 FLASH。
- 第三步：使用 `vm_read()` 接口读取写入的数据。

可参考” [VM 简单使用 demo](#)： ”

51.6 VM 掉电存储版本简单说明

目前 SDK 中 VM 有新旧两个版本可以选择：

备注：旧版 VM 功能：

1. 可存储 id 号数量：128 个；
 2. 单次写入最大数据长度：128 个 byte；
 3. 没有 cache 管理；
 4. 没有预擦除功能；
-

备注：新版 VM 功能

1. 可存储 id 号数量：512 个；
 2. 单次写入最大数据长度：4096 个 byte；（若 EEPROM 分配给 VM 区域只有 8K，则单次写入最大数据长度为：4092 个 byte）；
 3. 增加 cache 管理，提升读取速度；
 4. 增加预擦除功能。
-

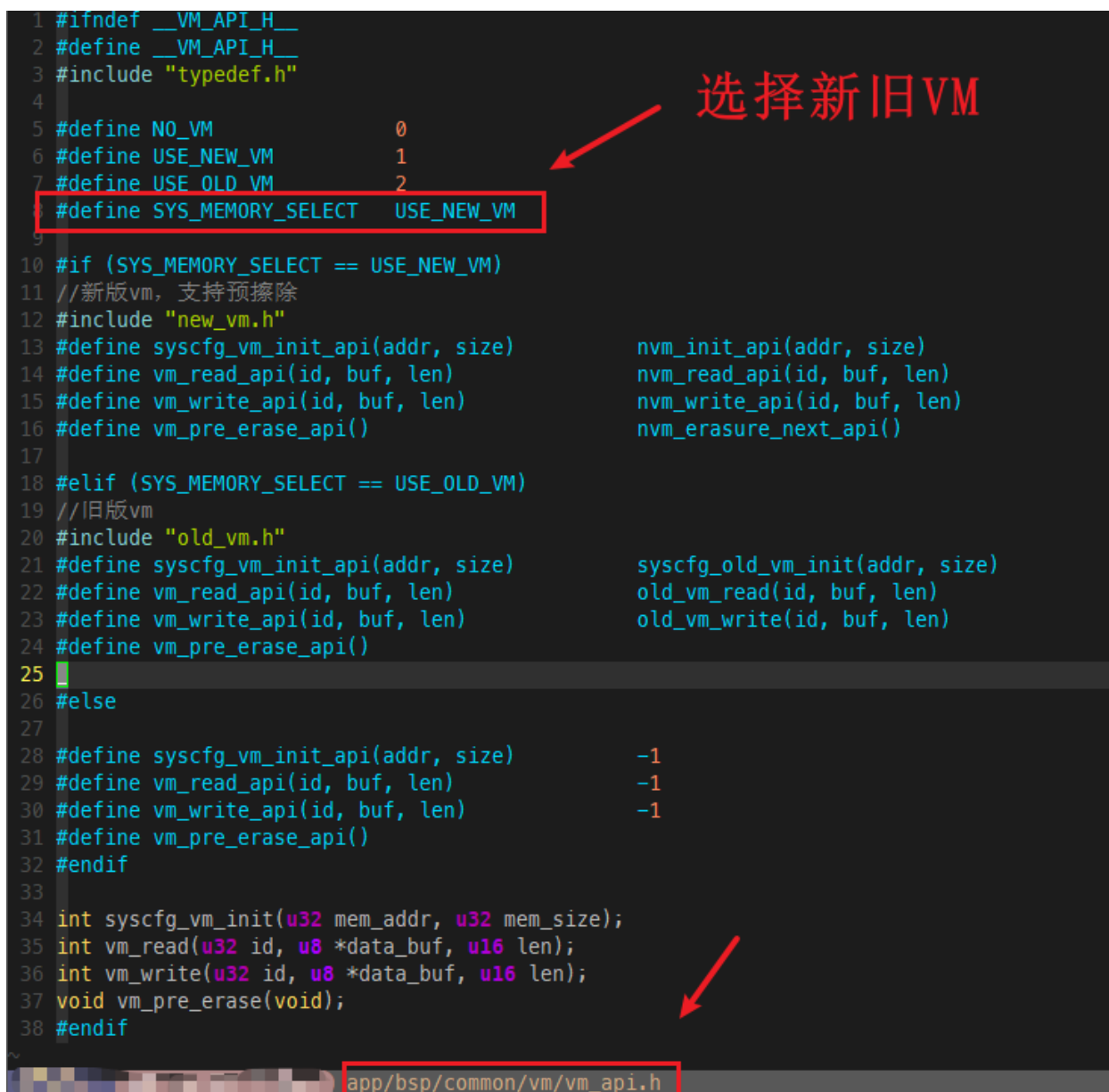
备注：新旧版本 VM 功能对比：

1. 新版 VM id 号数量增多；
 2. 新版 VM 单次写入最大数据长度增多；
 3. 新版 VM 增加 cache 管理，提升 VM 读取速度；
 4. 新版 VM 增加预擦除功能，可以在系统空闲时进行预擦除操作，可尽可能避免 VM 整理时关闭 flash 时间过长导致的卡音问题；
 5. 新版 ID 号与旧版 index 句柄含义相同；
-
-

51.7 新旧 VM 文件组成及版本选择

vm 模块由 vm_api.c 和 vm_api.h 文件构成 API 层，包含 ovm_api.c 和 nvm_api.c 文件；

备注：在 SDK 为 AD14/15/AC104 的 v1.7.0 版本（包括 1.7.0 版本）以前版本，用户可在 **vm_api.h** 文件里选择使用新 / 旧版 VM。



```

1 #ifndef __VM_API_H__
2 #define __VM_API_H__
3 #include "typedef.h"
4
5 #define NO_VM 0
6 #define USE_NEW_VM 1
7 #define USE_OLD_VM 2
8 #define SYS_MEMORY_SELECT USE_NEW_VM
9
10 #if (SYS_MEMORY_SELECT == USE_NEW_VM)
11 //新版vm, 支持预擦除
12 #include "new_vm.h"
13 #define syscfg_vm_init_api(addr, size) nvm_init_api(addr, size)
14 #define vm_read_api(id, buf, len) nvm_read_api(id, buf, len)
15 #define vm_write_api(id, buf, len) nvm_write_api(id, buf, len)
16 #define vm_pre_erase_api() nvm_erase_next_api()
17
18 #elif (SYS_MEMORY_SELECT == USE_OLD_VM)
19 //旧版vm
20 #include "old_vm.h"
21 #define syscfg_vm_init_api(addr, size) syscfg_old_vm_init(addr, size)
22 #define vm_read_api(id, buf, len) old_vm_read(id, buf, len)
23 #define vm_write_api(id, buf, len) old_vm_write(id, buf, len)
24 #define vm_pre_erase_api()
25
26 #else
27
28 #define syscfg_vm_init_api(addr, size) -1
29 #define vm_read_api(id, buf, len) -1
30 #define vm_write_api(id, buf, len) -1
31 #define vm_pre_erase_api()
32 #endif
33
34 int syscfg_vm_init(u32 mem_addr, u32 mem_size);
35 int vm_read(u32 id, u8 *data_buf, u16 len);
36 int vm_write(u32 id, u8 *data_buf, u16 len);
37 void vm_pre_erase(void);
38 #endif

```

选择新旧VM

app/bsp/common/vm/vm_api.h

新旧 VM 功能选择

备注：在 SDK 为 AD14/15/AC104 的 v1.7.0 版本以后，vm 配置选项更改位置，用户可在 **app_config.h** 文件里选择使用新 / 旧版 VM。

```

36
37 /*-----VM Mode Configuration-----*/
38 #define NO_VM                0
39 #define USE_NEW_VM           1
40 #define USE_OLD_VM           2
41 #define SYS_MEMORY_SELECT    USE_NEW_VM
42
43 /*-----FLASH Configuration-----*/
44 #define TCFG_FLASH_SPI_TYPE_SELECT 1//1:flash 选择硬件spi; 0:flash
45 #define HW_SPI_WORK_MODE           SPI_MODE_BIDIR_1BIT
46 #define SPI_READ_DATA_WIDTH        SPI_MODE_BIDIR_1BIT
47 #define SPI_CS_PORT_SEL             IO_PORTA_05
48 //port select for soft spi
49 #define A_CLK_BIT                   BIT(12)// set clk
50 #define A_CLK_PORT(x)               JL_PORTA->x
51 #define A_D0_BIT                    BIT(11)// set d0
52 #define A_D0_PORT(x)                JL_PORTA->x
53 #define A_D1_BIT                    BIT(11)// set d1
54 #define A_D1_PORT(x)                JL_PORTA->x
55 #define SOFT_SPI_WORK_MODE          SPI_MODE_UNIDIR_1BIT//只支持双向
56 #if TCFG_FLASH_SPI_TYPE_SELECT
57 #define SPI_HW_NUM                  1
58
59 /app/src/voice_toy/app_config.h
60 /voice_toy/app_config.h" 150L, 5153C

```

新旧 VM 功能选择

51.8 VM 相关参数介绍

对于用户，我们需要关心的参数分别为：

1. “ID 号 (*vm_api.h*) ”
2. “读/写入数据长度”
3. “*vm* 存储区域大小”

对于新版 VM

4. “新版 VM 格式整理时缓存空间设置 *new_vm_buff* (*new_vm.h*) ”
5. “新版 *vm* 格式整理后是否立即擦除旧半区区域功能”
6. “新版 *vm* 是否支持多段读功能”

下面逐一介绍：

51.8.1 ID 号 (vm_api.h)

```
u32 vm_write(u32 id, u8 *buf, u32 len)
```

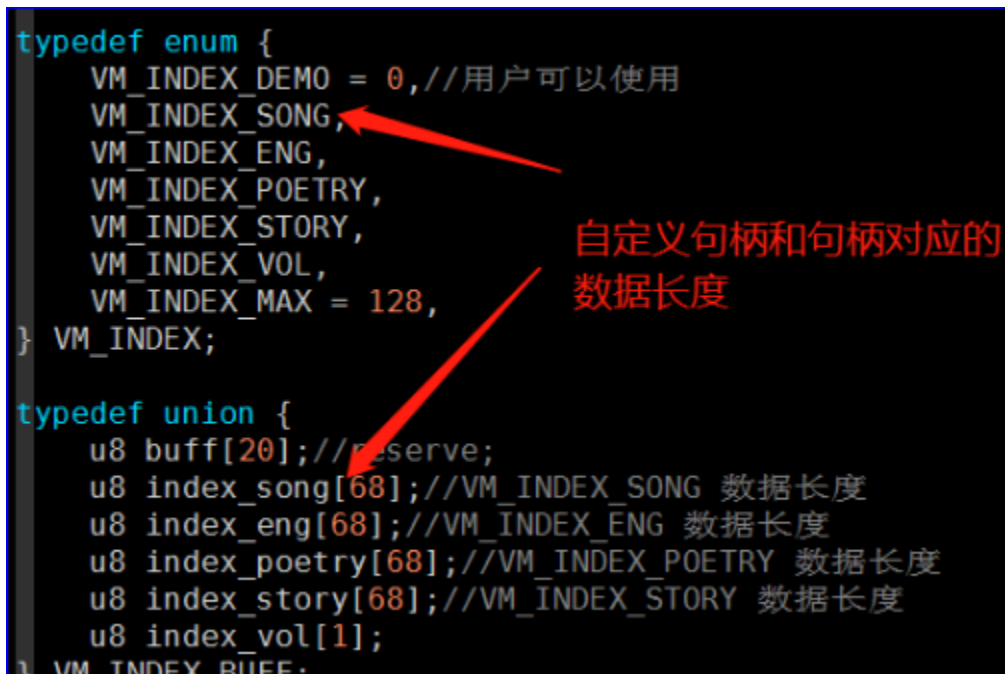
vm_write 参数

重要：旧版 VM ID 号最多 128 个，新版 VM ID 号最多 512 个；（旧版 VM index 号在新版 vm 仍可通用）

新版 VM ID 号最多支持 512 个，且不再需要自定义对应 ID 的数据长度。

用户可在 vm_api.h 的 VM_INDEX 里添加自己的 id 号。

注意：1~32 号 ID 为系统使用，用户不可修改其顺序



```
typedef enum {
    VM_INDEX_DEMO = 0, // 用户可以使用
    VM_INDEX_SONG,
    VM_INDEX_ENG,
    VM_INDEX_POETRY,
    VM_INDEX_STORY,
    VM_INDEX_VOL,
    VM_INDEX_MAX = 128,
} VM_INDEX;

typedef union {
    u8 buff[20]; // reserve;
    u8 index_song[68]; // VM_INDEX_SONG 数据长度
    u8 index_eng[68]; // VM_INDEX_ENG 数据长度
    u8 index_poetry[68]; // VM_INDEX_POETRY 数据长度
    u8 index_story[68]; // VM_INDEX_STORY 数据长度
    u8 index_vol[1];
} VM_INDEX_BUFF;
```

自定义句柄和句柄对应的数据长度

旧版 VM 掉电存储记忆项的定义 (vm_api.h)

51.8.2 读/写入数据长度

```
u32 vm_write(u32 id, u8 *buf, u32 len)
```

vm_write 参数 length

重要：旧版 VM 读/写长度最多支持 128 个 byte；

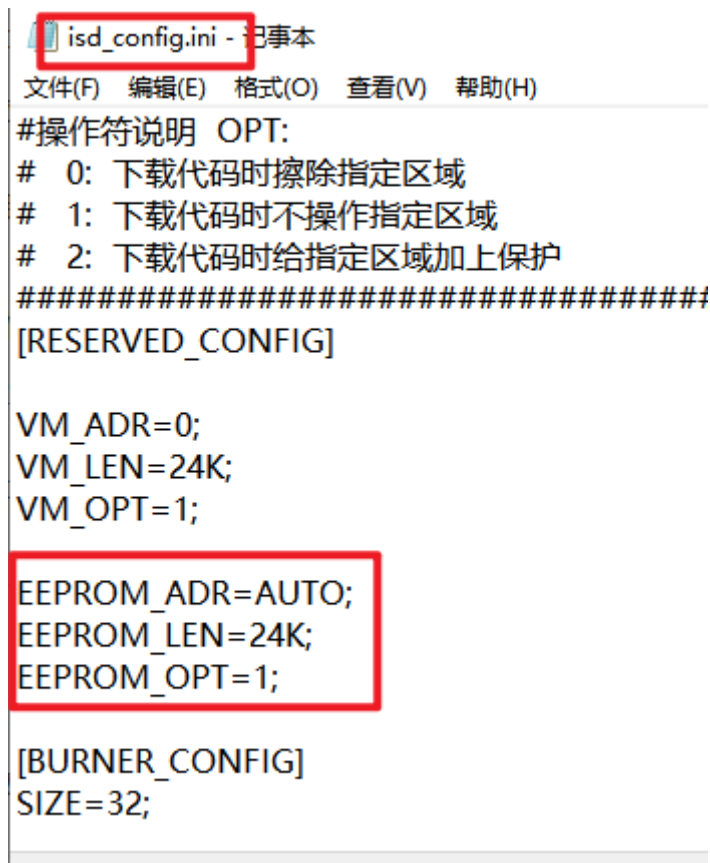
新版 VM 读/写长度最多支持 4096 个 byte；

51.8.3 vm 存储区域大小

重要： VM 记忆的数据是 EEPROM 区划分出一定空间大小给 VM 使用，该区域大小可由下载目录里的 isd_config.ini 文件配置。（该配置参数详细说明可参考” isd_config.ini 文件介绍”）

注意：

VM 存储区域大小分配的 EEPROM 空间不得低于 8K 字节，且需为 4K 字节的倍速，即 8K，12K，16K 等等。



```

isd_config.ini - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)
#操作符说明 OPT:
# 0: 下载代码时擦除指定区域
# 1: 下载代码时不操作指定区域
# 2: 下载代码时给指定区域加上保护
#####
[RESERVED_CONFIG]

VM_ADR=0;
VM_LEN=24K;
VM_OPT=1;

EEPROM_ADR=AUTO;
EEPROM_LEN=24K;
EEPROM_OPT=1;

[BURNER_CONFIG]
SIZE=32;

```

VM 区域大小设置”

51.8.4 新版 VM 格式整理时缓存空间设置 new_vm_buff (new_vm.h)

重要： new_vm_buff，该 buff 用于 vm 格式整理时数据的缓存。

该 buff 由 用户自定义最大 id 数量 (BIT_MAP_SIZE) + 单个 id 在整理时数据缓存的最大长度 (BIT_MAX_LEN) 两部分组成。

BIT_MAP_SIZE：即 BIT_MAP：vm 最多可以存储的 id 数量，固定为 512 个。

BIT_MAX_LEN：即 VM 格式整理时数据缓存的大小，该值越大，则整理速度越快，该值越小，则整理速度越慢。（注意：该 buf 与读/写数据最大长度为两个概念）

```

14 #include "log.h"
15 NEW_VM_OBJ g_nvm_obj;
16
17 u32 new_vm_buff[NVM_BUFF_SIZE / 4]; //需要32bit对齐
18 /**
19 **/
20 void *nvm_buf_for_lib(NEW_VM_OBJ *p_nvm, u32 *p_len)
21 {
22     *p_len = sizeof(new_vm_buff);
23 }
24
25 #define BIT_MAP (32 * 16)
26 #define BIT_MAP_SIZE (BIT_MAP / 8)
27
28 #define NVM_MAX_LEN 128
29 #define NVM_BUFF_SIZE (NVM_MAX_LEN * BIT_MAP_SIZE)
30
31 typedef struct __new_v 格式数据长度 最大id数
32 void *device;

```

VM 掉电存储记忆项的定义 (nvm_api.c 和 new_vm.h)

51.8.5 新版 vm 格式整理后是否立即擦除旧半区区域功能

```

27
28 #define NVM_CACHE_ENABLE 1
29 #define NVM_CACHE_NUMBER 6
30 const bool config_vm_multiple_read_en = 0;
31 const bool config_vm_erasure_after_format_en = 0;
32
33 #if NVM_CACHE_ENABLE
34
35 NVM_ENTRY g_nvm_entry[NVM_CACHE_NUMBER];
36 NORMAL }

```

config_vm_erasure_after_format_en

config_vm_erasure_after_format_en 用于控制新版 vm 格式整理后是否立即擦除旧半区区域;

0: 不立即擦除 (默认值)

1: 立即擦除

重要: 该功能直观影响为: 打开该功能后, vm 进行格式整理 会调用到 flash 擦除, 如果系统此时正在解码, 那么解码可能会出现卡音的现象。

而默认关闭该功能后, vm 进行格式整理 不会调用到 flash 擦除, 而是等到 系统空闲时程序员调用预擦除函数 进行擦除动作, 这样就不会影响到系统此时正进行的解码。

关闭该功能后, 未在系统空闲时进行擦除动作, 那么可能导致下次上电, VM 读取到老旧的数据。

51.8.6 新版 vm 是否支持多段读功能

```

27
28 #define NVM_CACHE_ENABLE    1
29 #define NVM_CACHE_NUMBER    6
30 const bool config_vm_multiple_read_en = 0;
31 const bool config_vm_erase_after_format_en = 0;
32
33 #if NVM_CACHE_ENABLE
34
35 NVM_ENTRY g_nvm_entry[NVM_CACHE_NUMBER];
NORMAL } <s/app/bsp/common/vm/nvm_api.c c

```

config_vm_multiple_read_en

config_vm_multiple_read_en 用于控制新版 vm 是否支持多段读功能；

0：不支持多段读（默认值）

1：支持多段读

51.9 VM 掉电记忆存储接口

51.9.1 函数 u32 syscfg_vm_init(u32 addr, u32 size)

此函数用来初始化 vm 虚拟存储系统，其中参数：

1. eeprom_saddr：vm 区域的起始地址。
2. eeprom_size：vm 区域的空间长度。
3. 返回值：

成功：0；

失败：错误号，可参考 errno-base.h；

51.9.2 函数 u32 vm_write(u32 id, u8 *buf, u32 len)

此函数用来把对应 hdl 的数据存储到 vm 虚拟存储系统，其中参数：

1. hdl：访问 vm 的 id。
2. buf：vm 读数据 buffer。
3. len：数据长度。
4. 返回值：

成功：写入长度；

失败：错误号，可参考 errno-base.h；

51.9.3 函数 u32 vm_read(u32 id ,u8 *buf,u32 len)

此函数用来把对应 hdl 的数据从 vm 虚拟存储系统读出来，其中参数：

1. id: 访问 vm 的 id。
2. buf: vm 读数据 buffer。
3. len: 数据长度。
4. 返回值:

成功: 写入长度;

失败: 错误号, 可参考 errno-base.h;

51.9.4 函数 u32 vm_pre_erase(void)

此函数用于预擦除设备，在系统空闲时主动调用提前擦除 flash，可尽量避免 VM 做整理时关 flash 时间过长导致卡音问题；

该函数只有在新版 VM 处生效，旧版 VM 没有该功能。

示例：音乐模式下，500MS 消息来时若系统处于空闲状态，则会进行预擦除动作：

```
case MSG_500MS:
    /* decoder_time(pctl[0].p_dec_obj); */
    UI_menu(MENU_MAIN);
    UI_menu(MENU_HALF_SEC_REFRESH);
    if (MUSIC_PLAY != get_decoder_status(pctl[0].p_dec_obj)) {
        vm_pre_erase();
        app_powerdown_deal(0);
    } else {
        app_powerdown_deal(1);
    }
}
```

图 2.1 系统空闲预擦除动作

51.9.5 函数 nvm_pre_erase_next(NEW_VM_OBJ *p_nvm, u16 using_next, u16 idle_next)

此函数为 NEW VM 中独有接口，由 2.4 函数 u32 vm_pre_erase() 调用，用于系统空闲时选择擦除当前 VM 已存储的数据往后的 flash 扇区，以及空闲 VM 需要往后擦除的 flash 扇区数，用户可自行修改需要擦除的 flash 扇区数，其中参数：

- 1、p_nvm: NEW VM 句柄指针；
- 2、using_next: 当前 VM 存储块需要往后擦除的 flash 扇区数；
- 3、idle_next: 空闲 VM 存储块需要往后擦除的 flash 扇区数；
- 4、返回值: 无

51.10 VM 常见问题介绍

51.10.1 VM 简单使用 demo:

用户可在 app/bsp/start/flash_init.c 里的 vm_init_api() 函数找到 demo 示例。

```

86
87
88 static struct vfs_attr eeprom_attr;
89 void vm_init_api(void)
90 {
91     u32 err;
92     void *pvfs = 0;
93     void *pvfile = 0;
94     void *device = 0;
95     u32 capacity = 0;
96
97
98     err = vfs_mount(&pvfs, (void *)NULL, (void *)NULL);
99     ASSERT(!err, "fil vfs mount : 0x%x\n", err)
100     err = vfs_openbypath(pvfs, &pvfile, "/app_area head/EEPROM");
101     ASSERT(!err, "fil vfs openbypath : 0x%x\n", err)
102     err = vfs_ioctl(pvfile, FS_IOCTL_FILE_ATTR, (int)&eeprom_attr);
103     ASSERT(!err, "fil vfs ioctl : 0x%x\n", err)
104     log_info("EEPROM size : 0x%x\nEEPROM sclust : 0x%x\n", eeprom_attr.fsize, eeprom_attr.sclust);
105     vfs_file_close(&pvfile);
106     vfs_fs_close(&pvfs);
107
108     syscfg_vm_init(eeprom_attr.sclust, eeprom_attr.fsize);
109     /* *demo */
110     #if 0
111     u8 data_buf[10];
112     for (int i = 0; i < sizeof(data_buf); i++) {
113         data_buf[i] = JL_RAND->R64L;
114     }
115     put_buf(data_buf, 10);
116     vm_write(VM_INDEX_DEMO, data_buf, 10);
117     memset(data_buf, 0x00, 10);
118     vm_read(VM_INDEX_DEMO, data_buf, 10);
119     put_buf(data_buf, 10);
120     #endif
121 }
122 struct vfs_attr *get_eeprom_attr_p(void)
123 {
124     return &eeprom_attr;
125 }

```

往VM_INDEX_DEMO id号里存入十个随机数

从VM_INDEX_DEMO id号里读取数据

vm_demo

51.10.2 调整 VM 区域大小设置:

节省 VM 占用系统空间可以从两个地方入手:

1. 减少 VM 区域大小。

具体参考上面”*vm 存储区域大小*”章节的 vm 存储区域大小部分

2. 新版 VM 可以减少 VM 格式整理时的缓存 new_vm_buff

可以适当减少 BIT_MAX_LEN 的长度。

具体参考上面”新版 VM 格式整理时缓存空间设置 new_vm_buff (new_vm.h)”章节的新版 VM 格式整理时缓存空间设置部分。

51.10.3 新版 VM 在关机或者掉电后可能存在数据丢失 BUG:

该问题原因可能有:

1. AD14v1.7.0、v1.8.0 voice_toy 工程 app_ld.c 文件缺少 (*.data) 段 (优先排查);

解决办法: FAQ: AD14v1.7.0、v1.8.0 voice_toy 工程 new_vm 存在掉电后读取数据错误问题?

2. 系统在关机前格式整理后没有做预擦除动作。

解决办法: FAQ: AD14/15/17/AC104 new_vm 功能在关机或者掉电后可能存在数据丢失 BUG

51.10.4 VM 格式整理原理:

VM 格式整理原理:

VM 功能将 EEPROM 区域划分为 A、B 两块, A 块记录 VM 数据, B 块处于空闲状态, 当 A 块数据接近饱和时, 会擦除 B 块并将 A 块的数据整合到 B 块, 后续 B 块记录 VM 数据, A 块处于空闲状态, 以此往复。

AD14N, AD15N, AD24N 支持的消息见图《消息列表图》

```

1  #include lib/
2  #include audio/
3  #include common/
4  #include cpu/
5  #include decoder/
6  #include dev_msg/
7  #include device/
8  #include encoder/
9  #include example/
10 #include fs/
11 #include liba/
12 #include msg/
13 #include msg.h
14 #include asm_type.h
15 #include audio_lib.a
16 #include common_lib.a
17 #include cpu_lib.a
18 #include cpu_sh54_lib.a
19 #include decoder_mge_lib.a
20 #include dev_msg_lib.a
21 #include effect_lib.a
22 #include example_lib.a
23 #include fs_lib.a
24 #include periphy_lib.a
25 #include sdmmc_lib.a
26 #include jenkins/
27 #include libs/
28 #include tools/
29 #include cscope.files
30
31 7 enum {
32 8  //SYS MSG_START LINE
33 9  MSG_0 = 0,
34 10 MSG_1,
35 11 MSG_2,
36 12 MSG_3,
37 13 MSG_4,
38 14 MSG_5,
39 15 MSG_6,
40 16 MSG_7,
41 17 MSG_8,
42 18 MSG_9,
43 19 MSG_PP_2,
44 20 MSG_RECDECODE_START,
45 21 MSG_RECDECODE_END,
46 22 //APP
47 23 MSG_500MS,
48 24 MSG_APP_SWITCH_ACTIVE,
49 25 MSG_LOW_POWER,
50 26 MSG_ENTER_IDLE,
51 27 MSG_POWER_OFF,
52 28
53 29 // MSG_VOL_UP,
54 30 // MSG_VOL_DOWN,
55 31 // MSG_VOL_KEY_UP,
56 32 // MSG_MUTE,
57 33 // MSG_POWER_ON,
58 34 // MSG_POWER_OFF,
59 35 MSG_PLAY_FILE1,
60
61  MSG_PLAY_FILE2,
62
63  ///音乐操作相关消息
64  MSG_PP,
65  MSG_NEXT_FILE,
66  MSG_PRIV_FILE,
67  MSG_NEXT_DIR,
68  MSG_A_PLAY,
69  MSG_VOL_DOWN,
70  MSG_VOL_UP,
71
72  ///录音操作相关信息
73  MSG_REC_MODE_SWITCH,
74  MSG_REC_SPEED_EN,
75
76  //midi
77  MSG_MIDI_MODE_SWITCH,
78  MSG_MIDI_OKON_GOON,
79
80  //others
81  MSG_NEXT_MODE,
82
83  //系统相关消息，库会使用到，不能更改
84  MSG_F1A1_FILE_END = 0x800,
85  MSG_F1A1_FILE_ERR,
86  MSG_F1A1_LOOP,
87  MSG_F1A2_FILE_END,
88  MSG_F1A2_FILE_ERR,
89  MSG_F1A2_LOOP,
90  MSG_MIDI_FILE_END,
91  MSG_MIDI_FILE_ERR,
92  MSG_A_FILE_END,
93  MSG_A_FILE_ERR,
94  MSG_A_LOOP,
95  MSG_MP3_FILE_END,
96  MSG_MP3_FILE_ERR,
97  MSG_MP3_LOOP,
98
99  MSG_COMMON_MAX, //common最大消息
100
101  MSG_CLEAN_ALL_MSG = 0x0ffe,
102  NO_MSG = 0x0fff,

```

消息列表图

AD14N, AD15N, AD24N 目前支持的事件见下图 10.2, 事件最多支持 64 种, 系统已经定义的事件如图《已有事件列表图》。事件发生与不发生的状态由两个 32 位的变量 event_buf_L、event_buf_H 表示, 其中变量的每个位表示一个事件的状态 (1: 事件发生; 0: 事件未发生)。当某个事件发生时对应的位将被置 1, 其它位状态不变。如果用户要增加其它事件可类似定义变量和事件宏, 用变量位表示其状态。

include_lib/	90	#define	MAX_EVENT	64
audio/	91			
common/	92	#define	NO_EVENT	0xffff
cpu/	93			
decoder/	94	#define	EVENT_F1A1_END	0
dev_mg/	95	#define	EVENT_F1A1_ERR	1
device/	96	#define	EVENT_F1A1_LOOP	2
encoder/	97	#define	EVENT_F1A2_END	3
example/	98	#define	EVENT_F1A2_ERR	4
fs/	99	#define	EVENT_F1A2_LOOP	5
liba/	100			
msg/	101	#define	EVENT_MIDI_END	6
msg.h	102	#define	EVENT_MIDI_ERR	7
asm_type.h	103	//		8
audio_lib.a	104	#define	EVENT_A_END	9
common_lib.a	105	#define	EVENT_A_ERR	10
cpu_lib.a	106	#define	EVENT_A_LOOP	11
cpu_sh54_lib.a	107			
decoder_mge_lib.a	108	#define	EVENT_MP3_END	12
dev_mg_lib.a	109	#define	EVENT_MP3_ERR	13
effect_lib.a	110	#define	EVENT_MP3_LOOP	14
example_lib.a	111			
fs_lib.a	112	#define	EVENT_APP_SW_ACTIVE	15
periphy_lib.a	113			
sdmmc_lib.a	114	#define	EVENT_HALF_SECOND	31

已有事件列表图

消息与事件的目录位于：APP/src/common/msg/msg.c、include_lib/msg/msg.h。其中主要的函数有：

```
void event_init(void);
void message_init();
u32 event2msg_api(u32 event);
int post_event(int event);
int post_msg(int argc, ...);
bool get_event_status(u32 event);
int get_msg(int len, int *msg);
void clear_all_message(void);
```

52.1 1 消息与事件使用一般流程

在开机时先调用 event_init()、message_init() 对消息系统初始化，主要初始化一些消息列表相关变量及事件状态变量。初始化后有用到消息的地方就可以设置消息或获取消息。

- a. 开机调用 event_init() 及 message_init() 完成对事件和消息的初始化；
- b. 调用 post_event(int event) 标记事件，调用 post_msg(int argc, ...) 标记消息；
- c. 调用 get_msg(int len, int *msg) 获取事件消息或其他消息；

52.2 2 消息与事件函数

52.2.1 2.1 void event_init(void)

此函数会初始化事件状态信息，将 event_buf_L、event_buf_H 事件状态清空，这一步一般在开机时进行。

52.2.2 2.2 void message_init()

此函数会初始化消息信息，将 msg_cbuf 结构体消息状态清空，这一步一般在开机时进行。

52.2.3 2.3 u32 event2msg_api(u32 event)

此函数实现根据事件 event 信息获取对应的消息信息的功能。

1、event, 事件编号（事件位置见图《事件消息对照表图》形参部分）可选事件：

- ① EVENT_F1A1_END, F1A1 文件播放结束事件;
- ② EVENT_F1A1_ERR, F1A1 文件播放错误事件;
- ③ EVENT_F1A1_LOOP, F1A1 文件播放循环事件;
- ④ EVENT_F1A2_END, F1A2 文件播放结束事件;
- ⑤ EVENT_F1A2_ERR, F1A2 文件播放错误事件;
- ⑥ EVENT_F1A2_LOOP, F1A2 文件播放循环事件;
- ⑦ EVENT_MIDI_END, MIDI 文件播放结束事件;
- ⑧ EVENT_MIDI_ERR, MIDI 文件播放错误事件;
- ⑨ EVENT_A_END, A 文件播放结束事件;
- ⑩ EVENT_A_ERR, A 文件播放错误事件;
- ❗ EVENT_A_LOOP, A 文件播放循环事件;
- ❗ EVENT_MP3_END, MP3 文件播放结束事件;
- ❗ EVENT_MP3_ERR, MP3 文件播放错误事件;
- ❗ EVENT_MP3_LOOP, MP3 文件播放循环事件;
- ❗ EVENT_APP_SW_ACTIVE, APP 切换活跃事件;
- ❗ NO_EVENT, 无事件发生;

2、返回值: 与事件对应的消息信息；! 事件与消息对照表如下图《事件消息对照表图》

include lib/	90 #define MAX_EVENT	64	// 系统相关消息, 库会使用到, 不能更
audio/	91 #define NO_EVENT	0xffff	MSG_F1A1_FILE_END = 0x800,
common/	93		MSG_F1A1_FILE_ERR,
cpu/	94 #define EVENT_F1A1_END	0	MSG_F1A1_LOOP,
decoder/	95 #define EVENT_F1A1_ERR	1	MSG_F1A2_FILE_END,
dev_mg/	96 #define EVENT_F1A1_LOOP	2	MSG_F1A2_FILE_ERR,
device/	97 #define EVENT_F1A2_END	3	MSG_F1A2_LOOP,
encoder/	98 #define EVENT_F1A2_ERR	4	MSG_MIDI_FILE_END,
example/	99 #define EVENT_F1A2_LOOP	5	MSG_MIDI_FILE_ERR,
fs/	100		MSG_A_FILE_END,
liba/	101 #define EVENT_MIDI_END	6	MSG_A_FILE_ERR,
msg/	102 #define EVENT_MIDI_ERR	7	MSG_A_LOOP,
msg.h	103 //	8	MSG_MP3_FILE_END,
asm_type.h	104 #define EVENT_A_END	9	MSG_MP3_FILE_ERR,
audio_lib.a	105 #define EVENT_A_ERR	10	MSG_MP3_LOOP,
common_lib.a	106 #define EVENT_A_LOOP	11	
cpu_lib.a	107		
cpu_sh54_lib.a	108 #define EVENT_MP3_END	12	
decoder_mge_lib.a	109 #define EVENT_MP3_ERR	13	
dev_mg_lib.a	110 #define EVENT_MP3_LOOP	14	MSG_COMMON_MAX, // common 最大消息
effect_lib.a	111		
example_lib.a	112 #define EVENT_APP_SW_ACTIVE	15	MSG_CLEAN_ALL_MSG = 0x0ffe,
fs_lib.a	113		NO_MSG = 0x0fff,
periphy_lib.a	114 #define EVENT_HALF_SECOND	31	MSG_APP_SWITCH_ACTIVE,
sdmcc_lib.a			

事件消息对照表图

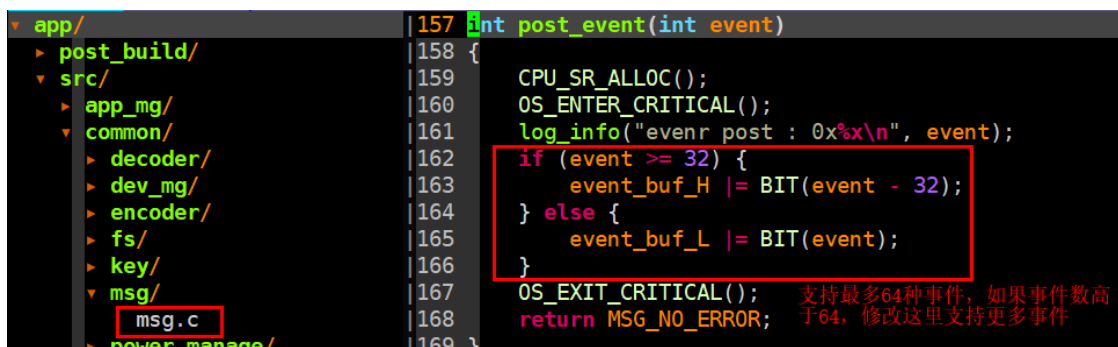
52.2.4 2.4 int post_event(int event)

当某个事件 event 发生时, 本函数实现将事件 event 对应的事件状态标号设置为 1。只有事件 event 状态位被置 1 后, 消息获取函数 get_msg 才能获取到与事件对应的消息。! 事件最多支持 64 种, 系统已经定义的事件如图《已有事件列表图》。事件发生与不发生的状态由两个 32 位的变量 event_buf_L、event_buf_H 表示, 其中变量的每个位表示一个事件的状态 (1: 事件发生; 0: 事件未发生)。当某个事件 event 发生时对应的位将被置 1, 其它位状态不变。如果用户需求事件量高于 64, 其事件可类似定义变量和事件宏, 用变量位表示其状态, 并修改文件中本函数 post_event、get_event 函数及事件消息对照数组 event2msg[MAX_EVENT] 等使其支持高于 64 种事件的事件。

1、event: 事件列表中的某个事件, 位置见上图《事件消息对照表图》形参, 可选事件:

- ① EVENT_F1A1_END, F1A1 文件播放结束事件
- ② EVENT_F1A1_ERR, F1A1 文件播放错误事件
- ③ EVENT_F1A1_LOOP, F1A1 文件播放循环事件
- ④ EVENT_F1A2_END, F1A2 文件播放结束事件
- ⑤ EVENT_F1A2_ERR, F1A2 文件播放错误事件
- ⑥ EVENT_F1A2_LOOP, F1A2 文件播放循环事件
- ⑦ EVENT_MIDI_END, MIDI 文件播放结束事件
- ⑧ EVENT_MIDI_ERR, MIDI 文件播放错误事件
- ⑨ EVENT_A_END, A 文件播放结束事件
- ⑩ EVENT_A_ERR, A 文件播放错误事件
- ⑪ EVENT_A_LOOP, A 文件播放循环事件
- ⑫ EVENT_MP3_END, MP3 文件播放结束事件
- ⑬ EVENT_MP3_ERR, MP3 文件播放错误事件
- ⑭ EVENT_MP3_LOOP, MP3 文件播放循环事件
- ⑮ EVENT_APP_SW_ACTIVE, APP 切换活跃事件
- ⑯ NO_EVENT, 无事件发生

2、返回值: 返回无误信息 MSG_NO_ERROR, 返回值无作用;



```

157 int post_event(int event)
158 {
159     CPU_SR_ALLOC();
160     OS_ENTER_CRITICAL();
161     log_info("event post : 0x%x\n", event);
162     if (event >= 32) {
163         event_buf_H |= BIT(event - 32);
164     } else {
165         event_buf_L |= BIT(event);
166     }
167     OS_EXIT_CRITICAL();
168     return MSG_NO_ERROR;
169 }

```

支持最多64种事件，如果事件数高于64，修改这里支持更多事件

事件状态改变函数

52.2.5 2.5 int post_msg(int argc, ...)

当某个消息发生时，本函数实现将消息放入消息列表。

- 1、argc：后面可变参数(...)的数量；
- 2、...：可变参数，参数个数可变，可带入一个或多个文件；

可变参数(...)有：

- (1) MSG_500MS, ticktime 时钟到 500ms 消息
- (2) MSG_PRIV_FILE, 播放前一个文件消息
- (3) MSG_NEXT_FILE, 播放下一个文件消息
- (4) MSG_NEXT_DIR, 播放下一个文件夹消息
- (5) MSG_PLAY_FILE1, 播放文件 1 消息
- (6) MSG_PLAY_FILE2, 播放文件 2 消息
- (7) MSG_PP, 播放文件 1 暂停消息
- (8) MSG_PP_2, 播放文件 2 暂停消息
- (9) MSG_VOL_UP, 播放声音增大消息
- (10) MSG_VOL_DOWN, 播放声音减小消息
- (11) MSG_A_PLAY, 播放文件 A 消息
- (12) MSG_REC_MODE_SWITCH, 开始录音与开始播放录音切换消息
- (13) MSG_REC_SPEED_EN, 录音速率模式使能消息
- (14) MSG_MIDI_MODE_SWITCH, midi 模式切换消息
- (15) MSG_MIDI_OKON_GOON, midi one key one note mode 下一音节消息
- (16) MSG_NEXT_MODE, app 模式切换消息
- (17) MSG_RECODE_START, 重新编码开始消息
- (18) MSG_RECODE_END, 重新编码结束消息
- (19) MSG_LOW_POWER, 低电消息
- (20) MSG_ENTER_IDLE, cpu 进入空闲消息
- (21) MSG_POWER_OFF, 关机消息
- (22) MSG_CLEAN_ALL_MSG, 清空所有消息的消息

- 3、返回值：MSG_NO_ERROR 无实际作用；

52.2.6 2.6 bool get_event_status(u32 event)

本函数实现获取事件 event 状态的功能。1、event: 事件序号, 位置见图《事件宏列表图》, 可选事件:

- ① EVENT_F1A1_END, F1A1 文件播放结束事件
- ② EVENT_F1A1_ERR, F1A1 文件播放错误事件
- ③ EVENT_F1A1_LOOP, F1A1 文件播放循环事件
- ④ EVENT_F1A2_END, F1A2 文件播放结束事件
- ⑤ EVENT_F1A2_ERR, F1A2 文件播放错误事件
- ⑥ EVENT_F1A2_LOOP, F1A2 文件播放循环事件
- ⑦ EVENT_MIDI_END, MIDI 文件播放结束事件
- ⑧ EVENT_MIDI_ERR, MIDI 文件播放错误事件
- ⑨ EVENT_A_END, A 文件播放结束事件
- ⑩ EVENT_A_ERR, A 文件播放错误事件
- ❏ EVENT_A_LOOP, A 文件播放循环事件
- ❏ EVENT_MP3_END, MP3 文件播放结束事件
- ❏ EVENT_MP3_ERR, MP3 文件播放错误事件
- ❏ EVENT_MP3_LOOP, MP3 文件播放循环事件
- ❏ EVENT_APP_SW_ACTIVE, APP 切换活跃事件
- ❏ NO_EVENT, 无事件发生

2、返回值: 1: 事件进行中; 0: 事件未执行

可选事件 event 列表:

```
include_lib/ 90 #define MAX_EVENT 64
└ audio/
└ common/
└ cpu/
└ decoder/
└ dev_mg/
└ device/
└ encoder/
└ example/
└ fs/
└ liba/
└ msg/
asm_type.h
audio_lib.a
common_lib.a
cpu_lib.a
cpu_sh54_lib.a
decoder_mge_lib.a
dev_mg_lib.a
effect_lib.a
example_lib.a
fs_lib.a
periph_lib.a
sdmmc_lib.a
101 #define EVENT_MIDI_END 6
102 #define EVENT_MIDI_ERR 7
103 // 8
104 #define EVENT_A_END 9
105 #define EVENT_A_ERR 10
106 #define EVENT_A_LOOP 11
107
108 #define EVENT_MP3_END 12
109 #define EVENT_MP3_ERR 13
110 #define EVENT_MP3_LOOP 14
111
112 #define EVENT_APP_SW_ACTIVE 15
113
114 #define EVENT_HALF_SECOND 31
```

事件宏列表图

52.2.7 2.7 函数 `int get_msg(int len, int *msg)`

本函数实现获取事件对应的消息信息以及消息列表中消息的功能当获取到事件消息时，不在获取消息列表中消息，当没有事件消息时，查询消息列表，如果有消息，这保存消息信息到形参 `msg`。形参 `msg` 需要自定义。

- 1、`len`：形参 `msg` 的长度，长度由实际要获取的消息长度决定，`msg` 长度小于要获取的消息长度时返回 `MSG_BUF_NOT_ENOUGH` 错误信息；
- 2、`msg`：保存获取到的消息信息（可获取到的消息如图图 10.1 消息列表图）；
- 3、返回值：

`MSG_NO_ERROR`：获取消息无误

`MSG_BUF_NOT_ENOUGH`：形参 `len` 长度小于实际消息长度

可获取到的消息如图《消息列表图》

52.2.8 2.8 函数 `void clear_all_message(void)`

本函数实现清除所有消息的功能，清除后消息列表为空，没有消息可获取。

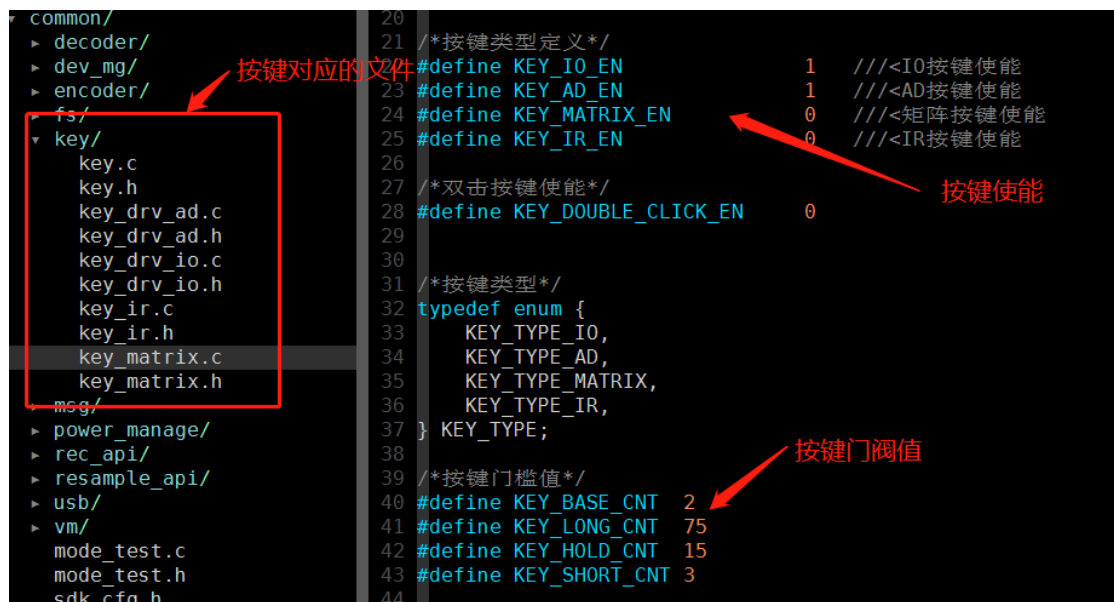
按键系统章节按照下面两个部分进行介绍：

1. 按键类型介绍
2. 按键接口介绍：

SDK 提供了系统按键 key 来进行人机交互。

SDK 包含的按键类型包括有 IO 按键、AD 按键、IR 按键、TOUCH 按键和 MATRIX 按键 5 类。

用户可以在 **key.h** 里选择单独开其中 1 类按键，也可以选择同时开启多类按键。按键可以调按键门阈值。



53.1 按键类型介绍

SDK 包含的按键类型共有 5 类，分别为：

1. *IO* 按键
2. *AD* 按键
3. *IR* 按键
4. *touch* 按键：
5. *MATRIX* 按键

53.1.1 IO 按键

IO 按键的基本配置在 “key_drv_io.h” 文件中。

IO 按键需要配置选择的 IO 口和 IO 是否需要内部上下拉。

小技巧： SDK 中配置了 3 个 IO 按键，其配置如下：

```
#define IS_KEY0_DOWN()      (!(JL_PORTA->IN & BIT(7)))
#define IS_KEY1_DOWN()      (!(JL_PORTA->IN & BIT(8)))
#define IS_KEY2_DOWN()      (!(JL_PORTA->IN & BIT(9)))

#define KEY_INIT()          do{\
    /**key0 init*/\
    JL_PORTA->PU |= BIT(7),\
    JL_PORTA->PD &= ~BIT(7),\
    JL_PORTA->DIR |= BIT(7);\
    /**key1 init*/\
    JL_PORTA->PU |= BIT(8),\
    JL_PORTA->PD &= ~BIT(8),\
    JL_PORTA->DIR |= BIT(8);\
    /**key2 init*/\
    JL_PORTA->PU |= BIT(9),\
    JL_PORTA->PD &= ~BIT(9),\
    JL_PORTA->DIR |= BIT(9);\
}while(0)
```

IO 按键配置

53.1.2 AD 按键

AD 按键的基本配置在“key_drv_ad.h”中。

AD 按键需要配置选择的 ADC 通道和外上拉电阻的阻值，如果外上拉电阻选择阻值为 0，即使选择内上拉电阻。

小技巧： SDK 中配置了 1 个 AD 按键，其配置如下：

```
#define AD_KEY_CH      ADC_CH_PA0
#define EXTERN_R_UP    100//220 -> 22k, 外挂上拉电阻, 0使用内部上拉, 内部上拉为10k
```

AD 按键配置

AD 按键的键值基本配置在“key_drv_ad.c”中。可根据按键连接电阻阻值配置键值。

```
35 #if EXTERN_R_UP
36 #define R_UP      EXTERN_R_UP//220      //22K
37 #else
38 #define R_UP      100      //内部上拉为10K
39 #endif
40
41 #define ADC10_33    (0x3ffL)
42 #define ADC10_30    (0x3ffL*1000/(1000 + R_UP))    //100K
43 #define ADC10_27    (0x3ffL*510/(510 + R_UP))      //51K
44 #define ADC10_23    (0x3ffL*240/(240 + R_UP))      //24K
45 #define ADC10_20    (0x3ffL*150/(150 + R_UP))      //15K
46 #define ADC10_17    (0x3ffL*100/(100 + R_UP))      //10K
47 #define ADC10_13    (0x3ffL*68/(68 + R_UP))         //6.8K
48 #define ADC10_10    (0x3ffL*47/(47 + R_UP))         //4.7K
49 #define ADC10_07    (0x3ffL*22/(22 + R_UP))         //2.2K
50 #define ADC10_04    (0x3ffL*10/(10 + R_UP))         //1K
51 #define ADC10_00    (0)
52
53 #define AD_NOKEY     ((ADC10_33 + ADC10_30)/2)
54 #define ADKEY1_0     ((ADC10_30 + ADC10_27)/2)
55 #define ADKEY1_1     ((ADC10_27 + ADC10_23)/2)
56 #define ADKEY1_2     ((ADC10_23 + ADC10_20)/2)
57 #define ADKEY1_3     ((ADC10_20 + ADC10_17)/2)
58 #define ADKEY1_4     ((ADC10_17 + ADC10_13)/2)
59 #define ADKEY1_5     ((ADC10_13 + ADC10_10)/2)
60 #define ADKEY1_6     ((ADC10_10 + ADC10_07)/2)
61 #define ADKEY1_7     ((ADC10_07 + ADC10_04)/2)
62 #define ADKEY1_8     ((ADC10_04 + ADC10_00)/2)
63
64 const u16 ad_key_table[] = {
65     ADKEY1_8, ADKEY1_7, ADKEY1_6, ADKEY1_5, ADKEY1_4,
66     ADKEY1_3, ADKEY1_2, ADKEY1_1, ADKEY1_0
67 };
68 #endif
```

AD 按键键值配置

53.1.3 IR 按键

IR 按键的基本配置在“key_ir.h”中。

IR 按键需要配置 IR 引脚和 TIMER 源。

小技巧： SDK 中配置了 1 个 IR 按键，其配置如下：

```
4 #include "gpio.h"
5 #include "irflt.h"
6
7 #define IR_KEY_IO          IR_KEY_IO_SEL
8 #define IR_KEY_TIMER      SEL_TIMER2
9
10 extern const key_interface_t key_ir_info;
11 #endif
12
```

IR 按键配置

53.1.4 MATRIX 按键

MATRIX 按键的基本配置在“ke_matrix.h”中。

MATRIX 按键需要配置 X 轴的 ADC 通道和 Y 轴的 IO 口，并且需要配置 ADC 通道检测到有按键的门阈值。

```
///X轴 io 要求是AD口，详细AD口看adc_drv.h
#define X_ADC_CH_INIT {ADC_CH_PA2,ADC_CH_PA3,ADC_CH_PA5}
///Y轴 io 要求是普通IO口
#define Y_PORT_INIT {IO_PORTA_10,IO_PORTA_11}

#define MATRIX_KEY_THRESHOLD      300
```

MATRIX 按键配置

53.1.5 touch 按键：

触摸按键识别的原理：

触摸按键引脚，其网络对地存在寄生电容。

当人手接触网络时，寄生电容会变大。

常态下，引脚为输出，并输出高，保证寄生电容满电。

当引脚突然设为输入，关上拉，开下拉，则寄生电容的电会下拉电阻放电，当程序读引脚电平为 0 时，则表示放电结束。

寄生电容越大则放电时间就越久（如人手触摸该引脚，相当于寄生电容增大）。

此时我们可以通过一个计数模块，来计算放电时间，通过时间的变化量来判断是否有人手触摸。

SDK 参数配置:

触摸按键共关心三个参数: 引脚, 标定周期, 变化阈值。通过 key_touch.h 修改。

```

1 #ifndef _KEY_TOUCH_H
2 #define _KEY_TOUCH_H
3
4 #include "key.h"
5 #include "gpio.h"
6
7 #define TOUCH_KEY_INIT          TOUCH_KEY_SEL
8
9 //滤波算法用到
10 #define TOUCH_VAL_CALIBRATE_CYCLE    100    //标定常态值的时间, 单位为key_scand的时间
11 #define TOUCH_DELDA                1000    //手按下时变化量的阈值, 大于阈值说明被按下
12
13 extern const key_interface_t key_touch_info;
14
15 #endif
16

```

key_touch.h

TOUCH_KEY_SEL //触摸用到的引脚自行定义

```

5 //TOUCH KEY
6 #define TOUCH_KEY_SEL          {IO_PORTA_09,IO_PORTA_10,IO_PORTB_03}
7

```

触摸按键 IO 配置

小技巧:

TOUCH_VAL_CALIBRATE_CYCLE

//标定常态值的时间, 单位为 key_scand 的时间, 默认配置是 1s 左右。

//这是 SDK 参考的标定常态值的算法用到, 如果用户有自己的一套算法, 该参数可以不用理会。

TOUCH_DELDA

//手按下时变化量的阈值, 大于阈值说明被按下。

//这个阈值是要调试出来的, 先给的很大的值, 保证不触发按键。下文会告诉怎么调试这个值。当然, 这是 SDK 参考的处理变化量的的算法用到, 如果用户有自己的一套算法, 该参数可以不用理会。

触摸按键调试

key.h 文件使能触摸按键:

```

20 typedef int (*key_emit_t)(us key_status, us key_num, us key_type);
21
22 /*按键类型定义*/
23 #define KEY_IO_EN          0    ///

```

key_touch.c 文件打开按键扫描的打印:

```

76  /*-----*/
77  static u8 get_touch_key_value(void)
78  {
79      u16 key_or = 0;
80      u32 cur_val = 0;
81      for (u8 i = 0; i < TOUCH_KEY_NUM; i++) {
82          cur_val = get_pl_cnt_value(i);
83          log_print("%d ", cur_val);
84          //简单滤波
85          if (touch_pre_value[i] == 0) {
86              touch_pre_value[i] = cur_val;
87          } else if (cur_val >= touch_pre_value[i]) {
88              touch_pre_value[i] = touch_pre_value[i] + (u32)((cur_val - touch_pre_value[i]) * 0.2f);
89          } else {
90              touch_pre_value[i] = touch_pre_value[i] - (u32)((touch_pre_value[i] - cur_val) * 0.2f);
91          }
92          //处理滤波之后的值
93          if (touch_pre_value[i] > (touch_normal_value[i] + TOUCH_DELDA)) {
94              key_or |= BIT(i);
95              touch_calibrate_cnt[i] = 0;
96          } else {
97              touch_calibrate_cnt[i] ++;
98          }
99          //定期标定常态下的基准值
100         if (touch_calibrate_cnt[i] > TOUCH_VAL_CALIBRATE_CYCLE) {
101             touch_normal_value[i] = touch_calibrate_tmp_value[i] / 10;
102             touch_calibrate_tmp_value[i] = 0;
103             touch_calibrate_cnt[i] = 0;
104         } else if (touch_calibrate_cnt[i] >= (TOUCH_VAL_CALIBRATE_CYCLE / 2)) {
105             if (touch_calibrate_cnt[i] < ((TOUCH_VAL_CALIBRATE_CYCLE / 2) + 10)) {
106                 touch_calibrate_tmp_value[i] += touch_pre_value[i];
107             }
108         } else {
109             touch_calibrate_tmp_value[i] = 0;
110         }
111     }
112     log_print("\n");
113     static u8 pre_1 = 0;

```

打开打印

下载程序，可以看到，按键的常态下的计数值，参考如下：

```

186 186 186
186 190 186
186 186 186
186 190 134
186 186 186
186 190 186
186 186 186
186 186 186
186 186 186
186 190 186
186 186 186
186 190 186
186 186 186
186 190 186
186 186 186
186 190 186
186 186 186
186 190 186
186 186 186

```

第一列是按键0

第二列是按键1

第三列是按键2

当按键被手触摸时，计数值会变大，参考如下：

```

288 186 3157
186 186 3157
186 186 3320
186 186 3186
186 186 3354
186 186 3176
186 186 3339
186 186 3176
186 186 3320
186 186 3258
186 186 3301
... ..

```

可以看出变化量有到 3000 左右，那么此时，我们可以将变化量的阈值设为 1000 左右就行，回到 key_touch.h 文件修改阈值：

```

#define TOUCH_DELDA 1000

```

触摸按键的数据处理

SDK 提供的是参考的示例，其中包括简单的滤波，滤波后的值的处理，常态值的标定，这些用户都可以自己修改，或者完全重写重新自定义。

主要涉及 pl_cnt.c 、 pl_cnt.h 、 key_touch.c 、 key_touch.h 文件

53.2 按键接口介绍：

53.2.1 函数 void key_init(void)

此函数根据 key 的使能类型来初始化不同的 key。

53.2.2 函数 void key_table_sel(void *msg_filter)

此函数用来注册一个 msg_filter 函数，来获取 key 事件对应的 msg。| 1、msg_filter：key 事件转换成消息的函数接口。

53.2.3 函数 void key_scan(void)

此函数在 tick_timer 中断中调用，用来扫描处理 key 事件。

Reserved_Area 区域说明

可以选择 FLASH 最后的 4K 作为一个特殊的保留区，这个保留区只有使用烧写器烧写程序才能进行擦写动作。一些希望烧写完程序后就不经常给擦写的信息可以选择放到这个区域里面。使用这个保留区需要在 isd_config.ini 文件中把 NEED_RESERVED_4K 置成 YES。
第三方数据 (认证码 / 三元组 / 序列号)



```
NEED_RESERVED_4K=YES; //关闭4K保留
AUTH_CODE=1;
```

图 1 isd_config 中配置保留区

54.1 认证码相关信息

认证码是一个烧写在保留区 (Reserved_Area 区域) 中的一个信息，用来给每一个产品烧写一个特殊的 ID 使用。

认证码功能需要搭配烧写器支持，而支持该功能的一拖二烧写器最低版本号为：2.27.9

54.2 认证码功能 isd_config_ini.c 配置

认证码使用的时候需要在 isd_config.ini 文件添加相关信息（如果不需要认证码功能，第 2、3 点需要注释掉）

- 1: 把 NEED_RESERVED_4K=YES;
- 2: 在 [SYS_CFG_PARAM] 段中的如图位置，增加一个 AUTH_CODE=1; 配置项。

3: 在 [BURNER_PASSTHROUGH_CFG] 段中的如图位置, 增加一个 AUTH_CODE_IN_FLASH=1; 配置项。

```

isd_config_ini.c
49
50
51 [CHIP_VERSION]
52 SUPPORTED_LIST=A
53
54 ##### UBOOT配置项
55 [SYS_CFG_PARAM]
56 // #clk [0-255]
57 // #data_width[0 1 2 3 4] 3的时候uboot自动识别2或者4线
58 // #mode:
59 // #      0 RD_OUTPUT,      1 cmd      1 addr
60 // #      1 RD_I/O,        1 cmd      x addr
61 // #      2 RD_I/O_CONTINUE] no_send_cmd x add
62 // #SPI = data_width, clk, mode;
63 AUTH_CODE=1;
64 SPI=2_3_0_0; #data_width, clk, mode;
65 // #OSC=btosc;
66 // #OSC_FREQ=24MHz; #[24MHz 12MHz]
67 // #SYS_CLK=48MHz; #[48MHz 24MHz]
68 UTTX=PA06; //uboot串口tx
69 UTBD=1000000; //uboot串口波特率
70 // #UTRX=DP; //串口升级 [PB00 PB05 PA05], 默认PB05
71 // #RESET=PB01_08_0; //port口_长按时间_有效电平 (长按时间有00、04、0
72 // #CACHE_WAY=1; // 范围1~4
73 // #WAIT_TIME=10; // * 100ms
74 #if TFG_EXT_FLASH_EN

```

图 2 isd_config 文件 [SYS_CFG_PARAM] 段增加 AUTH_CODE 配置项

```

isd_config_ini.c
85
86 #####
87 [TOOL_CONFIG]
88 1TO2_MIN_VER=2.27.1
89 1TO8_MIN_VER=3.1.17
90 //BTBOX_MIN_VER=1.2.4.f
91 #####
92
93 #####
94 //烧写器配置项
95 //OTP_BOOT 0, FLASH模式 1, OTP模式
96 #####
97 [BURNER_PASSTHROUGH_CFG]
98 AUTH_CODE_IN_FLASH=1;
99 LIGHTING2TYPE=0 //uboot_start
100 OTP_BOOT=0
101 // #FLASH_WRITE_PROTECT=NO;
102
103 #####
104 //烧写器配置项
105 //2.078v 2.188v 2.297v 2.406v 2.515v 2.624v 2.733v 2.840v

```

图 3 isd_config 文件 [BURNER_PASSTHROUGH_CFG] 段增加 AUTH_CODE_IN_FLASH 配置项

54.3 认证码信息接口

54.3.1 void rsv_auth_analysis(void)

该接口用于打印出烧写在芯片的认证码信息。
里面的 auth->data_p 即为烧录进去的认证码 (三元组 / 序列号)

```
24 *start addr:reserved area last 256 byte
25 data start:start addr + 80 byte
26 */
27
28 void rsv_auth_analysis(void)
29 {
30     auth_header_t *auth = (auth_header_t *)flash_addr2cpu_addr(rsv_flash_addr_get() + RESERVED_AREA_SIZE - 256 + 80);
31     u8 *crc_data = (u8 *)(&auth->data_len);
32     u32 crc_len = auth->data_len + sizeof(auth->data_len);
33     u16 crc16 = CRC16(crc_data, crc_len);
34
35     if (crc16 != auth->crc16) {
36         log_error("no auth data !!!\n ");
37         return;
38     }
39
40     log_info("auth data:");
41     log_info_hexdump((u8 *)auth->data_p, auth->data_len);
42 }
43
```

对应烧写在芯片的认证码 / 三元组 / 序列号

图 4 认证码信息

54.4 认证码文件说明

认证码文件的格式是：认证码格式头 + 数据。

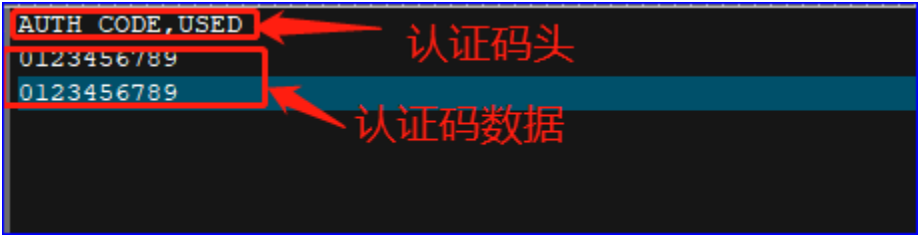


图 5 认证码文件格式

认证码的数据烧写到固件的时候会有 2 种格式

当文件的后缀名为 “*.csv” 时候，数据格式按照 ASCII 方式读取烧写。
当文件的后缀名为 “*.hex” 时候，数据格式按照 HEX 方式读取烧写。

多录音分区 (AD14N、AD24N)

AD14N、AD24N 目前支持多录音分区功能，能在各个分区中独立的进行录音操作。本章着重介绍录音的多分区功能实现流程。将会涉及到 `app_record.c` 以及 `nor_fs_resource.c`。

55.1 1 多录音分区一般流程

在开机初始化中会调用 `void norfs_init_api(void)`，此函数实现对录音区域的初始化，会初始化录音的起始地址、结束地址以及字节大小等变量。在分区前需要确保有足够的空间能够存放至少 256 字节。此流程中以将原本的录音地址一分为二，实现双录音分区为例。

55.1.1 1.1 创建录音分区初始化

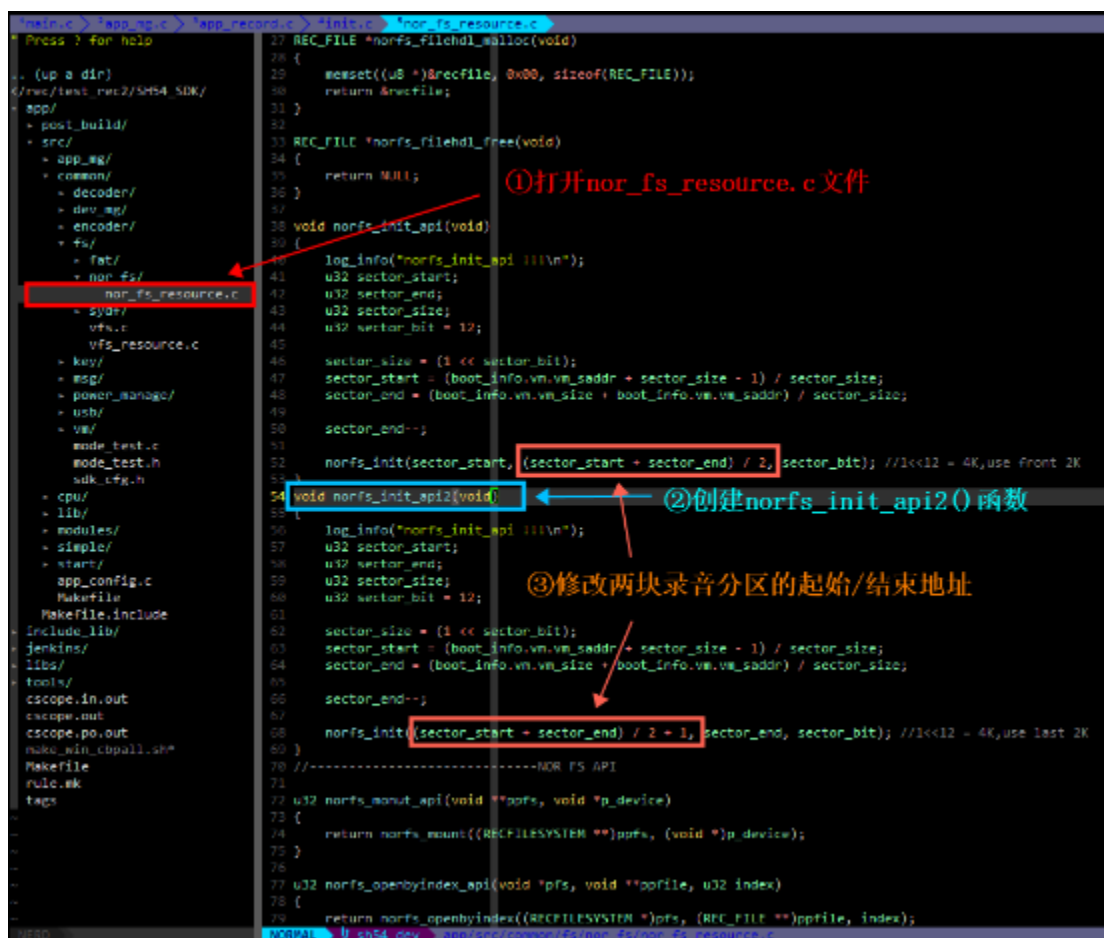


图1 创建录音分区初始化

创建双录音分区初始化步骤：

- ① 打开nor_fs_resource.c文件；
- ② 仿照函数norfs_init_api()创建录音分区2的初始化函数norfs_init_api2()；
- ③ 修改录音分区1函数norfs_init_api()中，norfs_init()传入的结束地址。本章中
→ 将其改为原来长度的一半；
- ④ 修改录音分区2函数norfs_init_api2()中，norfs_init()传入的起始地址。本章中
→ 将其改为同录音分区1同长度；

详情步骤可参考图1。同理，多录音分区也可根据上述步骤修改进行录音分区初始化。

若不选择默认录音地址，可在isd_config.ini文件中自行开辟地址。

55.1.2 1.2 创建录音切换分区消息事件并设置键位

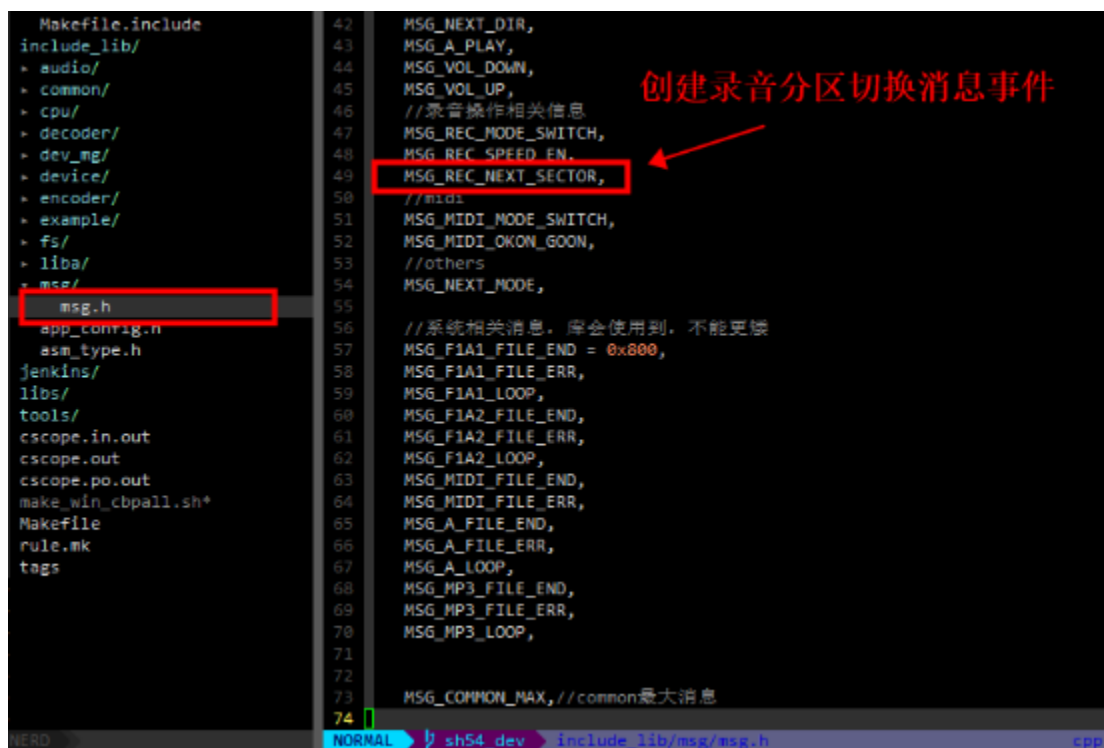


图 2 创建录音切换分区消息事件



图 3 设置录音切换分区键位

创建录音切换分区消息事件并设置键位步骤：

- ① 打开msg.h文件；
- ② 如图2，在录音操作相关信息处添加录音切换分区消息事件

MSG_REC_NEXT_

(续下页)

(接上页)

- ↪SECTOR;
- ③ 打开app_record_key.c文件;
- ④ 如图3, 在IOKEY_RECORD_SHORT_UP定义中添加IOKEY 3作为切换录音分区按键;

可自行命名切换消息事件和选择切换的键位。

55.1.3 1.3 在 app 录音任务中实现录音分区切换操作



图 4 在 app 录音任务中实现录音分区切换操作

在 app 录音任务中实现录音分区切换操作步骤:

- ① 打开nor_fs_resource.c文件;
- ② 创建录音分区切换标志`norfs_sector_flag`;
- ③ 关闭正在编码或者正在播放的录音任务, 防止任务干扰分区的切换;
- ④ 根据标志选择初始化的录音分区;
 - a. 标志为0时, 切换至分区2, 同时将标志置1;
 - b. 标志为1时, 切换至分区1, 同时将标志置0;
- ⑤ 播放所在分区最新的录音文件;
 - a. 初始化解码器;
 - b. 打开录音设备;
 - c. 进行录音解码, 解码会自动选择该分区最新的文件进行播放;

详情步骤可参考图 4。同理, 多录音分区也可根据上述步骤, 通过增加标志的置来选择多录音分区切换。

AD15N 从 v1.7.0 版本 SDK 开始提供了虚拟 RTC 功能示例，其功能包含读写时钟、闹钟，可在 powerdown、softoff 情况下走时并且响应闹钟。使用时需要开启 app_modules.h 中的 rtc 功能宏，相关应用示例位于 app/bsp/common/rtc/rtc.c。

目前支持虚拟 RTC 的芯片有：AD15N

注：虚拟 rtc 在 powerdown、softoff 情况下运行会六个小时自动唤醒一次，否则时间会不准！

56.1 功能接口介绍

56.1.1 int rtc_simulate_init(const struct vir_rtc_data *arg)

该函数主要实现虚拟 rtc 功能初始化。其中参数：

1、vir_rtc_data：初始化的时钟、闹钟和回调信息；

56.1.2 void read_sys_time(struct sys_time *curr_time);

该函数实现读取当前 rtc 运行的时钟。其中参数：

1、curr_time：读取到的时钟信息，函数会将读取到的时钟更新到该结构体；

56.1.3 void write_sys_time(struct sys_time *curr_time);

该函数实现写入时钟。其中参数：

1、curr_time：读取到的时钟信息，函数会将读取到的时钟更新到该结构体；

56.1.4 void read_alarm(struct sys_time *alarm_time);

该函数实现读取闹钟。其中参数：

1、alarm_time：读取到的闹钟信息，函数会将读取到的时钟更新到该结构体；

56.1.5 void write_alarm(struct sys_time *alarm_time);

该函数实现写入闹钟。其中参数：

1、alarm_time：写入的闹钟信息；

56.1.6 void vir_set_alarm_ctrl(u8 set_alarm);

该函数实现设置闹钟使能。其中参数：

1、set_alarm：闹钟使能标志；1：闹钟使能， 0：闹钟失能。

56.1.7 u8 vir_get_alarm_ctrl(void);

该函数实现获取闹钟使能标志。其中参数：：

1、返回值：闹钟使能标志。1：闹钟使能， 0：闹钟失能。

57.1 本内存管理驱动特性与限制

本驱动基于 **FREE RTOS** 的 `heap_2.c`，在此基础上修改 增加了对数据类型的定义。

由于采用纯软件方式管理内存分配，无法解决因频繁申请与释放导致的内存碎片问题。

基于此限制，本驱动 仅用于为系统分配各类句柄；

增加的数据类型定义，在释放内存时，驱动并不实际释放内存，而是仅将该内存块标记为可用。该内存块在后续同一数据类型申请时可被复用，但不能用于不同类型的数据申请。

57.1.1 内存管理方式

- 纯软件管理内存分配
- 无法解决频繁申请释放导致的内存碎片问题
- 仅用于为系统分配各种句柄
- 增加了数据类型定义支持

57.1.2 内存释放机制

注意：本驱动的内存释放机制与常规内存管理不同：

1. 不真正释放内存
2. 仅将释放的内存标记为可用
3. 释放后的内存只能再次分配给同类型的数据

57.1.3 工作流程示例

1. 申请 TypeA 类型内存 → 分配新内存块
2. 释放 TypeA 类型内存 → 标记为 "TypeA 可用"
3. 再次申请 TypeA 类型 → 复用步骤2的内存
4. 申请 TypeB 类型内存 → 分配新内存块 (不可复用 TypeA 内存)

备注:

1. **碎片问题:** 由于不真正释放内存, 长期运行仍可能产生碎片
2. **类型隔离:** 不同类型内存完全隔离, 不能交叉使用
3. **适用场景:** 适合句柄类固定大小、类型明确的数据分配
4. **性能影响:** 减少实际分配次数, 但增加类型匹配开销

57.2 函数接口说明

57.2.1 函数 void vPortInit(void *pAddr, uint32_t xLen)

此函数为指定内存管理单元分配物理区域:

1. pAddr: 分配给内存管理单元的起始位置;
2. xLen: 分配给内存管理单元的字节长度;
3. 返回值:
 - 成功 0;
 - 失败 错误请查看 errno-base.h。

57.2.2 函数 void vPort_disable()

此函数标记内存单元为未初始化, 调用此函数后, 可以重新调用函数 vPortInit, 实现内存管理的复位。

备注:

用户若调用 vPort_disable() + vPortInit(), 内存管理完全复位;
那么之前申请到的所有内存将失效, 客户不可以再使用之前申请的内存。
为避免产生野指针问题, 不建议客户使用此方法。

57.2.3 函数 void *pvPortMalloc(size_t xWantedSize, mm_type type)

申请内存:

1. xWantedSize: 申请的长度;
2. type: 申请的数据类型;
3. 返回值:
 - 成功 内存指针;
 - 失败 0。

57.2.4 函数 void vPortFree(void *pv)

释放内存：

1. pv：内存指针；

备注：

内存释放时，本驱动并不执行实际的物理释放操作，而是将该内存块标记为逻辑可用。此内存块仅可被后续同数据类型的申请请求复用，不支持跨类型分配。

以指针方式直接访问系统 Flash 中的资源文件

系统上电初始化后，CPU 会通过内存控制器将系统 Flash 的映射到内存地址空间。这部分映射区域可以通过 Cache 进行加速访问。通过文件系统（VFS）获取到资源文件在系统 Flash 中的 ** 物理起始簇号 ** 后，结合已知的系统 Flash 映射基地址，即可计算出该文件内容在内存中的 ** 直接访问地址 **。

58.1 示例代码

首先，需要通过虚拟文件系统（VFS）打开目标文件并获取其关键属性。

```
1 void *pvfs = 0;    // 文件系统句柄
2 void *pvfile = 0;  // 文件句柄
3 // ... 此处为打开文件 (pvfs, pvfile) 的代码 ...
4
5 struct vfs_attr attr;           // 文件属性结构体
6 vfs_get_attr(pvfile, &attr);    // 获取文件属性
7 u8* pdata = (u8 *) (boot_info.sfc.app_addr + attr.sclust);
8 u32 file_len = attr.fsize;
```

58.2 关键变量说明

- boot_info.sfc.app_addr: 系统启动信息中记录的应用程序区 Flash 映射到内存的 ** 基地址 **。
- attr.sclust: 通过 vfs_get_attr 获取的文件 ** 起始簇号 ** (Cluster Number)。
- attr.fsize: 通过 vfs_get_attr 获取的 ** 文件实际大小 ** (字节)。
- pdata: 计算得到的，指向文件内容首字节的 ** 直接内存指针 **。

58.3 重要注意事项

注意:

1. Cache 管理范围限制

使用此方式访问的数据受到 CPU Cache 管理范围的限制。不同芯片型号的可用地址空间大小不同:

- **AD14N**: 通常为 **32M 字节**
- 其他 AD 系列 (如 AD15N, AD17N 等): 通常为 **64M 字节**
- 具体限制请以项目链接脚本 (**.ld 文件**) 中的定义为准, 超出范围的地址可能无法通过 Cache 正确访问。

注意:

2. 地址的持久有效性

一旦成功通过 `pdata = (u8 *) (boot_info.sfc.app_addr + attr.sclust)` 计算出地址, 该指针在文件关闭后依然有效并可以继续使用。这意味着你可以安全地关闭文件句柄 (`pvfile`) 以释放系统资源, 而不会影响通过 `pdata` 对文件数据的读取。

58.4 相关资源

- **演示视频**: 杰理 AD 用指针方式直接访问系统 flash 中的资源文件 (<https://www.bilibili.com/video/BV1grqnBiEZS/>)

CHAPTER 59

Indices and tables

- `genindex`
- `modindex`
- `search`